

MODELLING OF COMPLEX SYSTEMS

Prof. dr. Marc Denecker

March 19, 2025

Compiled March 19, 2025.

“There is nothing as practical as a good theory.”
– Kurt Lewin

Contents

1	Introduction	1
1.1	Concepts in modelling	2
1.1.1	Conclusion: “ <i>Get the models right!!</i> ”	12
1.2	A modelling logic $\mathcal{L}$	14
1.3	Use of Information: computational problems and inference	20
1.4	Use of modelling in software engineering	25
1.5	Conclusions of this chapter and goals of the course	28
1.6	Important for the exam	28
2	Predicate Logic and extensions	31
2.1	Short history	31
2.2	First-order Logic (FO)	32
2.2.1	Formal Syntax	32
2.2.2	Formal semantics	38
2.2.3	Informal semantics	44
2.2.4	Derived semantical concepts of the modelling logic FO	48
2.2.5	Useful forms of inference	48
2.2.6	Basic properties	48
2.2.7	Laws of thought and pushing negation inside	50
2.2.8	Some examples	52
2.3	Pragmatics, implicatures and expressivity of FO	55
2.3.1	Unary versus binary quantification	56
2.3.2	Common implicatures and their expression in FO	57
2.3.3	Definitions of concepts in FO	60
2.3.4	Material implication $\Rightarrow$	62
2.3.5	Conclusion	63
2.3.6	Exercises	63
2.4	Extensions of Classical Logic	65
2.4.1	Extending FO with types: FO(Types)	65
2.4.2	Extending FO with arithmetic: FO(Types,Arit)	67
2.4.3	Second and higher order logic	68
2.4.4	Extending FO with identifiers and domain/type closure	69
2.4.5	Extending FO with aggregates: FO(Types,Arit,Agg)	70
2.4.6	Extending FO with partial functions	73
2.4.7	Extending FO with definitions: FO(ID)	74
2.4.8	Summary	92
2.5	An example of FO($\cdot$): Steel oven scheduling	92
2.6	Axiomatising complex domains	94
2.6.1	Generalising UNA and DCAin FO($\cdot$)	94
2.6.2	Databases from the logical point of view	98

2.6.3	Peano arithmetic	102
2.6.4	Logic Programming as a declarative language	105
2.7	Important for the exam	107
3	Modelling dynamic systems in classical logic	109
3.1	Introduction: Modelling dynamic systems in LTC	109
3.2	A historical note on the frame problem in early AI	112
3.3	Expressing inertia through causal laws	114
3.3.1	Discussion of LTC	120
3.3.2	Generalizations of the LTC	122
3.3.3	Temporal reasoning in the 1970s: the Yale turkey shooting problem	128
3.4	Inference on LTC	132
3.4.1	Applications of Finite Model Generation	132
3.4.2	Progression inference and (interactive) simulation inference	134
3.4.3	Lightweight verification by finite model generation	135
3.4.4	Heavyweight verification of invariants	136
3.4.5	Example: Combining light- and heavyweight verification	141
3.4.6	Reasoning on programs	143
3.5	LogicBlox: software by “running” specifications	147
3.6	Important for the exam	149
4	LTL, CTL and verification by model checking	151
4.1	Model checking	151
4.1.1	ProB	154
4.2	Linear-Time Logic (LTL)	156
4.2.1	Syntax and semantics	156
4.2.2	Translation to FO	159
4.2.3	Practical patterns of specifications	160
4.2.4	Important equivalences of LTL	161
4.2.5	Example: mutual exclusion (locks)	162
4.2.6	Mutual exclusion in ProB	164
4.3	Fairness constraints	166
4.4	CTL and CTL*: branching time logics	167
4.4.1	Syntax of CTL*	167
4.4.2	Semantics of CTL*	168
4.4.3	Embedding LTL in CTL*	169
4.5	CTL: a subformalism of CTL*	169
4.5.1	Syntax and semantics CTL	169
4.5.2	Practical patterns of specifications	172
4.5.3	Logical analysis of CTL	174
4.5.4	On the use of LTL and CTL and CTL*	176
4.6	Important for exam	177
5	Refinement in Event-B	179
5.1	Introduction	179
5.2	Real-world examples	181
5.3	Refinement	182
5.3.1	Event-B and Rodin	183
5.3.2	Building a copier by refinement	184
5.3.3	Proof obligations	185
5.3.4	Event refinement	186
5.3.5	Building a copier by refinement, continued	187

5.3.6	Correctness of Refinement	189
5.4	More about proof obligations	191
5.5	Compiling Event-B to programs (not for exam)	194
5.6	Conclusion	197
5.7	Important for the exam	198
6	Classical results on Predicate Logic: Provability, Expressivity, Decidability, Incompleteness	199
6.1	Proof theory	199
6.1.1	Background: deductive inference	199
6.1.2	Hilbert Proof systems	200
6.1.3	Soundness and completeness	203
6.2	Expressivity analysis in FO: an introduction	205
6.2.1	Proving expressivity	205
6.2.2	Inexpressivity analysis	207
6.2.3	Summary of inexpressivity results	214
6.2.4	Facultative material: about non-standard models of PA^{FO}	215
6.3	Undecidability and Gödel's incompleteness theorem	216
6.3.1	Undecidability of deduction	216
6.3.2	Gödel's Incompleteness Theorem	217
6.3.3	Undecidability of the natural numbers	218
6.3.4	Incompleteness of SO and FO(ID)	219
6.4	Implications for “deductive logic”	222
6.5	Important for exam	223
7	Inference and the FO(.)-KB project	225
7.1	Motivation	225
7.2	Why FO as a foundation?	227
7.3	The knowledge base paradigm	228
7.4	A KB-solution for Interactive Configuration	229
7.5	Inference with definitions	233
7.6	Imperative + Declarative Programming in IDP3	234
7.7	Various forms of inference in IDP	235
7.8	Reasoning on LTC theories	237
7.9	Conclusion	238
7.10	Important for the exam	238
8	Algorithms for finite satisfiability checking in propositional and predicate logic	241
8.1	Complexity of satisfiability in propositional logic	241
8.1.1	Propositional logic	241
8.1.2	Satisfiability inference in PC	242
8.1.3	Complexity of tautologicity checking in CNF	245
8.2	SAT algorithms	249
8.2.1	DPLL: a basic SAT solver	249
8.2.2	Conflict-Driven Clause learning (CDCL)	252
8.2.3	Optimizations of CDCL	258
8.2.4	Summary	259
8.3	Other forms of inference in PC	260
8.4	Finite model expansion in IDP	260
8.4.1	Type inference.	261
8.4.2	Extend the input structure	261

8.4.3	Simplify the input theory	262
8.4.4	Ground	263
8.4.5	Solve	263
8.4.6	Post-evaluation	263
8.5	Important for exam	264
9	Algorithms for CTL and LTL model checking	265
9.1	CTL model checking algorithm	265
9.1.1	Background	265
9.1.2	Without fairness constraints	266
9.1.3	With fairness constraints	267
9.2	An LTL model checking algorithm	270
9.2.1	Intuition	271
9.2.2	Summary of the method	277
9.3	Important for exam	278

Foreword

History of this textbook

The course Modelling of Complex Systems has been taught in the master of Computer Science of KU Leuven since the academic year 2007-2008. In the years before, DTAI and Distrinet had both advocated to organize a course on formal modelling and knowledge representation. This collaboration strongly influenced the course. While the perspectives on modelling in formal systems and knowledge representation in AI are quite different, there is obviously a big overlap: it is both about modelling information in a formal language. As such this course arose as a synthesis of these two perspectives and reconciliates them. Chapter 2 and 3 take the knowledge representation point of view, Chapters 4 and 5 take the formal modelling point of view, while the rest focus on theoretical expressivity results and algorithms. A strong influence on this course was the book of Huth and Ryan, *Logic in Computer Science: modelling and reasoning in systems*. This book covers many aspects of modelling and reasoning, but suffers the price of its breadth, in that it is rather shallow. Over the years, it led me to take more and more distance, and increasing my own contribution to the topic, e.g., in the form of the logic FO(.) and the usage of the IDP system.

Initially the course materials consisted of the book of Huth and Ryan, and slides. When the distance between the book and the content of the course grew, I decided to compile the slides into course notes. The course notes still bears the consequences of this origin, e.g., the many paragraph titles in certain parts of it stem from titles of slides. This is less than optimal but hopefully it should not affect the readability and clarity of the text.

Another factor that has strongly affected the structure of these course notes is the pressing need induced by the exercise sessions and the project. Students are trained to formalize knowledge using the logics in the course. Therefore, these logics (FO, FO(.), Event-B, and temporal logics LTL, CTL) must be introduced early in the course. It meant that theoretical analysis and algorithms of FO is delayed until after LTL and CTL. That is the reason for this way of structuring the material, but it is clear that this is not the most logical structure.

The systems of the course: IDP and ProB

In the course, two systems are used: IDP3 and ProB.

The IDP3 system is a knowledge base system supporting a number of different forms of inference for the FO(.) language. The FO(.) language is the name for the extension of classical first-order logic, denoted FO, with various extensions: types, aggregates, arithmetic, inductive definitions. IDP3 is used in exercise sessions of Chapter 2 and 3 and in the project.

ProB is an inference system for a formal language called Event-B, and for Linear Time Logic (LTL). Event-B is a formal logic developed for formal modelling and for supporting

the development methodology of refinement. ProB is used for the exercises of Chapter 4 and 5 and in the second part of the project.

For installing these systems, I refer to the Toledo-pages of the course.

The course is accompanied by many online examples in IDP. The names of those online examples will appear as buttons in the margin. For readers of the digital edition of this book, these are clickable; others can manually browse to the example by prefixing its name by `https://idp.cs.kuleuven.be/idp/?present=` in a web browser.

Mathematical theorems

All mathematical textbooks possess definitions. As for theorems, often distinction is made between different sorts: *Theorem*, *Proposition*, *Lemma*, *Corollary*. They differ in importance. This textbook takes the following approach:

- *Theorem*: important proposition, a milestone of the subject.
- *Proposition*: less important proposition but still useful later.
- *Lemma*: an auxiliary theorem, helpful mainly to prove a Theorem or Proposition.
- *Corollary*: a proposition of interest follows very easily from another Theorem or Proposition.

MARC DENECKER, 2023.

Chapter 1

Introduction

Judging by its title, this course is about modelling systems.

The word “system” should be understood in a broad sense: a system can be any domain involving a collection of any sorts of objects and sorts of relations between them. Time may be one of the type of objects, in which case evolving objects and relations may be part of the system. It then is a dynamic domain, including, e.g., agents performing actions that change the world, such as a planning domain, or a piece of software. But a system may also be a static system, such as the design of a seal, the positioning of boxes in a container, a schedule, a study program, etc.

Such systems will be modelled. To “model” is a verb, used in this course as a synonym of terms such as “to specify”, “to represent knowledge”. The result of this act is “a modelling”, “a knowledge base”, “a specification”, “a logic theory”. These are terms used in different research communities, but they will be synonyms here.

Perhaps the most evident place where modellings are built, is in empirical formal sciences. These are sciences that use mathematical theories to model an observable reality. E.g., we could say that Newton’s theory of gravitation is a mathematical modelling of gravitation. A more standard term for “a modelling” is the word “a model”, which is a term often used in engineering and empirical science, as in a “a climate model” or as “the mathematical model of a certain combustion engine”. However, we cannot use this term here in this course, since in the context of logic, the word “a model” has a different meaning from how it is used elsewhere in science, and we will use the term “a model” in the logical sense. In the logical sense, a “model” is a mathematical, abstract representation of a state of affairs that satisfies a “modelling”. E.g., Newton’s gravitation theory is a modelling, and any mathematical solution of it describing trajectories of n objects possessing mass, is a “model” of Newton’s “modelling”. This conforms the way these terms are interpreted in logic.

The modellings in this course are expressed in a logic. The course will study several logics in some detail (classical first-order logic (FO) and extensions of it, linear-time logic (LTL), computation tree logic (CTL), Event-B). As such, logics are an important topic of this course, and the course aims to give an introduction to several theoretical aspects of them: syntax, formal semantics, informal semantics, expressivity, different sorts of inference. If these terms are not clear to you now, I certainly hope they will be clearer at the end of the course.

Modellings, e.g., scientific theories, are clearly not programs: they are not executable, they cannot be run, they don’t do anything. So, why then is a course on modelling a compulsory

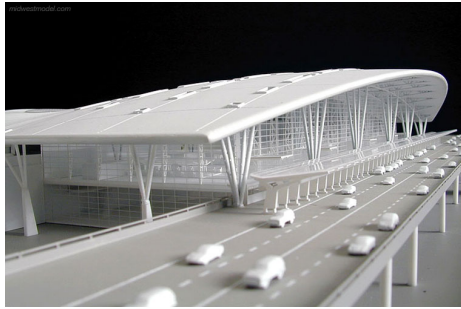


Figure 1.1 – A scale model of the Indianapolis international airport

course in the Master of Computer Science? This is because modellings describe the application domain, and the information they contain may be useful to solve problems. One of the goals of this course is to develop an idea of how modellings can be used to solve a range of computational problems. That will bring us close to the topic of declarative programming languages (e.g., Prolog, Haskell) and even closer to declarative problem solving paradigms (e.g., constraint solving languages, semantic web languages).

The goals of the course are not only on the theoretical side. One goal is of a practical nature: to develop practical skills to use logics for modelling, and to develop an understanding how these modellings can be used to solve problems. These skills are mainly taught in the exercise sessions and in the project, so make sure that you attend these.

The rest of this first chapter develops a view on modelling based on formal science. It uses Newtons theory of gravitation as an example to explore the principles underlying mathematical modelling. The example is taken from physics, but I believe mathematical modelling works much the same in every formal science. The same principles are (or should be) underlying also formal modelling languages. The chapter may be of general philosophical interest in how mathematical modelling works, and how this is related to scientific and mathematical concepts such as consistency, entailment, equivalence, information and how it is formalized, and how information is used to solve problems.

1.1 Concepts in modelling

What is modelling? A *modelling* is an artifact, made to capture certain aspects of some *application domain*, up to some level of abstraction. A modelling never reflects all aspects of the domain; it is an *abstraction* of the application domain. E.g., a scale model of a building does not model its internal structure, or the electrical wiring or plumbing inside.

Examples of non-mathematical modellings that come to mind are: scale models of a building (Figure 1.1), a blueprint of a machine; a flow chart for a program; a road map. An example of a mathematical modelling is: a set of differential equations.

Sometimes, a modelling serves to describe an existing application domain. E.g., in empirical formal science, scientific theories are modellings of the existing empirical reality. Sometimes the application domain exists only in the imagination. E.g., a scale model of a building, or a blueprint of a machine are typically developed before the objects that they represent become real (and they may never become real).

As for mathematical modellings, its use depends on the remarkable power of mathematical objects to serve as abstract representations of real objects. E.g., in Newtons theory of gravitation, mathematical objects serve to describe a star, a planet, or an apple, in the

form of its mass and its trajectory function. E.g., in a theory about graph colorings, a graph coloring can be an abstraction of a geographical map, where nodes are abstractions of countries, edges are abstractions of a border between countries, and colors of nodes are the colors of the countries in the map. In another abstraction, a node abstracts a radio station, an edge expresses that the distance between stations is less than some threshold value, and a color represents the frequency band of the station. A graph coloring then represents an assignment of frequency bands that avoids interference.

Modelling in formal science A formal empirical science is a science that studies existing, observable (“empirical”) phenomena, and makes mathematical (“formal”) modellings for it, e.g., sets of equations. It is the area by excellence where mathematical modellings are used. The prototypical such science is physics.

To obtain a better understanding of how mathematical modelling works, let us investigate this in detail in the context of a concrete physical theory. As an example, we select a famous case: Newton’s gravitation theory, limited to two bodies. The application domain is that of two bodies b_1, b_2 that freely move in each others gravitation field.^{1 2 3} Newton’s law can be informally represented as: *The acceleration of b_i is proportional to the mass of b_j ($i \neq j$) and inverse proportional to the square of the distance between b_i and b_j .* Stated in mathematical equations,

$$\begin{aligned}\forall t \in \mathbb{R}^+ : \frac{d^2 \vec{r}_1(t)}{dt^2} &= G \frac{m_2}{\|\vec{r}_2(t) - \vec{r}_1(t)\|^2} \cdot \vec{e}_{12}(t) \\ \forall t \in \mathbb{R}^+ : \frac{d^2 \vec{r}_2(t)}{dt^2} &= G \frac{m_1}{\|\vec{r}_1(t) - \vec{r}_2(t)\|^2} \cdot \vec{e}_{21}(t)\end{aligned}$$

where bodies b_i are abstracted as pairs $(m_i, \vec{r}_i)$ with $m_i \in \mathbb{R}^+$ representing the mass of b_i and $\vec{r}_i : \mathbb{R}^+ \rightarrow \mathbb{R}^3$ representing the trajectory of b_i , i.e., the position as a function of time. Here, $\vec{e}_{12}(t)$ is a vector of length 1 from the position of the first body towards the second body at time t ; it indicates the direction of the acceleration. Likewise for $\vec{e}_{21}(t)$ ($\vec{e}_{21}(t) = -1 \cdot \vec{e}_{12}(t)$; they are inverse vectors).

This theory will be denoted as T . It is a mathematical modelling of a physical domain. It has many applications, Sun and planet, Earth and Moon, Earth and ISS (International Space Station), Earth and Newton’s apple, In many of them, the theory achieves extreme accuracy. As such, the theory contains valuable information about such physical systems and has been used to solve an impressive number of problems (see below).

But how do such formal scientific theory work as a modelling? What are the ingredients and principles? Below, we investigate these questions in some detail. This is useful since the same principles are found in virtually all formal scientific theories, and they are also applied in the modelling logics used in this course.

¹Newton’s theory extends to an arbitrary number of bodies. Another important special case is where b_2 is fixed in the center of the universe ($\vec{r}_2 = (0, 0, 0)$) and the second equation is dropped.

²Since the actual universe contains an immense number of bodies, the two-body problem does not apply in principle to our universe. However, the two-body problems is *approximately* correct in a broad range of real situations, namely when the gravitation of one object on a second dominates that of all other. E.g., the gravitation of the Earth on Newton’s apple falling from the tree or on the Moon turning around the Earth dominates the gravitation of the Sun and planets on them. The gravitation of the Sun on the Earth dominates that of all other planets.

³For the sake of the argument, the discussion below takes an idealized and simplified view on science. In particular, we ignore the fact that science needs to work with approximations. E.g., mass, time, position can be determined only approximately, and Newton’s theory is approximately correct, as Einstein taught us.

Three different aspects of this theory need to be distinguished: **syntax**, **formal semantics**, and **informal semantics**.

Syntax : the scientific theory T is presented to us as a set of *strings* of symbols. The strings are composed of a collection of symbols. There are symbols of different types:

- $m_i, \vec{r}_i, \vec{e}_{ij}, G$,
- $=, -, \dot{\cdot}, \cdot^2, \frac{d^2}{dt^2}, \cdot, \|\cdot\|, \in, \mathbb{R}^+$,
- finally the symbol t .

Strings are composed from these symbols according to strict syntactical rules to form terms (e.g., $\vec{r}_2(t) - \vec{r}_1(t)$) and equations. The symbol t is a universally quantified variable. The mathematical symbols $=$ and $\forall$ are also symbols of the logic that is considered in the next chapter.

Not all mathematical statements are equations. Also natural language statements such as “each multiple of 3 is an odd number”, or the sentence “at $t = 0$, $\vec{r}_1$ belongs to the unit sphere” are mathematical statements. Any precise statement about mathematical objects is a mathematical statement.

Formal semantics To evaluate a mathematical theory, all symbols in it need to be assigned a value. Such an assignment of values to symbols is called a **structure**. Structures will be denoted using the symbol $\mathfrak{A}$. A structure $\mathfrak{A}$ is a joint assignment of **values** to the symbols of the theory. It consists of:

- a mathematical **domain** $D_{\mathfrak{A}}$, also called the **universe**; this is a set;
- an assignment of a mathematical value $\sigma^{\mathfrak{A}}$ from this domain to each symbol σ . Depending on the type of the symbol, this value may be an element of $D_{\mathfrak{A}}$, or it may be a compound object, e.g., a subset or n-ary relation or n-ary function on $D_{\mathfrak{A}}$, or even a relation or function on sets or functions, etc.

E.g., in the two-body problem, structures $\mathfrak{A}$ have as domain $D_{\mathfrak{A}} = \mathbb{R} \cup \mathbb{R}^3$, or a superset of this. Admissible values of symbols are listed below:

- $m_i^{\mathfrak{A}}$ is a positive real number > 0 .
- $\vec{r}_i^{\mathfrak{A}}$ and $\vec{e}_{ij}^{\mathfrak{A}}$ are functions $\mathbb{R}^+ \rightarrow \mathbb{R}^3$, where $\vec{e}_{ij}^{\mathfrak{A}}$ is a vector of length 1.
- $G^{\mathfrak{A}}$ is the number $6,6743 \times 10^{-11}$. This is called the *gravitational constant*.

Furthermore, the structure assigns also the standard values to mathematical symbols:

- $=^{\mathfrak{A}}$ is the identity relation.
- $-^{\mathfrak{A}}, \dot{\cdot}^{\mathfrak{A}}$ and $(\cdot)^2^{\mathfrak{A}}$ are the subtraction, division and square functions on $\mathbb{R}$.
- $\frac{d^2}{dt^2}^{\mathfrak{A}}$ is the second derivative operator.
- $\|\cdot\|^{\mathfrak{A}}$ is the function mapping vectors in $\mathbb{R}^3$ to their length.
- $\cdot^{\mathfrak{A}}$ is the scalar product between real numbers and vectors of $\mathbb{R}^3$.

The universally quantified variables t in the two statements cannot be assigned a value since the propositions state that for *any* value in $\mathbb{R}^+$ assigned to t , the equation should be satisfied.

We see that some symbols are **constant**, also called **rigid** in the sense that they have the same value in every admissible structure: the symbol G and all mathematical symbols. Other symbols have a different value in different structures: $m_i, \vec{r}_i$. Any well-typed mathematical value assigned to these symbols results in a structure. So, there are structures where the values of $\vec{r}_i$ correspond to nice smooth orbits around each other, but also structures where the values of $\vec{r}_i$ correspond to zigzagging or teleporting trajectories. The value of $\vec{e}_{ij}$ can be defined in terms of $\vec{r}_i$ and $\vec{r}_j$ as the vector of length 1 pointing from the location of b_i to that of b_j :

$$\vec{e}_{ij} = \frac{1}{\|\vec{r}_j - \vec{r}_i\|} \cdot (\vec{r}_j - \vec{r}_i)$$

In this course, the collection of non-constant symbols in the theory is called the **vocabulary** of the theory. Vocabularies will be denoted as Σ . In science and engineering, they are often called *variables*, but in a logical course we cannot call them this way since there are symbols that are even more “*variable*”: the quantified variables such as t . Instead, symbols that are not constants and not variables are called **ontology symbols** or **vocabulary symbols**.

To define a structure, it suffices to specify the domain and the values of the ontology symbols. An example of a structure $\mathfrak{A}$ is :

- Its domain $D_{\mathfrak{A}}$ is $\mathbb{R} \cup \mathbb{R}^3$;
- the value $m_1^{\mathfrak{A}}$ of m_1 is 78;
- the value $\vec{r}_1^{\mathfrak{A}}$ of $\vec{r}_1$ is the function $\mathbb{R}^+ \rightarrow \mathbb{R}^3 : t \mapsto (t, 0, 0)$;
- the value $m_2^{\mathfrak{A}}$ of m_2 is 54.4;
- the value $\vec{r}_2^{\mathfrak{A}}$ of $\vec{r}_2$ is the function $\mathbb{R}^+ \rightarrow \mathbb{R}^3 : t \mapsto (0, 0, 0)$.

Below, we use the shorthand notation $\mathfrak{A} = [m_1 : 78, \vec{r}_1 : (\lambda t \in \mathbb{R}^+ : (0, 0, 0)), m_2 : 54.4, \vec{r}_2 : (\lambda t \in \mathbb{R}^+ : (t, 0, 0))]$ where $\lambda t \in \mathbb{R}^+ : (t, 0, 0)$ is shorthand notation for $\mathbb{R}^+ \rightarrow \mathbb{R}^3 : t \mapsto (t, 0, 0)$.

Terms and formulas of T can be mathematically **evaluated** in a structure $\mathfrak{A}$. Terms evaluate to mathematical objects and formulas evaluate to **true** or **false** in a structure. This is a big difference. It is a **category clash** to confuse terms with formulas.

We use the following terminology in this course.

- Some structures $\mathfrak{A}$ **satisfy** T ; we say that T is **true** in $\mathfrak{A}$. In logical terminology, $\mathfrak{A}$ is called a **model** of T .
- Notation: $\mathfrak{A} \models T$.
- Otherwise, we say that $\mathfrak{A}$ does **not satisfy** T , or that T is **false** in $\mathfrak{A}$, $\mathfrak{A}$ **violates** T , $\mathfrak{A}$ **falsifies** T .
- Notation: $\mathfrak{A} \not\models T$.

Pay attention with terminology: in science and engineering, what in this course is called a “model” of T is often called a “solution” of T . What in this course is called a “modelling” or “theory” is often called a “model”. We will follow the terminology of logic: a model is a structure that satisfies the theory.

Informal semantics A mathematical theory (e.g., a set of differential equations) may be an interesting mathematical object, but it is meaningless as a scientific theory unless we know the **abstraction relation** between math and reality. This abstraction relation is represented here through the symbol $\mathcal{I}$ and it is called the **informal interpretation**. It lays a connection between symbols σ of the theory and **concepts** $\mathcal{I}(\sigma)$ in the application domain, and between equations φ and propositions $\mathcal{I}(\varphi)$ about the application domain.

It also lays a connection between structures $\mathfrak{A}$ and potential states of affairs $\mathcal{I}(\mathfrak{A})$ of the application domain and between mathematical values v and entities $\mathcal{I}(v)$ appearing in a state of affairs. Thus, this informal interpretation $\mathcal{I}$ involves all components of the syntax and formal semantics:

a ontology symbol c	a concept $\mathcal{I}(c)$
Σ	an ontology $\mathcal{I}(\Sigma)$ (set of concepts)
an equation φ	a proposition $\mathcal{I}(\varphi)$
a structure $\mathfrak{A}$ of Σ	a state of affairs $\mathcal{I}(\mathfrak{A})$
a value v in a structure	a value $\mathcal{I}(v)$ in a state of affairs

For an introduction of these terms, take the following sentence in the application of the two-body theory to the Sun (b_1) and the Earth (b_2): *the trajectory of the Earth would be different if the mass of the Sun would be 10 times lower.* The underlined expressions are **concepts** that are represented by symbols $\vec{r}_2$ and m_1 . The statement itself compares two different **states of affairs** of our solar system that satisfy the theory: the actual state of affairs and one where the mass of the Sun is ten times lower. The claim is made that in these states of affairs, the trajectory of the Earth would be different.

Below, these ideas are elaborated in the context of Newton's theory.

- A symbol σ of the vocabulary Σ of a scientific theory is a reference to a **concept** $\mathcal{I}(\sigma)$ in the application domain. These concepts exist in the mind of the scientist. We call $\mathcal{I}(\sigma)$ the **informal interpretation** of σ . E.g., the informal interpretation of the symbol m_1 is the concept of mass of object b_1 ; the informal interpretation of the symbol $\vec{r}_1$ is the concept of the trajectory of b_1 .

Also mathematical symbols may receive an informal interpretation. E.g., $\mathcal{I}(\|\vec{r}_i(t) - \vec{r}_j(t)\|)$ is the concept of physical distance between the bodies at time t and $\mathcal{I}(\frac{d\vec{r}_i}{dt}(t))$ and $\mathcal{I}(\frac{d^2\vec{r}_i}{dt^2}(t))$ are the concepts of speed, respectively acceleration of b_i at time t .

The collection $\mathcal{I}(\Sigma) = \{\mathcal{I}(\sigma) | \sigma \in \Sigma\}$ of concepts is called the **ontology** of the theory in the application domain. The ontology is the collection of concepts that we use to understand the application domain. They are the lens through which we see the application domain.⁴

- The mathematical values v (assigned to ontology symbols in structures) are abstract representations of entities (or properties of entities) $\mathcal{I}(v)$ in the application domain. E.g., the informal interpretation of the real number 54.4 (used as a mass) is a weight of 54.4 kg. The informal interpretation of a triple (x_1, x_2, x_3) of real numbers is a position in space (relative to some given coordinate system in space). The informal interpretation of a function $\mathbb{R}^+ \rightarrow \mathbb{R}^3$ is a trajectory through space, i.e., a range of positions in space varying with time. Etc.
- A structure $\mathfrak{A}$ is an abstract representation of a **state of affairs** $\mathcal{I}(\mathfrak{A})$ of the application domain. It represents the state of affairs in which each concept $\mathcal{I}(\sigma)$ has the value $\mathcal{I}(\sigma^{\mathfrak{A}})$. Thus, when $\sigma^{\mathfrak{A}} = v$, then in the corresponding state of affairs $\mathcal{I}(\sigma)$, the concept $\mathcal{I}(\sigma)$ takes on the value $\mathcal{I}(v)$. Vice versa, a state of affairs S can be abstracted into a structure $\mathfrak{A} = \mathcal{I}^{-1}(S)$.

⁴It is remarkable how much information is abstracted away in Newton's theory. E.g., the exact size and form and density of a spatial object, its geological and chemical structure; in case one body is an astronaut, its education, age, gender . . . None of these matter for the trajectory. For us, this seems evident, but we have had 300 years to get used to it. But in the middle ages, planets were seen as divine objects, orchestrated by the 9 muses in a celestial harmony (look up "Musica universalis" if you want to know more). Imagine the excitement when it was discovered that such aspects could be made abstraction of, and that with so few concepts, such precise predictions could be made.

E.g., in the 2-body problem, a structure is an abstract representation of a state of affairs of two bodies each following some trajectory in space. The structure $\mathfrak{A} = [m_1 : 78, \vec{r}_1 : (\lambda t \in \mathbb{R}^+ : (0, 0, 0)), m_2 : 54.4, \vec{r}_2 : (\lambda t \in \mathbb{R}^+ : (t, 0, 0))]$ is an abstraction of a state of affairs where b_1 has mass 78 kg, sits still in the center of the universe, while b_2 has mass 54.4, is moving away from the center along the x -axis with a constant velocity of 1 m/s.

In our imagination, we can think of truly bizarre states of affairs. It is important that also these states of affairs can be abstracted into structures. E.g., we can imagine a potential state of affairs where the Sun (b_1) and the Earth (b_2) are zigzagging through space; or another one where every second, they swap positions (hence, performing teleporting). We would like that our scientific theory says that such states of affairs are impossible. Therefore, even such a strange state of affairs S should have a mathematical abstraction in the form of a structure $\mathcal{I}^{-1}(S)$. It is up to our scientific theory then to reject such a structure.

There is a difference between a concept and a value for it in a state of affairs. In philosophical logic, this is the distinction between the **intension** of a symbol and its **extension** in a state of affairs. The intension is the concept assigned to the symbol, and the extension (in a state of affairs) is the entity that the symbol denotes in that state of affairs. E.g., the symbol “king of Belgium” has as intension the concept of king of Belgium. In the actual state of affairs (at the time of writing this course), the extension is the person Filip I. In several former states of affairs, the extension has been the persons known as Leopold I, II, and III, Albert I and II, and Boudewijn I. Formally, for symbol $\sigma \in \Sigma$, its intension is its informal interpretation $\mathcal{I}(\sigma)$ which is the concept that it represents in the application domain. Its extension in the state of affairs $\mathcal{I}(\mathfrak{A})$ is $\mathcal{I}(\sigma^{\mathfrak{A}})$, the informal interpretation of the value of σ in $\mathfrak{A}$.

The informal interpretation $\mathcal{I}$ further extends to mathematical propositions and theories, turning, e.g., mathematical equations, into precise propositions about the application domain.

- Mathematical propositions ψ (e.g., differential equations) become precise informal propositions $\mathcal{I}(\psi)$ about the application domain. E.g., the informal interpretation of Newton’s differential equation ψ_i is the physical proposition $\mathcal{I}(\psi_i)$ that the acceleration of b_i is proportional to the mass of b_j ($i \neq j$) and inverse proportional to the square of the distance.
- A structure represents a **potential** state of affairs. The mathematical theory T partitions structures $\mathfrak{A}$ over Σ into models and non-models. Models of T are abstractions of **possible states of affairs** according to the scientific theory; non-models are abstractions of **impossible states of affairs** according to the scientific theory.

E.g., structures $\mathfrak{A}$ of Newton’s theory that contain zigzagging or discontinuous functions $\vec{r}_i$ do not satisfy the differential equations. Correspondingly, state of affairs $\mathcal{I}(\mathfrak{A})$ involving zigzagging or teleporting stellar objects are impossible according to Newton’s theory.

- A scientific theory consisting of a mathematical theory T and an informal interpretation $\mathcal{I}$ is **correct** if the models $\mathfrak{A}$ of T correspond to possible states of affairs $\mathcal{I}(\mathfrak{A})$ of the application domain, and the non-models $\mathfrak{A}$ of T correspond to impossible states of affairs $\mathcal{I}(\mathfrak{A})$. Stated differently, if $\mathcal{I}(T)$ is true in the possible states of affairs, and false elsewhere.

This describes the way formal theories express information about the application domain.

It also shows that science cannot exist without a precise informal interpretation $\mathcal{I}$.

Experiment and prediction in formal science A formal science theory is evaluated by experiments. **Experiments** can be set up by computing **predictions** from T and performing **measurements** in the application domain. If the measurements correspond to the predictions, the experiment **supports** or **corroborates** the scientific theory; otherwise it **refutes** it. Notice the asymmetry: no finite set of successful experiments can ever prove the correctness of a scientific theory. The correctness of a scientific theory can never be proven. But one failed experiment suffices to **refute** the scientific theory.

Suppose an astronomer applies Newton's theory for the Sun and Uranus. Now, b_1 is the Sun and b_2 is Uranus. Based on data, the scientist computes a model $\mathfrak{A}$ for these objects which provides a trajectory $\vec{r}_2^{\mathfrak{A}}$ for Uranus. He calculates some real numbers t and x, y, z such that $\vec{r}_2^{\mathfrak{A}}(t) = (x, y, z)$. Under $\mathcal{I}$, this is the **prediction** that on time $\mathcal{I}(t)$, Uranus will be at location $\mathcal{I}((x, y, z))$. The scientist decides to evaluate this prediction by an **experiment**. The scientist translates the number t to a specific time point $\mathcal{I}(t)$ (say, tonight at 20:34:51), and the triple (x, y, z) to a specific direction for his telescope. This must be done with great precision. He then looks in his telescope, at that time, in that direction. He should verify if Uranus is visible and **measure** when it occurs, and compare it with the prediction. If there is a match, the experiment supports the theory; otherwise, the experiment refutes the theory. Thus, aspects of formal science, such as prediction, experiment and measurement strongly depend on $\mathcal{I}$.

Clearly, a **scientific theory** is more than a mathematical theory T . It must also comprise an **informal theory** describing the informal interpretation $\mathcal{I}$. It should describe how to translate observations into mathematical objects; vice versa, how to translate mathematical objects into (properties or facts about) non-formal entities. E.g., in physics this informal theory will describe a.o.t. the ways to measure the mass of an object, the time, the position and speed of objects, Since reality is not a mathematical object, this part of a scientific theory cannot be entirely a mathematical theory. Nevertheless, it needs to be precise. This informal part of the science plays an essential role in vital aspects of formal science: predictions, experiments, measurements.

A final comment is on an idealizing assumption that was made so far. It was assumed that a scientific theory T under informal interpretation $\mathcal{I}$ is true in states of affairs of the application domain. But, this condition rarely holds. In general, scientific theories are approximations of reality. E.g., some underlying assumptions of Newton's two-body gravitation is that the universe contains only two bodies and that gravitation is the only force. Furthermore, it does not take Einstein's relativity in account. Still, under fairly general conditions, Newton's theory is approximately correct, sometimes with extreme precision. These conditions include: that orbits are not significantly influenced by 3rd bodies (e.g., the Sun on the orbit of the Moon around the Earth, or Jupiter on the orbit of the Earth around the Sun), or by other forces (e.g., electric or magnetic forces) or by relativistic effects that bend space-time (high mass or speed).

On the subjectivity of the informal interpretation A formal scientific theory is a combination of a mathematical theory T and an informal interpretation $\mathcal{I}$. The theory T has a hard objective mathematical content while $\mathcal{I}$ is subjective and exists only in our mind. T does not prescribe or constrain the informal interpretation $\mathcal{I}$ of symbols and values. We can change our informal interpretation $\mathcal{I}$ anyway we want, and different scientists can hold different informal interpretations $\mathcal{I}, \mathcal{I}'$ of the symbols and values in T .

E.g., we interpret m_i as the mass of b_i , but nothing stops us from interpreting it as b_i 's

age, and the 3 numbers in $\vec{r}_i(t) = (x, y, z)$ as respectively the total mass of gold, silver and copper in the body b_i at time t . But what use would it have? None. Under this alternative informal interpretation, the theory predicts that the age and amount of these substances in one body influences the amount of hem in the other body. This is rubbish. The equations have zero predictive power under this $\mathcal{I}$.

When different scientists hold different informal interpretations on the same theory, it spells confusion or even disaster. A famous example is that of the Mars Climate Orbiter in 1999. This was a collaboration between scientists of Nasa and Lockheed Martin. They unknowingly disagreed about the informal interpretation of certain numbers computed by software. The first interpreted them as impulse in Newton-seconds, the second as impulse in pound-seconds. You can guess what happened: the orbiter crashed on Mars.

Clearly, a scientific field depends on a precise, common, objective understanding of the informal interpretation $\mathcal{I}$ of symbols and mathematical values. Without a well-understood informal interpretation, there is no science, there is only mathematics. Likewise, the informal interpretation $\mathcal{I}$ is of critical importance in all modelling applications. Without a well-understood informal interpretation, there is no modelling.

5

The knowledge in a scientific theory A scientific theory (if it is correct) gives us **information**, or **knowledge** about the application domain. How does it work?

- Its mathematical theory T partitions structures in models and non-models. Using $\mathcal{I}$, it induces a partition of **potential** states of affairs in **possible** and **impossible** states of affairs. A scientific theory distinguishes between models and non-models and, through the informal interpretation $\mathcal{I}$, between possible and impossible states of affairs. **This is the way a scientific theory conveys information about the application domain.**

E.g., any structure with zigzagging or discontinuous $\vec{r}_i$'s does not satisfy Newton's equations. Hence, Newton's theory tells us that planets do not zigzag or teleport under influence of gravitation.

- The information in a theory is **inverse proportional with the class of models** and **proportional with the class of non-models**: the less models a theory has, the more information it contains. Adding formal information to a theory reduces the class of models. Correspondingly, it increases the class of non-models.

E.g., if we extend Newton's theory T with *additional information* in the form of equations expressing the observed position of a planet at some time, or with equations describing some initial conditions of the system, the class of models and the induced class of possible states of affairs decreases.

- A **tautological** theory (or proposition) T is satisfied in every structure $\mathfrak{A}$; it does not eliminate any potential state of affairs. There is no information in it. It contains the least amount of information.

⁵I find it an interesting question if there are mathematical theories that are sensible scientific theories under different informal interpretations $\mathcal{I}$. In my limited knowledge of physics, I am aware of one such a theory: the equations of harmonic oscillators. These equations can be applied in various ways to mechanical systems (e.g., springs) and electrical systems (e.g. RLC circuitry). In different informal interpretations $\mathcal{I}$, one and the same symbol is interpreted as the mass of an object (in systems of mechanical translation), the moment of inertia (in systems of mechanical rotation), the inductance (in systems of electrical series), and the capacitance (in systems of electrical parallel circuits). These are very different sorts of physical entities indeed. See [30].

- A theory has **incomplete knowledge** if the theory allows for multiple models. In the running example, the theory T represents incomplete knowledge since it has a large set of models. If we add only few pieces of information, e.g., the positions of the bodies at a few time points, the class of models drastically reduces.
- A theory has **complete knowledge** if it has a unique model $\mathfrak{A}$. Such a theory is called **categorical**. A categorical theory states that the unique possible state of affairs is $\mathcal{I}(\mathfrak{A})$; all other states of affairs are impossible.

E.g., Newton's two-body theory is not categorical since it admits many models $\mathfrak{A}$. However, if we extend it with an exact description of the initial state of the two bodies⁶, the resulting theory has one model and is categorical. It is this property, and similar ones elsewhere in physics, that led to the *deterministic world view*.⁷

In the context of informatics and software engineering, categorical data sources are omni-present in the forum of data bases. Indeed, the typical aim of a data base is to specify the unique actual state of affairs of its application domain.

- A theory with maximal knowledge, even more than a categorical theory, is an **inconsistent or contradictory theory**: it has no models. E.g., if we extend the 2-body theory T with two statements stating that b_1 is on two locations at the same time, an inconsistent theory is obtained. As explained later, an inconsistent theory entails every proposition as well as its negation.

Can a contradictory theory have any practical use? This depends on its aim. If a theory T is build as a formal modelling of an existing reality, then inconsistency is clearly an error. Indeed, there is an actual state of affairs, and its abstraction should be a model of T . E.g., if Newton's gravitation theory would be inconsistent, it would tell us that the planets cannot orbit around the Sun, while in fact, they do. But not all theories are built to model an existing reality. E.g., if the aim of T was to explore some imaginary situation (e.g., the situation that in a fraction of a second, Mars jumps from one side of the Sun to the opposite side) then finding that the theory T extended with this condition is contradictory tells us that this situation is impossible. This is correct and useful.

- **The methodology of modelling** A modelling process corresponds to a process of iteratively adding formal propositions to the theory T , where each new proposition is true in all possible states of affairs and is false in some impossible states of affairs. Thus, adding this proposition reduces the class of models. E.g., in the running example, any linear combination of the two differential equations is true in every state of affairs but it makes no sense to add it to the theory, since this does not reduce the class of models. The modelling process goes on as long as the scientist is aware of impossible states of affairs whose abstractions $\mathfrak{A}$ satisfy his theory. Ideally, when the process stops, any model of T represents a state of affairs $\mathcal{I}(\mathfrak{A})$ that is possible according to the scientist and vice versa, each structure $\mathfrak{A}$ such that $\mathcal{I}(\mathfrak{A})$ is possible as far as the scientist knows, is a model of T .

Derived semantical concepts Scientists and mathematicians use basic semantic concepts to describe properties of propositions and theories, and relations between them.

- A proposition or theory T is **consistent** if it has a model. In the context of logic, this is called **satisfiable**.

⁶I.e., values for $m_1, m_2, \vec{r}_1(0), \vec{r}_2(0)$, and for all derivatives $\frac{d^n \vec{r}_1(t)}{dt^n}(0), \frac{d^n \vec{r}_2(t)}{dt^n}(0)$.

⁷<https://en.wikipedia.org/wiki/Determinism>

Newton's theory is consistent.

Inversely, a proposition or theory is called **inconsistent**, **contradictory** or **unsatisfiable** if it has no models.

- A proposition or theory is a **tautology** if it is satisfied in every structure.
- A proposition or theory is **categorical** if it has a unique model.
- A proposition or theory T **entails** a proposition or theory φ if φ is true in every model of T .

E.g., Newton's theory entails that under certain initial conditions trajectories are elliptic. In every model of the theory where these conditions are met, the trajectories are elliptic functions.

- Two scientific theories are **equivalent** if they have the same models.

These semantic concepts are fundamental concepts in formal science and mathematics. They are defined in terms of the distinction between models and non-models.

Also in logic, the concepts of consistency, entailment and equivalence are important derived semantical concepts. It is important to understand that these concepts were not invented in logic, but logic merely took these concepts from mathematics and formal science, and formalized them.

Analyzing information Following ideas of the philosopher Wittgenstein, the partition of potential states of affairs into the class that satisfies it, and the class that does not satisfy it, is to a high degree characteristic for the information: if two pieces of information yield the same partition, they convey the same information, otherwise they don't. This suggests an informal technique, known as *possible world analysis*, to analyze the meaning of natural language statements. Below two examples are given.

This suggests an informal technique, known as *possible world analysis*, to analyze the meaning of natural language statements. Below two examples are given.

Example 1.1.1

Consider two statements: (1) "All men love some film star" versus (2) "Some film star is loved by all men". Do they mean the same? I claim they do not and will show it by sketching a state of affairs in which (1) is true, and (2) is false. Take a state of affairs in which everybody except me loves Angelina Jolie and not Meryl Streep. I love only Meryl Streep. In this world the statement (1) is true: all men love some film star. On the other hand, (2) is not true. Hence, possible world analysis shows these statements are not equivalent.

Exercise 1.1.1 We all know that the stress in a sentence can be important for the meaning of the sentence. Stress on a different word may result in a different meaning. The following natural language sentences only differ on where the stress is put. Say the sentences below and listen to yourself: you will hear that they mean something different. Apply a possible world analysis to show that they are not equivalent:

- "I" did not drive your car.
- I did not drive "your" car.

- *I did not drive your “car”.*

The stressed words are those between quotes. Note that in the three cases, the proposition is the same, but the stress adds an implicit message to the sentence. Such an implicit message is called an **implicature**. Give the implicatures of the sentences. Then for each pair of these sentences, it suffices to find one state of affairs in which I did not drive your car but also, the implicature of one sentence is false and the implicature of the other sentence is true.

Calculus! ⁸

Models are *solutions* of the scientific theory. To computer scientists, the word “solution” rings a bell. Could we say that a scientific theory is a program? To compute models? Is a theory an encoding of a computational problem? Of which the models are the answers? Not in my view. To scientists, a scientific theory is a representation of knowledge, of information. Information is not a computational problem. In fact, one scientific theory induces many types of computational problems, the answers of which are all determined by the scientific theory and can be computed using a variety of techniques and algorithms.

Here are some examples of problems that were solved using Newton’s theory:

- *Computing a trajectory*, given a set of observed positions of the bodies.
- *Proving* that trajectories are elliptical.
- *Computing the escape velocity* of a rocket from Earth.
- *Computing the distance from Earth for a geostationary orbit*.
- *Computing the position and mass of an unobserved planet* given some irregularities in the trajectory of another planet.⁹

As for the relevance for this course, the point is that a modelling is not a program, it is not even a specification of a computational problem, but it contains information, and this information might be useful for solving a range of computational problems. Although the same information can be used to solve different problems, the algorithms to compute solutions from it are different of course. Also, some of these computational problems may be easy, some may be hard or even unsolvable (confer complexity and computability theory).

For a good example of what a *declarative theory* is, as opposed to a *program*, think of a formal scientific theory.

1.1.1 Conclusion: “*Get the models right!!*”

In formal science, we use mathematics for modelling some non-formal reality. Relevant concepts of the application field are identified and mathematical symbols are chosen as representations of them. Mathematical objects are designed as mathematical abstractions of the values of these concepts. Theories are built that characterize a class of satisfying structures called *models*. The quality of a theory is in the match between the models and

⁸This exclamation means “Let us calculate!” and comes from the philosopher Leibniz. His idea was to build a universal language, called *characteristica universalis*, in which human knowledge could be expressed. Leibniz argued that given such a theory, all kinds of problems could be solved by *calculation*.

⁹This is the way Neptune was discovered. The physicist Le Verrier predicted its position from irregularities in the trajectory of Uranus, and Neptune was discovered in 1846, exactly when and where Le Verrier had calculated it to be in 1845. It was a sensational moment of 19th-century science, and a dramatic confirmation of Newtonian gravitational theory: Le Verrier had discovered a planet “with the point of his pen.”. See https://en.wikipedia.org/wiki/Discovery_of_Neptune.

the possible states of affairs of application. The modelling methodology of formal science can be summarized in a 4 word sentence:

Get the models right!!

We end with a list of concepts relevant to modelling, extracted from this section.

Terminology 1.1.1: Summary of modelling concepts

- **application domain:** a modelling is about an application domain. Within this domain, a number of distinguished concepts emerge as relevant to us. Our modellings focus on these relevant concepts and make abstraction of the rest.
- **ontology:** this is the set of relevant concepts of the application domain.
- **state of affairs:** we can imagine the domain to be in one of many potential states of affairs. One state of affairs is characterized by the values of the concepts in that state of affairs. We observe the state of affairs through the lens of the selected ontology, and see the value of concepts in the state of affairs. Different states of affairs differ in the values that concepts have in it. Different states of affairs are mutually exclusive: if the domain is in one state of affairs, it is not in another.

The application domain may be real or imaginary. When the domain is real, the **actual state of affairs** is the one that is the case. Usually, we do not know the actual state of affairs, although we have information about it. Often the domain is imaginary. E.g., when we build a specification of a piece of hardware to be built in the future (or not).

- **information:** according to a piece of information, some states of affairs are **possible**, the others are **impossible**. If the state of affairs is possible we also say it **satisfies** that given information. Vice versa, we say that a piece of information is **true** or **false** in a given state of affairs. The information content of a proposition is characterized by the partition of possible and impossible states of affairs. Two pieces of information with the same possible and impossible states of affairs have the same information content. They are called **equivalent**.
- **proposition:** a statement (not necessarily in a logic) about the domain in terms of the given ontology, expresses a piece of information.
- **a modelling, a specification, a theory:** a set of statements in natural language or in mathematics or in some logic, providing propositions about the domain of discourse and aiming to describe the possible states of affairs of the domain. A modelling is **true** in a state of affairs if the state of affairs belongs to the possible states of affairs of the modelling. Otherwise the modelling is **false** in it.
- **symbols** are strings, (formal) **values** are mathematical objects (atomic objects, numbers, sets, relations, functions). Symbols and strings are mathematical objects too and they will sometimes be used as values. But in general, mathematical objects are not symbols or strings. E.g., a number is not a string; we distinguish between the number 5 and the symbol 5. A symbol for a number, e.g., 5, is called a **numeral**.
- **informal interpretation $\mathcal{I}$:** we use symbols to refer to objects and concepts in the domain. An informal interpretation $\mathcal{I}$ specifies for each symbol of the vocab-

ulary what concept or object in the application domain it expresses. As such, the informal interpretation captures the abstraction relation between mathematical symbols and the entities and interrelations in the application domain. Applying an informal interpretation $\mathcal{I}$ to a mathematical proposition returns a proposition about the application domain. Applying an informal interpretation $\mathcal{I}$ to a mathematical structure $\mathfrak{A}$ returns a state of affairs of the application domain.¹⁰

- **structure**: an assignment of mathematical values to symbols. They serve as mathematical abstractions of states of affairs. Mathematical objects serve as abstractions of objects in the informal domain. A structure is sometimes called an *interpretation*.
- **a model** of a theory: a structure that satisfies the theory. Given an informal interpretation $\mathcal{I}$, the models of a theory are abstractions of possible states of affairs.
- Derived semantical relations.
 - Two propositions are **equivalent** if they are true in the same states of affairs.
 - One proposition **entails** a second proposition if each state of affairs satisfying the first also satisfies the second.
 - A proposition is **consistent** if it is true in at least one state of affairs.
 - A proposition is **inconsistent/contradictory** if false in every state of affairs.
- The information in a formal scientific theory can be used to solve many computational problems, each with its own algorithm. A scientific theory versus an algorithm. This shows the difference between **declarative** and **procedural** entities.
- Given an informal interpretation $\mathcal{I}$, a formal scientific theory is **correct** if the models of the theory correspond to possible states of affairs of the application domain and vice versa. In many cases, theories are only approximations.
- **Possible world analysis** is a technique to analyze the meaning of statements by showing states of affairs where they are true or where they are false.

1.2 A modelling logic $\mathcal{L}$

The language that formal scientists use in scientific text is a mixture of natural language and mathematical expressions. E.g., the statement “*the acceleration of b_i is proportional to the mass of b_j ($i \neq j$) and inversely proportional to the square of the distance*” is a natural language statement but a precise statement, of the kind that Newton himself originally used to specify his laws. Certainly, a competent physicist would be able to translate it into the corresponding differential equation.

In this section, we introduce what a *modelling logic* is in this course and we define a toy example of one. This and the next sections serve (1) to specify the blueprint for logics in which the modelling methodology of formal empirical science can be applied, (2) to formally define, once and for all, the important semantic concepts such as consistency, entailment and equivalence, (3) to create awareness of the unbounded range of computational problems

¹⁰Or better, a class of states of affairs that are the same up to the chosen level of abstraction and are abstracted into $\mathfrak{A}$.

that – in principle – can be solved using a modelling, and (4) to define the concept of inference as a domain-independent computational problem (and problem solving method) that takes modellings as input.

Future chapters will introduce several new logics. They follow the same layout: syntax, formal semantics, informal semantics.

Definition 1.2.1

A *modelling logic* $\mathcal{L}$ as defined in this course, is a trinity of:

1. A definition of the **formal syntax** of formulas and expressions; a theory is a set of formulas.
2. A **formal semantics** defining the following notions:
 - a formal notion of structure $\mathfrak{A}$ as an assignment of *values* $\sigma^{\mathfrak{A}}$ to *symbols* σ ; a structure *interprets* a set Σ of symbols, or an expression e , if it assigns a value to each symbol in it;
 - a satisfaction relation $\models_{\mathcal{L}}$ between structures $\mathfrak{A}$ and formulas ψ interpreted by $\mathfrak{A}$.

For a theory T consisting of formulas, we define $\mathfrak{A} \models_{\mathcal{L}} T$ if for every $\psi \in T$, it holds that $\mathfrak{A} \models_{\mathcal{L}} \psi$.

3. A theory of **informal semantics** which explains the meaning of expressions and formulas of the logic for any informal interpretation $\mathcal{I}$ of all (free) symbols occurring in them.

Terminology 1.2.1

Given that $\mathfrak{A}$ interprets formula (or theory) φ , the following statements are synonyms:

- $\mathfrak{A}$ *satisfies* φ ;
- $\mathfrak{A} \models \varphi$;
- φ is *true* in $\mathfrak{A}$;
- $\mathfrak{A}$ is a *model* of φ .

Given that $\mathfrak{A}$ interprets formula (or theory) φ , the following statements are synonyms:

- $\mathfrak{A}$ does not *satisfy* φ ;
- $\mathfrak{A} \not\models \varphi$;
- φ is *false* in $\mathfrak{A}$.

The third case is that $\mathfrak{A}$ does not interpret φ . All the above statements are undefined then.

Furthermore, we define the truth function for $\mathcal{L}$:

- $\varphi^{\mathfrak{A}} = \mathbf{t}$ if $\mathfrak{A}$ interprets φ and $\mathfrak{A} \models \varphi$;
- $\varphi^{\mathfrak{A}} = \mathbf{f}$ if $\mathfrak{A}$ interprets φ and $\mathfrak{A} \not\models \varphi$;
- $\varphi^{\mathfrak{A}}$ is undefined if $\mathfrak{A}$ does not interpret φ .

Derived semantical concepts The derived semantical concepts of entailment, equivalence, consistency and contradiction are all as explained in the previous section, in terms of the satisfaction relation $\models_{\mathcal{L}}$.

The definitions below give a formal definition of these concepts in the context of a modelling

logic $\mathcal{L}$. They will apply to the logics in this course.

Let $\text{Mod}(T)$ be the collection of all models of a formula or theory T .

Definition 1.2.2

T is **satisfiable** or **logically consistent** if $\text{Mod}(T) \neq \emptyset$, i.e., at least one structure $\mathfrak{A}$ satisfies T . T is **contradictory** or **unsatisfiable** or **logically inconsistent** if $\text{Mod}(T) = \emptyset$, i.e., there is no structure that satisfies T .

T is **tautological** (notation $\models_{\mathcal{L}} T$) if $\text{Mod}(T)$ is the collection of all structures that interpret T . In the logic literature, a tautology is often called **logically valid**. [Pay attention: in colloquial language, a proposition is sometimes called “valid” if it is true in the application domain. E.g., “the age of a human is less than 130 years”. But this proposition is not a tautology, it is not “logically valid”.]

T is **logically equivalent** to T' (notation $T \equiv_{\mathcal{L}} T'$) if for all structures $\mathfrak{A}$ that interpret both T and T' , it holds that $\mathfrak{A} \models T$ iff $\mathfrak{A} \models T'$. Or equivalently, $T^{\mathfrak{A}} = T'^{\mathfrak{A}}$.

T **logically entails** T' (notation $T \models_{\mathcal{L}} T'$) if for every structure $\mathfrak{A}$ interpreting both T and T' , if $\mathfrak{A} \models T$ then $\mathfrak{A} \models T'$.

T contains **complete knowledge** if T has exactly one Σ -model (Σ being the set of symbols of T). Such a theory is called **categorical**.

Logical equivalence “almost” means $\text{Mod}(T) = \text{Mod}(T')$ except when T and T' do not contain the same symbols. E.g., the propositional logic tautologies $P \vee \neg P$ and $R \vee \neg R$ are logically equivalent but they do not have the same models since they are not interpreted by the same structures.

Pay attention that the symbol $\models_{\mathcal{L}}$ is overloaded. The binary relation $\mathfrak{A} \models_{\mathcal{L}} T$ is the satisfaction relation. The binary relation $T \models_{\mathcal{L}} T'$ (between theories or formulas) is the entailment relation. The unary relation $\models_{\mathcal{L}} T$ is the tautology property.

The semantical concepts are strongly interrelated. E.g., it is easy to show that T is logically equivalent to T' iff¹¹ T entails T' and T' entails T . Other properties emerge in logics $\mathcal{L}$ possessing an *objective negation connective* $\neg$. This is a connective with the property that for every formal proposition ψ , it holds that $\mathfrak{A} \models_{\mathcal{L}} \neg\psi$ iff it is not the case that $\mathfrak{A} \models_{\mathcal{L}} \psi$ (which is usually written in mathematical text as $\mathfrak{A} \not\models_{\mathcal{L}} \psi$). Intuitively, $\neg\psi$ is true in a state of affairs if and only ψ is false in it. In such a logic, it holds that ψ is logically valid iff $\neg\psi$ is contradictory iff $\neg\psi$ is not satisfiable. The following property reduces an entailment question to the satisfiability question. This reduction is exploited in almost all theorem proving systems.

Proposition 1.2.1

T logically entails φ iff $T \cup \{\neg\varphi\}$ is unsatisfiable.

Proof. T logically entails φ

iff for every structure $\mathfrak{A}$ interpreting T and φ , if $\mathfrak{A}$ satisfies T , $\mathfrak{A}$ satisfies φ

iff there is no structure $\mathfrak{A}$ interpreting T and φ such that $\mathfrak{A} \models T$ and $\mathfrak{A} \not\models \varphi$

iff there is no structure $\mathfrak{A}$ satisfying T and $\neg\varphi$

iff $T \cup \{\neg\varphi\}$ is unsatisfiable. ■

¹¹The word “iff” is short for “if and only if”.

Monotonicity When putting more information in a theory T , it gets fewer models. A consequence of this is a property called **monotonicity**: a property entailed by a theory is entailed by every extension of it. Or, the larger the theory, the more entailed formulas.

Proposition 1.2.2: monotonicity

If $T \models_{\mathcal{L}} \psi$ then $T \cup T' \models_{\mathcal{L}} \psi$.

Proof. If $T \models_{\mathcal{L}} \psi$, then ψ is satisfied in every model of T and hence in every model of an extension $T \cup T'$ of T . Hence, $T \cup T' \models_{\mathcal{L}} \psi$. ■

A toy modelling logic $\mathcal{Lin}$ To illustrate the above concepts, we now introduce the (toy) modelling logic $\mathcal{Lin}$.

The symbols used in expressions of $\mathcal{Lin}$ are the following:

- Rigid constants, also called **logic symbols**. Both informal interpretation and mathematical value is fixed in the logic.
 - An infinite set of *numerals*: symbols of the form $\dots, -2, -1, 0, 1, 2, \dots$
 - Binary arithmetic function symbols $+, \times$;
 - Binary comparison symbols: $=, <, >, \leq, \geq, \neq$.
- Constants often denoted as $x, y, \dots$. In an application, their informal interpretation as a numerical concept will be known, but not their mathematical value. The values assigned to them will be integer numbers, i.e., elements of $\mathbb{Z}$.
- There is no quantification in $\mathcal{Lin}$, hence, there are no (quantified) variables.

Definition 1.2.3: Syntax of $\mathcal{Lin}$

A *vocabulary* Σ is a (finite or infinite) set of constants.

A $\mathcal{Lin}$ -*term* is a finite string t of the form

$$\mathbf{a}_1 \times x_1 + \dots + \mathbf{a}_n \times x_n$$

where $\mathbf{a}_1, \dots, \mathbf{a}_n$ are numerals, $x_1, \dots, x_n$ are constants.

A $\mathcal{Lin}$ -*sentence* is a finite string of the form

$$t \sim \mathbf{a}$$

where t is a $\mathcal{Lin}$ -term, $\mathbf{a}$ is a numeral and $\sim$ is an element of $\{=, <, >, \leq, \geq, \neq\}$.

A $\mathcal{Lin}$ -*expression* is a $\mathcal{Lin}$ -term or sentence.

A $\mathcal{Lin}$ -*theory* T is a finite or infinite set of $\mathcal{Lin}$ -sentences.

A $\mathcal{Lin}$ -*expression over* Σ is a $\mathcal{Lin}$ -expression such that all its constants belong to Σ . Idem for a $\mathcal{Lin}$ -sentence and a $\mathcal{Lin}$ -theory.

We now define the formal semantics. Let Σ be a vocabulary.

Definition 1.2.4: Semantics of $\mathcal{Lin}$

The value $\sigma^{\mathcal{Lin}}$ of a logic symbol $\sigma \in \{+, \times, =, <, \dots\}$ is the numerical function or

operator standardly denoted by the same symbol.

E.g., $+\mathcal{Lin}$ is the plus function in the integer numbers, standardly denoted by the same symbol $+$.

A Σ -structure $\mathfrak{A}$ is an assignment of integer numbers to the symbols of Σ . I.e., it is a function $\Sigma \rightarrow \mathbb{Z}$.

Notation: the structure $\mathfrak{A}$ assigning 5 to x and -12 to y is denoted as $[x : 5; y : -12]$. Hence, the *value* (aka the *interpretation*) of x in $\mathfrak{A}$, denoted $x^{\mathfrak{A}}$, is 5.

The *value* $t^{\mathfrak{A}}$ of a $\mathcal{Lin}$ -term t of the form $\mathfrak{a}_1 \times x_1 + \dots + \mathfrak{a}_n \times x_n$ over Σ is the integer number that is the result of computing $\mathfrak{a}_1^{\mathcal{Lin}} \times^{\mathcal{Lin}} x_1^{\mathfrak{A}} +^{\mathcal{Lin}} \dots +^{\mathcal{Lin}} \mathfrak{a}_n^{\mathcal{Lin}} \times^{\mathcal{Lin}} x_n^{\mathfrak{A}}$. This is also called the *interpretation of t in $\mathfrak{A}$* .^a

The satisfaction relation $\models_{\mathcal{Lin}}$ between structures $\mathfrak{A}$ and $\mathcal{Lin}$ -sentences $t \sim \mathfrak{a}$ over Σ is defined as follows: $\mathfrak{A} \models t \sim \mathfrak{a}$ if $(t^{\mathfrak{A}}, \mathfrak{a}^{\mathcal{Lin}})$ belongs to $\sim^{\mathcal{Lin}}$.

^aOr, interpreting $+$ and $\times$ as numerical operators, $t^{\mathfrak{A}}$ is $\mathfrak{a}_1^{\mathcal{Lin}} \times x_1^{\mathfrak{A}} + \dots + \mathfrak{a}_n^{\mathcal{Lin}} \times x_n^{\mathfrak{A}}$.

Informal semantics of $\mathcal{Lin}$ The informal semantics of $\mathcal{Lin}$ is straightforward. Given some informal interpretation $\mathcal{I}$ of constants x as numerical entities in some application domain, the informal semantics of a $\mathcal{Lin}$ -sentence reads as a numerical expression about the informal interpretations of the constants. We illustrate this in Example 1.2.1.

Example 1.2.1

Consider the following puzzle. “*I am one year older than twice the age of my son. The sum of our ages is between 70 and 80. What are our ages?*” This puzzle contains information and a problem. We introduce vocabulary $\Sigma = \{x, y\}$ and the informal interpretation $\mathcal{I}(x)$: my age and $\mathcal{I}(y)$: my son’s age. These are numerical concepts; their value is not given. For a structure, e.g., $\mathfrak{A} = [x : 100, y : 1]$, $\mathcal{I}(\mathfrak{A})$ is the (non-actual) state of affairs where I am hundred years old, and my son is 1 year old.

The modelling of the information in $\mathcal{Lin}$ is the following theory T_{age} :

$$\begin{aligned} 1 \times x + -2 \times y &= 1 \\ 1 \times x + 1 \times y &\geq 70 \\ 1 \times x + 1 \times y &\leq 80 \end{aligned}$$

Given the informal interpretation $\mathcal{I}$, for each of these equations e , $\mathcal{I}(e)$ is a precise proposition about our ages. E.g., the first one expresses that my age minus two times my son’s age is 1; i.e., that I am one year older than twice the age of my son. The three sentences of this theory correspond to the two pieces of the puzzle.

The structures satisfying T_{age} are solutions of the equations. One can verify that there are four models with my age ranging around 50, and that of my son around 25. Thus, the puzzle has four models which correspond to four possible states of affairs. ■

Example 1.2.2

In $\mathcal{Lin}$, the following derived semantical properties hold and are provable:

- The sentence $2 \times x > 4$ is satisfiable. The structure $[x : 3]$ is a model of it.
- The sentence $1 \times x + -1 \times x = 0$ is a tautology (or is logically valid). Equivalently, $1 \times x + -1 \times x \neq 0$ is contradictory.
- The sentence $2 \times x > 4$ is equivalent to $1 \times x \geq 3$.
- The sentence $2 \times x > 4$ entails $1 \times x > -10$.
- The theory $\{1 \times x = 23, 1 \times y = 54\}$ is categorical. The theory T_{age} of the age puzzle is not categorical.

A modelling logic as a formal scientific theory of information This chapter began with a discussion of modelling in formal science, and what can be learned from this about modelling logics. But the field of logic itself is considered to be part of formal science. If so, what is the subject studied in this field? Many view logic as the formal science of *reasoning*. This cannot be the full story. All reasoning requires a resource: information. Therefore, a formal science theory of reasoning requires a formal language of information.¹² It should not surprise that the formal study of reasoning went hand in hand with the development of formal languages to express information. For example, the philosopher Leibniz hoped to develop a reasoning system called *the calculus ratiocinator*, and intended to develop a universal language called the *characteristica universalis* to express the information. Frege developed a formalized notion of mathematical proof, and the *Begriffsschrift* to express the information. This language is considered to be the first version of the Predicate Logic.

In what sense can a modelling logic, e.g., Frege's predicate logic (which is introduced in chapter 2), be seen as a formal scientific study of *information*? Consider the following three statements:

- All Greeks are Europeans.
- All men are humans.
- All laptops are computers.

These propositions express totally different informations. However, it can be seen also that each of these statements express the same “logical” relationship between two concepts, the concepts of Greek and European, respectively the concepts of man and human, and laptop and computer. Each expresses that the first concept is a subset of the second. The study of logic, seen as a formal science of information, does not formally study the information proper in natural language statements, but the logical relationships between groups of concepts that can be expressed using the language constructs of the logic. Take the following sentence of predicate logic:

$$\forall x(P(x) \Rightarrow Q(x))$$

Depending on the informal interpretation $\mathcal{I}(P)$ and $\mathcal{I}(Q)$, this logical sentence could be interpreted as any of the above three NL propositions. Predicate logic can and should be viewed in the first place, as a formal scientific study of the connectives *and*, *or*, *not*, *if ... then ...*, *... is equivalent to ...*, *for all* and *there exists*, and the logical relationships that can be expressed by combinations of them. The same holds for modelling languages in general.

If a modelling logic is a formal scientific theory about some types of information, what are the *experiments* by which it can be evaluated? A modelling logic has a formal and an

¹²Moreover, if the scientific study is about forms of reasoning of mathematical precision, then the language must express information of mathematical precision.

informal semantics. The scientific hypothesis underlying the modelling logic is that these two semantics correspond. This means that any combination of a formal proposition or theory T and an informal interpretation $\mathcal{I}$, and a structure $\mathfrak{A}$ can be taken as an experiment. The expected correspondence is displayed in Figure 1.2.

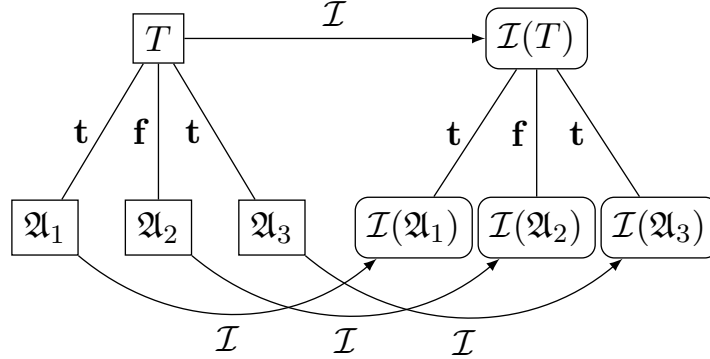


Figure 1.2 – The abstraction relation induced by the informal interpretation $\mathcal{I}$

The scientific “hypothesis” underlying a modelling logic is that $\mathfrak{A} \models_{\mathcal{L}} T$ iff $\mathcal{I}(T)$ is satisfied in the state of affairs $\mathcal{I}(\mathfrak{A})$. A *semantic correspondence problem* occurs when this correspondence between formal and informal semantics is broken. That is, if we can find a logical sentence or theory T , informal interpretation $\mathcal{I}$ and a structure $\mathfrak{A}$ such that the truth value of T in $\mathfrak{A}$ disagrees with the truth of the proposition $\mathcal{I}(T)$ in $\mathcal{I}(\mathfrak{A})$. Such a triple would be a refuting experiment of the modelling logic.

As often in formal science, this way of “refuting” a logic should be taken with a grain of salt. E.g., imagine an experiment where T is a proposition of size 10^{10} symbols. Then it would take more than a life time just to read the formula, let alone to understand it. Also, it is well-known that already fairly simple logical formulas are difficult for us to understand. For example, nested alternating quantifiers $\forall\exists\forall\exists$ are hard to understand for the human mind, and the maximal nesting is not much more than 4. It is easy to set up “experiments” where T is simply too complex to make sense of. However, take T to be $\forall x(P(x) \Rightarrow Q(x))$. If it would be possible to find $\mathcal{I}$ and $\mathfrak{A}$ such that $\mathcal{I}(\mathfrak{A})$ is a scenario with a group of people of diverse nationality in which all Greeks are Europeans but $\forall x(P(x) \Rightarrow Q(x))$ is false in $\mathfrak{A}$, then there would be good reasons to be skeptical about the logic.

1.3 Use of Information: computational problems and inference

With a program, we think of a Turing Machine, or any program in a Turing complete programming language, which describes an execution of a computing process from any given input. Programs are used to solve problems.

A theory in a modelling logic $\mathcal{L}$ is not a program but it represents information. Information determines the solutions to potentially many problems. In this section, we attempt to sketch the difficult connection between information and computation.

Recalling basic concepts of computability theory First, we (informally) define a few basic concepts. These concepts are about sets and functions on mathematical objects that can be encoded as finite strings (e.g., numbers by strings such as ‘42’).

Definition 1.3.1: Computational problems, decidability, recognisability, enumerability

A *computational problem* is a function $\mathcal{M}$ between two types of (finitely encodable) mathematical objects:

- input objects are elements of some domain $In(\mathcal{M})$
- output objects are elements of some domain $Out(\mathcal{M})$.

An *instance* of the problem $\mathcal{M}$ is given by a specific input object $i \in In(\mathcal{M})$. Its solution is $\mathcal{M}(i)$.

A computational problem $\mathcal{M}$ is *computable* if there is a program¹³ that for every instance i , returns $\mathcal{M}(i)$ and terminates. This program *solves* $\mathcal{M}$.

A computational problem $\mathcal{M}$ is a *decision problem* if $\mathcal{M}$ is a Boolean function, i.e., $Out(\mathcal{M}) = \{\mathbf{t}, \mathbf{f}\}$.

A decision problem $\mathcal{M}$ is *decidable* if it is computable. It is *recognisable* if a program exists that for each input i returns $\mathbf{t}$ if $\mathcal{M}(i) = \mathbf{t}$ and otherwise, returns $\mathbf{f}$ or does not terminate. Recognisable problems are sometimes called **semi-decidable**. This refers to the fact that $\mathcal{M}(i)$ for some input i will be decided in finite time if $\mathcal{M}(i)$ is true; the problem may not get decided if $\mathcal{M}(i)$ is false.

A set S of (finitely encodable) mathematical objects is *recursively enumerable* if there exists a program that generates all and only elements of S , potentially in infinite time.

Computational problems may be uncomputable (such as the halting problem) or computable. Decision problems can be decidable or undecidable. The class of recognisable (semi-decidable) decision problems includes all decidable and a subclass of undecidable problems. A decision problem that is not decidable but is recognisable will be called *strictly semi-decidable*. Strictly semi-decidable decision problems are not computable. Decidable decision problems can be further partitioned in complexity classes, e.g., P, NP, Exp, NExp, ...

The notions of recursive-enumerability and recognisability are tightly connected. Any set S of finitely encodable objects corresponds to the computational problem $\mathcal{M}_S$ with input an object x , and output $\mathbf{t}$ if $x \in S$ and $\mathbf{f}$ otherwise. A basic result from computability theory is the following.

Proposition 1.3.1

S is recursively enumerable iff $\mathcal{M}_S$ is recognisable.

¹³In this definition, the programming language is not specified, in accordance with **Church's hypothesis** that, in principle, the class of computational problems solvable by programs in any sufficiently rich, effective programming language is the same.

Solving computational problems using information through inference Information allows us to solve (computational) problems. Typically, not just one particular type of problem, but a range of problems. We discussed this already for Newton’s gravitation theory. In the context of a theory of a modelling logic, different forms of inference are needed.

But what is inference? Below we define it as a sort of computation based on the information expressed in an input theory.

Definition 1.3.2

An *inference problem* of a modelling logic $\mathcal{L}$ is a computational problem $\mathcal{M}$ that takes a theory (or logical proposition) of $\mathcal{L}$ as one of its input arguments such that if T and T' are logically equivalent theories of $\mathcal{L}$, then the output $\mathcal{M}(T, \bar{i})$ and $\mathcal{M}(T', \bar{i})$ is the same or is equivalent.

The idea of inference is that only the information content of the input theory matters, not the precise syntactical form. Thus, computing the number of symbols of an input theory T is *not* an inference problem. Theories that are equivalent have the same information content, hence substituting one for another in the input should not have an impact on the output.

Below, we define some inference problems. This list is not exhaustive. The definitions hold for all modelling logics with a model-theoretic semantics.

Definition 1.3.3: Some inference problems

Evaluation inference $\mathcal{M}_{eval}$

Input: a Σ -structure $\mathfrak{A}$, and term or expression or theory e over Σ

Output: $e^{\mathfrak{A}}$. If e is a Boolean expression, the outcome is **t** (“true”) if e is satisfied in $\mathfrak{A}$, and **f** (“false”) otherwise. This case is often called *model checking inference*.

Satisfiability checking $\mathcal{M}_{sat}$

Input: theory T

Output: **t** if T is satisfiable, **f** otherwise.

¹⁴A sketch of the classic proof (not for exam).

- ($\Rightarrow$) It is given that S is enumerated by some program P . We build a program P' for solving $\mathcal{M}_S$ as follows: when called on input x , it runs P and waits to see if x is produced. If so, it returns **t**, otherwise it does not return an answer. It is easy to see that P' is a recognizer of $\mathcal{M}_S$.
- ($\Leftarrow$) There is a countable sequence of codes $c_0, c_1, \dots$ such that S corresponds to a subset of them. Suppose we have a recognising program P' for $\mathcal{M}_S$. A recursively enumerating program P for S is as follows: it lets a counter N run from 0 to ∞ . For every value assigned to N , for each code c_i with $i \leq N$, it lets P' run on c_i for at most N steps. If P' terminates with **t** on c_i in less than N steps, P outputs c_i . P clearly only outputs elements of S . That P outputs each element of S follows from the fact that for each code $c_i \in S$, the program P' needs only a finite number n of steps to compute **t**. When during execution of P , N is assigned the maximum of i and n , then P' will be called on c_i and will have enough time to compute **t**, so P will output c_i .

Model generation inference $\mathcal{M}_{modgen}$ *Input:* theory T *Output:* “Unsat” if T is unsatisfiable, otherwise a model $\mathfrak{A}$ of T (variant: all models).**Model expansion inference** $\mathcal{M}_{modexp}$ *Input:* theory T , structure $\mathfrak{A}$ representing data*Output:* “Unsat” if $\mathfrak{A}$ cannot be expanded into a model $\mathfrak{A}'$ of T ; otherwise a model $\mathfrak{A}'$ of T that expands $\mathfrak{A}$.**Minimization inference** $\mathcal{M}_{minim}$ *Input:* theory T , arithmetical term t *Output:* “Unsat” if T is unsatisfiable, otherwise a model $\mathfrak{A}$ of T such that for any model $\mathfrak{A}'$ of T , $t^{\mathfrak{A}} \leq t^{\mathfrak{A}'}$.**Deduction inference** $\mathcal{M}_{ded}$ *Input:* theory T , boolean expression e *Output:* **t** if T logically entails e , **f** otherwise.**Validity inference** $\mathcal{M}_{valid}$ *Input:* theory T *Output:* **t** if T is a tautology/logically valid, **f** otherwise.**Possible-values inference** $\mathcal{M}_{posval}$ *Input:* theory T , a term t *Output:* the set of all values $t^{\mathfrak{A}}$ of t in all models $\mathfrak{A}$ of T .**Proposition 1.3.2**

Each problem defined in Definition 1.3.3 is an inference problem.

Proof. When replacing the theory T or the expression e in the input by an equivalent theory T' or formula e' , we need to prove that the output does not change. This is easy to verify. E.g., take the deductive inference problem $\mathcal{M}_{ded}$, substitute an equivalent pair of theory and formula T', e' for T, e . Then T' logically entails e' iff every model of T' is a model of e' iff every model of T is a model of e . Hence, $\mathcal{M}_{ded}(T, e) = \mathcal{M}_{ded}(T', e')$. ■

The above inference problems are well-defined computational problems in every modelling logic. In the current state of the art, Evaluation inference is implemented in database query languages, Satisfiability checking in the field of SAT, Model expansion inference and Minimization inference in Constraint Programming languages, Deduction inference and/or Validity inference in deductive logics and logic programming.

Example 1.3.1: Example 1.2.1, continued

Below is a list of problems that can be answered using the information in T_{age} , and a classification of these problems as a form of inference. Importantly, take notice how the solution is defined in terms of the semantics of T_{age} .

1. Decide if it is possible that I am 46 and my son 22.
This is the instance $([x : 46, y : 22], T_{age})$ of *evaluation inference* $\mathcal{M}_{eval}$, more precisely *model checking*.
2. Decide if the puzzle is consistent/satisfiable.
This is the instance T_{age} of *satisfiability inference* $\mathcal{M}_{sat}$.
3. Return all possible solutions to the puzzle.
This is the instance T_{age} of *model generation inference* $\mathcal{M}_{modgen}$.
4. Compute a solution in which I am 50 (and check the age of my son in it).
This is the instance $(T_{age}, [x : 50])$ of *model expansion inference* $\mathcal{M}_{modexp}$.
5. Given the additional clue that the right solution is the one where my age is maximal, determine our ages.
This is the instance $(T_{age}, (-1 \times x))$ of the minimization inference $\mathcal{M}_{minim}$: by minimising the term $(-1 \times x)$, a model with maximal age x is searched for.
6. Decide if the puzzle entails that my son is adult.
This is the instance $(T_{age}, 1 \times y \geq 18)$ of *deduction inference* $\mathcal{M}_{ded}$.
7. Compute possible values of my son.
This is the instance (T_{age}, y) of $\mathcal{M}_{posval}$.
8. Explain why $[x : 53; y : 26]$ is a solution; or why my son is adult, or why 34 is not a possible age of my son, ... What is the maximal age of my son?

For such a tiny puzzle, the list of problems solvable using the information in the age puzzle is surprisingly long and heterogenous.

Not all computational problems in a domain can be classified as straight application of some form of inference of declarative knowledge about the application domain. When that is the case then still, often subproblems of the main computational problem can be solved by logic inference.

The discussion above gives in the first place a way to relate computational problems to declarative theories that contain the relevant information to solve these problems. It is a specification of what the program should compute.

It also points to the potential of formal modelling for problem solving: the possibility of reusing a theory of T expressing declarative knowledge of the application domain, to solve a range of different types of problems.¹⁵ This sort of reuse cannot be achieved with standard programs. Indeed, programs tend to be problem-specific: they are implemented to solve a specific problem.

However, there is no free lunch in the world. To solve a specific problem instance applying a specific sort of inference on a domain theory T , we need an efficient inference program that takes as input T and reasons on its information. To develop such programs is often the million dollar problem. For some forms of inference, efficient inference programs exist. E.g. one could argue that database systems efficiently solve *evaluation inference*, given that a query is a special term and a database DB is a structure. But for other forms of inference arising in a particular application domain, the domain specific program written by a human

¹⁵The IDP system used in this course is a research tool to explore this hypothesis.

programmer is often more efficient than state of the art inference engines. More work is needed to close the gap.

The following proposition shows a useful property of categorical theories.

Proposition 1.3.3

Let T be a categorical theory and $\mathfrak{A}$ its model. Then for any formula φ , it holds that $T \models \varphi$ iff $\mathfrak{A} \models \varphi$.

In words, φ is true in all models of T iff φ is true in $\mathfrak{A}$. This is trivial since $\mathfrak{A}$ is the only model of T . This property shows that deductive inference problems of T can be solved using evaluation inference on $\mathfrak{A}$. In general, deductive inference problems are much harder to solve than evaluation inference problems in a concrete structure. But for categorical theories, the two types of inference problems coincide and are equally hard, or easy. That is great.

Exercise 1.3.1 *Think of some practical applications of this property.*

Computational complexity The computational complexity of different sorts of inference may vary enormously. E.g., for predicate logic FO (first-order logic) seen in the next chapter, the model checking inference problem of verifying whether a formula is satisfied in a finite structure is polynomial in the size of the structure, but the satisfiability inference problem for a formula is undecidable and the deduction inference problem is recognisable but not decidable (hence, it is *strictly semi-decidable*)¹⁶. Thus, evaluating an FO proposition in a structure may take a fraction of a second, while verifying that the same proposition is satisfiable or tautological may not terminate. The time complexity of computational inference problems varies widely, from efficient to undecidable. It means that we cannot associate a time complexity class to a modelling logic (unless we fix the sort of inference). Such is the nature of declarative knowledge.

1.4 Use of modelling in software engineering

Software systems are becoming more and more complex. The process of building complex software often goes through a number of iterations of the following steps:

- *design, specification* In this phase, ideas are collected of different aspects of the system to be built:
 - declarative aspects of the application domain
 - functional and non-functional requirements
 - analysis of algorithms and complexity of the computational tasks
 - ...

These ideas are analysed and written down in specifications.

- *implementation*
- *testing, verification, documentation.*

Developing complex software systems is extremely costly in time and money. Quite a few projects simply fail and are canceled. Those projects that lead to operational software, often produce software that contain bugs. Such bugs may be fairly insignificant, and most

¹⁶This is known as the undecidability and semi-decidability of deduction of FO.

of us can live with them, but there are applications where bugs are unacceptable. Such applications are sometimes called *mission-critical*. They are applications where bugs can be extremely costly, in terms of money, resources, or human lives. A few costly bugs taken from https://en.wikipedia.org/wiki/List_of_software_bugs:

- In the 80s, several cancer patients died due to overdoses of radiation resulting from a race condition between concurrent tasks in the Therac-25 software.
- On 4 June 1996, the Ariane 5 crashed after 40 seconds on its maiden flight. It was caused by a faulty software exception routine resulting from an overflow while converting a 64-bit floating point to a 16-bit integer.
- On 23 September 1999, the Mars Climate Orbiter crashed on Mars due to the fact that some modules expressed propulsion power in pound-seconds, while other modules interpreted these numbers in newton-seconds.
- Millenium bug problems.
- The Pentium FDIV bug in the Intel P5 Pentium floating point unit discovered in 1994 made Intel lose \$475 million dollars.

The common solution to avoid bugs is “testing, testing, testing”, but this does not guarantee correctness of software in all eventualities. Certainly for mission-critical, stronger guarantees are needed.

How to use Modellings? What is or could be the role of modelling in software development, now and in the future? ¹⁷ Could modelling help to build *correct* software? Could it help to build software solutions in a faster way?

Below, we discuss the use of modellings for three main purposes during the software development process: (1) modellings can be useful for specification and prior analysis (2) modellings can be used to develop provably correct systems, (3) modellings can be used directly, without the development of software, by inference engines that use the modellings to solve problems and perform tasks.

As for (1), a modelling provides precise *specification* and *documentation* of the modeled domain. When complex systems are built, precise documentation is crucial for maintaining them. This is not only the case in software, but also in hardware. E.g., there is a rumor that after the space shuttle program collapsed due to accidents with the Challenger and Columbia, NASA could not rebuild the reliable Saturn 5 rockets used in the Apollo program because documentation had not been maintained. In the context of software, the Unified Modelling Language (UML) is mainly concerned with (1). UML consists of different formal languages to describe different aspects of a software system.

Sometimes, specifications contain bugs. It is well-known that the earlier a bug is found in the software development process, the cheaper it is to solve it. According to the Systems Sciences Institute at IBM it costs $6\times$ more to fix a bug found during implementation than during design, and $15\times$ more during the testing phase. The cost of solving a bug in operational software may be exponentially much higher.¹⁸ Therefore, it would be useful to have tools to perform a priori analysis of the specifications. However, such tools require that specifications are expressed as formal modellings that can be tested for correctness using inference engines. When correctness is essential, there is no other way than through formal modelling.

¹⁷Some of the arguments below do not only apply to software development but more generally to hardware development.

¹⁸<https://www.synopsys.com/blogs/software-security/cost-to-fix-bugs-during-each-sdlc-phase/>

For example, the Alloy community¹⁹ uses the formal specification language Alloy to specify system components, and run a range of so called *light weight* verifications on it. We discuss this in Chapter 3. This way, bugs can be removed during the design phase and the confidence of the designer grows that the design is correct and that, eventually, the program implementing the formally specified design, will behave correctly. However, the Alloy specifications built in this stage only serve to analyze the specification; they are not used to implement the software. The implementation restarts from scratch and is a manual process; consequently, there are no guarantees of correctness of the program.

The use of (2) goes one step further and uses formal modelling for development of formally verified computer programs. Different systems follow this road to different length.

One way is found in *program verification*. The general idea is that programmers attach correctness conditions to their program components (e.g., pre- and postconditions). These conditions are expressed in some declarative logic. Verification systems are built capable to prove or disprove that program components satisfy the attached correctness conditions. Or perhaps weaker, the verification systems are proof checkers that serve to verify human-written formal proofs of the correctness conditions of programs. In case of success, the program is guaranteed to be correct.²⁰

An even stronger version of method (2) is found in the B-method, and its descendant Event-B which will be studied in Chapter 5. These logic systems are designed to build *executable* specifications of systems by iterated *refinement* of logic specifications. In this method, the human experts describe the system at different levels of abstraction. It starts with (simple) specifications at a high abstraction level, and gradually decreases the abstraction level until eventually, an entirely concrete specification is obtained that can be transformed into an executable program. An early application of this approach is the control system of Metro 14 in Paris, a fully automatic metro system operational since 1998. The control software was developed by Siemens. It was compiled from a sequence of more and more refined formal specifications, each of which had been formally checked for correctness with respect to the previous specification. The result was a program of 86 000 lines of Ada/C/Java code. In comparison, the specification in the B-method was 110 000 lines in the B language, showing that its development was extremely labor-intensive. However, this system achieved something exceptional: during the first ten years of operation, until it was upgraded to a new version of the hardware, not a single bug was found.²¹ In Chapter 5, we introduce Event-B, a descendant of B and the refinement methodology that it supports.

Use (3) of modelling is the most spectacular: computational problems are directly solved using the modelling, by applying generic (domain independent) inference programs to them. No more (domain specific) programming is required. This idea is implemented in databases, where queries are declarative expressions of propositions or relations of which the user wants to know the value. Other instances going in this direction are the *declarative programming paradigms*: logic and functional programming, Constraint Programming, and several other young and dynamic fields such as Answer Set Programming and the Semantic Web. Other approaches following (3) are executable declarative work flow languages such as LogicBlox.²² Theories in LogicBlox specify workflow processes consisting of interactions between system, users, databases and other actors. These logic theories can be used for various purposes, including the most obvious one: *executing* the specified process in the run-time environment.

¹⁹[https://en.wikipedia.org/wiki/Alloy_\(specification_language\)](https://en.wikipedia.org/wiki/Alloy_(specification_language))

²⁰Although, we should not forget that a correctness result for a program is a theoretical result. It does not ensure 100% that physically running the program will compute the correct result. Running a correct program can still go wrong, due to defect hardware, hacking, cosmic rays, ...

²¹http://deploy-eprints.ecs.soton.ac.uk/8/1/fm_sc_rs_v2.pdf

²²<http://www.logicblox.com/>

Chapter 7 presents a view on this way of using modellings.

Using formal modelling and verification may still be very expensive and time-consuming. E.g., the method (2) based on proving correctness of man made programs, or refinement is very expensive. However, in certain types of applications, formal modelling can also be very time saving. This is particularly the case with applications of method (3). Method (3) is the most difficult but also seems to be the most promising method. Most certainly, where it works, it has achieved sometimes immense reduction of development cost. It is easy to forget that there used to be a time where database systems did not exist, and companies built their own data storage system such that every query over its data, had to be manually programmed. Database systems achieved huge progress here. Also, in the field of Constraint Programming, it may be possible to reduce development time with a huge factor compared to manual programming. E.g., a company that uses a constraint solver to solve a complex scheduling or configuration problem using a declarative specification of the configuration domain, likely may have saved much time and money by not building its own (domain-specific) scheduling program ($10\times?$, $50\times?$).

In this course, we provide an overview of principles of a few different approaches in (1), (2) and (3). Of course, it is not possible to introduce all these languages and systems. However, it is possible to explain many principles in terms of the (few) languages used in this course.

1.5 Conclusions of this chapter and goals of the course

The goal of this chapter was to explore how a scientific theory conveys information on the application domain. We saw syntax, formal semantics, informal semantics, and how they work together to let a scientific theory convey information about the application domain. These concepts induce well-known semantical concepts of mathematics and science, such consistency, entailment, contradiction, equivalence. Information in a scientific theory can be used to solve a range of problems using different forms of inference. As such a formal scientific theory may be viewed as a prototypical **declarative theory**. Such a theory distinguishes itself from programs in the following ways : that such a theory is true or false in the application domain, that it provides information about the application domain, that it cannot be executed to solve a specific problem, that it does not define a problem, but that the information stored in it may be useful for solving a broad range of computational problems. This is interesting in its own right, but the interest doubles for this course since the same principles apply also in the modelling logics of this course. As such I sincerely hope that this chapter provides the students with a good foundation for the topics of course. I am not aware of any other book in logic that follows this approach.

This chapter may also lead to better understanding of the complex and heterogeneous landscape of computational logic and declarative programming. In the past half a century, a trend in computational logic has been to develop new logics specialized for one specific form of inference, and sometimes even for a specific inference algorithm. Clear cases are logic programming (with its SLDNF algorithm), functional programming, query languages, and production rule languages, constraint programming languages, answer set programming languages, model checking languages. In some of these cases, the algorithm induces an operational semantics on the logic, allowing to interpret expressions as executable program code. With all the benefits that trend brought us, it certainly also had a disadvantage: it blurred the question of the declarative meaning, the informal interpretation, of these logics. In this course, we see an expressive modelling logic that extends, or at least overlaps, with many of these declarative programming languages and offers an explanation of their

declarative interpretation. I am not aware of any other book in logic that aims at this goal.

As for the exam, only a few questions of this chapter could be asked.

1.6 Important for the exam

Questions for the exam:

- definition of a modelling logic
- definitions of the derived semantics concepts (tautology, equivalence, entailment, ...); !
- definition of what inference is, and the definitions of different sorts of inference, and understanding of what they can be used for.
- the different kind of use of modellings for software engineering.

Chapter 2

Predicate Logic and extensions

2.1 Short history

The ancient Greeks invented *mathematical reasoning*. *Aristotle* (384-322 BC) observed that there were patterns in correct mathematical reasoning, and he studied some of them in his *syllogisms*. Medieval philosophers constructed mnemonics for each syllogistic pattern [14]. The following example was called “Barbara” because its three lines start with “all”:

$$\begin{array}{l} \text{All humans are mortal.} \\ \text{All Greeks are humans.} \\ \hline \text{All Greeks are mortal.} \end{array}$$

He started the scientific study of this form of reasoning: deductive reasoning – a great exploit. Philosophers and mathematicians have been searching for more than 2000 years to further formalise and extend these patterns. Only in the 19th century substantial progress was made over Aristotle.

Gottfried Wilhelm Leibniz (1646-1716) was a genius German philosopher, mathematician and diplomat. Leibniz’s greatest work is the differential and integral calculus, which he developed (largely) independently of Newton. But he was active in many other domains, including logic. He was convinced that:

- Complex *thoughts* in our mind are composed from simpler thoughts, using mathematical operators analogous to arithmetical operators $+$, $\times$, $-$...
- Once we make our thoughts explicit in terms of these operators, it will be possible to study the rules of correct reasoning using mathematics.

Leibniz phrased and partially formalised these ideas in letters but he did not publish his results. Only around 1840, his work was published and his ideas became broadly known and started to influence the development of logic.¹ Leibniz had a strong belief in logic and believed that it would allow to solve all diplomatic disagreements by rational reasoning. That may have been a little naive.

George Boole (1815-1864)² identified a number of these operators, such as $\wedge, \vee, \neg$ in a paper “*An Investigation of the Laws of Thought (1854)*”. In this paper, he developed

¹<https://plato.stanford.edu/entries/leibniz-logic-influence>

²He was a brilliant, humble and decent man, who died at middle age and got fame only after his death. His death was due to superstition. In his time, some believed that a disease could be cured by repeating the cause of the disease. One day, he caught a cold during a walk through heavy rain. When he was in bed with high fever, his wife doused him with a bucket of cold water. He died of pneumonia.

Boolean algebra, work that laid the foundations for digital circuits. In logic, it is known as *propositional logic*.

Gottlob Frege extended this work by adding *quantification* ($\forall$ and $\exists$), and by extending the set of inference rules of correct reasoning. His *Begriffsschrift* (1879) counts as the first presentation of classical logic. His logic included First and Second Order Logic. Classical logic, or Predicate logic corresponds to First-order Logic (FO, or sometimes FOL). *Kurt Gödel* was probably the main logician of the first part of the 20th century, and resolved a number of fundamental questions regarding FO and the foundation of mathematical reasoning. One result is his completeness theorem: that each proposition that is entailed by a classical logic theory, has a formal proof. Another famous result is his incompleteness theorem: that every enumerable/computable classical logic theory T of the natural numbers is necessarily incomplete in the sense that there are true propositions about the natural numbers that cannot be proven from T . Chapter 6 discusses these results (and some others).

Classical logic as a modelling language (knowledge representation) was a by-product of the study of deductive inference (reasoning). Indeed, *information* is the raw material for any form of reasoning. To formally study reasoning, one needs a formal language to express information. Leibniz, Boole, Frege, Gödel and so many others not only contributed to deductive reasoning but also to the development of a language in which to express the resource for all reasoning: information. This led to development of the language of classical logic as a modelling language.

Classical logic is by no means an ideal modelling language. It is very unsuitable for modelling in the sense of **getting the models right**. However, classical logic is a base language in the sense that its language constructs are indispensable for modelling. But they are simply not enough. This chapter will propose a few very useful extensions of classical logic.

2.2 First-order Logic (FO)

In this chapter, we apply the ideas of Chapter 1 to build First-order logic, also called predicate logic. We abbreviate this logic as FO (FOL is also commonly used). In later sections, a number of extensions of this logic are presented.

2.2.1 Formal Syntax

In FO, one distinguishes between two sorts of symbols: **logical symbols** and **non-logical symbol**.

Logical symbols of FO are enumerated in the following table, together with their informal interpretation:

symbol σ	name	fixed informal interpretation $\mathcal{I}(\sigma)$
$=$	equality,	... is equal to ...
$\wedge$	conjunction,	... and ...
$\vee$	disjunction,	... or ...
$\neg$	negation,	not ...
$\Rightarrow$	material implication,	if ... then ...
$\Leftrightarrow$	equivalence,	... if and only if ...
$\exists$	existential quantor,	there exists $\langle$ variable $\rangle$ such that ...
$\forall$	universal quantor,	for all $\langle$ variable $\rangle$, ...

Table 2.1 – Table of logical symbols and their informal interpretation

Logical symbols are constant symbols: their value and informal interpretation is fixed in FO. They serve as language constructs to build compound propositions from more primitive propositions (in the spirit of Leibniz). They are well-known from mathematical texts where they are used in the same meaning.

In FO, quantifiers $\forall, \exists$ quantify over all elements of the universe. The name “first order” stems from this. In other, more expressive logics, quantifiers may quantify over compound, “second order” values such as sets, relations and functions. The resulting logic is called Second order logic (SO).

A **non-logical symbol** σ is either an *object symbol*, a *function symbol* or a *predicate symbol*. Each symbol σ has an arity n . A function symbol of arity 0 is an *object symbol*. A predicate symbol of arity 0 is a *propositional symbol*.

In Table 2.2, the notation for predicate, object and function symbols σ is illustrated with a few examples, and for each, a potential informal interpretation $\mathcal{I}(\sigma)$ is given. With symbol $\#1, \#2, \dots$, we refer to the first, second, $\dots$ argument of the symbol. Notice that function symbols get a trailing colon, e.g., $\text{Sam}/0:$, $\text{AgeOf}/1:$.

non-logical symbol σ	informal interpretation $\mathcal{I}(\sigma)$
$\text{Human}/1$	$\#1$ is a person
$\text{CourseResult}/3$	student $\#1$ did exam of course $\#2$ obtaining score $\#3$
$\text{Sam}/0:$	Sam, one particular student
$\text{PrimeMinister}/0:$	the prime minister of Belgium
$\text{AgeOf}/1:$	the age of person $\#1$
$+/2:$	the sum of $\#1$ and $\#2$

Table 2.2 – Table of non-logical symbols and their informal interpretation

Another type of symbol is the variable, used in quantified formulas. A variable is an object symbol, of arity 0; a variable obviously has no informal interpretation, since its value ranges over a range of values.

Definition 2.2.1

A vocabulary is a set Σ of **non-logical symbols** (variables not included).

Terms and formulas

Definition 2.2.2: Syntax of FO

We assume that an infinite collection of object, function and predicate symbols of all arities is given. We define a *term* and *formula* **by induction**:

- Terms:
 - an object symbol C is a term;
 - if f is a function symbol of arity n and $t_1, \dots, t_n$ are terms, then $f(t_1, \dots, t_n)$ is a term;
- Formulas:
 - if P is a propositional symbol, then P is a formula, called an atom;
 - if P/n is a predicate symbol of arity n and $t_1, \dots, t_n$ are terms, then $P(t_1, \dots, t_n)$ is a formula, also called an *atom*;

- if t_1 and t_2 are terms, then $(t_1 = t_2)$ is a formula, called an (*equality*) *atom*;
- if α, β are formulas, then $(\neg\alpha)$, $(\alpha \wedge \beta)$, $(\alpha \vee \beta)$, $(\alpha \Rightarrow \beta)$, $(\alpha \Leftrightarrow \beta)$ are formulas;
- if x is an object symbol and α is a formula then $(\exists x \alpha)$ and $(\forall x \alpha)$ are formulas; here, x is called a (quantified) variable.

An *expression* is a term or a formula.

Remark 2.2.1: What does this inductive definition mean?

Definition 2.2.2 is an example of an *inductive definition*. Such a definition is often presented as a collection of rules and defines one or more sets, relations or functions. This one defines the sets of terms and formulas.

Rules often contain variables. E.g., in the first rule, C is a (meta-)variable that ranges over object symbols; f/n is a (meta-)variable ranging over function symbols (or arity n), $t_1, \dots, t_n$ are (meta-)variables ranging terms; the symbol P is a (meta-)variable ranging over propositional or predicate symbols. α and β are meta-variables ranging over formulas and x is a meta-variable ranging over ... variables.

Some rules are *base rules*: their condition is absent or does not depend on a defined concept. E.g., the first rule in the definition of term and formula are base rules. The rule deriving that an object symbol C is a term, has no condition. The rule deriving atomic formulas has a condition which does not depend on a defined concept.

Other rules are *inductive rules*: their condition depends on some defined concept.

The definition defines the set of terms and formulas by describing how to construct these sets. The idea of this construction is natural for a computer scientist. The idea is to use the rules of the definition as a kind of *bottom up program*. We start by initialising the set of terms and the set of formulas to the empty set. Then we iteratively apply the rules of the inductive definition in a bottom up way: for rule instances with a body that is satisfied with the current set of terms and formulas, its conclusion is made true by adding a string to the set of terms or to the set of formulas. This process is iterated as long as needed, until no more new terms or formulas can be produced. This may require infinitely many steps. But the result of this process is two sets of strings. They are the sets of terms and of formulas defined by this definition. This construction process is called the *induction process*.

We show a few steps in this process for the object symbol a and the binary function symbol g .

- In the initial step, we apply the instance of the first rule for $[C : a]$ to derive that the object symbol a is a term. a is added to the set of terms.
- Now, a is given to be a term, and the condition of the instance of the second rule for $[f : g, t_1 : a, t_2 : a]$ is satisfied. Applying the rule results in adding the string $g(a, a)$ to the set of terms.
- In the third step, the application of another instance of the second rule derives $g(g(a), a)$.
- etc.

One can see that this process will create infinitely many terms.

Definition 2.2.2 is called a **monotone inductive definition**. This means that the conditions of the rules are positive. I.e., a rule only applies due to the presence of elements in the defined set, not due to the absence of some elements in the defined set.

Question Do infinite terms exist, according to this inductive definition? E.g., could the infinite string $g(a, g(a, g(a, g(a, \dots))))$ be a term? This string can only be a term if during the induction process, some rule application derives it to be term. However, it is straightforward to see that this is impossible. Indeed, for the inductive rule to produce an infinite term of the form $f(t_1, \dots, t_n)$, at least one of the component terms $t_1, \dots, t_n$ must be infinite. So, with finite terms only new finite terms can be produced. Stated differently, finiteness of terms is an **invariant** of the induction process. It follows that no infinite string can be derived to be a term. The same holds for formulas.

Question Could a term or formula contain strictly more brackets “(” than brackets “)”? The answer is negative and the argument is similar. It is easy to see that having an equal number of “(” and “)” brackets is an invariant of the induction process. Therefore, all terms and formulas possess equal numbers of these brackets.

Example 2.2.1

This example shows a number of strings which the induction process can or cannot derive to be terms or formulas. Take the function symbols $f/2$, $a/0$, $b/0$ and predicate symbol $P/2$.

- $f(a, f(a, b))$: not an expression; missing bracket “)”.
- $f(a)$: not an expression; mismatch arity f and number of arguments;
- $f(a, b)$: a term, not a formula;
- $P(a, b)$: a formula, not a term;
- $f(P(a, b), b)$: not an expression; $P(a, b)$ is not a term;
- $f(f(a, b), a)$: a term, not a formula;
- $P(f(a), a)$: not an expression; $f(a)$ is not a term;
- $P(a, P(a, b))$: not an expression; 2nd argument not a term;
- $(\exists x)$: seems to express that some object exists, but not a formula;
- $(\exists x(x = x))$: a formula;
- $(P(a, a) = P(b, b))$: not an expression; equality takes terms as argument, not formulas;
- $(P(a, a) \Leftrightarrow P(b, b))$: a formula.

Special formulas

- A formula $\alpha \wedge \beta$ is called a *conjunction* and α, β are called its *conjuncts*.
- A formula $\alpha \vee \beta$ is called a *disjunction* and α, β are called its *disjuncts*.
- A formula $\alpha \Rightarrow \beta$ is called a (*material*) *implication*, α is called its *premise* or *antecedent* or *condition* and β its *conclusion* or *consequent*.

- A formula $\alpha \Leftrightarrow \beta$ is called an *equivalence*.

Free and bound occurrences of symbols A symbol may occur multiple times in an expression.

Definition 2.2.3

An occurrence of a non-logical symbol σ in a subformula $(\exists\sigma\alpha)$ or $(\forall\sigma\alpha)$ of an expression e is called a *bound occurrence* of σ in e . Any occurrence of a symbol σ in an expression e that is not a bound occurrence is called a *free occurrence* of e .

A symbol σ is called a *free symbol* of expression e if σ is non-logical (e.g., different from $=$) and has at least one free occurrence in e .

Example 2.2.2

Free occurrences of non-logical symbols in the following formula are underlined twice, bound occurrences only once:

$$(\forall y(\underline{P}(\underline{x}, y) \vee \exists \underline{x}\underline{Q}(\underline{x}, \underline{C}, y)))$$

Consequently, the free symbols of this formula are $\underline{P}/2$, $\underline{x}/0$, $\underline{C}/0$, $\underline{Q}/3$.

Notice that the symbol x occurs both bound and free in this formula. This can be very confusing and is best avoided by renaming the bound variables.

Terms and sentences over vocabulary Σ To interpret a formula φ in an application domain, we should know the informal interpretation of all free symbols of φ . There is no need to know the informal interpretation for a bound symbol, since this symbol is assigned a range of values through quantification and whatever informal interpretation we had for this symbol, it is ignored. Thus, if Σ is the set of symbols for which we possess an informal interpretation, the set of formulas that can be interpreted as statements in the problem domain are those such that each free symbol belongs to Σ . This motivates the following definition:

Definition 2.2.4

- An expression e is called an *expression over Σ* if the free symbols of e belong to Σ .
- A *term over Σ* is an expression over Σ that is a term. The set of terms over Σ is denoted $\mathbf{Terms}(\Sigma)$.
- A formula φ is a *sentence over Σ* if the free symbols of φ belong to Σ .
- A theory T is a *theory over Σ* if it is a set of sentences over Σ .

Notice that if e is an expression over Σ , then also over any superset of Σ .

Given a vocabulary τ of object and function symbols (and possibly also predicate symbols, but these do not play a role here), $\mathbf{Terms}(\tau)$ is the set of terms built from τ . This set can be defined inductively as follows:

- C belongs to $\mathbf{Terms}(\tau)$ if C is an object symbol in τ ;
- $f(t_1, \dots, t_n)$ belongs to $\mathbf{Terms}(\tau)$ if function symbol f/n belongs to τ and $t_1, \dots, t_n \in \mathbf{Terms}(\tau)$.

$\text{Terms}(\tau)$.

Exercise 2.2.1 What is $\text{Terms}(\tau)$ if ...

- if τ contains no object symbols?
- if $\tau = \{C_1, \dots, C_n\}$?
- if $\tau = \{A, S/1:\}$?
- if $\tau = \{A, B, l/2:\}$?

Example 2.2.3

Recall that $=$ is a logical symbol. Formulas where the only predicate symbol is the equality symbol are formulas over every vocabulary, including the empty vocabulary. An example is:

$$(\exists x(\exists y(\neg(x = y))))$$

It expresses that two different objects exist.

Notational conventions Too many parentheses in a formula can harm its readability. This is the same in logic as it is in arithmetical expressions. A number of techniques exist to reduce the number of parenthesis. They are similar as in arithmetic. The idea is that parentheses may be dropped if it is clear where to reinsert them.

- Parentheses at the outside of the formula can always be dropped. E.g., $(\forall x(\forall y(x = y)))$ can be rewritten as $\forall x(\forall y(x = y))$.
- For inner parentheses, the binding strength of connectors and quantifiers determines the position of implicit parentheses. Parentheses are inserted closer to connectives with higher binding strength.

The binding precedence of connectives and quantifiers is as follows:

$$\neg > \wedge > \vee > \Rightarrow > \Leftrightarrow > \forall > \exists$$

Further more, all binary connectives are right associative. This means that $P \wedge Q \wedge R$ is shorthand for $(P \wedge (Q \wedge R))$, not for $((P \wedge Q) \wedge R)$.

We apply the idea on the following string:

$$\neg P \wedge Q \vee R \vee S$$

Since $\neg$ has the highest binding strength, the first parentheses are inserted around $\neg P$. The second highest binding strength is for $\wedge$, so the next parentheses are inserted around $(\neg P) \wedge Q$. Due to right associativity, parentheses are inserted around $R \vee S$. The final parentheses are the outer ones. We obtain:

$$(((\neg P) \wedge Q) \vee (R \vee S))$$

Some other examples:

- $\alpha \wedge \beta \vee \neg \alpha \wedge \neg \beta$ is shorthand notation for $((\alpha \wedge \beta) \vee ((\neg \alpha) \wedge (\neg \beta)))$
- $\exists c P(c) \wedge Q(c, d)$ is shorthand for $(\exists c(P(c) \wedge Q(c, d)))$.

Example 2.2.4

The following string

$$\exists x \exists y \exists z \neg(x = y) \wedge \neg(x = z) \wedge \neg(y = z)$$

is a shorthand notation for the formula:

$$(\exists x(\exists y(\exists z(\neg(x = y) \wedge (\neg(x = z) \wedge \neg(y = z))))))$$

More notational conventions: pre-fix or in-fix notation Standard atoms $P(t_1, \dots, t_n)$ and terms $f(t_1, \dots, t_n)$ are written in pre-fix notation. However, there is also much use of in-fix notation, in particular in case of the logical symbols: the binary predicate $=$ and the connectives $\wedge, \vee, \Rightarrow$ and $\Leftrightarrow$. Later in this chapter, FO will be extended with numerical predicate symbols $\leq, <, \geq, >, \neq$ and numerical function symbols $+, \times, \div$. Also these symbols are written in in-fix, as usual in mathematics. E.g., $2 \times 5 > 3 + 4$.

2.2.2 Formal semantics**Values and structures****Definition 2.2.5**

A *structure* $\mathfrak{A}$ of vocabulary Σ (in short, a Σ -structure) consists of

- a non-empty set $D_{\mathfrak{A}}$, called the *domain* or the *universe* of $\mathfrak{A}$;
- for each symbol $\sigma \in \Sigma$ an appropriate value denoted $\sigma^{\mathfrak{A}}$, appropriate in the sense that the types of σ and $\sigma^{\mathfrak{A}}$ should match:
 - for an object symbol $C \in \Sigma$, a domain element $C^{\mathfrak{A}} \in D_{\mathfrak{A}}$;
 - for an n -ary function symbol $F \in \Sigma$, an n -ary (total) function $F^{\mathfrak{A}} : D_{\mathfrak{A}}^n \rightarrow D_{\mathfrak{A}}$;
 - for a propositional predicate symbol $P \in \Sigma$, a truth value $P^{\mathfrak{A}} \in \mathbb{B} = \{\mathbf{t}, \mathbf{f}\}$;
 - for an n -ary predicate symbol $P \in \Sigma$, an n -ary relation $P^{\mathfrak{A}} \subseteq D_{\mathfrak{A}}^n$.

The set of symbols interpreted by structure $\mathfrak{A}$ is denoted as $\Sigma(\mathfrak{A})$. We say that $\mathfrak{A}$ interprets Σ if $\mathfrak{A}$ interprets at least every symbol of Σ (and possibly more symbols).

Propositional structures For vocabularies Σ consisting of propositional symbols, the definition of structure can be simplified since it makes no sense to include a domain in the structure.

Definition 2.2.6

Given a propositional vocabulary Σ , a Σ -structure is a function $\mathfrak{A} : \Sigma \rightarrow \mathbb{B}$.

Notational conventions for denoting propositional structures are as follows. Take $\Sigma = \{P/0, Q/0, R/0\}$. The structure in which P and Q are true and R is false is written as a set of tuples $\{(P, \mathbf{t}), (Q, \mathbf{t}), (R, \mathbf{f})\}$. Alternatively, if Σ is clear from the context, a propositional structure may be written as the set of propositional symbols that are true in it. Under this convention, the above structure is written as $\{P, Q\}$.

Beware: a set of propositional symbols could be intended as a vocabulary, or as a structure. What is meant should be derived from the context.

Notation for structures (!) A notation for structures is needed. This is illustrated in the following examples.

Example 2.2.5

Consider the vocabulary $\Sigma = \{0/0:, S/1:, ST/0, P/1, Q/2\}$. We define a Σ -structure $\mathfrak{A}$ as follows:

- $D_{\mathfrak{A}} = \{a, b, c, d\}$
- $O^{\mathfrak{A}} = a$
- $S^{\mathfrak{A}} = \{(a, b), (b, c), (c, a), (d, d)\}$ representing a total function as a set of pairs;
- $ST^{\mathfrak{A}} = \mathbf{t}$
- $P^{\mathfrak{A}} = \{a, b, c\}$
- $Q^{\mathfrak{A}} = \{(a, a), (a, b), (b, c)\}$

Notice that a, b, c, d are not non-logical symbols. Without symbols, a page remains blanc. We introduced these symbols to denote domain elements. **Do not confuse these symbols with non-logical symbols.**

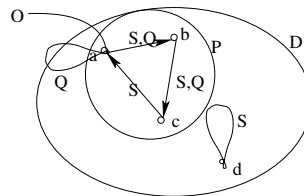
Example 2.2.6

We define a second, infinite Σ -structure $\mathfrak{A}$ for the same vocabulary:

- $D_{\mathfrak{A}} = \mathbb{N}$
- $O^{\mathfrak{A}} = 0$
- $S^{\mathfrak{A}} = \{(n, n+1) | n \in \mathbb{N}\}$
- $ST^{\mathfrak{A}} = \mathbf{f}$
- $P^{\mathfrak{A}} = \{n \in \mathbb{N} | n \text{ is a prime number}\}$
- $Q^{\mathfrak{A}} = \{(n, m) \in \mathbb{N}^2 | n < m\}$

Example 2.2.7: Graphical notation

For simple vocabularies Σ and finite structures $\mathfrak{A}$, the structure can be represented in a graphical notation. The domain and values of unary predicate symbols are represented as Venn-diagrams. The value of unary function symbols and binary predicate symbols are represented as sets of arcs labeled by the symbol. The value of an object symbols is written by an arc from the symbol to its value in the domain. This notation does not work for function and predicate symbols of greater arity or for propositional symbols. The following symbol graph represents the structure $\mathfrak{A}$ of Example 2.2.5 except for the propositional symbol ST :



Notations of vocabularies and structures in IDP3 notation The IDP3 knowledge base system used in this course supports an extension of FO. Vocabularies and structures are denoted similarly as in the mathematical notation. While FO is untyped, IDP3 supports a typed version of FO. Untyped vocabularies and structures can be simulated in a

typed logic by using a single type. In the following example, the vocabulary Σ and the Σ -structure $\mathfrak{A}$ of Example 2.2.5 is described in IDP3 notation. The example includes an informal interpretation $\mathcal{I}$ for each symbol.

Example 2.2.8

```

vocabulary V {
  type D
  O: D      // O means 'King of Belgium'
  S(D): D   // S(x) means 'Father of x'
  ST        // ST means 'it is sunny weather today'
  P(D)      // P(x) means 'x is a liberal'
  Q(D,D)    // Q(x,y) means 'x is a grandparent of y'
}

structure S:V {
  D = {a; b; c; d}
  O = a
  S = {a-> b; b->c; c->a; d->d}
  ST = true
  P = {a; b; c}
  Q = {(a,a); (a,b); (b,c)}
}

```

Notice that in the IDP3 notation, we write $\sigma = \dots$ rather than $\sigma^{\mathfrak{A}} = \dots$. That is as standard mathematical practice where symbols are systematically used as references to a value. In logic semantics, we write $\sigma^{\mathfrak{A}} = \dots$ to stress that it is not the symbol that satisfies this equality but the value assigned to the symbol. E.g., when writing $O = a$, it is obviously not meant that the symbol O is the same as the symbol a , but that a is the value assigned to the symbol O . In exams, use the mathematical notation for a structure.

Expanding a structure Given is a structure $\mathfrak{A}$, a symbol σ that may be interpreted by $\mathfrak{A}$ or not, and an appropriate value v for σ on the domain of $\mathfrak{A}$.

Definition 2.2.7

The **expansion** $\mathfrak{A}[\sigma : v]$ of $\mathfrak{A}$ is the structure identical to $\mathfrak{A}$ except that it assigns the value v to σ . The **restriction of $\mathfrak{A}$ to vocabulary Σ** , denoted $\mathfrak{A}|_{\Sigma}$, is identical to $\mathfrak{A}$ except that the value of each symbol not in Σ is dropped.

If $\mathfrak{A}$ interprets σ , then $\mathfrak{A}[\sigma : v]$ is identical to $\mathfrak{A}$ except that the value of σ is overridden.

The satisfaction relation of FO (!) We now define the value of expressions in structures: the value of terms and the satisfaction of formulas (which tells us the truth value of formulas).

Definition 2.2.8: Semantics of FO

Let t be a term interpreted by structure $\mathfrak{A}$. The *interpretation* (or *value*) $t^{\mathfrak{A}}$ of t in $\mathfrak{A}$ is defined by induction on the structure of t :

- If t is an object symbol, then $t^{\mathfrak{A}}$ is the domain element assigned by $\mathfrak{A}$ to t .
- If $t = f(t_1, \dots, t_n)$ then $t^{\mathfrak{A}}$ is the domain element $f^{\mathfrak{A}}(t_1^{\mathfrak{A}}, \dots, t_n^{\mathfrak{A}})$, that is, the domain element obtained by applying the function $f^{\mathfrak{A}}$ to the tuple of values $(t_1^{\mathfrak{A}}, \dots, t_n^{\mathfrak{A}})$.

Otherwise, if some free symbol of t is not interpreted in $\mathfrak{A}$ then $t^{\mathfrak{A}}$ is undefined.

[Satisfaction or truth relation] Let φ be a formula interpreted by $\mathfrak{A}$. We define that $\mathfrak{A}$ *satisfies* φ (denoted $\mathfrak{A} \models \varphi$) by an inductive case analysis on the structure of φ :

- $\mathfrak{A} \models P(t_1, \dots, t_n)$ if $(t_1^{\mathfrak{A}}, \dots, t_n^{\mathfrak{A}}) \in P^{\mathfrak{A}}$;
- $\mathfrak{A} \models (t = s)$ if $t^{\mathfrak{A}} = s^{\mathfrak{A}}$;
- $\mathfrak{A} \models (\alpha \wedge \beta)$ if $\mathfrak{A} \models \alpha$ and $\mathfrak{A} \models \beta$;
- $\mathfrak{A} \models (\alpha \vee \beta)$ if $\mathfrak{A} \models \alpha$ or $\mathfrak{A} \models \beta$ (or both);
- $\mathfrak{A} \models (\neg \alpha)$ if $\mathfrak{A} \not\models \alpha$; (that is, $\mathfrak{A}$ does not satisfy α);
- $\mathfrak{A} \models (\alpha \Rightarrow \beta)$ if $\mathfrak{A} \not\models \alpha$ or $\mathfrak{A} \models \beta$ (or both);
- $\mathfrak{A} \models (\alpha \Leftrightarrow \beta)$ if $\mathfrak{A} \models \alpha$ and $\mathfrak{A} \models \beta$ or $\mathfrak{A} \not\models \alpha$ and $\mathfrak{A} \not\models \beta$;
- $\mathfrak{A} \models (\exists x \alpha)$ if there exists $d \in D_{\mathfrak{A}}$ such that $\mathfrak{A}[x : d] \models \alpha$;
- $\mathfrak{A} \models (\forall x \alpha)$ if for all $d \in D_{\mathfrak{A}}$, it holds that $\mathfrak{A}[x : d] \models \alpha$.

If $\mathfrak{A}$ does not interpret φ , then $\mathfrak{A} \models \varphi$ is undefined. Also, $\varphi^{\mathfrak{A}}$ is **t** if $\mathfrak{A} \models \varphi$, it is **f** if $\mathfrak{A} \not\models \varphi$. If $\mathfrak{A}$ does not interpret φ , then $\varphi^{\mathfrak{A}}$ is undefined.

Remark 2.2.2: What does this inductive definition mean?

In this definition, $t^{\mathfrak{A}}$ and $\mathfrak{A} \models \varphi$ are defined **by induction on the structure of t and φ** . This is a new type of inductive definition. What does it mean?

It means that according to this definition, $t^{\mathfrak{A}}$ is determined from the values $s^{\mathfrak{A}}$ of the component terms s of t , and the satisfaction of φ in $\mathfrak{A}$ is determined from the satisfaction of component formulas φ' of φ . In particular, this definition is an example of **definition by induction on an induction order $<_i$** . Such a definition defines a function or relation for some object in terms of the value of the function or relation for objects that are strictly smaller in the induction order $<_i$. For $t^{\mathfrak{A}}$, the induction order is the subterm order: $s <_i t$ if s is a strict subterm of t . For $\mathfrak{A} \models \varphi$, the induction order is the subformula order. E.g., we define $\mathfrak{A} \models \neg \varphi$ if $\mathfrak{A} \not\models \varphi$, and indeed, φ is a subformula of $\neg \varphi$: $\varphi <_i \neg \varphi$. Induction on a component relation is also called **definition by structural induction**. Induction on an induction order is used for the factorial function $n!$:

We define the factorial function $! : \mathbb{N} \rightarrow \mathbb{N}$ by induction on the natural numbers:

- $0! = 1$
- $(n+1)! = (n+1) \times n!$

Here, the induction order $<_i$ is the standard order of the natural numbers, and indeed, $!(n+1)$ is derived from $n!$.

Definition 2.2.8 of $\models$ is formulated as a collection of rules. Also this definition admits an induction process which starts from the empty binary relation and proceeds by iterated rule application until eventually, after infinitely many steps, all rules are satisfied. However, something important is different compared to the induction process of monotone definitions as described in Remark 2.2.1.

For monotone definitions such as the definition of term in Definition 2.2.2, the rules can be applied in any way that is possible. Any order of rule application will produce the same defined set. However, Definition 2.2.8 is a **non-monotone** definition: it contains

non-monotone rules such as the $\neg$ -rule “ $\mathfrak{A} \models \neg\varphi$ if $\mathfrak{A} \not\models \varphi$ ”: this rule is applicable if $(\mathfrak{A}, \varphi)$ is *absent* from the satisfaction relation. For such definitions, the order of rule application matters.

During the induction process of $\models$, this relation grows. In early phases, it may be that for some formula φ , the condition $\mathfrak{A} \not\models \varphi$ is satisfied so that the $\neg$ -rule instance for deriving $\mathfrak{A} \models \neg\varphi$ seems applicable. E.g., in the initial step, when the set is still empty, the condition of the $\neg$ -rule is satisfied for *any* $\neg\varphi$ since $(\mathfrak{A}, \varphi) \notin \emptyset$. Obviously, this is dangerous. We should wait to apply the $\neg$ -rule until it is safe and derive $\mathfrak{A} \models \neg\varphi$ only once the induction process has had the chance to derive whether $\mathfrak{A} \models \varphi$ or not. If we apply the rule to derive $\mathfrak{A} \models \neg\varphi$ too early, later rule applications might still derive $\mathfrak{A} \models \varphi$ in which case we have derived both $\mathfrak{A} \models \neg\varphi$ and $\mathfrak{A} \models \varphi$, which is a contradiction.

In definitions over an induction order, the order of rule applications is not arbitrary but must follow the induction order $<_i$. In the case of $\models$, a rule instance deriving $\mathfrak{A} \models \varphi$ cannot be applied as long as there are subformulas α of φ such that $\mathfrak{A} \models \alpha$ is derivable using one of the rules of the definition, but has not been derived yet. This idea applies to all definitions by induction on a well-founded order.

Remark 2.2.3

It is important to understand Definition 2.2.8, not only the defined concept of satisfiability $\models$ itself, but also the definitional mechanism that is used to define it as explained in the previous remark. In the next chapter, we will routinely express definitions of this type in an extension of FO.

Now that the truth relation, or the satisfaction relation, has been defined for FO, recall Terminology 1.2.1):

Terminology 2.2.1

Assume that $\mathfrak{A}$ interprets formula φ . The following terms are synonyms:

- $\mathfrak{A}$ *satisfies* φ ;
- $\mathfrak{A} \models \varphi$;
- φ is *true* in $\mathfrak{A}$;
- $\mathfrak{A}$ is a *model* of φ .

The following terms are synonyms:

- $\mathfrak{A}$ does not *satisfy* φ ;
- $\mathfrak{A} \not\models \varphi$;
- φ is *false* in $\mathfrak{A}$.

A third case is where $\mathfrak{A}$ does not interpret φ . All the above statements are undefined then.

Values of connectives In Definition 2.2.8, the rules define the meaning of formal connectives in terms of the corresponding natural language connectives. E.g., the rule “ $\mathfrak{A} \models \alpha \wedge \beta$ if $\mathfrak{A} \models \alpha$ and $\mathfrak{A} \models \beta$.”. An commonly used, equivalent, but more formal way is to specify the value of connectives through truth tables. Below are the truth tables of FO’s connectives.

$\neg$		$\wedge$	t	f	$\vee$	t	f	$\Rightarrow$	t	f	$\Leftrightarrow$	t	f
t	f	t	t	f	t	t	t	t	t	f	t	t	f
f	t	f	f	f	f	t	f	f	t	t	f	f	t

Frege’s compositionality principle It can be seen from Definition 2.2.8 that FO satisfies *Frege’s compositionality principle*: the value of a composite expression in a structure is functionally determined by the value of the symbol at the top of the expression and the values of its subexpressions. E.g., $(\alpha \wedge \beta)^{\mathfrak{A}}$ is computed by applying the semantic boolean function associated with $\wedge$ on $\alpha^{\mathfrak{A}}$ and $\beta^{\mathfrak{A}}$.

This principle is often but not always satisfied in natural language. E.g., assume a state of affairs where first I went home, then I called my wife. If natural language was fully compositional, then the following sentences should have the same truth value: “I went home and I called my wife”, and “I called my wife and I went home”. But only the first is true. Here “and” is used to express conjunction and temporal precedence of the first argument.

Overloading and ambiguity Several natural languages connectives are overloaded and have more than one interpretation. E.g., “or” can be inclusive or exclusive, “and” can be pure conjunction or temporal conjunction as explained in the previous paragraph. This may lead to ambiguity of a natural language text, although often the text can be disambiguated using the context. In Definition 2.2.8, the meaning of the logical connectives is defined in terms of the corresponding natural language connectives. Care is taken that the latter are non-ambiguous. E.g., the rule for $\vee$ states “ $\mathfrak{A} \models \alpha$ or $\mathfrak{A} \models \beta$, or both”. This shows that the inclusive “or” is meant here. So, the meaning of the logical connectives is well-defined.

Exercise 2.2.2 Compare the disjunction in the following sentences:

- *Bill or his wife will be home.*
- *All employees of company XYZ receive as a legal benefit a company car or a train subscription.*

We cannot know for sure. Still, is the disjunction in both sentences intended to be inclusive or exclusive?

What φ states about $\mathfrak{A}$ expressed as $NL_{\mathfrak{A}}(\varphi)$ For many people it is simple: mathematics is precise and natural language is vague and ambiguous. It is not that simple, as demonstrated below.

Definition 2.2.8 defines the values of terms and formulas in structures. There are several ways in which this definition can be used to determine whether some structure $\mathfrak{A}$ satisfies a sentence φ . One way is by reasoning bottom up. We start by evaluating expressions at the bottom of the sentence, and then work the way up. This simulates the part of the induction process of the satisfaction relation $\models$ that is relevant for φ . There is also the way of reasoning backwards, or top down. We start from $\mathfrak{A} \models \varphi$ and iteratively apply the rules of Definition 2.2.8 backwards, to simplify φ . In the end we obtain a condition $NL_{\mathfrak{A}}(\varphi)$ which is a natural language statement about the values of symbols in $\mathfrak{A}$ that is mathematically equivalent with $\mathfrak{A} \models \varphi$. Thus, $\mathfrak{A} \models \varphi$ iff $NL_{\mathfrak{A}}(\varphi)$ is a true NL statement about $\mathfrak{A}$. We illustrate the top down method with an example.

Take the following formula:

$$\varphi = \exists x(\neg(Q(x, S(x))))$$

By repeated application of the rules of Definition 2.2.8, we can *unfold* occurrences of $\models$:

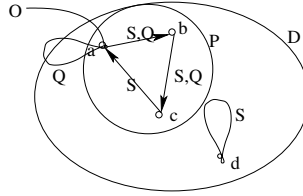
$\mathfrak{A} \models \varphi$ iff there exists $x \in D_{\mathfrak{A}}$ such that $\mathfrak{A} \models \neg(Q(x, S(x)))$
 iff there exists $x \in D_{\mathfrak{A}}$ such that it is not the case that $\mathfrak{A} \models Q(x, S(x))$
 iff there exists $x \in D_{\mathfrak{A}}$ such that it is not the case that $(x, S^{\mathfrak{A}}(x)) \in Q^{\mathfrak{A}}$
 iff there exists $x \in D_{\mathfrak{A}}$ such that $(x, S^{\mathfrak{A}}(x)) \notin Q^{\mathfrak{A}}$.

Thus, we have proven here that φ is true in $\mathfrak{A}$ iff the following proposition holds:

there exists $x \in D_{\mathfrak{A}}$ such that $(x, S^{\mathfrak{A}}(x)) \notin Q^{\mathfrak{A}}$.

Example 2.2.9

Take once more the formula $\varphi = \exists x(\neg(Q(x, S(x))))$, and evaluate it in the structure $\mathfrak{A}$ of Example 2.2.5 graphically represented as:



It holds that $(c, a) \notin Q^{\mathfrak{A}}$ and $a = S^{\mathfrak{A}}(c)$, therefore there is an element $x \in D_{\mathfrak{A}}$ such that $(x, S^{\mathfrak{A}}(x)) \notin Q^{\mathfrak{A}}$ (take x to be c). Hence, we have $\mathfrak{A} \models \varphi$.

Exercise 2.2.3 In Example 2.2.6, a second structure interpreting the same vocabulary is proposed. Verify whether $NL_{\mathfrak{A}}(\exists x(\neg(Q(x, S(x))))$ holds in that structure $\mathfrak{A}$.

It should be clear that the method of iterated application of Definition 2.2.8 can be applied to any FO sentence φ , for any structure $\mathfrak{A}$. In a number of steps depending on the size of φ , it translates φ in a provably equivalent mathematical statement $NL_{\mathfrak{A}}(\varphi)$ in natural language that is true in a structure $\mathfrak{A}$ iff $\mathfrak{A} \models \varphi$.

FO emerges as formal notation for a particular class of precise mathematical statements in natural language.

Example 2.2.10: A final structure

A **final** or **terminal** Σ -structure $\mathfrak{A}$ has domain $D_{\mathfrak{A}} = \{a\}$, for each n -ary function symbol f ($n \geq 0$), $f^{\mathfrak{A}}$ is $\{((a, \dots, a), a)\}$, and for each n -ary predicate symbol P , $P^{\mathfrak{A}}$ is $\{(a, \dots, a)\}$. All final Σ -structures are isomorphic. Final structures will often be useful to show unexpected properties of sentences. Every sentence over Σ without $\neg$, $\Rightarrow$ and $\Leftrightarrow$ is true in $\mathfrak{A}$.

Exercise 2.2.4 Verify this claim. The proof is by induction on the structure of formulas (but is very simple).

2.2.3 Informal semantics

When we write a formal proposition in a specification of an application domain, what proposition about the application domain have we expressed? This is undeniably an important question for modelling languages. We define here a *theory of informal semantics* for FO that addresses this question.

The key concept is that of **informal interpretation** $\mathcal{I}$. It serves to relate symbols to concepts of the application domain, formal sentences to propositions about states of affairs of the application domain, structures as abstractions to states of affairs, and values in structures to entities in states of affairs.

To use FO for modelling in an application domain, the knowledge engineer selects a finite set of n -ary relational and functional concepts ($n \geq 0$) in the application domain. Symbols are chosen to represent these concepts. This creates a vocabulary Σ and an informal semantics $\mathcal{I}$ mapping symbols $\sigma \in \Sigma$ to concepts $\mathcal{I}(\sigma)$ of the corresponding type. For example, a binary predicate symbol $Fof/2$ is mapped to the binary relational concept between a father and his child.

According to Leibniz, a proposition is a thought. A more “material” way of expressing the thought $\mathcal{I}(\varphi)$ that is expressed by a sentence φ is obtained by translating φ into a natural language sentence. For this, the informal interpretation of an ontology symbol need to be expressed as a natural language pattern: a noun-phrase with n parameters for an n -ary function symbol ($n \geq 0$); a verb-phrase with n -parameters for an n -ary relation symbol ($n \geq 0$).

It is then possible to translate each logic formula φ over Σ , into a syntactically correct, meaningful natural language string $NL_{\mathcal{I}}(\varphi)$ that expresses $\mathcal{I}(\varphi)$. This works the same way as the translation $NL_{\mathfrak{A}}(\varphi)$ defined in the previous section on page 44. The only difference is that the translation uses the natural language patterns expressing $\mathcal{I}(\sigma)$ rather than $\sigma^{\mathfrak{A}}$. It is illustrated in the following example.

Example 2.2.11: Royal family

Suppose the application domain is the royal family. We define a vocabulary Σ and an informal interpretation $\mathcal{I}$:

- KoB : “the king of Belgium”
- $Filip$: Filip (Filip Leopold Lodewijk Maria van België) .
- $Fof(x, y)$: “ x is the father of y ”
- $AgeOf(x)$: “the age of x ”
- $x + y$: “the sum of x and y ”
- $KisO$: “The king of Belgium is old”
- $Q(x, y)$: “ x is a grandparent of y ”.

Every FO formula φ over Σ can now be translated into a natural language sentence expressing $\mathcal{I}(\varphi)$.

$$KoB = Filip$$

“The king of Belgium is Filip”.

$$\exists x(Q(x, KoB) \wedge \exists y(Fof(y, KoB) \wedge AgeOf(x) < AgeOf(y)))$$

“There exists x in the universe such that x is a grandparent of the king of Belgium and there exists y in the universe that is the father of the king and the age of x is less than the age of y ”. Or, the king of Belgium has a grandparent that is younger than his father. I do not know if this is true in the actual state of affairs.

$$\forall x \forall y \forall z (Fof(x, y) \wedge Fof(y, z) \Rightarrow Q(x, z))$$

“For all x, y, z in the universe, if x is the father of y and y is the father of z then x is

a grandparent of z". Or, the father of a father is a grandparent.

$$\exists x (Fof(x, KoB) \wedge (AgeOf(x) = AgeOf(KoB) + AgeO(KoB)))$$

"There exists x in the universe, x is the father of the king of Belgium and the age of x is the sum of the age of the king of Belgium and the age of the king of Belgium". Or, the king's father is twice as old as the king.

$$(\exists x Q(KoB, x)) \Rightarrow KisO$$

Or, if the king is a grandparent, he is old.

Exercise 2.2.5 Take the application domain of students and courses which contains the following ontology symbols, together with natural-language patterns expressing the informal interpretation $\mathcal{I}$:

- T : Tom;
- M : MCS;
- $P(x)$: x is a passing grade;
- $G(x, y, z)$: student x has grade y for the exam of course z .

What is $\mathcal{I}(\exists g(G(T, M, g) \wedge P(g)))$?

The informal semantics of a logical sentence under an informal interpretation is sometimes also called its *declarative or intuitive reading*.

Informal interpretation of a structure The informal interpretation $\mathcal{I}(\mathfrak{A})$ of a structure $\mathfrak{A}$ is a potential state of affairs. There is a one-to-one connection between the elements of the domain of $\mathfrak{A}$ and the collection of entities in that state of affairs, and between the values of the symbols in $\mathfrak{A}$ and the values of the concepts in the state of affairs.

Example 2.2.12

Take the following structure $\mathfrak{A}$ interpreting the vocabulary of the Royal Family Example 2.2.11:

- $D_{\mathfrak{A}} = \{a\}$
- $KoB^{\mathfrak{A}} = Filip^{\mathfrak{A}} = a$
- $AgeOf^{\mathfrak{A}} = \{(a, a)\}$, the function that maps a to a
- $+^{\mathfrak{A}} = \{((a, a), a)\}$, the function that maps (a, a) to a
- $KisO^{\mathfrak{A}} = \mathbf{t}$
- $Fof^{\mathfrak{A}} = Q^{\mathfrak{A}} = \{(a, a)\}$

Then $\mathcal{I}(\mathfrak{A})$ would be a state of affairs where the royal family consists of one entity $\mathfrak{A}(a)$ which is king, is Filip, is its own age, is the sum of itself and itself, and last but not least is both its own father and grandparent. Quite impossible indeed.

Exercise 2.2.6 Think of properties that make the Σ -structures of the Royal Family vocabulary more sensible abstractions of (royal) families and express as FO sentences in the proposed vocabulary.

The semantic correspondence question Could there be a mismatch between the formal and the informal semantics of FO? Following section 1.2, such a problem arises if a logical sentence φ , informal interpretation $\mathcal{I}$ and a structure $\mathfrak{A}$ can be found such that the

truth value of φ in $\mathfrak{A}$ disagrees with the truth of the proposition $\mathcal{I}(\varphi)$ in $\mathcal{I}(\mathfrak{A})$.

E.g., assume that φ contains disjunction $\vee$, which is interpreted in the formal semantics as inclusive disjunction, and in the informal semantics as exclusive disjunction, then there would be semantic correspondence problem with $P \vee Q$, the structure $\mathfrak{A} = \{P, Q\}$ in which P and Q is true, any informal interpretation $\mathcal{I}$ and $\mathcal{I}(\mathfrak{A})$. This is obvious. However, this problem does not occur for FO, since $\vee$ is interpreted as inclusive disjunction, both in formal and informal semantics.

To get hold of this question, consider that what is expressed by φ about a structure $\mathfrak{A}$ is given a mathematical statement $NL_{\mathfrak{A}}(\varphi)$, and what is expressed by φ about a state of affairs $\mathcal{I}(\mathfrak{A})$ can be phrased as a natural language statement $NL_{\mathcal{I}}(\varphi)$. These statements are identical, except that the first talks about mathematical values $\sigma^{\mathfrak{A}}$ where the second talks about the relational or functional concept $\mathcal{I}(\sigma)$. It is therefore highly unlikely that semantic mismatches can be found.

Example 2.2.13

Consider the royal family vocabulary Σ with its informal interpretation $\mathcal{I}$ given in Example 2.2.11, and take the following sentence:

$$\forall x \forall y \forall z (Fof(x, y) \wedge Fof(y, z) \Rightarrow Q(x, z))$$

It is true in a structure $\mathfrak{A}$ if the following sentence is true:

$$\text{For all } x, y, z \in D_{\mathfrak{A}}, \text{ if } (x, y), (y, z) \in Fof^{\mathfrak{A}} \text{ then } (x, z) \in Q^{\mathfrak{A}}$$

It is true in a state of affairs $\mathcal{I}(\mathfrak{A})$ if :

For all x, y, z in the universe, if x is the father of y and y is the father of z then x is a grandparent of z

This is a slightly artificial way of saying that a father of a father is a grandparent. It is not possible to find a $\mathfrak{A}$ and the corresponding $\mathcal{I}(\mathfrak{A})$ such that the two statements have different truth values.

Here are a few more meta-comments about the above question. As explained in chapter 1, one cannot prove that a scientific theory is correct but one might be able to prove that it is incorrect. This is done by performing an experiment that refutes the theory.

Here the scientific theory is that formal semantics of FO matches with its informal semantics. That cannot be proven, for the simple reason that the informal interpretation of a sentence is not a formal object. An experiment could be a choice for an application domain, $\Sigma, \mathcal{I}$, a Σ -structure $\mathfrak{A}$, and finally a proposition φ over Σ . The experiment consists then to determine whether $\mathfrak{A} \models \varphi$ iff $\mathcal{I}(\varphi)$ is true in $\mathcal{I}(\mathfrak{A})$. If that is the case, the experiment supports FO and its informal semantics. Otherwise, it refutes FO and its informal semantics.

Above we explained why it is unlikely to find such problems. FO is constructed from the most precise natural language connectives, and these are used in the definition of the formal semantics and in the informal semantics. It seems impossible that a semantic correspondence problem could arise.

Nevertheless, one potentially weak point is the fact that we defined:

$$\mathfrak{A} \models (\alpha \Rightarrow \beta) \text{ if } \mathfrak{A} \not\models \alpha \text{ or } \mathfrak{A} \models \beta.$$

On the informal side, this was rephrased as “if α then β ”. Imagine that in natural language

“if ... then ...” is overloaded and has an interpretation that does not correspond with $\neg \dots \vee \dots$. Then there might be cases where a semantic mismatch problem would seem to arise. We will later see examples like this.

2.2.4 Derived semantical concepts of the modelling logic FO

FO has a formal syntax, formally defined satisfaction relation, and a well-understood informal semantics. As such it is a *modelling logic*.

In Chapter 1, Definition 1.2.2 defined derived semantical concepts for modelling logics, formalising informal concepts already existing in mathematics and formal science. These concepts are defined also for FO. The following definitions remain the same, with one exception:

- T is **satisfiable** or **logically consistent** if there exists a structure that satisfies T .
- T is **unsatisfiable** or **logically inconsistent** or contradictory if there is no structure that satisfies T .
- T is **tautologically true** or **logically valid** (notation $\models T$) if T is satisfied in every structure interpreting T .
- T is **logically equivalent** to T' (notation $T \equiv T'$) if T and T' have the same truth value in every structure that interprets both of them.
- T **logically entails** T' (notation $T \models T'$) if T' is satisfied in every structure $\mathfrak{A}$ that interprets both T and T' and in which T is satisfied.
- A FO theory T over Σ is **categorical** in Σ if T has a unique Σ -model.

Only one concept needs to be refined: categoricity. The intuition of a categorical theory is that it characterises a unique state of affairs (think of a database). In the next section, we will see that in FO, even theories that characterise a unique state of affairs have infinitely many models, though all of them have the same internal structure: they are isomorphic. The definition of categorical theory will be redefined at the end of the next section.

2.2.5 Useful forms of inference

The forms of inference of Definition 1.3.3 are all relevant for FO. Most of them are implemented in the IDP system. Examples will be seen later in this course, in the exercise sessions and in the project.

2.2.6 Basic properties

When formal scientists subjectively evaluate the importance of theorems, they tend to use the following criteria: how difficult was it to prove it? How surprising is the theorem? For example, Gödel's incompleteness theorem, still considered to be one of the most important theorems in mathematics, was very surprising and very difficult to prove (see chapter 6). But some important properties are not surprising and they are easy to prove. However, their importance would become very obvious from the “disasters” it would create if they would *not* hold. We see two such properties here.

The first is that the value of an FO expression e in a structure $\mathfrak{A}$ depends only on the value of the free symbols of e in $\mathfrak{A}$. If this property would not hold, then the value of e in structure $\mathfrak{A}$ could change by expanding $\mathfrak{A}$ with a value for a new symbol. E.g., in one structure $\mathfrak{A}$ it might hold that $Grade(Tom, MCS, AA)$ but after expanding $\mathfrak{A}$ with a value for a new predicate $TelephonenNumberOf/2$, the fact becomes false. That would be a catastrophe.

Proposition 2.2.1

Let $\mathfrak{A}$ and $\mathfrak{A}'$ be two structures with the same domain and the same value for all the free symbols of expression e (term or formula). It holds that $e^{\mathfrak{A}} = e^{\mathfrak{A}'}$.

Proof. The proof is by induction on the structure of e .

Base case: if e is an object or propositional symbol, then e is a free symbol of e and hence, the values of e in $\mathfrak{A}$ and $\mathfrak{A}'$ are identical. It follows that $e^{\mathfrak{A}} = e^{\mathfrak{A}'}$.

Inductive cases. Let e be a compound expression. The induction hypothesis is that the equality $e'^{\mathfrak{A}} = e'^{\mathfrak{A}'}$ holds for all subexpressions e' of e . We must prove that it also holds for e . For all expressions e except quantified expressions, the argument is the same: the component expressions of e have the same value in $\mathfrak{A}$ and $\mathfrak{A}'$, and the value of e in $\mathfrak{A}$ and $\mathfrak{A}'$ is obtained from these values by applying the same function to it, namely the value of a free function symbol or of a logical symbol ($=, \neg, \wedge, \dots$). It follows that $e^{\mathfrak{A}} = e^{\mathfrak{A}'}$.

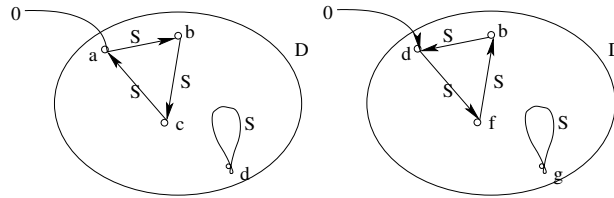
This argument does not work for quantified formulas $\exists x \alpha$ and $\forall x \alpha$ since the free symbols of the main formula and of its subformula α are not the same. Take e to be $\exists x \alpha$. The free symbols of α are x and those of e . Let d be an arbitrary value from the domain of $\mathfrak{A}$ or $\mathfrak{A}'$ (they have the same domain). It holds that $\mathfrak{A}[x : d]$ and $\mathfrak{A}'[x : d]$ have the same domain and the same values for all free symbols of α . The induction hypothesis entails that $\alpha^{\mathfrak{A}[x:d]} = \alpha^{\mathfrak{A}'[x:d]}$. Since this holds for arbitrary $d \in D_{\mathfrak{A}}$, it follows easily that $e^{\mathfrak{A}} = e^{\mathfrak{A}'}$. A similar reasoning applies to $\forall x \alpha$. ■

In particular, if $\mathfrak{A}$ and $\mathfrak{A}'$ are structures with the same domain and the same value for the free symbols of a first order formula or theory φ , then $\mathfrak{A} \models \varphi$ iff $\mathfrak{A}' \models \varphi$. Intuitively, it means that φ contains only information about its free symbols. According to φ , the value of any other symbol is *arbitrary*. It entails that the class of models of φ is closed under expansions. Recall, $\mathfrak{A}'$ is an expansion of $\mathfrak{A}$ if $\mathfrak{A}'$ is obtained from $\mathfrak{A}$ by adding an interpretation to symbols that are not interpreted by $\mathfrak{A}$.

Corollary 2.2.1

If $\mathfrak{A}$ satisfies φ then any expansion $\mathfrak{A}'$ of $\mathfrak{A}$ satisfies φ .

A second proposition is about structures that have the same internal structure. E.g., consider the following structures $\mathfrak{A}$, respectively $\mathfrak{A}'$ for the vocabulary $\{0/0; S/1\}$.



While the left structure $\mathfrak{A}$ and the right one $\mathfrak{A}'$ are different, they clearly possess the same internal patterns. But what does that mean *exactly*? Well, consider the following mapping $b : D_{\mathfrak{A}} \mapsto D_{\mathfrak{A}'}$:

$$b = \{(a, d), (b, f), (c, b), (d, g)\}$$

This is a bijection. Importantly, b maps the value of each symbol σ in the first structure to its value in the second structure: $b(\sigma^{\mathfrak{A}}) = \sigma^{\mathfrak{A}'}$. Indeed, we have $b(0^{\mathfrak{A}}) = b(a) = d = 0^{\mathfrak{A}'}$. We apply (an extension of) b to the set $S^{\mathfrak{A}}$ by applying b to all domain elements inside it, and observe that $b(S^{\mathfrak{A}}) = b(\{(a, b), (b, c), (c, a), (d, d)\}) = \{(d, b), (b, f), (f, d), (g, g)\} = S^{\mathfrak{A}'}$.

This formalises the idea that the two structures have the same internal patterns. We say that b is an *isomorphism*.

In general, given a function $b : D \mapsto E$, let X be some composite object (tuples, sets, functions, ...) in which elements of D appear. Then we obtain $b(X)$ by substituting any occurrence of an element $x \in D$ within X by $b(x)$.

Definition 2.2.9

A structure $\mathfrak{A}$ is Σ -isomorphic to structure $\mathfrak{A}'$ (notation $\mathfrak{A} \simeq_{\Sigma} \mathfrak{A}'$) if

- they both interpret all symbols of Σ (and possibly others), and
- there exists a bijection $b : D_{\mathfrak{A}} \rightarrow D_{\mathfrak{A}'}$ such that for each symbol $\sigma \in \Sigma$, $b(\sigma^{\mathfrak{A}}) = \sigma^{\mathfrak{A}'}$.

Two structures are isomorphic (notation $\mathfrak{A} \simeq \mathfrak{A}'$) if they interpret the same vocabulary Σ and they are Σ -isomorphic.

Every structure is isomorphic to infinitely many other structures. All isomorphic structures are abstract representations of the same states of affairs. Therefore, the truth value of a formula should be the same in all isomorphic structures that interpret it.

Proposition 2.2.2: Truth in isomorphic structures

If $\mathfrak{A} \simeq_{\Sigma} \mathfrak{A}'$ then for every formula φ over Σ , it holds that $\mathfrak{A} \models \varphi$ iff $\mathfrak{A}' \models \varphi$.

Exercise 2.2.7 [advanced] Prove this proposition. Hint: the proposition follows from the more general proposition that for every expression e over Σ , for all Σ -structures $\mathfrak{A}$ and $\mathfrak{A}'$, for every Σ -isomorphism b from $\mathfrak{A}$ to $\mathfrak{A}'$, it holds that $b(e^{\mathfrak{A}}) = e^{\mathfrak{A}'}$. Prove this by induction on the structure of e . Then use the fact that for every isomorphism b , $b(\mathbf{t}) = \mathbf{t}$ and $b(\mathbf{f}) = \mathbf{f}$.

It follows that if a theory has a model, it has infinitely many models.

In view of Corollary 2.2.1 and Proposition 2.2.2, we see that if a theory has one model, it has infinitely many: all isomorphic structures and all its expansions. However, isomorphic Σ -structures are abstractions of the same state of affairs. This induces to redefine the notion of categorical theory.

Definition 2.2.10: Categoricity

A FO theory over Σ is categorical in Σ iff all its models are Σ -isomorphic.

2.2.7 Laws of thought and pushing negation inside

There is a great number of properties of rigorous natural language statements that can be proven in the context of logic. For more information, see [15]. A few important ones are incorporated as the following list of **tautologies**:

- Idempotence, commutativity, associativity, distributivity of $\wedge$ and $\vee$:

$$\begin{aligned}
\varphi \wedge \varphi &\Leftrightarrow \varphi \\
\varphi \vee \varphi &\Leftrightarrow \varphi \\
\varphi \wedge \psi &\Leftrightarrow \psi \wedge \varphi \\
\varphi \vee \psi &\Leftrightarrow \psi \vee \varphi \\
\varphi \wedge (\psi \wedge \gamma) &\Leftrightarrow (\varphi \wedge \psi) \wedge \gamma \\
\varphi \vee (\psi \vee \gamma) &\Leftrightarrow (\varphi \vee \psi) \vee \gamma \\
\varphi \wedge (\psi \vee \gamma) &\Leftrightarrow (\varphi \wedge \psi) \vee (\varphi \wedge \gamma) \\
\varphi \vee (\psi \wedge \gamma) &\Leftrightarrow (\varphi \vee \psi) \wedge (\varphi \vee \gamma)
\end{aligned}$$

- Laws of quantification:

$$\begin{aligned}
\forall x \forall y \varphi &\Leftrightarrow \forall y \forall x \varphi \\
\exists x \exists y \varphi &\Leftrightarrow \exists y \exists x \varphi \\
\forall x (\varphi \wedge \psi) &\Leftrightarrow (\forall x \varphi) \wedge (\forall x \psi) \\
\exists x (\varphi \vee \psi) &\Leftrightarrow (\exists x \varphi) \vee (\exists x \psi) \\
\forall x (\varphi \vee \psi) &\Leftrightarrow (\forall x \varphi) \vee \psi \text{ (if } x \text{ does not occur free in } \psi) \\
\exists x (\varphi \wedge \psi) &\Leftrightarrow (\exists x \varphi) \wedge \psi \text{ (if } x \text{ does not occur free in } \psi)
\end{aligned}$$

- Laws of negation: law of excluded middle, and distributing negation over connectives:

$$\begin{aligned}
\varphi \vee \neg \varphi & \\
\neg \neg \varphi &\Leftrightarrow \varphi \\
\neg(\psi \vee \varphi) &\Leftrightarrow (\neg \psi \wedge \neg \varphi) \\
\neg(\psi \wedge \varphi) &\Leftrightarrow (\neg \psi \vee \neg \varphi) \\
\neg(\psi \Rightarrow \varphi) &\Leftrightarrow (\psi \wedge \neg \varphi) \\
\neg(\psi \Leftrightarrow \varphi) &\Leftrightarrow (\psi \Leftrightarrow \neg \varphi) \\
\neg(\exists x \varphi) &\Leftrightarrow \forall x (\neg \varphi) \\
\neg(\forall x \varphi) &\Leftrightarrow \exists x (\neg \varphi)
\end{aligned}$$

All tautologies in this list express equivalence of two formulas. They can be used for rewriting formulas in equivalent formulas. In particular the laws of negation are useful for such analysis. To understand that a complex proposition φ is false is not the same as to understand what is true. If φ is complex, it may not be clear what $\neg \varphi$ means. A way to get insight is to apply the laws of negation on $\neg \varphi$, by **pushing negation inside**. That is, to distribute $\neg$ over connectives and quantifiers of φ till $\neg$ occurs only in front of atomic formulas. The result is a positive statement that explains what is true.

Example 2.2.14

Verify that with the laws of negation, the following sentence

$$\neg [\forall x (Vertex(x) \Rightarrow \exists y (Edge(x, y) \wedge \neg(x = y)))]$$

is translated in:

$$\exists x (Vertex(x) \wedge \forall y (\neg Edge(x, y) \vee (x = y)))$$

Introducing $\Rightarrow$, this is logically equivalent to:

$$\exists x (Vertex(x) \wedge \forall y (Edge(x, y) \Rightarrow (x = y)))$$

Explain what these sentences mean, under the informal interpretation suggested by the predicate symbols.

These laws of thought are easy to prove, e.g., propositional tautologies using truth tables. As for the tautologies with quantifiers, consider the case of $\neg(\exists x \varphi) \Leftrightarrow \forall x(\neg\varphi)$ is logically valid. To prove this, it suffices to show that a structure $\mathfrak{A}$ satisfies the left hand iff it satisfies the right hand.

$\mathfrak{A} \models \neg(\exists x \varphi)$
 iff $\mathfrak{A} \not\models \exists x \varphi$
 iff there is no $d \in D_{\mathfrak{A}}$ such that $\mathfrak{A}[x : d] \models \varphi$
 iff for all $d \in D_{\mathfrak{A}}$, it holds that $\mathfrak{A}[x : d] \not\models \varphi$
 iff for all $d \in D_{\mathfrak{A}}$, $\mathfrak{A}[x : d] \models \neg\varphi$
 iff $\mathfrak{A} \models \forall x(\neg\varphi)$.

Exercise 2.2.8 [basic, easy] Select another equivalence and prove it.

Exercise 2.2.9 [basic, easy] The laws of negation work also in natural language. Take the following NL proposition about a language L (a set of strings over some vocabulary Σ).

There exists a pumping length d such that for each string $s \in L$ of length at least d , there exists a partitioning of s into three parts x, y, z such that $s = xyz$ and for all $n \geq 0$, the string $xy^n z$ belongs to L .

What would be true if this proposition is false: negate this proposition and push negation inside. This proposition is known to be true when L is a regular language. This theorem is called the pumping lemma. One technique to prove that L is not regular is to prove that the negation of the proposition is true for L .

2.2.8 Some examples

Expressing the size of a predicate or the universe

In FO, constraints on the size of predicates or the universe can be expressed.

The predicate $P/1$ contains at least $n = 2$ elements:

$$\exists x_1 \exists x_2 (P(x_1) \wedge P(x_2) \wedge \neg(x_1 = x_2))$$

The universe contains at least $n = 2$ elements (only using the logical equality predicate):

$$\exists x \exists y \neg(x = y)$$

Exercise 2.2.10 Extend this for $n=3, n=4, \dots$ arbitrary n . The sentences get longer and longer. How fast do they grow with n ?

The predicate $P/1$ contains at most $n = 2$ elements:

$$\forall x_1 \forall x_2 \forall x_3 (P(x_1) \wedge P(x_2) \wedge P(x_3) \Rightarrow (x_1 = x_2) \vee (x_1 = x_3) \vee (x_2 = x_3))$$

The universe contains at most $n = 2$ elements:

$$\forall x_1 \forall x_2 \forall x_3 ((x_1 = x_2) \vee (x_1 = x_3) \vee (x_2 = x_3))$$

Exercise 2.2.11 *Extend this for $n=3$, $n=4$, ... arbitrary n . How fast does the formula grow?*

Exercise 2.2.12 *Express in the empty vocabulary, that the universe is infinite. With a finite FO theory? With an infinite FO theory? Using the answer for one of the previous questions?*

Exercise 2.2.13 *Given $Registered/2$ representing which students are registered for which courses, a predicate $CourseInRoom/2$ representing in which room a course is taught, and a function symbol $Capacity/1$: mapping rooms on their capacity. How would you express in FO that the number of students following a course has to be less than the capacity of the room of the course. Hint: If after trying a few minutes, you find out that it is more difficult than it looks, you can stop. It was proven that such constraints cannot be expressed in FO.*

Conclusion Constraints on the size of a set with a constant bound (i.e., a given natural number) can be expressed, but not in a convenient way. **Numerical constraints on the size of sets with a non-constant bound cannot be expressed in FO.**

Group theory

A *group* is a mathematical structure $\langle G, \odot, n \rangle$ consisting of a domain G (a set), a total binary operator $\odot : G \times G \mapsto G$ and an element $n \in G$ that satisfies four properties: the operator $\odot$ is internal in G , associative, has a neutral element n and each element has an inverse element. A well-known group is $\langle \mathbb{Z}, +, 0 \rangle$.

An appropriate vocabulary is $\Sigma = \{\text{op}/2, \text{e}/0\}$ with informal interpretation $\mathcal{I}$:

- $\mathcal{I}(\text{op})$ = the binary group operator $\odot$
- $\mathcal{I}(e)$ = the neutral element n

We build a theory T_{group} over this vocabulary Σ . No axioms are needed to express that *plus* represents a total binary function internal in the domain, since FO implicitly assumes this. The proposed theory is as follows:

$$\begin{aligned} \forall x \forall y \forall z (op(x, op(y, z)) &= op(op(x, y), z)) && \text{(associativity)} \\ \forall x (op(x, e) &= x \wedge op(e, x) = x) && \text{(neutral element)} \\ \forall x \exists y (op(x, y) &= e \wedge op(y, x) = e) && \text{(inverse element)} \end{aligned}$$

An online version of this theory written in IDP syntax is found in this footnote ³. It also shows that (finite) models of T_{group} can be computed by applying (finite) model generation and model expansion.

We now prove that this theory correctly expresses the concept of group. First, there is a one-to-one correspondence between Σ -structures $\mathfrak{A}$ and triplets $\langle G, \odot, n \rangle$ such that $D_{\mathfrak{A}} = G$, $op^{\mathfrak{A}} = \odot$, $e^{\mathfrak{A}} = n \in G$. Thus, the class of Σ -structures is suitable to represent the concept of group and reason about it.

Second, take a triplet $\langle G, \odot, n \rangle$ and the corresponding structure $\mathfrak{A}$. We can prove that $\langle G, \odot, n \rangle$ is a group iff $\mathfrak{A} \models T_{\text{group}}$. With a few applications of the rules of Definition 2.2.8, we reason as follows:

³ <https://idp.cs.kuleuven.be/idp/?present=group>

$\mathfrak{A} \models \forall x \forall y \forall z (op(x, op(y, z)) = op(op(x, y), z))$
 iff for all $d, e, f \in D_{\mathfrak{A}} : \mathfrak{A}[x : d, y : e, z : f] \models op(x, op(y, z)) = op(op(x, y), z)$
 iff for all $d, e, f \in G : d \odot (e \odot f) = (d \odot e) \odot f$
 iff $\odot$ is associative.

Similar arguments exist for the laws of neutral and inverse elements.

As such, **the theory T_{group} gets the models right.**

Graph colouring

A graph colouring of an undirected graph is a mapping from vertices to colours such that adjacent vertices have different colour. Typically, in a graph colouring computational problem, the input is the graph and a set of colours, and the output is a graph colouring of that graph. Perhaps the best known instance of this problem is the map colouring problem: colouring maps such that adjacent countries have different colours.

The proposed vocabulary is

$$\Sigma = \{\text{Vertex}/1, \text{Edge}/2, \text{Col}/1, \text{Colouring}/1:\}$$

with informal interpretation $\mathcal{I}$:

- $\mathcal{I}(\text{Vertex}(x)) = \text{“}x \text{ is a vertex”}$
- $\mathcal{I}(\text{Col}(x)) = \text{“}x \text{ is a colour”}$
- $\mathcal{I}(\text{Edge}(x, y)) = \text{“there is an edge from } x \text{ to } y\text{”}$
- $\mathcal{I}(\text{Colouring}(x)) = \text{“the colouring of (vertex) } x\text{”}$

A small problem is that the graph is undirected while the predicate symbol $\text{Edge}/2$ is interpreted as a directed binary relation. A standard way to resolve this by viewing an undirected graph as a symmetric relation (there is an edge from x to y iff there is an edge from y to x). Below the theory T_{gcol} ⁴ is given. The first axiom expresses symmetry of $\text{Edge}/2$. The other axioms express the essential properties of correct graph colourings.

$$\begin{aligned}
 &\forall x \forall y (\text{Edge}(x, y) \Rightarrow \text{Edge}(y, x)) \\
 &\forall x (\text{Vertex}(x) \Rightarrow \text{Col}(\text{Colouring}(x))) \\
 &\forall x \forall y (\text{Edge}(x, y) \Rightarrow \neg(\text{Colouring}(x) = \text{Colouring}(y)))
 \end{aligned}$$

Figure 2.1 is a graphical representation of a model $\mathfrak{A}$.

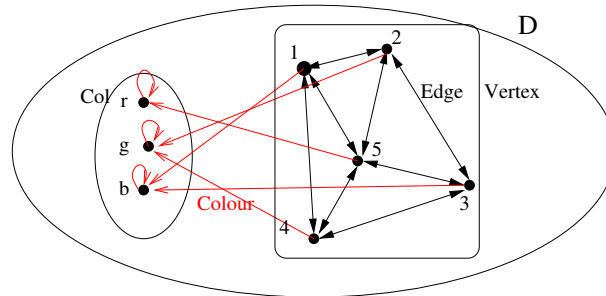


Figure 2.1 – A model of the graph colouring theory

The next question is how to use this theory for solving graph colouring problems. The input of such a problem is the graph and a set of colours, and the output is a graph colouring of

⁴ <https://idp.cs.kuleuven.be/idp/?present=grafekleur>

that graph. One way to solve it is through *model expansion*. This form of inference takes as input a theory T and an structure $\mathfrak{A}_{inp}$ interpreting part of the symbol of T , and returns as output one or more expansions $\mathfrak{A}$ of $\mathfrak{A}_{inp}$ that satisfy T . Here, the input is T_{gcol} and a structure $\mathfrak{A}_{inp}$ interpreting *Vertex*, *Edge* and *Col* with values corresponding to the graph and the collection of colours. Model expansion then computes models $\mathfrak{A}$ of T_{gcol} expanding $\mathfrak{A}_{inp}$. In a model $\mathfrak{A}$, the value $Colouring^{\mathfrak{A}}$ is a solution to the graph colouring problem.

Does this theory get the models right? Not entirely, as can be seen in the picture. There is a small semantic mismatch. As shown in the figure, colours are coloured in $\mathfrak{A}$, e.g., $Colouring^{\mathfrak{A}}(r) = r$ (the colouring of red is red). In the intended states of affairs, only vertices should be coloured. The reason is that in FO, functions symbols are assumed to represent total functions on the domain, which here entails that also colours have a colouring.

This semantic mismatch did not appear in the group theory of the previous section because the domain of group theory is the uniform set of group elements and the function *op* represents a total function on this domain. But in applications where the domain is heterogeneous and consists of objects of different types, as in the graph colouring case, functions are almost always partial. FO does not support partial functions. One solution is to use a predicate symbol to represent the partial function, and express that it is a partial function.

Exercise 2.2.14 Elaborate T_{gcol} using the predicate symbol *HasColour*/2. Express that it is a function from vertices to colours. Implement and try it out with IDP.

Exercise 2.2.15 Verify whether the following sentence is true in the structure of Figure 2.1.

$$\exists x(Vertex(x) \wedge \forall y(Edge(x, y) \Rightarrow (x = y)))$$

2.3 Pragmatics, implicatures and expressivity of FO

The term *pragmatics* refers to the practical methodology of modelling in a logic. Information comes to us expressed in natural language. Natural language is a complex language, extremely expressive, sometimes precise, sometimes vague, sometimes ambiguous, quite unsystematic in some aspects. So, the first step for the knowledge engineer is to read the text, and take care to understand it as it is intended. Then it must be translated in the modelling language. In the case of FO, this is not as simple as it seems. Far from it, to be honest. There are several reasons.

First, one reason has nothing to do with FO itself; it is the problem that much information is implicit. Such implicit information is called an **implicature**. The problem is that we are often not consciously aware of the implicatures. Another is that implicatures are communicated in subtle ways. Recall from Exercise 1.1.1 the implicature due the pronunciation of “I did not ride your bike” (that I rode someone else’s bike). The implicature may also depend on context, as shown in the next exercise.

Exercise 2.3.1 Consider two conversational scenarios:

Scenario A) Person 1 asks Person 2: “Tell me about your siblings.”

Person 2 answers: “I have a sister called Ann.”

Scenario B) Person 1 tells: “I like the name ‘Ann’.”

Person 2 responds: “I have a sister called Ann.”

The answer of Person 2 is the same in both scenarios, but the implicatures are different. Therefore, Person 2 returns different information in both scenarios. Explain the implicatures, think of a state of affairs where one is true and the other is false, and express in FO using $\Sigma = \{I/0:, Ann/0:, SisterOf/2, BrotherOf/2\}$.

One difficult aspect of modelling is to make the implicatures explicit. A few important forms of implicatures will be seen in this section.

Second, once the knowledge engineer found out what information is available, there is the problem of choosing Σ and $\mathcal{I}$. Typically, there are many choices, and they have a strong impact on the representation.

Third, the information need to be translated into FO. There is no algorithmic way to translate natural language text into an FO-style specification.

Exercise 2.3.2 Consider the following NL sentences:

- Jan and Mieke come to the meeting at 10am.
- Only Jan and Mieke come to the meeting at 10am.
- Jan and Mieke come only to the meeting at 10am.

The NL sentences have the same structure, but their expression in FO do not. Use $\Sigma = \{AttendsMeeting/2, Jan, Mieke, Meeting@10am\}$ with the obvious informal interpretation to express the statements in FO and compare their structure.

The translation in FO might be tricky or verbose due to the lack of expressivity of FO's language constructs. For example, the statement “*There are at most 4 attendants in the meeting*” can be expressed in FO, but not in a compact way, as seen in section 2.2.8.

Last but not least, it is possible that **the information cannot be expressed in FO**. We saw several examples, and more will follow.

In the next subsections, we see some frequent problems with expressing knowledge in FO.

2.3.1 Unary versus binary quantification

In FO, quantification $\forall, \exists$ is *unary*, i.e., globally over the entire universe. This works fine when the universe is uniform, i.e., when all objects are of the same type. This is the case in many mathematical applications, e.g., group theory, theory of the natural numbers. But in real applications, the universe is often heterogeneous. In such cases, we quantify over the entities of a specific class or type or set. E.g.,

“All lectures of this course take place in the first semester”

“Some lecture of this course takes place in the first semester”

Also in mathematical text, quantification is binary, as in “ $\forall x \in \mathbb{N}: x < x + 1$ ”.

To use FO in heterogeneous domains, binary quantification must be simulated using unary quantification:

- Universal binary quantification: “All Ps are Qs” is expressed as:

$$\forall x(P(x) \Rightarrow Q(x))$$

- Existential binary quantification: “Some P is a Q” is expressed as:

$$\exists x(P(x) \wedge Q(x))$$

Exercise 2.3.3 Prove that the negation of the binary quantification for $\forall$ is a binary quantification for $\exists$ and vice versa. In particular prove that $\neg(\forall x(P(x) \Rightarrow Q(x)))$ is logically equivalent to $\exists x(P(x) \wedge \neg Q(x))$. Hint: push negation inside.

There is a striking asymmetry between simulating binary universal and binary existential quantification. This probably explains why students make so many mistakes in expressing quantification. They use the method for simulating one binary quantifier to express the other. E.g., to express that some P is a Q, many write:

$$\exists x(P(x) \Rightarrow Q(x))$$

This is completely wrong.

Exercise 2.3.4 Explain the difference between:

- $\forall x(P(x) \Rightarrow Q(x))$ and $\forall x(P(x) \wedge Q(x))$.
- $\exists x(P(x) \wedge Q(x))$ and $\exists x(P(x) \Rightarrow Q(x))$.

For both cases, show with a structure that the two formulas are not equivalent. Hint: there is a structure with one domain element that shows the non-equivalence of both pairs of formulas. Hint: use IDP3 model expansion to generate that structure.

2.3.2 Common implicatures and their expression in FO

In this section, we see three common implicatures.

Unique names (UNA) and structured objects axioms In many problem domains, there is a given set of entities which need to be given a name to identify them. To avoid confusion, it is important to introduce exactly one name per entity. In a modelling context, the knowledge engineer introduces a collection $\tau \subseteq \Sigma$ of object symbols that serve as *names* or *identifiers* for these entities. This gives rise to an implicature: that for any pair $C_1, C_2 \in \tau$ of distinct symbols, it holds that:

$$\neg(C_1 = C_2)$$

There may be other object symbols in Σ that represent concepts, to which this implicature is not made and that should not be included in τ .

Example 2.3.1

The application area in this example is about the employees of a company XYZ, and the informal interpretation $\mathcal{I}(P)(x)$ is that x is an employee of XYZ. We use three object symbols *Albert*, *Bertrand* and *CEO*. In the informal interpretation as suggested by these three object symbols, there is a clean and clear difference: $\mathcal{I}(\textit{Albert})$ and $\mathcal{I}(\textit{Bertrand})$ will be specific and different entities in the actual state of affairs, while $\mathcal{I}(\textit{CEO})$ is a concept, that of CEO of XYZ.

Consider the following FO sentence ψ :

$$P(\textit{Albert}) \wedge P(\textit{Bertrand})$$

Since our subconscious is inclined to make the UNA implicature, it may expect that the company has at least two employees. However, ψ does not entail this, since it is satisfied by the **final** structure $\mathfrak{A}$ with $D_{\mathfrak{A}} = \{a\} = P^{\mathfrak{A}}$ and $\textit{Albert}^{\mathfrak{A}} = \textit{Bertrand}^{\mathfrak{A}} = a$

in which the UNA implicature is broken (see Example 2.2.10 for final structures). On the other hand, $\mathcal{I}(CEO)$ is a concept, and our subconscious is not likely to extend the UNA implicature to this object symbol. It is perfectly possible that the CEO of XYZ is Albert or Bertrand.

The unique name axiom for a set τ of object symbols, denoted $\text{UNA}(\tau)$, is the conjunction of all disequalities $\neg(C_1 = C_2)$ where C_1, C_2 are distinct symbols of τ .

An area where the unique name implicature is systematically used is that of databases. Imagine a table in a database that expresses the instructors of courses:

Instructor	
Ray	CS230
Hec	CS245
Mar	HD87

Typically, the unique name implicature is intended for all object symbols in this table.

Domain Closure (DCA) In the previous point, we considered the situation where the knowledge engineer was aware of a set of entities existing in the state of affairs, and has chosen a unique name for each of them resulting in the set τ of object symbols. Now it will often be the case that these entities are **the only entities** in the universe (that are considered in this modelling application). This is a frequent implicature. It is called **Domain Closure**.

Example 2.3.2: Continuation of Example 2.3.1

We are still in the setting of the company XYZ with its employees Albert and Bertrand and its unspecified CEO. Our theory T is

$$\begin{aligned} &\neg(\text{Albert} = \text{Bertrand}) \\ &P(\text{Albert}) \wedge P(\text{Bertrand}) \end{aligned}$$

Assume a (trivial) riddle with unique clue: “the CEO is not Albert”.

$$\neg(\text{CEO} = \text{Albert})$$

Who is the CEO? The evident solution is that it is Bertrand.

This is not entailed by the theory, as shown by the structure $\mathfrak{A}$ with $D_{\mathfrak{A}} = \{a, b, c\} = P^{\mathfrak{A}}$, $\text{Albert}^{\mathfrak{A}} = a$, $\text{Bertrand}^{\mathfrak{A}} = b$, and $\text{CEO}^{\mathfrak{A}} = c$. It satisfies our theory T and the clue, but Bertrand is not the CEO in the state of affairs represented by it.

The solution that Bertrand is the CEO is based on another implicature: that the universe in this application consists of only the two mentioned persons. This assumption is called the Domain Closure Assumption (DCA). This assumption is made explicit by the following sentence (which is not satisfied in $\mathfrak{A}$):

$$\forall x(x = \text{Albert} \vee x = \text{Bertrand})$$

After adding this sentence and the axiom expressing the clue to T , it is entailed that $\text{CEO} = \text{Bertrand}$.

Likewise, since T states that the employees Albert and Bertrand are elements of P , one might expect that T entails $\forall x P(x)$. Well, it does not (why not? show a small

structure that proves this). But after adding the domain closure axiom, the theory does entail $\forall x P(x)$.

Definition 2.3.1

Given a set $\tau = \{C_1, \dots, C_n\}$ of object symbols, $DCA_{FO}(\tau)$ is the sentence

$$\forall x (x = C_1 \vee \dots \vee x = C_n)$$

In many applications where the UNA implicature about τ holds, the DCA implicature holds as well.⁵ But there are also many applications where the UNA implicature about τ holds, but not the DCA implicature. This is the case when the Knowledge Engineer (KE) does not know the universe, i.e., considers it possible that some entities exist than are not named by any object symbol in τ .

Example 2.3.3

E.g., a one night Albert and Bertrand work late in the office. In the morning, Bertrand is found in the office, murdered! Who is the murderer? The murderer was in the office and it is not Bertrand. Was it Albert or was there another person that entered the office? The police investigator may not know.

In some applications, neither UNA or DCA hold. E.g, the theories of section 2.2.8 expressing groups and graph colourings are generic, and can serve to characterise all groups, respectively all graphs with graph colourings. **It is a great virtue of FO that it can express such generic theories.**

But now, assume that a concrete graph colouring problem arises (e.g., to compute a colouring of the map of Europe). In this setting, there is additional data specifying colours, vertices and the graph. Hence, a set τ of identifiers can be chosen for the vertices and the colours. In the new setting, UNA and DCA hold for τ . In FO, these axioms should be explicitly stated to get the models right.

Exhaustive enumerations Information (data) is often conveyed through tables, for example, in databases. Take a table expressing the lecturers of courses in a first bachelor.

Instructor	
Ray	CS230
Hec	CS245
Mar	HD87

What information is expressed by a table? What implicatures are in it?

Take the FO vocabulary Σ consisting of all symbols in the table.

$$\Sigma = \{Instructor/2, Ray/0 :, Hec/0 :, Mar/0 :, CS230/0 :, CS245/0 :, HD87/0 :\}$$

The first idea to express the information in the table might be the following sentence φ :

$$Instructor(Ray, CS230) \wedge Instructor(Hec, CS245) \wedge Instructor(Mar, HD87)$$

However, the table almost certainly intends to convey much more information than φ .

⁵The A in DCA stands for Assumption or Axiom. Depending on the situation, both make sense.

Exercise 2.3.5 Before reading on, think about the following questions. In what Σ -structures is the table satisfied according to your intuition? Do these structures match with the models of this formula φ ? Do you see implicatures that are not expressed by this formula?

One fairly obvious implicature missing in φ is the Unique Name Axiom for

$$\tau = \{Ray, Hec, Mar, CS230, CS245, HD87\}.$$

Indeed, these object symbols were clearly chosen as names for lecturers and courses.

The table also contains a second, extremely common implicature: the **exhaustive enumeration implicature**. The sentence φ expresses that three facts are true but says nothing about other instructor facts. A table, in most cases, contains the implicature that the contained facts are the only true ones; all others are false. Any instructor fact that is not in the table is false. The table is intended as an *exhausted enumeration* of true facts. This is missing in φ .

How to express that the three entries in the table of *Instructor* are the only true instructor facts? This can be done in a compact way using an equivalence:

$$\begin{aligned} \forall x \forall y (Instructor(x, y) \Leftrightarrow & (x = Ray \wedge y = CS230) \vee \\ & (x = Hec \wedge y = CS230) \vee \\ & (x = Mar \wedge y = HD87)) \end{aligned} \quad (2.1)$$

The left to right part of this equivalence expresses that if x is instructor of y then (x, y) occurs in the table. The right to left part expresses that each entry in the table occurs in the instructor relation.

This formula serves as a *definition* of *Instructor* in the sense that it determines a unique value for it.

2.3.3 Definitions of concepts in FO

In many modelling applications, we introduce auxiliary concepts that can be defined in terms of other concepts. The latter are called the **parameters** of the definition. E.g., the concept of a brother can be defined as a different male child of the same parents. The parameters are the concepts of male, and parent. Or the concept of a mother as a female parent. The parameters are the concepts of female and parent.

In FO, non-inductive definitions can be expressed using a sort of FO sentence called “**an explicit definition**”.

Definition 2.3.2: Explicit definition

An **explicit definition** of a predicate symbol P/n is a formula of the form:

$$\forall x_1 \dots \forall x_n (P(x_1, \dots, x_n) \Leftrightarrow \psi)$$

where $x_1, \dots, x_n$ are *distinct* variables and ψ is a formula not containing P/n . The free symbols of ψ different from $x_1, \dots, x_n$ are called the parameter symbols of this explicit definition.

Notice that the defined symbol P/n should not occur in ψ , which means that an inductive or recursive definition for P cannot be expressed this way.

Example 2.3.4

We express the definition of *Sibling*/2 using the parameters *Parent_of*/2:

$$\forall x \forall y (Sibling(x, y) \Leftrightarrow \neg(x = y) \wedge \exists z (Parent_of(z, x) \wedge Parent_of(z, y)))$$

Notice that also a half brother or sister is defined here as a sibling. (Refine this definition so that siblings share two parents.)

Example 2.3.5

We express the definition of *Passes*/2 in terms of *Grade*/3 and *PassingGrade*/1:

$$\forall x \forall y (Passes(x, y) \Leftrightarrow \exists g (Grade(x, y, g) \wedge PassingGrade(g)))$$

Example 2.3.6

The sentence Eq. 2.1 was an explicit definition of *Instructor*.

Even small deviations of the format of explicit definition results in completely wrong propositions as shown in the next exercise.

Example 2.3.7

We intend to define that a person has fever if its temperature is more than 37 and write:

$$\forall x \forall t (Fever(x) \Leftrightarrow Temperature(x, t) \wedge t > 37) \quad (2.2)$$

^a Check that this sentence is not an explicit definition (check the position of $\forall t$). The error seems small, but the sentence is completely wrong. The formula is equivalent with the conjunction of two material implications.

$$\forall x \forall t (Fever(x) \Leftarrow Temperature(x, t) \wedge t > 37)$$

$$\forall x \forall t (Fever(x) \Rightarrow Temperature(x, t) \wedge t > 37)$$

The first implication is correct, the second is completely wrong. Applying the laws of thought, we can move the quantification $\forall t$ inside to obtain:

$$\forall x (Fever(x) \Rightarrow \forall t (Temperature(x, t) \wedge t > 37))$$

Now the error is clear. It states that if *x* has fever, then all objects *t* of the universe are temperature of *x* and are strictly larger than 37. Nonsense!

Correct is the explicit definition:

$$\forall x (Fever(x) \Leftrightarrow \exists t (Temperature(x, t) \wedge t > 37))$$

^a <https://idp.cs.kuleuven.be/idp/?present=Fever>

Exercise 2.3.6 Given two sets *P* and *Q* of natural numbers, suppose that we want to

define the set R consisting of all numbers that are the sum of an element of P and an element of Q . We express it as follows:

$$\forall n \forall m (R(n + m) \Leftrightarrow P(n) \wedge Q(m))$$

Explain why this is not an explicit definition. Explain why this is completely wrong. Show this using a structure. You can use IDP for this, but it is better for you to solve it by thinking. Write up the correct explicit definition for R .

2.3.4 Material implication $\Rightarrow$

The natural language connectives corresponding to FO's binary connectives are all overloaded. The most difficult case is the conditional "If ... then ...". Linguistic studies showed the existence of many kind of conditionals in natural language. It is complicated. To avoid problems with ambiguity, Definition 2.2.8 defined:

$$\mathfrak{A} \models (\alpha \Rightarrow \beta) \text{ if } \mathfrak{A} \not\models \alpha \text{ or } \mathfrak{A} \models \beta \text{ (or both)}$$

rather than the potentially ambiguous:

$$\mathfrak{A} \models (\alpha \Rightarrow \beta) \text{ if it holds that if } \mathfrak{A} \models \alpha \text{ then } \mathfrak{A} \models \beta$$

It should be expected that in some applications, FO's material implication $\Rightarrow$ is not the right conditional. Below is an example. Compare the following NL propositions and their formalisation in FO:

- "If you succeed for all courses, then you get the degree."

We propose to formalise it as:

$$(\forall c \text{ Succ}(c)) \Rightarrow \text{Degree}$$

- "There is a course such that, if you succeed for it, you get the degree."

We propose to formalise it as:

$$\exists c (\text{Succ}(c) \Rightarrow \text{Degree})$$

Are the NL statements equivalent? No! E.g., the master computer science at our university satisfies the first but not the second NL statement: there is no course such that, if you succeed for it, you get the degree. Even the most important course, the master thesis, is not important enough to let students obtain the degree if they succeed for it and not for other courses.

Are the proposed formalisations logically equivalent? Yes! The following derivation is a correct proof using laws of thought:

$$\begin{aligned} & \exists c (\text{Succ}(c) \Rightarrow \text{Degree}) \\ & \equiv \exists c (\neg \text{Succ}(c) \vee \text{Degree}) \\ & \equiv (\exists c (\neg \text{Succ}(c))) \vee \text{Degree} \\ & \equiv (\neg \forall c \text{ Succ}(c)) \vee \text{Degree} \\ & \equiv (\forall c \text{ Succ}(c)) \Rightarrow \text{Degree} \end{aligned} \tag{2.3}$$

Given that the NL-propositions are clearly not equivalent, it follows that at least one of these FO sentences does not correctly formalise the corresponding NL-proposition. The problem is with $\exists c (\text{Succ}(c) \Rightarrow \text{Degree})$. This statement intends to say that there is a course such

that *in every state of affairs* where the student succeeds for the course, the student obtains the degree. The states of affairs that are quantified over, are the conceivable, legal situations at the university for a student making exams. The fact that the statement is false means that for every course, there is a conceivable, legal state of affairs where the student succeeds for that course but does not obtain the degree (e.g., the student fails for all other exams).

This implicit quantification over states of affairs cannot be expressed in FO but can be represented in modal logic. This paradox is similar to *Smullyan's drinkers paradox*⁶.

2.3.5 Conclusion

This section gave insight in common implicatures in modelling applications fields, and how to express them. Together with the previous section, it also showed a number of serious weaknesses of FO. This includes inconvenience (e.g., no types, unary quantification, size constraints, unique name axioms), semantic mismatches (e.g. partial functions) and lack of expressivity (domain closure, size constraints with variable bound, inductive definitions). In the next section, FO will be extended to overcome some of these weaknesses.

2.3.6 Exercises

Exercise 2.3.7 Has the word “and” the same meaning in the following sentences?

- “Earth is a planet and our sun is a star.”
- “I got up this morning and I brushed my teeth.”

Exercise 2.3.8 Has the word “or” the same meaning in the following sentences?

- “Bill or Monica will be home.”
- “You may choose a book or a CD.”

Exercise 2.3.9 Consider the following statement:

“There exists a number m such that $n < m$, and this for every natural number n .”

Notice the order of the quantifiers. Express this statement in FO or in FO-like natural language.

Exercise 2.3.10 What is meant with the following statement:

“For newyear, there will be a gift for every employee.”

Make the quantification explicit. Is the quantification ambiguous? Is it the same gift that is given to the employees?

Exercise 2.3.11 Quantifiers can be nested and implicit. E.g.,

“At least one car drives faster than every car that is conducted by a police man”

It contains three nested quantifiers. What is the order? Formalize this using

$$\Sigma = \{Car/1, Police/1, Conducts/2, DrivesFasterThan/2\}$$

⁶https://en.wikipedia.org/wiki/Drinker_paradox

Exercise 2.3.12 Compare the following statements:

- In front of the house is a Mercedes: $\exists x(\text{Mercedes}(x) \wedge \text{InFront}(x, \text{theHouse}))$
- A Mercedes is expensive: $\forall x(\text{Mercedes}(x) \Rightarrow \text{Expensive}(x))$

The NL word “a” can serve both as universal or as existential quantifier. It is not easy to explain.

Exercise 2.3.13 Natural language contains other binary quantifiers.

- No (passenger survived)
- Not all (passengers survived)
- At least n , at most n , exactly n (passengers survived)
- Most, few (passengers survived)

Translate the new quantifiers to FO. Do you see other quantifiers in NL? “Most” and “few” are vague quantifiers not expressible in FO. What is the meaning of “Most Ps are Qs”? This is not an easy question.

Exercise 2.3.14 Reconsider the statements of Example 1.1.1 with the role of the different stress in three statements:

- “I” did not drive your car.
- I did not drive “your” car.
- I did not drive your “car”.

Each statement has a different implicature. Choose a vocabulary and express in FO while making the implicatures explicit. For at least one pair of statements, give a structure showing the non-equivalence of them.

Exercise 2.3.15 This is a linguistic question. Suppose we the following statements:

- The king of France is bald.
- The king of France is not bald.
- It is not true that the king of France is bald.”

Given that France has no king, do you consider the sentences true or false? Or perhaps undefined?

Exercise 2.3.16 A functional dependency is defined as follows.

Definition 2.3.3

A relation R of n arguments contains a functional dependency $i \rightarrow j$ (with $1 \leq i \neq j \leq n$) if for each pair of tuples $\bar{d}, \bar{e} \in R$, if $d_i = e_i$ then $d_j = e_j$. Hence, the value at position i determines the value at position j .

(a) Express that the predicate symbol $P/3$ satisfies the following functional dependencies: $1 \rightarrow 2$ and $1 \rightarrow 3$. Write down one or more axioms such that, in each model $\mathfrak{A}$ of them, $P^{\mathfrak{A}}$ satisfies these functional dependencies.

(b) Given that $P/3$ has the functional dependencies $1 \rightarrow 2$, and $1 \rightarrow 3$, $P/3$ can be decomposed in two separate new predicates $Q/2$ and $R/2$ that are the projections of P on respectively the 1st and 2nd argument, and the 1st and 3rd. Define $Q/2$ and $R/2$ in terms of $P/3$ in FO. Express in FO that this decomposition is equivalence-preserving.

Exercise 2.3.17 The zoo has four monkeys, Sam, Anna, Harriet and Mike. Each one

eats a different fruit at its own favorite resting place. Sam, who doesn't eat bananas, likes sitting on the grass. The monkey who sat on the rock ate the apple. The monkey who ate the pear didn't sit on the tree branch Anna sat by the stream but she didn't eat the pear Harriet didn't sit on the tree branch. Mike doesn't eat oranges. Which ape ate what and where?

Express all information in FO. Think about the implicatures. Think about the needed kind of inference to solve the puzzle from your theory. You can use the partial IDP specification ^a.

^a <https://idp.cs.kuleuven.be/idp/?present=Zoo>

2.4 Extensions of Classical Logic

FO has been extended in many ways. Below we define a number of extensions of FO, mostly those that are supported by the IDP system.

2.4.1 Extending FO with types: FO(Types)

First-order Logic is an untyped language. In applications with different types of entities, types are to be represented as unary predicates. E.g., *Vertex/1* and *Colour/1* in the graph coloring application. By introducing types, the quality of modelling can be improved.

What is the nature of types, and the difference between types and ordinary unary predicates? E.g., which of the following unary predicate symbols (with obvious informal interpretation) represent types and which represent only unary relations: *Human/1*? *Adult/1*? *Employed/1*? *Colour/1*, *Fruit/1*? *Juicy/1*? My intuition says that the types are *Human/1*, *Colour/1* and *Fruit/1*. Does that match with yours?

In many applications, the universe is heterogeneous and consists of different kinds of entities. There are (almost) no concepts that range over all entities of the universe. Our brain seems to deal with this by partitioning the universe in different more homogeneous *categories* of entities. Each concept is applicable only to entities of certain categories. Applying concepts to objects of the wrong category is a *category clash*. E.g., the functional concept of shoesize is applicable only to humans, not to colours or fruits. To talk about the shoesize of an apple, or the colouring of a colour, is a category clash. On the other hand, the concepts behind *Adult/1*, *Employed/1* and *Juicy/1* do not seem to serve as the categories on which other concepts are defined.

Another -but related- way to distinguish types from unary relations is that in case of a unary relation, we can imagine states of affairs where some entity belongs to the relation, and other states of affairs where it does not belong to it. E.g., we can imagine a human individual as a child or as an adult, or as employed or unemployed. On the other hand, we cannot imagine a human individual as a colour or a fruit. There are almost no concepts that apply to each of these categories. Therefore, when imagining a human as a fruit, the human loses all its essential properties. These properties need to be replaced by a range of traits that make no sense for a human (e.g. taste, being ripe or not, etc.).

This motivates to add *types* in FO. Adding types improves the quality of the modelling. E.g., it removes a class of semantic mismatches between states of affairs and structures, such as the mismatch between graph colourings and structures where elements of type colour are coloured themselves. Adding types also allows to detect and eliminate a class of common modelling errors: type errors, that are formal category clashes.

A common terminology in classical logic is to call a type a *sort*. The simple form of typed logic in which sorts are disjunct, is called *many-sorted logic*. Below, we describe *multi-sorted FO* $\text{FO}(\text{Types})$, an extension of FO with a simple multi-sorted type system, based on the view of types as categories.

In multi-sorted FO, the universe is partitioned over a finite set Tb of base types. A function type is of the form $\mathbf{b}_1 \times \cdots \times \mathbf{b}_n \rightarrow \mathbf{b}_{n+1}$ ($n \geq 0$) where $\mathbf{b}_i$ are base types. A predicate type is of the form $\mathbf{b}_1 \times \cdots \times \mathbf{b}_n \rightarrow \mathbb{B}$ ($n \geq 0$). Such types are often called *signatures*.

A *vocabulary* Σ of multi-sorted FO consists of a set Tb of base type symbols, a set of function and predicate symbols, and a signature over Tb for each function and predicate symbol.

Structures assign domains to types and values of the right type to predicate and function symbols. Formally, a structure $\mathfrak{A}$ of vocabulary Σ consists of:

- per base type symbol $\mathbf{b} \in \Sigma$, a domain $\mathbf{b}^{\mathfrak{A}}$,
- per function symbol f of type $\mathbf{b}_1 \times \cdots \times \mathbf{b}_n \rightarrow \mathbf{b}$, a value $f^{\mathfrak{A}} : \mathbf{b}_1^{\mathfrak{A}} \times \cdots \times \mathbf{b}_n^{\mathfrak{A}} \rightarrow \mathbf{b}^{\mathfrak{A}}$,
- per predicate symbol p of type $\mathbf{b}_1 \times \cdots \times \mathbf{b}_n \rightarrow \mathbb{B}$, a value $p^{\mathfrak{A}} \subseteq \mathbf{b}_1^{\mathfrak{A}} \times \cdots \times \mathbf{b}_n^{\mathfrak{A}}$.

The many-sorted logic $\text{FO}(\text{Types})$ is defined as the following extension of FO:

Definition 2.4.1: Syntax

An expression is an $\text{FO}(\text{Types})$ expression if it is an FO-expression and it is *well-typed*: the type of a subexpression matches the type of the argument position in which it occurs.

Definition 2.4.2: Semantics, extension of Definition 2.2.8

The definition of term evaluation and satisfaction is the same as in FO, but for two modifications in the inductive rules for the quantifiers. Below, we assume that variable x has type $\mathbf{b}$.

- $\mathfrak{A} \models \forall x \varphi$ if for all $d \in \mathbf{b}^{\mathfrak{A}}$, $\mathfrak{A}[x : d] \models \varphi$.
- $\mathfrak{A} \models \exists x \varphi$ if for some $d \in \mathbf{b}^{\mathfrak{A}}$, $\mathfrak{A}[x : d] \models \varphi$.

An extension of multi-sorted logic is *order-sorted logic* where sorts can have subsorts. The type system of the logic $\text{FO}(\cdot)$ supported by IDP is a simple order-sorted type system. It is illustrated with an example. For more information, see the IDP manual.

Example 2.4.1

The IDP system supports order-sorted logic. We illustrate it in the context of graph-colouring:

```

vocabulary V {
  type Vertex
  type Col
  type BrightCol isa Col
  G(Vertex,Vertex)
  Colouring(Vertex): Col
}
theory T: V {
  !x[Vertex] y[Vertex] : (G(x,y) => Colouring(x) ~= Colouring(y))
.

```

```

    !u v : (G(u,v) => Colouring(u) ~= Colouring(v)).
    ! u: G(u,u) => BrightCol(Colouring(u)).
    !x y : (G(x,y) => Colouring(x) = y).
  }

```

We make the following observations:

- Using the `isa` statement, a subtype of `Col` is defined.
- The first axiom `!x[Vertex] y[Vertex]: (G(x,y) => Colour(x) ~= Colour(y))` has explicitly typed variables `x`, `y`.
- In the second axiom (which is a copy of the first), no explicit types for `u`, `v` are given, but type inference infers that `u`, `v` have type `Vertex`.
- The third axiom states that reflexive nodes are brightly coloured. The type `BrightCol` is used as a unary predicate.
- For the fourth axiom, type inference derives a type error on the variable `y` since it occurs on arguments positions of type `Vertex` and `Col`.

In the context of a typed logic, implicatures such as the unique name axiom (UNA) and the domain closure axiom (DCA) need to be expressed on the level of types, not the entire domain. E.g., in some setting of graph colouring, we may know all entities of type colour but not the entities of type vertex.

2.4.2 Extending FO with arithmetic: FO(Types,Arit)

FO(Types) forms a good basis for adding interpreted types. FO(Types,Arit) is obtained by adding the integer numbers as a type, and adding operators over it as interpreted symbols:

- type symbols *int* and *nat*
- numerals: $\dots -2, -1, -, 1, 2, \dots$ (cfr. Chapter 1)
- arithmetic operators $+, \times, ^, \leq, \geq, \dots$

Their values in every structure $\mathfrak{A}$ are the natural ones.

- E.g., the value $12^{\mathfrak{A}}$ of numeral `12` in $\mathfrak{A}$ is the number 12.
- E.g., the value $<^{\mathfrak{A}}$ of symbol `<` in $\mathfrak{A}$ is the standard strict order relation $<$ on integers, that is, $\{(n, m) \in \mathbb{Z}^2 \mid n < m\}$.

In FO(Types,Arit), integer arithmetic can be used in theories. E.g., Fermat's last theorem is formalised as follows:

$$\neg \exists x[nat] \exists y[nat] \exists z[nat] \exists n[nat] (n > 2 \wedge x^n + y^n = z^n)$$

Fermat's theorem is easy to express, but difficult to prove. It was proven to be true, but it took humanity 300 years.

Reasoning in a logic with integer arithmetic is computationally complex. Models -if they exist- are infinite. In full generality, many inference problems, such as deduction and satisfiability checking, become uncomputable in the sense of being neither recognizable nor co-recognizable. See Section 6.3.

Arithmetic in IDP3 The logic FO($\cdot$) supported by the IDP3 system is a conservative extension of FO and shares all incomputability results for general forms of inference of FO. However, IDP3 itself is a finite domain reasoner. IDP3 implements several forms of inference, but most of them take an input that restricts classes of structures to a fixed finite domain.

As for arithmetic, IDP mainly supports (mainly) finitely bounded arithmetic. Every numerical symbol has to be declared to be an element of a subtype of integers. This subtype is described as a finite subset in an input structure. An example is as follows.

Example 2.4.2

```

vocabulary V {
  type someIntegers isa int
  C1: someIntegers
  C2: someIntegers
}
theory T: V {
  C1 + C1 = C2.
}
structure S: V {
  someIntegers = {1..5}
}

```

The vocabulary V specifies that *someIntegers* is a subtype of the integers, without saying more about it. But in the (partial) structure S , the value of this subtype is set to $\{1, 2, 3, 4, 5\}$.

Most inference procedures of IDP3 take as input not only a theory, but also a structure with a finite domain. The system then reasons about expansions of this finite structure that satisfy the theory. In the above case, the structure S has a finite domain and it interprets only a subset of the symbols in V , in particular *someInteger* but not $C1, C2$. Inference calls with this input will typically reason about V -structures (that do interpret $C1, C2$) that satisfy the theory and expand S , hence, have the same finite domain as S .

2.4.3 Second and higher order logic

This extension is not supported by IDP.

In FO, quantification is over elements of the domain. Second-order logic (SO) is an extension of FO in which quantification over predicates (of some signature $\mathbf{b}_1 \times \dots \times \mathbf{b}_n \rightarrow \mathbb{B}$) and functions (of some signature $\mathbf{b}_1 \times \dots \times \mathbf{b}_n \rightarrow \mathbf{b}$) is allowed.

The extension of syntax and semantics of FO to SO is straightforward. For syntax, we add a new rule to the definition of expression:

Definition 2.4.3

- ...
- If φ is a formula, and P an n -ary function or predicate symbol, then $(\forall P \varphi)$ and $(\exists P \varphi)$ are formulas.

As for semantics, we extend Definition 2.2.8:

Definition 2.4.4

- ...
- $\mathfrak{A} \models \exists P \varphi$ if P is a predicate symbol with signature $\mathbf{b}_1 \times \dots \times \mathbf{b}_n \rightarrow \mathbb{B}$ and for some $X \subseteq \mathbf{b}_1^{\mathfrak{A}} \times \dots \times \mathbf{b}_n^{\mathfrak{A}}$ it holds that $\mathfrak{A}[P : X] \models \varphi$.

- ... (similar rules for function variables and for universal quantifier.)

Example 2.4.3

We express in SO, using *Boy*/1, *Girl*/1 and *DancesWith*/2 that there is a group of boys and girls that went dancing, and each of the boys danced with each of the girls.

$$\exists P(\forall x(P(x) \wedge \text{Boy}(x) \Rightarrow \forall y(P(y) \wedge \text{Girl}(y) \Rightarrow \text{DancesWith}(x, y))))$$

Example 2.4.4

Let *Parent*(*x*, *y*) denote that *x* is a parent of *y*. In First-order Logic, we can't express the property that A(lbert) is an ancestor of B(ert). We can express this in second-order logic (although not conveniently). We can say that every set of people containing B and closed under the Parent relation, contains A. A set *P* is closed under the Parent relation if for every *x* in *P*, the parents of *x* belong to *P*.

$$\forall P((P(B) \wedge \forall x \forall y((P(y) \wedge \text{Parent}(x, y)) \Rightarrow P(x)) \Rightarrow P(A))$$

Explain to yourself why this is correct. Hint: prove that the set of Bertrand and his ancestors is the smallest set of people containing Bertrand and closed under the parent relation.

Exercise 2.4.1 Given $\Sigma = \{0 : U, S : (U \rightarrow U)\}$ of an object symbol *0* and a unary function *S*, express that the entire universe is a subset of every set that contains *0* and is closed under application of *S*. (Which of course entails that the universe is the only such set.)

Exercise 2.4.2 Dedekind's definition of an infinite set is : a set *S* is infinite iff there exists a strict subset *A* of *S* and a bijection *b* between *S* and *A*. Use Dedekind's idea to express that the universe is infinite in SO (Second order logic). The solution is a straightforward translation of Dedekind's condition for the case of the universe.

Higher order logic In FO and SO, predicates and functions are sets consisting of (tuples of) atomic elements of the universe. In Higher Order logic (HO), the elements in predicates and functions do not need to be atomic, but can be sets (or relations or functions), or even sets (or relations or functions) of the latter, and so on. So, HO possesses relations of a vastly more complex nature than FO and SO. We will not define this logic here.

The expressivity of SO is (much) higher than of FO. That of HO is (much) higher than of SO. Later in this course, we will see more properties that can be expressed in SO but not in FO.

2.4.4 Extending FO with identifiers and domain/type closure

In many applications, many object symbols serve as *identifiers*: they serve as unique names for certain entities known to exist in the state of affairs. The theory of unique names axioms $\text{UNA}(\tau)$ holds for the collection τ of them. Often some form of Domain Closure Axiom $\text{DCA}(\tau)$ holds for them. Other objects symbols serve to represent concepts, and UNA or DCA should not be applied to them.

The logic $FO(\cdot)$ supported by the IDP system supports identifiers. It offers a construct that combines *UNA* and *DCA* in one declaration: the *constructed type*. An example is as follows.

```
vocabulary V {
type day constructed from { mon, tue, wed, thu, fri, sat, sun }
myFreeDay: day
type dim isa int
type position constructed from { posit(dim,dim) }
}
```

The object symbols `mon`, ..., `sun` are clearly identifiers. They serve as unique names for certain days. This declaration induces $UNA(\{\text{mon}, \text{tue}, \text{wed}, \text{thu}, \text{fri}, \text{sat}, \text{sun}\})$ and $DCA(\{\text{mon}, \text{tue}, \text{wed}, \text{thu}, \text{fri}, \text{sat}, \text{sun}\})$ for the type `day`. On the other hand, the object symbol `myFreeDay` is not an identifier but $\mathcal{I}(\text{myFreeDay})$ is a concept, the concept of my free day (I work part time). Its value in a structure that satisfies the constructed-from declaration is the same as the value of one of `mon`, ..., `sun`.

The type `position` denotes the set of positions on a `dim` $\times$ `dim` board with constructor `posit` that maps two elements of `dim` to the corresponding position on the board. This also induces a *UNA* and *DCA* axiom. Notice that the values of `position` and `posit` depend on the value of `dim`.

Definition 2.4.5: Semantics

A structure $\mathfrak{A}$ satisfies a declaration

`type b constructed from  $\{C_1, \dots, C_n\}$`

if there exists a one-to-one mapping $h : \{C_1, \dots, C_n\} \rightarrow \mathfrak{b}^{\mathfrak{A}}$ from the set of object symbols to the value $\mathfrak{b}^{\mathfrak{A}}$ of the type, such that $C_i^{\mathfrak{A}} = h(C_i)$.

The following proposition links up such type declarations with *UNA* and *DCA*.

Proposition 2.4.1

A structure $\mathfrak{A}$ satisfies the above constructed type declaration iff it satisfies $UNA(\{C_1, \dots, C_n\})$ and the type closure axiom $DCA_{FO}(\{C_1, \dots, C_n\})$.

Exercise 2.4.3 *Prove this proposition.*

2.4.5 Extending FO with aggregates: $FO(\text{Types}, \text{Arit}, \text{Agg})$

We have seen that in *FO* it is impossible to express propositions about cardinalities of sets of objects that satisfy certain conditions. There are many situations where this would be very useful. E.g., assume we want to express that the room in which a course is taught must be larger than the number of students enrolled in the course. It would be useful to express:

$$\forall c \forall r (Course(c) \wedge Room(c, r) \Rightarrow \#\{y : Enrolled(y, c)\} \leq Capac(r))$$

This sentence cannot be expressed in *FO*.

Here $\#$ is the cardinality aggregate symbol and represents the (partial) function from sets to the number of elements of the set. The symbol takes as argument a set expression $\{(x_1, \dots, x_n) : \varphi\}$. Useful aggregate expressions are:

- $\#\{(x_1, \dots, x_n) : \varphi\}$: represents the number of elements.
- $Min\{x : \varphi\}$ represents the least number of a set of integers.
- $Max\{x : \varphi\}$ represents the largest element of a set of integers.

- $Sum\{(x, x_1, \dots, x_n) : \varphi\}$ represents the sum of the first arguments of these tuples:

$$Sum\{(x, x_1, \dots, x_n) : \varphi\} = \sum_{\{(x, x_1, \dots, x_n) : \varphi\}} x$$

In a set expression, $\{(x_1, \dots, x_n) : \varphi\}$, the $x_1, \dots, x_n$ are different fresh variables whose scope is exactly the set expression.

Notice that aggregate expressions are numeric expressions; therefore, the extension with aggregates is of the logic $FO(\text{Types}, \text{Arit})$.

We extend the inductive Definition 2.2.2 of term and formula of FO with new inductive rules and a new sort of expression.

Definition 2.4.6: Syntax

Extend Definition 2.2.2 of *term* and *formula* of FO with the following inductive rules:

- For $\bar{x}$ a tuple distinct object symbols and φ a formula, the expression $\{\bar{x} : \varphi\}$ is a *set expression* (also called a *set comprehension*). Symbols $x_1, \dots, x_n$ in $\bar{x}$ are distinct variables with scope $\{\bar{x} : \varphi\}$. All occurrences of them in this expression are *bound*.
- If s is a set expression and Agg is an aggregate symbol (e.g., $\#, Sum, Min, \dots$) then $Agg(s)$ is a term (called an aggregate term).

Remark 2.4.1

The original Definition 2.2.2 was an example of a *stratified* definition: it defines the concept of *term* and, on top of that, the concept of *formula* in terms of the parameter concept of *term*. E.g., when defining an atomic formula as $P(t_1, \dots, t_n)$ where $t_1, \dots, t_n$ are terms and P/n a predicate symbol. On the other hand, the extended Definition 2.4.6 defines *term* in terms of *formula* as in the set term $\{x : Q(x)\}$. Therefore, the stratification is broken, and the extended definition is a clear example of a *definition by mutual or simultaneous induction*.

The following example serves to illustrate the scope of a variable in a set expression.

Example 2.4.5

Can you figure out the meaning of the following sentence? And whether it is true or not?

$$\exists x(x = 5 \wedge \#(\{x : x = 4\}) = 1)$$

It all depends on whether the x outside the set expression is the same as the x inside the set expression. If it is, then the value of the set expression (in any structure) is $\emptyset$ since $5 \neq 4$ and the sentence is false. If the x outside and inside the set expression are occurrences of distinct variables, then the value of the set expression is $\{4\}$, and the sentence is true. The correct answer is the second. Indeed, the scope of x in the set expression is limited to the set expression.

To extend the evaluation function and satisfaction relation $\models$, we assume that each aggregate function Agg has a value $Agg^{\mathfrak{A}}$ in structure $\mathfrak{A}$. E.g., $\#^{\mathfrak{A}}$ is the function that maps sets of (tuples of) domain elements to the number of its elements. Then we add two inductive rules to Definition 2.2.8 to define the interpretation of a set expression and an aggregate expression in a structure,

Definition 2.4.7: Semantics

Extend Definition 2.2.8 of satisfaction $\models$ of FO with the following inductive rules:

- $\{\bar{x} : \varphi\}^{\mathfrak{A}} = \{\bar{d} \in D_{\mathfrak{A}}^n \mid \mathfrak{A}[\bar{x} : \bar{d}] \models \varphi\}$
- $Agg(s)^{\mathfrak{A}} = Agg^{\mathfrak{A}}(s^{\mathfrak{A}})$

Thus, the value of a set expression is a set of tuples of domain elements, and the value of an aggregate expression is the aggregate function applied to this set. That is the main idea to obtain a well-defined syntax, satisfaction relation, informal semantics and hence, a well-defined modelling logic.

Example 2.4.6

We express that the study load of a student should be less than 65 study points using the sum aggregate:

$$\forall s (Sum\{(l, c) : SelCourse(s, c) \wedge StudyPoints(c, l)\} \leq 65)$$

In words: the sum of the first arguments of all tuples in the set of tuples (l, c) such that l is the number of study points of a course followed by a student s should be less than 65, for each student s .

We sum only the first argument of tuples in the set expression; why then do we need the other arguments? This is to be able to sum up multiple occurrences of the same number. This is explained in the next example.

Example 2.4.7

Compare the following two propositions:

$$\forall s (Sum\{(\ell, c) : SelCourse(s, c) \wedge StudyPoints(c, \ell)\} \leq 65) \quad (2.4)$$

$$\forall s (Sum\{\ell : \exists c (SelCourse(s, c) \wedge StudyPoints(c, \ell))\} \leq 65) \quad (2.5)$$

In the first, the set expression denotes a set of pairs of study loads and courses; in the second, the set expression denotes a set of study loads. Is this equivalent? Certainly not.

Imagine that a student *Bob* follows 10 courses $c_1, \dots, c_{10}$ of 8 study points each. His total load is 80, far above the threshold. Now compare the two sets included in these expressions, for $s = Bob$: the first one is of the form $\{(8, c_1), \dots, (8, c_{10})\}$ and has sum 80, while the second one is of the form $\{8\}$ and has sum 8. Clearly, the correct expression is the first one. Indeed, by adding more arguments, we can distinguish between different occurrences of the same study load for different courses.

Remark 2.4.2

In IDP, aggregates are represented in a slightly different syntax, as illustrated here:

- Number of elements of P
 $\#\{x, y : P(x, y)\}.$
- Sum of all pairs of elements x and y in P

$$\text{sum}\{x, y : P(x, y) : x+y\}.$$

This corresponds to the expression $\text{Sum}\{(z, x, y) : P(x, y) \wedge z = x + y\}$.

– Minimum of the elements x in both Q and R :

$$\min\{x : Q(x) \& R(x) : x\}.$$

– Maximum of the elements x in both Q and R :

$$\max\{x : Q(x) \& R(x) : x\}.$$

– Nesting is allowed, as in:

$$\text{sum}\{x[\text{num}] : x = \#\{y : Q(x, y)\} : x\}.$$

Exercise 2.4.4 ^a For the above aggregate, IDP is capable to perform computations with different input/output. Try this.

- Compute the value of aggregate expressions from values for P, R .
- Compute values of P, R from the value of aggregates expressions.
- Compute a minimal structure satisfying the aggregates expressions.

^a <https://idp.cs.kuleuven.be/idp/?present=Agg>

2.4.6 Extending FO with partial functions

In heterogeneous domains, almost all functions are partial. The fact that FO assumes that all functions are total creates a semantic mismatch problem between the models and the state of affairs that they serve to abstract. This could be seen in the models of the graph colouring theory in section 2.2.8, where also colours need to be coloured. That problem could be solved by introducing types *Vertex* and *Colours*, and typing *Colouring* : *Vertex* → *Colour*. Indeed, in the standard graph colouring problem, all and only vertices have a colour. But often the domain of a functional concept is not a type. E.g., the function mapping a person to its spouse or to its shothesize. Not every person has a spouse, not everybody has feet. The domains of these functions are not natural types (unless every set is a type). It is useful to add partial functions to the logic.

One simple way to extend FO with partial functions is the so-called *relational way*. When F/n is declared to be a partial function, then any atomic formula

$$P(\dots F(t_1, \dots, t_n) \dots)$$

is interpreted as:

$$\exists y (F(t_1, \dots, t_n) = y \wedge P(\dots y \dots))$$

Here, $F(t_1, \dots, t_n) = y$ is false for every y in case $F(t_1, \dots, t_n)$ is undefined.

Example 2.4.8

An example dating from Bertrand Russell is as follows. Consider the NL proposition:

The king of France is bald.

Given that in the current state of affairs, France has no king, the question is what is the meaning of this sentence? Is it true? There are different views on this. The relational way is one of them. Consider $\Sigma = \{KoF/0 :, Bald/1\}$ under the obvious

informal interpretation. Take the FO proposition:

$$\text{Bald}(\text{KoF})$$

Interpreted in the relational way, it reads as:

$$\exists y(\text{KoF} = y \wedge \text{Bald}(y))$$

This states: there exist x in the universe that is Bald and is the king of France. This is definitely false. Thus, the relational way is one consistent view of handling partial functions.

An extension of this approach has been implemented in IDP3, as described in [10]. The relational way is applied to all occurrences of partial functions terms in $\text{FO}(\cdot)$ formulas. This includes rule bodies of formal definitions.

The one exception is in for partial functions terms in the head of definitional rules. There another interpretation is used. A rule

$$\forall \dots (P(\dots F(t_1, \dots, t_n) \dots) \leftarrow \dots)$$

is interpreted as:

$$\forall \dots \forall y (P(\dots y \dots) \leftarrow F(t_1, \dots, t_n) = y \wedge \dots)$$

Example 2.4.9

Assume we e

In IDP3 syntax, a partial function F/n is declared as:

$$\text{partial } F(T_1, \dots, T_n) : T$$

For a small example in IDP3, see, e.g., ⁷.

2.4.7 Extending FO with definitions: FO(ID)

Definitions are an important form of knowledge in every domain of human expertise. E.g., in the context of Newton's two-body theory, base concepts are the mass m_i and the trajectory $\vec{r}_i$ of body b_i . On top of these concepts, one *defines* the velocity, the acceleration, the impulse, the kinetic energy, ..., of b_i .

Definition 2.4.8

A formal expression Δ of a modelling logic $\mathcal{L}$ **defines** a set Σ_d of defined symbols in terms of a (disjoint) set Σ_p of parameter symbols if (1) any symbol occurring free in Δ belongs to $\Sigma_d \cup \Sigma_p$, and (2) each Σ_p -structure $\mathfrak{A}_p$ has a unique $\Sigma_d \cup \Sigma_p$ -structure $\mathfrak{A}$ expanding $\mathfrak{A}_p$ that satisfies Δ .

In words, Δ specifies for any value of the parameters a unique value of the defined symbols. This seems to capture the essence of what a definition should do.

In FO, some definitions can be expressed through *explicit definitions* as defined in Definition 2.3.2. They are formulas of the form:

$$\forall x_1 \dots \forall x_n (P(x_1, \dots, x_n) \Leftrightarrow \varphi)$$

⁷ <https://idp.cs.kuleuven.be/idp/?present=PartialFun>

where P/n does not occur in φ . The set of parameters of an explicit definition was defined to be the set Σ_p of symbols that occur free in φ and are different from $x_1, \dots, x_n$.

Proposition 2.4.2

An explicit definition defines $\{P/n\}$ in terms of Σ_p . In particular, for any structure $\mathfrak{A}_p$ interpreting Σ_p , the unique model of the explicit definition that expands $\mathfrak{A}_p$ is $\mathfrak{A}_p[P : \{(x_1, \dots, x_n) : \varphi\}^{\mathfrak{A}_p}]$.

Exercise 2.4.5 Prove this proposition.

E.g., we can express the definition of the sibling concept in terms of the parent concept by the following explicit definition:

$$\forall x \forall y (Sibling(x, y) \Leftrightarrow \neg(x = y) \wedge \exists z (Parent_of(z, x) \wedge Parent_of(z, y)))$$

So far, these course notes contain around 25 mathematical definitions, most of them non-inductive. A few of them are inductive. One is the inductive Definition 2.2.2 of term (and formula):

- An object symbol is a term.
- The string $f(t_1, \dots, t_n)$ is a term if f is a function symbol of arity n , and $t_1, \dots, t_n$ are terms.

Another is the definition of the satisfaction relation by structural induction. In the propositional case:

- $\mathfrak{A} \models P$ if $P \in \mathfrak{A}$;
- $\mathfrak{A} \models \alpha \wedge \beta$ if $\mathfrak{A} \models \alpha$ and $\mathfrak{A} \models \beta$;
- $\mathfrak{A} \models \alpha \vee \beta$ if $\mathfrak{A} \models \alpha$ or $\mathfrak{A} \models \beta$ (or both);
- $\mathfrak{A} \models \neg \alpha$ if $\mathfrak{A} \not\models \alpha$ (i.e., it is not the case that $\mathfrak{A} \models \alpha$).

Both are syntactically represented as a collection of rules. Some are non-inductive rules, some are inductive. The latter can be partitioned in monotone and non-monotone rules. A monotone rule uses elements of the defined concept to construct a new element. A non-monotone rule uses a non-element of the defined concept to construct a new element. The one non-monotone rule here is the last one, defining $\mathfrak{A} \models \neg \alpha$ if $\mathfrak{A} \not\models \alpha$.

Semantically, there is a strong resemblance between these definitions, as explained in Remark 2.2.1 and Remark 2.2.2. They both define the defined concept (*term*, respectively $\models$) by describing how to *construct* the value of the defined concept from the value of the parameters. The construction process is called the *induction process*. This process starts by initially assuming that the defined set is the empty set, and then proceeds by iterated bottom up rule application until all rules are satisfied. In case of a non-monotone rule, it is essential to delay its application until it is *safe* to do so: until its condition cannot be made false anymore later in the induction process. This is why non-monotone definitions in mathematics are specified together with an induction order $\prec_i$. The rules need to be applied *along the induction order* $\prec_i$. This means that rules deriving smaller elements in this order, need to be applied before rules deriving larger elements. The induction order $\prec_i$ is carefully chosen such that the resulting induction process is safe. E.g., in the definition of $\models$, the order $\prec_i$ is the structural relation between formulas and larger formulas that contain the first. By following this order, rules deriving satisfaction of component formulas of $\neg \alpha$ will be applied before the rule deriving $\mathfrak{A} \models \neg \alpha$. As a consequence, the induction process is safe: it is impossible to derive $\mathfrak{A} \models \neg \alpha$ and in a later stage to derive that $\mathfrak{A} \models \alpha$.

Syntax of FO(ID) This logic is an extension of FO in which a theory consists of FO axioms and FO(ID) definitions. An FO(ID) definition is a collection of definitional rules.

Definition 2.4.9: Syntax of FO(ID) definitions

A definition Δ is a *set of definitional rules*:

$$\forall \bar{x}(P(\bar{t}) \leftarrow \varphi)$$

where

- $\bar{x}$ is a sequence of distinct variable object symbols $x_1 \dots x_m$,
- $\bar{t}$ is a tuple of n terms, with n the arity of P ,
- $P(\bar{t})$ is an atomic formula, called the head of the rule,
- φ is a FO formula, called the body of the rule,
- the free variables of $\bar{t}$ and φ belong to $\bar{x}$.

A predicate symbol P in the head $P(\bar{t})$ of a rule is called a *defined predicate* of Δ . Any other non-logical symbol that occurs free in Δ is called a *parameter symbol* of Δ . The symbol $\leftarrow$ is called the *definitional implication*. A definition is called *inductive* or *recursive* if one of its defined predicates occurs in the body of a rule. A definition is non-monotone if it contains a non-monotone rule. A rule $\forall \bar{x}(P(\bar{t}) \leftarrow \varphi)$ is non-monotone if a defined predicate symbol occurs inside a negated subformula $\neg\psi$ within the rule body φ .

The definitional implication is a conditional, but a different one than the (reversed) material implication.

Definition 2.4.10: Syntax of FO(ID)

An FO(ID) theory T is a set of FO sentences and FO(ID) definitions.

Before considering the semantics, a few examples serve to illustrate syntax and informal semantics of FO(ID) definitions and theories.

Remark 2.4.3

The following rules of syntax are important:

- A definitional rule never contains $\neg$ in the head. The head is always an atomic formula of a defined predicate.
- An FO(ID) definition never contains FO formulas. It contains only definitional rules.
- An FO(ID) theory never contains definitional rules. It contains only FO formulas and FO(ID) definitions (which are sets of definitional rules).

Exercise 2.4.6 Is something wrong with the following FO(ID) theory?

$$\begin{aligned} \forall x \forall y (G(x, y) \Rightarrow G(y, x)) \\ \forall x \forall y \forall z (G(x, y) \leftarrow P(x, z) \wedge Q(y, z)) \end{aligned}$$

Exercise 2.4.7 Is something wrong with the following definitional rule:

$$\forall x (\neg \text{Healthy}(x) \leftarrow \text{Sick}(x))$$

Exercise 2.4.8 *Is something wrong with the following definition?*

$$\left\{ \begin{array}{l} \forall x \forall y (R(x, y) \leftarrow G(x, y)) \\ \forall x \forall y \forall z (R(x, z) \Leftarrow R(x, y) \wedge R(y, z)) \end{array} \right\}$$

Formal semantics A definition Δ will be satisfied in a structure $\mathfrak{A}$ if the value of a defined predicate symbol is the value constructed by the induction process of Δ in $\mathfrak{A}$.

To run an induction process for definition Δ in a structure $\mathfrak{A}$, $\mathfrak{A}$ should interpret at least all parameter symbols of Δ . If P/n is the unique defined predicate symbol of Δ , an induction process of Δ in $\mathfrak{A}$ is a (potentially infinite) sequence:

$$P_0 \rightarrow P_1 \rightarrow \dots \rightarrow P_i \rightarrow P_{i+1} \rightarrow \dots \rightarrow P_\alpha$$

consisting of n -ary relations P_i in the domain of Δ (i.e., $P_i \subseteq D_{\mathfrak{A}}^n$), such that

- $P_0 = \emptyset$;
- P_{i+1} is obtained from P_i by applying one or more **safely applicable** rule instances of Δ in $\mathfrak{A}[P : P_i]$, the structure that extends $\mathfrak{A}$ by interpreting symbol P as the relation P_i .
- P_α is saturated: it is closed under application of rules of Δ . That is, if the condition of a rule instance of Δ is satisfied in $\mathfrak{A}[P : P_\alpha]$, then also its head is satisfied in $\mathfrak{A}[P : P_\alpha]$.

The above description is not entirely complete, since it is not explained how to determine when a rule instance is *safely applicable*. This can be formally defined using three-valued logic [11, 12, 13]. It would lead too far for this course to include this here. Below, an explanation by example is given of this concept.

Definition 2.4.11: Satisfaction of an FO(ID) definition in a structure

A structure $\mathfrak{A}$ interpreting all free (defined and parameter) symbols of an FO(ID) definition Δ , satisfies Δ (notation $\mathfrak{A} \models \Delta$) if, for all defined predicates P/n of Δ , $P^{\mathfrak{A}}$ is the limit of the sequence of values of P/n in a safe induction process of Δ in $\mathfrak{A}$.

This definition is not mathematically complete, since the concept of safe induction process of Δ in $\mathfrak{A}$ is missing. It is out of the scope of this course to define this concept here, since it relies on three- and four-valued logic.⁸

Definition 2.4.12: Semantics of FO(ID)

A structure $\mathfrak{A}$ satisfies a FO(ID) theory T (notation $\mathfrak{A} \models T$) if

1. $\mathfrak{A} \models \varphi$ for every FO sentence $\varphi \in T$ (according to Definition 2.2.8), and
2. $\mathfrak{A} \models \Delta$ for every definition $\Delta \in T$ (according to Definition 2.4.11).

⁸The intuition is not so difficult. During the induction process, the new third truth value **u** serves to indicate for a defined fact P that P has not yet been safely derived to be true or false. This will be the case if there are applicable rules r that can derive P but not yet in a safe way (because the truth of the condition of r depends on unsafe facts, with value **u**). So, some rules are applicable to derive P , but none is safely applicable and their application needs to be delayed till they are safely applicable, or until no more applicable rules are left, in which case P can safely be derived to be false. This idea was formalised for the first time in [13].

Definitional rules are not truth conditional. This statement means that the satisfaction relation is not defined for definitional rules $\forall \bar{x}(P(\bar{t}) \leftarrow \varphi)$: Definition 2.4.11 does not contain a rule defining $\mathfrak{A} \models \forall \bar{x}(P(\bar{t}) \leftarrow \varphi)$. However, a definition Δ is truth conditional, since there is a rule defining $\mathfrak{A} \models \Delta$. Thus, it makes sense to say that a definition is satisfied in a structure (or in a state of affairs), but not that a rule is satisfied in it.

With each definitional implication $\forall \bar{x}(P(\bar{t}) \leftarrow \varphi) \in \Delta$, a unique FO material implication can be assigned: $\forall \bar{x}(P(\bar{t}) \Leftarrow \varphi)$.

Proposition 2.4.3

If Δ contains definitional rule $\forall \bar{x}(P(\bar{t}) \leftarrow \varphi)$, then Δ logically entails $\forall \bar{x}(P(\bar{t}) \Leftarrow \varphi)$.

Proof. If $\mathfrak{A} \models \Delta$ then the values of defined predicates P of Δ are the limit of a safe induction process. This process ends in saturation: all rule instances of Δ with body satisfied in $\mathfrak{A}$ have a head that is true in $\mathfrak{A}$. It follows from Definition 2.2.8 that $\mathfrak{A} \models \forall \bar{x}(P(\bar{t}) \Leftarrow \varphi)$. ■

Thus, in a model of Δ , all material implications corresponding to its rules are satisfied. But the inverse direction is not the case at all. It is perfectly possible that in a structure $\mathfrak{A}$, all the material implications corresponding to Δ 's rules are satisfied, and yet, $\mathfrak{A}$ does not satisfy Δ . Here are two propositional examples.

Take $\Delta = \{P \leftarrow \mathbf{f}\}$ and $\mathfrak{A} = \{P\}$. For this case, $\mathfrak{A}$ satisfies $P \Leftarrow \mathbf{f}$ but not Δ , since the induction process is $\langle \emptyset \rangle$ deriving the value $\mathbf{f}$ for P .

Take $\Delta = \{P \leftarrow P\}$ and $\mathfrak{A} = \{P\}$. For this case, $\mathfrak{A} \models P \Leftarrow P$, and even $\mathfrak{A} \models P \Leftrightarrow P$ but the induction process is again $\langle \emptyset \rangle$ deriving the value $\mathbf{f}$ for P . In particular, $\mathfrak{A} \not\models \Delta$.

In the series of examples below, the semantics of FO(ID) are explored and explained. It starts from propositional definitions, then monotone predicate definitions, then non-monotone predicate definitions.

Example 2.4.10

We begin with some propositional FO(ID) definitions, their induction processes and what they define:

- $\{P \leftarrow \mathbf{t}\}$ defines P , has no parameters, is non-inductive. Its unique induction process is $\emptyset \rightarrow \{P\}$. Hence, Δ is satisfied (or true) in a structure $\mathfrak{A}$ iff P is true in $\mathfrak{A}$.
- $\{P \leftarrow \mathbf{f}\}$ defines P , has no parameters, is non-inductive and monotone. The rule can never be applied, hence its induction process is $\langle \emptyset \rangle$. Hence Δ is satisfied in $\mathfrak{A}$ iff P is false in $\mathfrak{A}$.
- $\left\{ \begin{array}{l} P \leftarrow \mathbf{t} \\ P \leftarrow \mathbf{f} \end{array} \right\}$ defines P , has no parameters and is non-inductive. Δ has one rule that derives P and one rule that never applies. Therefore, its induction process is $\emptyset \rightarrow \{P\}$. Δ is satisfied in any structure in which P is true.
- $\{P \leftarrow P\}$ has no parameters, is inductive and monotone. Its unique induction process is $\langle \emptyset \rangle$. Thus, P is defined to be false.

- $\left\{ \begin{array}{l} P \leftarrow \mathbf{f} \\ Q \leftarrow P \end{array} \right\}$ defines P and Q , no parameters, inductive, monotone. It is inductive since P occurs in the body of a rule. Its unique induction process is $\langle \emptyset \rangle$. Hence, Δ defines P and Q to be false.
- $\left\{ Dead \leftarrow \neg Alive \right\}$ defines $Dead$ in terms of parameter $Alive$. It is non-inductive and monotone. In each $\mathfrak{A}$, it has a unique induction process, being $\langle \emptyset \rangle$ if $Alive^{\mathfrak{A}} = \mathbf{t}$ and $\langle \emptyset \rightarrow \{Dead\} \rangle$ if $Alive^{\mathfrak{A}} = \mathbf{f}$. Hence, $\mathfrak{A}$ satisfies Δ iff $Dead^{\mathfrak{A}}$ is the negation of $Alive^{\mathfrak{A}}$.
- $\left\{ \begin{array}{l} P \leftarrow \mathbf{t} \\ Q \leftarrow \neg P \end{array} \right\}$ defines P and Q , is inductive and non-monotone. Its unique safe induction process is $\langle \emptyset \rightarrow \{P\} \rangle$. Hence, Δ defines P to be true and Q to be false. Another process is $\langle \emptyset \rightarrow \{Q\} \rightarrow \{P, Q\} \rangle$. However, the second rule was applied unsafely, and its condition $\neg P$ is violated at the second step.
- $\left\{ \begin{array}{l} P \leftarrow P \\ Q \leftarrow \neg P \end{array} \right\}$ defines P and Q , is inductive and non-monotone. Its induction process is $\langle \emptyset \rightarrow \{Q\} \rangle$. It defines P to be false and Q to be true. Notice that the second rule can be safely applied since there is no way to derive P .

The induction order $\prec_i$ of a definition Δ serves to order the defined facts such that if rule application follows $\prec_i$, then the induction process is safe. To obtain this, $\prec_i$ should match the rules of the definition, in the sense that defined facts in the body, should be lower in the induction order than facts in the head. We explore this idea in the following example.

Example 2.4.11: Determining an induction order

Take the following non-monotone definition Δ :

$$\left\{ \begin{array}{l} P \leftarrow Q \\ Q \leftarrow P \\ R \leftarrow R \\ R \leftarrow \neg P \end{array} \right\}$$

When a monotone rule is applicable, it is always safe to apply it. There is one non-monotone rule $R \leftarrow \neg P$. It is crucial that the induction process for P is finished before this rule is applied. In this case, nor P , nor Q can be derived, hence it is safe to apply the non-monotone rule. We obtain the model $\{R\}$.

We compare Δ with the FO theory T_{Δ} consisting of the four material implications corresponding to Δ 's rules. This theory has 3 models: $\{R\}$, $\{P, Q\}$ and $\{P, Q, R\}$ and is clearly not equivalent to Δ . Notice also that $\{R\}$ is not the least model of T_{Δ} since $\{R\}$ is not a subset of $\{P, Q\}$.

In this example, we can derive an induction order. P depends on Q , Q depends on P , and R depends on P and R . The direct and indirect dependencies are caught by the least transitive relation $\preceq_i$ that includes these dependencies:

$$\preceq_i = \{(P, Q), (Q, P), (P, R), (Q, R), (R, R)\}$$

It shows that R depends indirectly on Q . For the unique non-monotone rule $R \leftarrow \neg P$, we have $P \preceq_i R$ but fortunately not $R \preceq_i P$. Thus, by finishing the induction process for P and Q before applying this rule, we will obtain a safe induction process. A good induction order $\prec_i$ is obtained by defining $x \prec_i y$ if $x \preceq_i y$ and not $y \preceq_i x$. The result is $\{(P, R), (Q, R)\}$. The induction process along $\prec_i$ is safe:

$$\langle \emptyset \rightarrow \{R\} \rangle$$

In the next series of examples, non-inductive predicate definitions are explored.

Example 2.4.12

The following is an FO(ID) definition by exhaustive enumeration of the predicate symbol *Prerequ*:

$$\begin{cases} \text{Prerequ}(CS230, M100) \leftarrow \mathbf{t} \\ \text{Prerequ}(M100, M200) \leftarrow \mathbf{t} \end{cases} \quad (2.6)$$

Such rules are often written as in $\text{Prerequ}(CS230, M100) \leftarrow$. This definition defines *Prerequ*/2 in terms of parameter symbols $CS230, M100, M200$. It is non-inductive (hence monotone).

The definition Δ can be evaluated in every structure that interprets its parameters. For example, take the structure $\mathfrak{A}$ with $D_{\mathfrak{A}} = \{a\}$, $\text{Prerequ}^{\mathfrak{A}} = \{(a, a)\}$, and $CS230^{\mathfrak{A}} = M100^{\mathfrak{A}} = M200^{\mathfrak{A}} = a$. In this structure, the induction process is $\emptyset \rightarrow \{(a, a)\}$. The latter set is the set defined by Δ in this $\mathfrak{A}$.

One can evaluate Δ also in a term structure $\mathfrak{A}$ with $\text{Prerequ}^{\mathfrak{A}} = \{(CS230, M100)\}$ and each object symbol interpreted by itself. Both rules are applicable and can be applied in any order or simultaneously. Three safe induction processes exist.

$$\begin{aligned} \emptyset &\rightarrow \{(CS230, M100)\} \rightarrow \{(CS230, M100), (M100, M200)\} \\ \emptyset &\rightarrow \{(M100, M200)\} \rightarrow \{(CS230, M100), (M100, M200)\} \\ \emptyset &\rightarrow \{(CS230, M100), (M100, M200)\} \end{aligned}$$

Each constructs the same relation $\{(CS230, M100), (M100, M200)\}$. This is the relation defined by Δ in $\mathfrak{A}$. This relation differs from $\text{Prerequ}^{\mathfrak{A}}$, so Δ is not satisfied in $\mathfrak{A}$.

Example 2.4.13

The following theory over Σ consists of a definition of *European*/1 and the FO sentence.

$$\begin{cases} \forall x(\text{European}(x) \leftarrow \text{Albanian}(x)) \\ \dots \\ \forall x(\text{European}(x) \leftarrow \text{Ukrainian}(x)) \end{cases} \quad (2.7)$$

$$\text{European}(\text{George})$$

Informally, under the suitable informal interpretation $\mathcal{I}$, the theory expresses a definition of *European* (± 50 non-inductive cases), and the assertion that George is European.

It follows that George has one of the European nationalities Albanian, ..., Belgian, ..., Ukrainian, although it cannot be deduced from this theory which nationality George has.

Formally, a Σ -structure $\mathfrak{A}$ satisfies the definition if $European^{\mathfrak{A}}$ is the result of some induction process. Many induction processes exist, but the order of rule application is immaterial; all of them construct the same set $Albanian^{\mathfrak{A}} \cup \dots \cup Ukrainian^{\mathfrak{A}}$. Thus, $\mathfrak{A}$ satisfies the definition iff $European^{\mathfrak{A}} = Albanian^{\mathfrak{A}} \cup \dots \cup Ukrainian^{\mathfrak{A}}$. $\mathfrak{A}$ satisfies the theory iff in addition, $George^{\mathfrak{A}}$ belongs to $European^{\mathfrak{A}}$ which is the case only if for some symbol σ representing a European country, $George^{\mathfrak{A}}$ is an element of $\sigma^{\mathfrak{A}}$.

The example clarifies the distinction between definitional and assertional knowledge. Imagine that we remove the FO assertion $European(George)$, and add the definitional rule to the definition:

$$European(George) \leftarrow$$

This would express that George is *defined* to be European, no matter what his nationality is. The resulting theory would entail that George is European but not that his nationality is a European nationality. The expanded definition has models in which George is Brazilian or Japanese, and yet, he is also European. Clearly, this would not be correct.

Exercise 2.4.9 [Continuation of Exercise 2.3.6] In the previous exercise, the goal was to define the set R as the set of sums of an element of P and an element of Q . The question was what is wrong with the following sentence:

$$\forall n \forall m (R(n + m) \Leftrightarrow P(n) \wedge Q(m))$$

Here is another question. Is there something wrong with:

$$\Delta = \left\{ \forall n \forall m (R(n + m) \leftarrow P(n) \wedge Q(m)) \right\}$$

Try out both cases in the structure with $P^{\mathfrak{A}} = \{1\}$ and $Q^{\mathfrak{A}} = \{2\}$. This example suggests that definitions are 4 more naturally represented in FO(ID) than in FO.

Now, we come to monotone predicate definitions.

Example 2.4.14: The prototypical inductive definition

Let domain D possess an object O and a function $S : D \rightarrow D$. We inductively define the set $N \subseteq D$:

- $O \in N$;
- For all $n \in D$, $S(n) \in N$ if $n \in N$.

What set is defined here? It is the set containing O and all entities that can be reached by iterated application of S on O . To express it in FO(ID), we use $\Sigma = \{O, S/1:, N/1\}$ under the obvious informal interpretation $\mathcal{I}$. The definition is expressed as:

$$\Delta = \left\{ \begin{array}{l} N(O) \leftarrow \\ \forall n (N(S(n)) \leftarrow N(n)) \end{array} \right\}$$

In each Σ -structure $\mathfrak{A}$, the induction process of Δ will be:

$$\emptyset \rightarrow \{O^{\mathfrak{A}}\} \rightarrow \{O^{\mathfrak{A}}, S^{\mathfrak{A}}(O^{\mathfrak{A}})\} \rightarrow \{O^{\mathfrak{A}}, S^{\mathfrak{A}}(O^{\mathfrak{A}}), S^{\mathfrak{A}}(S^{\mathfrak{A}}(O^{\mathfrak{A}}))\} \rightarrow \dots$$

For a structure $\mathfrak{A}$ with domain $D_{\mathfrak{A}} = \{a, b, c\}$, $O^{\mathfrak{A}} = a$, $S^{\mathfrak{A}} = \{(a, b), (b, b), (c, c)\}$, the induction process ends after three steps. Verify that $\mathfrak{A}$ satisfies this definition iff $N^{\mathfrak{A}} = \{a, b\}$.

For a structure $\mathfrak{A}$ with domain $D_{\mathfrak{A}} = \mathbb{R}$, $O^{\mathfrak{A}} = 0$, $S^{\mathfrak{A}} = \{(x, x+1) | x \in \mathbb{R}\}$, the induction process ends after ω steps. Verify that $\mathfrak{A}$ satisfies Δ iff $N^{\mathfrak{A}} = \mathbb{N}$.

Example 2.4.15

The **transitive closure** R of a binary relation G is defined as the least transitive relation that extends G . It is also called the **reachability relation** of G because a pair (x, y) belongs to R iff there is a finite path from x to y through G . More precisely, there is a sequence $x_0, \dots, x_n$ ($n > 0$) such that $x = x_0, y = x_n$ and for each $i < n$, $(x_i, x_{i+1}) \in G$. A third way of defining R is through an inductive definition:

- for all x, y , $(x, y) \in R$ if $(x, y) \in G$;
- for all x, y, z , $(x, z) \in R$ if $(x, y) \in R$ and $(y, z) \in R$.

^a The inductive definition can be expressed using $\Sigma = G/2, R/2\}$ in the following way:

$$\left\{ \begin{array}{l} \forall x \forall y (R(x, y) \leftarrow G(x, y)) \\ \forall x \forall y \forall z (R(x, z) \leftarrow R(x, y) \wedge R(y, z)) \end{array} \right\} \quad (2.8)$$

Notice that the inductive rule resembles the transitivity law. So, this definition literally closes G under the transitivity law.

- Take $\mathfrak{A}_1$ with $D_{\mathfrak{A}_1} = \{a, b\}$ and $G^{\mathfrak{A}_1} = \{(a, a), (b, b)\}$. The induction process consists of applications of the two instances of the first rule. Here is a slow and a fast induction process:

$$\emptyset \rightarrow \{(a, a)\} \rightarrow \{(a, a), (b, b)\}$$

$$\emptyset \rightarrow \{(a, a), (b, b)\}$$

$\mathfrak{A}_1$ is a model of the definition iff $R^{\mathfrak{A}_1} = \{(a, a), (b, b)\}$.

- Take $\mathfrak{A}_2$ with $D_{\mathfrak{A}_2} = \{a, b, d\}$ and $G^{\mathfrak{A}_2} = \{(a, b), (b, a), (d, d)\}$. The fastest induction process is obtained by applying at each stage all applicable rules. This results in the following sequence:

$$\emptyset \rightarrow \{(a, b), (b, a), (d, d)\} \rightarrow \{(a, b), (b, a), (d, d), (a, a), (b, b)\}$$

- Take $\mathfrak{A}_3$ with $D_{\mathfrak{A}_3} = \mathbb{N}$ and $G^{\mathfrak{A}_3} = \{(n, n+1) | n \in \mathbb{N}\}$. For this instance, the transitive closure is nothing else than the standard strict order $<$ on the natural numbers.

Now, all induction processes are infinite, and most are transfinite. For example, here is a transfinite one that derives one pair (n, m) at the time. We present it as a sequence of the newly derived pairs, although in reality, it is a sequence of sets

of pairs.

$$\begin{aligned}
& (0, 1) \rightarrow (1, 2) \rightarrow (2, 3) \rightarrow \dots \rightarrow (n, n+1) \rightarrow \dots \\
& \rightarrow (0, 2) \rightarrow (0, 3) \rightarrow (0, 4) \rightarrow \dots \rightarrow (0, n) \rightarrow (0, n+1) \rightarrow \dots \\
& \rightarrow (1, 3) \rightarrow (1, 4) \rightarrow (1, 5) \rightarrow \dots \rightarrow (1, n) \rightarrow (1, n+1) \rightarrow \dots \\
& \rightarrow (2, 4) \rightarrow (2, 5) \rightarrow (2, 6) \rightarrow \dots \rightarrow (2, n) \rightarrow (2, n+1) \rightarrow \dots \\
& \dots
\end{aligned}$$

In the first row, $G^{\mathfrak{A}_3}$ is “copied” in $R^{\mathfrak{A}_3}$ tuple by tuple. In the second row, all tuples of the form $(0, n)$ are derived using the inductive rule. Etc.. This process takes $\omega \times \omega$ steps.

On the other hand, the fastest induction process is the one that applies in each step all applicable rule instances at once. Then only ω steps are required. No matter how the induction process is executed, each constructs the same relation, namely $<$ on $\mathbb{N}$.

^aIt should be proven that these three definitions are equivalent. Do this yourself.

Exercise 2.4.10 [Important] It will be proven in chapter 6 that the transitive closure relation cannot be defined in FO. Most students study this theorem (by heart?) but many do not understand what is wrong with using the following FO theories or formulas to “define” R :

1. $\forall x \forall y (R(x, y) \Leftarrow G(x, y))$
 $\forall x \forall y \forall z (R(x, z) \Leftarrow R(x, y) \wedge R(y, z))$
2. $\forall x \forall y (R(x, y) \Leftrightarrow G(x, y) \vee (\exists z (R(x, z) \wedge R(z, y)))$
3. $\forall x \forall y (R(x, y) \Leftrightarrow G(x, y) \vee (\exists z (G(x, z) \wedge R(z, y)))$
4. $\forall x \forall y (R(x, y) \Leftrightarrow G(x, y) \vee (\exists z (R(x, z) \wedge G(z, y)))$

Take the following structure $\mathfrak{A}$ (equal to $\mathfrak{A}_1$ in the previous example): $D_{\mathfrak{A}} = \{a, b\}$ and $G^{\mathfrak{A}} = \{(a, a), (b, b)\}$. Show that none of the FO sentences in this list **gets the models right**. Hint: give a structure that satisfies each of the FO sentences but not the inductive definition.

The transitive closure of the parent relation is the ancestor relation.

The following example shows a monotone definition with a universal quantifier in the body.

Example 2.4.16: Terminating nodes of a graph

Given a graph G , we call a node s *terminating* if there is no infinite path $s \rightarrow s_1 \rightarrow s_2 \rightarrow \dots$ through G . We define this concept through an FO(ID) definition using $\Sigma = \{G/2, T/1\}$.

$$\left\{ \forall x (T(x) \leftarrow \forall y (G(x, y) \Rightarrow T(y))) \right\}$$

This is a monotone definition since T occurs positively in the rule body. Its rule body contains a universal quantifier. It may seem that there is no base case, but that is a false impression. A base case of T ’s definition is a node d without outgoing edges in G . For such node d , it trivially holds that $\mathfrak{A}[x : d] \models \forall y (G(x, y) \Rightarrow T(y))$. Thus, the induction process can derive $T(d)$ for all every such node d . In later stages of the

induction process, the rule can be applied to derive $T(e)$ for nodes e with outgoing paths of maximally length 1, then maximally length 2, etc. The limit of this process is the set of all nodes that have only outgoing paths of finite length.

Example 2.4.17: traveling salesman

In the *travelling salesman* problem, there is a map of cities and roads, represented by a road relation $Road(x, y)$. Assume the city A (Antwerp of course) represents the starting point of the traveling salesman. The problem is to compute a path for the salesman from A that passes at every city exactly once and returns to A .

We express the path by a predicate $N/2$ such that $N(x, y)$ means that the salesman goes from city x to y . In addition, we introduce one auxiliary predicate $R/1$ representing the set of cities that the salesman visits on his path. This is the proposed theory:

$$\begin{aligned} & \forall x \forall y (N(x, y) \Rightarrow Road(x, y)) \\ & \forall x \forall y \forall z (N(y, x) \wedge N(z, x) \Rightarrow y = z) \\ & \forall x \forall y \forall z (N(x, y) \wedge N(x, z) \Rightarrow y = z) \\ & \forall x R(x) \\ & \left\{ \forall x (R(x) \leftarrow N(A, x)) \forall x \forall y (R(x) \leftarrow R(y) \wedge N(y, x)) \right\} \end{aligned}$$

The theory expresses that the path follows roads, that each city is entered at most once, and left at most once, that all cities are reached on the path. R is defined inductively as the set of all cities that are reached from A with at least one step. Since A itself must be reached, the path is a cycle that begins and ends with A , and all cities of the universe lie on the path.

The computational problem here is a model expansion problem. Input is the theory and a structure $\mathfrak{A}$ with as universe the set of cities, $A^{\mathfrak{A}}$ is some city, and $Road^{\mathfrak{A}}$ is a road map. The output is the value $N^{\mathfrak{A}}$ in a model of this theory that expands $\mathfrak{A}$.

Notice that in solving this problem, the defined predicate $R/1$ is known, and it is the parameter N of its definition that must be computed. It shows that IDP cannot only reason on definitions by bottom up execution. It can also reason top down, backwards, from the defined predicate to the parameter concepts.

Exercise 2.4.11 Complete this specification, implement your solution in IDP and check if it is correct for some examples.

A definition may define multiple concepts simultaneously.

Example 2.4.18

Here is a definition of predicates $Even/1$ and $Odd/1$ by simultaneous induction:

$$\left\{ \begin{array}{l} Even(0) \leftarrow \\ \forall x (Odd(S(x)) \leftarrow Even(x)) \\ \forall x (Even(S(x)) \leftarrow Odd(x)) \end{array} \right\} \quad (2.9)$$

This definition can be evaluated in any structure that interprets the parameters 0/0: and $S/1$:. In the context of the structure $\mathbb{N}$, the induction process is the following infinite sequence of pairs (E_i, O_i) :

$$(\emptyset, \emptyset) \rightarrow (\{0\}, \emptyset) \rightarrow (\{0\}, \{1\}) \rightarrow (\{0, 2\}, \{1\}) \rightarrow (\{0, 2\}, \{1, 3\}) \rightarrow \dots$$

The limit of this process is the pair of the set of even and the set of odd numbers. Hence, this definition is satisfied in an extension $\mathfrak{A}$ of $\mathbb{N}$ iff $Even^{\mathfrak{A}}$ is the set of even numbers, and $Odd^{\mathfrak{A}}$ is the set of odd numbers.

Exercise 2.4.12 What are the defined values for *Even* and *Odd* in the following structure $\mathfrak{A}$, with $D_{\mathfrak{A}} = \{a, b, c, d\}$, $0^{\mathfrak{A}} = a$, and $S^{\mathfrak{A}} = \{(a, b), (b, c), (c, a), (d, d)\}$?

Now we present a non-monotone definition of even numbers.

Example 2.4.19

Take the following uncommon but sensible definition.

Definition 2.4.13: Non-monotone definition of even numbers

We define even numbers (by induction on the standard order $<$ of $\mathbb{N}$):

- 0 is even.
- $n + 1$ is even if n is **not** even.

Its non-monotone inductive rule is uncommon, but similar to the $\neg$ -rule in the definition of the satisfaction relation:

$$\mathfrak{A} \models \neg\varphi \text{ if } \mathfrak{A} \not\models \varphi.$$

The definition can be expressed using $\Sigma = \{0, S/1:, Even/1\}$:

$$\Delta = \left\{ \begin{array}{l} Even(0) \leftarrow \\ \forall n (Even(S(n)) \leftarrow \neg Even(n)) \end{array} \right\} \quad (2.10)$$

When evaluating Δ in $\mathbb{N}$, the key issue is safe application of the inductive rule. Otherwise, many erroneous facts could be derived. E.g., in the initial phase when the value of *Even* is still empty, *every* atom $Even(n)$ can be derived using the inductive rule. However, only the base rule can be safely applied. If we analyse the direct and indirect dependencies of the defined facts $Even(n)$ of this definition, we see that $Even(n)$ depends on $Even(m)$ iff $n > m$. A safe induction process should follow this order, which is the standard order of natural numbers. The one and only correct safe induction process is:

$$\emptyset \rightarrow \{0\} \rightarrow \{0, 2\} \rightarrow \{0, 2, 4\} \rightarrow \dots$$

Initially, the base rule is the only one that can be safely applied. After deriving 0, it becomes impossible to derive evenness of 1, which in turn leads to safe derivation of 2, making it impossible to derive 3, etc.

Exercise 2.4.13 Construct an unsafe induction by deriving $Even(3)$ in the first step and see what you get.

Example 2.4.20: Company control

One of my favourite non-formal inductive definitions is the following:

Definition 2.4.14

A company A controls company B if the total sum of the shares in company B owned by A or by companies controlled by A is more than 50%.

In $\text{FO}(\cdot)$, the inductive definition construct is further extended allowing aggregate expressions in the body. Consider the following vocabulary:

- $Ct(x, y)$: company x controls company y .
- $OwnsSh(x, y, s)$: company x owns s percent shares of the company y .

The following inductive definition in $\text{FO}(\text{Types}, \text{Arit}, \text{Agg}, \text{ID})$ defines the controls relation:

$$\left\{ \forall a \forall b (Ct(a, b) \leftarrow \text{Sum}\{(s, c) : (c = a \vee Ct(a, c)) \wedge OwnsSh(c, b, s)\} > 0.50) \right\} \quad (2.11)$$

Exercise 2.4.14 Consider the situation where company A owns 60% of the shares in B , and A and B both own 30% of the shares in C . Formalize by a structure $\mathfrak{A}$ and elaborate an induction process.

Example 2.4.21: Compressing lists

The following formal non-monotone definition defines the compression relation between finite lists by deletion of double occurrences of elements of x . We choose the vocabulary consisting of the empty list $[]$, the list constructor $[\cdot]$ (in Prolog style), and two predicate symbols $member/2$ and $compress/2$ where $compress(l, l')$ means that l' is the compression of l .

$$\left\{ \begin{array}{l} compress([], []) \leftarrow \\ \forall x \forall y \forall z (compress([x|y], [x|z]) \leftarrow compress(y, z) \wedge \neg member(x, z)) \\ \forall x \forall y \forall z (compress([x|y], z) \leftarrow compress(y, z) \wedge member(x, z)) \\ \forall x \forall y (member(x, [x|y]) \leftarrow) \\ \forall x \forall y \forall z (member(x, [y, z]) \leftarrow member(x, z)) \end{array} \right\}$$

The second rule is non-monotone since it contains the negative condition $\neg member(x, z)$. Nevertheless, this is a non-problematic definition. Safe induction processes are those that first construct the $member$ relation, and only later the $compress$ relation.

This definition corresponds to a working Prolog program.

So far, the list of non-monotone definitions is fairly short. In the next chapter, where definitions are used to describe dynamic systems, non-monotone definitions are all over the place.

There is an obvious correspondence between $\text{FO}(\text{ID})$ definitions and the way definitions are commonly phrased in mathematical text. This correspondence is the basis for the informal interpretation $\mathcal{I}(\Delta)$ of definitions. At this point we obtain that the language $\text{FO}(\text{ID})$ is a modelling logic according to Definition 1.2.1. Hence, the concepts of tautology, logical entailment, logical equivalence, etc. are well-defined for $\text{FO}(\text{ID})$.

Some semantic properties of definitions

Simplifying the head of a definitional rule In a definition Δ , it is always possible to simplify the head of a definitional rule

$$\forall x_1 \dots \forall x_m (P(t_1, \dots, t_n) \leftarrow \varphi)$$

by turning it into:

$$\forall y_1 \dots \forall y_n (P(y_1, \dots, y_n) \leftarrow \exists x_1 \dots \exists x_m (y_1 = t_1 \wedge \dots \wedge y_n = t_n \wedge \varphi))$$

It is easy to see that if one rule is applicable during the induction process in structure $\mathfrak{A}$, then the other is applicable as well, and both derive the same tuple for P/n .

This transformation can be used to eliminate terms from the head, or eliminate variables quantified in front of the rule, as in the transformation from:

$$\forall x \forall y \forall z (R(x, z) \leftarrow R(x, y) \wedge R(y, z))$$

to

$$\forall x \forall z (R(x, z) \leftarrow \exists y (R(x, y) \wedge R(y, z)))$$

Compacting a group of rules into one In a definition Δ , a group of rules of the following form:

$$\left\{ \begin{array}{l} \dots \\ \forall x_1 \dots \forall x_n (P(x_1, \dots, x_n) \leftarrow \varphi_1) \\ \dots \\ \forall x_1 \dots \forall x_n (P(x_1, \dots, x_n) \leftarrow \varphi_n) \\ \dots \end{array} \right\}$$

can be transformed into one rule:

$$\left\{ \begin{array}{l} \dots \\ \forall x_1 \dots \forall x_n (P(x_1, \dots, x_n) \leftarrow \varphi_1 \vee \dots \vee \varphi_n) \\ \dots \end{array} \right\}$$

E.g., we can combine these two translations to turn the definition:

$$\left\{ \begin{array}{l} Prerequ(CS230, CS232) \leftarrow \\ Prerequ(M100, M200) \leftarrow \end{array} \right\}$$

into the logically equivalent definition:

$$\left\{ \forall x \forall y (Prerequ(x, y) \leftarrow (x = CS230 \wedge y = CS232) \vee (x = M100 \wedge y = M200)) \right\}$$

E.g., in the same way, the FO(ID) definition of Eq. 2.8:

$$\left\{ \begin{array}{l} \forall x \forall y (R(x, y) \leftarrow G(x, y)) \\ \forall x \forall y \forall z (R(x, z) \leftarrow R(x, y) \wedge R(y, z)) \end{array} \right\}$$

can be translated into the equivalent definition:

$$\left\{ \forall x \forall y (R(x, y) \leftarrow G(x, y) \vee \exists z (R(x, z) \wedge R(z, y))) \right\}$$

Translating a definition Δ in its FO completion $\text{compl}(\Delta)$ Using the methods of the previous paragraphs, any finite definition Δ can be translated into a logically equivalent definition Δ' with exactly one rule

$$\forall x_1 \dots \forall x_n (P(x_1, \dots, x_n) \leftarrow \varphi)$$

for every defined predicate P/n . Then Δ' can be translated in an FO theory consisting of:

$$\forall x_1 \dots \forall x_n (P(x_1, \dots, x_n) \Leftrightarrow \varphi)$$

The resulting FO theory is called the completion of Δ and is denoted $\text{compl}(\Delta)$.

Proposition 2.4.4
 Δ logically entails $\text{compl}(\Delta)$.

Proof. Translate Δ in the format with one rule per defined predicate:

$$\forall x_1 \dots \forall x_n (P(x_1, \dots, x_n) \leftarrow \varphi)$$

During an induction process in structure $\mathfrak{A}$, a tuple $(d_1, \dots, d_n)$ is added to $P^{\mathfrak{A}}$ at stage α of the induction process only if φ is satisfied for the assignment $[x_1 : d_1, \dots, x_n : d_n]$ at that stage. Because the induction process is safe, φ is still satisfied for this assignment in the limit of the induction process. Hence, if $\mathfrak{A}$ is a model of Δ then

$$\mathfrak{A} \models \forall x_1 \dots \forall x_n (P(x_1, \dots, x_n) \Rightarrow \varphi)$$

For the other direction, in the limit of the induction process, the constructed value for P is saturated for the rule. Hence, if $\mathfrak{A}$ satisfies Δ this means that

$$\mathfrak{A} \models \forall x_1 \dots \forall x_n (P(x_1, \dots, x_n) \Leftarrow \varphi)$$

■

Each finite definition Δ logically entails the FO theory $\text{compl}(\Delta)$. The question is whether or when Δ is logically equivalent to $\text{compl}(\Delta)$.

We already saw that $\{ P \leftarrow P \}$ is not logically equivalent to its completion $P \Leftrightarrow P$ since the boolean structure $\{P\}$ satisfies the completion but not the definition.

In Exercise 2.4.10, the goal was to show that the inductive definition of the transitive closure of graph $G/2$ is not equivalent to its completion. **It is imperative that you make this exercise** to understand where $\text{compl}(\Delta)$ goes wrong.

Proposition 2.4.5
 A definition Δ is hierarchical if there is a strict order $<$ on the defined predicates such that if some predicate P occurs in the head of a rule and Q in the body, then $Q < P$. In this case Δ and $\text{compl}(\Delta)$ are logically equivalent. In this case, $\text{compl}(\Delta)$ is a hierarchally structured set of explicit definitions.

There are many other interesting cases where Δ and $\text{compl}(\Delta)$ is logically equivalent but further discussion of this topic is out of the scope of this course.

Formalising monotone definitions in SO It is well-known that for any monotone inductive definition in mathematics, the defined concept is the least set satisfying the material implications corresponding to the definition rules.

This is not the case in general with non-monotone definitions. A small example showing this is the following:

$$\left\{ \begin{array}{l} P \leftarrow \mathbf{f} \\ Q \leftarrow \neg P \end{array} \right\}$$

The FO theory consisting of implications has three models $\{P\}, \{Q\}, \{P, Q\}$. None is the least, since two of them are minimal. The only safe induction process is $\langle \emptyset \rightarrow \{Q\} \rangle$.

Monotone inductive definitions expressible in $\text{FO}(\cdot)$ can be expressed also in SO. We illustrate this with the definition of the transitive closure (reachability relation) of Eq. 2.8

$$\left\{ \begin{array}{l} \forall x \forall y (R(x, y) \leftarrow G(x, y)) \\ \forall x \forall y \forall z (R(x, z) \leftarrow R(x, y) \wedge R(y, z)) \end{array} \right\}$$

Example 2.4.22

The formalisation of this definition in SO is the following theory:

$$\begin{array}{l} \forall x \forall y (R(x, y) \leftarrow G(x, y)) \\ \forall x \forall y \forall z (R(x, z) \leftarrow R(x, y) \wedge R(y, z)) \\ \forall X \left(\begin{array}{l} \forall x \forall y (X(x, y) \leftarrow G(x, y)) \wedge \\ \forall x \forall y \forall z (X(x, z) \leftarrow X(x, y) \wedge X(y, z)) \\ \Rightarrow \forall x \forall y (R(x, y) \Rightarrow X(x, y)) \end{array} \right) \end{array}$$

It states that R contains G and is transitive and is contained in each X that contains G and is transitive.

Definitional paradoxes A definition defines a concept by describing how to construct it. For non-monotone definitions, there is occasionally a snake under the grass. Sometimes, the induction process stops in a state where rules are applicable but none is safely applicable. The simplest example where this occurs is as follows:

$$\left\{ P \leftarrow \neg P \right\}$$

At the start of the induction process, P is false, making the rule applicable. However, the rule is not safely applicable since applying it will cause its own condition to become false. Thus the induction process stalls here. There is no defined set, and this definition has no model. It is unsatisfiable.

One consequence is that the definition in $\text{FO}(\text{ID})$ does not fully satisfy the fundamental property of definitions: it defines at most one value, but (for some values of the parameters) it may be that no value for the defined predicates exists. In the above case, the definition should define a value for P but it does not. This occurs only with certain non-monotone definitions.

There is a positive side to it. Namely that the logic allows to express *definitional paradoxes*. Paradoxes are fascinating topics. The above definition is arguably a paradox. We define a

concept to be its own negation: if it is true, it must be false, and if it is false it must be true.

Berry's paradox is a definitional paradox in natural language. The paradox is the following definition. We define the Berry number as *the least natural number that cannot be defined in less than thirteen words*. Do you spot the paradox? To make it more visible consider the following table.

Defining string	defined value
"the least natural number"	0
...	...
"the least natural number that cannot be ..."	??
...	...

In its left column, the table contains strings of maximally 13 words that define a number. The right column contains the defined number. We see that Berry's number is defined in terms of this table (using predicate *Defined13W*/2) by the following rule:

$$\begin{aligned} \forall n(\text{Defined13W}(\langle \text{Berry's string} \rangle, n) \leftarrow \\ \neg \exists w(\text{Defined13W}(w, n) \\ \forall m(m < n \Rightarrow \exists w(\text{Defined13W}(w, m) \end{aligned}$$

The paradox emerges as a loop over negation $\text{Defined13W}(\dots) \leftarrow \neg \exists \dots \text{Defined13W}(\dots)$.

The following example shows a sensible definition where depending on the value of the parameters, the definition can be paradoxical or not.

Example 2.4.23: access control

Consider the following access control scenario: in a company there is one file with a group of potential users. The file has owner O . A user x can delegate access to the file to another user y (*delegate*(x, y)), or can block access for another user y (*blocks*(x, y)). Delegation and blocking is only effective in case the issuing user has access. Thus, a user x has access if x is owner or there is a path $x_0, \dots, x_{n+1}$ of users, with x_0 the owner, x_{n+1} the user x , such that each $x_i, i \leq n$ has access, and each x_i delegates to x_{i+1} and x is not blocked by a user with access. We can express this definition in FO(ID):

$$\left\{ \begin{array}{l} \text{access}(O) \leftarrow \\ \forall x(\text{access}(x) \leftarrow \exists y(\text{access}(y) \wedge \text{delegates}(y, x)) \wedge \neg \exists z(\text{access}(z) \wedge \text{blocks}(z, x))) \end{array} \right\} \quad (2.12)$$

It is non-monotone due to the second condition of the inductive rule. If the *blocks* relation is empty, this definition defines *access* as the set of users that can be reached from the owner in the relation *delegates*.

Interestingly, this definition allows for paradoxes. E.g., suppose the owner delegates to users A and B, but A blocks B and B blocks A. Who has access? A has access if B has not and B has access if A has not. There is no way out. The induction process of this definition stalls in a state where there are applicable rules none of which is safely applicable. This is a paradox.

Another paradox arises if A blocks itself.

There is a natural solution to this problem: by imposing the rule that a user can only block its subordinates in the corporate hierarchy. Then the inverse of the corporate

hierarchy acts as an induction order $\prec_i$ for this definition.

^aYou can experiment with this domain online.

^a <https://idp.cs.kuleuven.be/idp/?present=access>

The topic of paradoxes is fascinating in its own right. But in modelling applications, designers need to avoid paradoxes at all cost. Paradoxes will not play a role anymore in this course, and we will not spend more attention on it.

Definitions: declarative or procedural?

An intriguing aspect of definitions formalised in FO(ID) is that they are declarative, yet they also have a procedural interpretation corresponding to iterated safe bottom up rule application. This procedural interpretation is non-deterministic, but “computes” always the same defined relation. The induction process correspond to a run of the procedure. Of course, the induction process is not always a computation, for example when it takes transfinitely many steps.

The “procedure” associated with a definition Δ can only be physically computed on computer in special circumstances: when the universe and the values of the parameters are given and finite. In that case, iterated safe rule application computes answers for the following inference problem:

Δ-model expansion inference <i>Input:</i> definition Δ, finite structure $\mathfrak{A}$ interpreting parameter symbols <i>Output:</i> value of defined symbols of Δ in the model of Δ expanding $\mathfrak{A}$.

This form of inference is implemented in quite some applications, e.g., in computing answers to queries or to view-predicates in databases.

At the same time, definitions are clearly also declarative. They serve to specify a precise logical relationship between the defined concept and the set of parameter concepts. To know this correspondence is valuable in so many situations, even when the above procedure cannot be executed. E.g., in mathematics, inductive definitions are used to construct highly infinite entities where physical computation is impossible. Definitions are reasoned upon not only bottom up but also top down as in Prolog. In some cases, such as in the traveling salesman problem Example 2.4.17, it is the value of the defined predicate that is input, and it is the value of the parameter of the definition that needs to be computed. Indeed, in this problem, the set of cities is given and is equal to the reachability relation $R^{\mathfrak{A}}$. The desired output of the computational problem is a value for the parameter N of the definition, i.e., the path. The computational reasoning process for this problem is totally unlike the procedural interpretation of the definition.

So, despite their “procedural” format, definitions are truly declarative pieces of knowledge on which a range of different forms of inference can be applied. The non-computational “procedural” understanding of definitions (as a description of the induction process) seems to help the human brain to understand the logical relation between parameters and the defined concepts, in arbitrary contexts.

2.4.8 Summary

A number of extensions of FO were described to obtain a more powerful modeling language that better supports the pragmatics of modelling (and to get the models right). It is customary in the literature to denote extensions of FO as $F(\langle \text{ext} \rangle)$ with $\langle \text{ext} \rangle$ the extension. This section introduced extensions $FO(\text{Types})$, $FO(\text{Types}, \text{Arit})$, $FO(\text{Agg})$, $FO(\text{Identifier})$, $FO(\text{ID})$, $FO(\text{PF})$ and combinations of them. Strictly speaking, with $FO(\cdot)$, we mean the family of logics obtained by extending FO. The system IDP supports one such a language, which corresponds more or less to $FO(\text{Types}, \text{Arit}, \text{Agg}, \text{Identifier}, \text{ID}, \text{PF})$. In this course, by small abuse of terminology, we use $FO(\cdot)$ to denote this extension of FO.

The most complex extension is that with (inductive) definitions. I argued that the definition construct that was added in $FO(\text{ID})$ is suitable to express the most common forms of definitions of sets found in mathematics: monotone definitions and non-monotone definitions over some induction order are correctly expressed in $FO(\text{ID})$. This is because its formal semantics implements the idea of iterated safe rule application. The definitional implication symbol introduced in $FO(\text{ID})$ is another conditional, clearly distinguishable from material implication. And also from the conditional used in $\exists c(\text{Succ}(c) \Rightarrow \text{Degree})$. The (inductive) definition will play an important role in the next section, when $FO(\cdot)$ is used to model dynamic systems.

Quite a few rule-based formalisms exist. E.g., Prolog systems, business rules systems, production rule systems. What these languages have in common is that rules and rule sets are viewed as procedural entities. E.g., in production rule systems, rules are run as bottom up procedures. In Prolog, rules are run in a top down way. A distinguishing feature of $FO(\text{ID})$ is that rule sets are viewed in a purely declarative way, as definitions. They specify information, not procedures, and this information can be used to solve multiple forms of problems.

2.5 An example of $FO(\cdot)$: Steel oven scheduling

Below, a scheduling application is worked out. The elaboration comments on the choice of vocabulary, and the effect it has on the modelling.

Assume we have three types of resources: 3 platforms, 2 ovens, and 1 cooler. There are a number of tasks. A task is to construct some steel object. Each task uses a single platform for: one hour loading, then some hours of oven, and then five hours cooling, one hour deloading. We want to do 20 tasks in the minimal amount of time: nine of them need 10 hours of oven, five need 12h, one needs 15h, two need 16h, and three need 22h.

Diagrammatically, the manufacturing cycle looks like:

load	oven	cooling	unload
1 hr	x hrs	5 hrs	1 hr

We start with the design of a suitable vocabulary. There are certainly many ways to do this. One option is as follows.

- The following types are relevant: *Platform, Oven, Cooler, Task, Time*.
 - *Time* is *int* or a subset of it.
 - *Cooler*: we have only one cooler. Types with only one element can always be eliminated. The effect on predicate and function symbols is that the argument of this type is dropped and becomes implicit. The advantage is simplicity and

efficient reasoning. The disadvantage is on the level of robustness and extensibility: if later coolers are added, the theory needs to be rewritten. So, we keep the type *Cooler*.

- Durations: the load, bake and cool operations of a task have durations. Only the duration of the baking phase varies from task to task. We choose a function symbol to represent the baking phase: $dur(Task) : Time$. We leave the durations of the load and cool phase implicit. The disadvantage of this is a lower robustness of our theory: if new tasks show up that need more time to load than others, the theory must be extended with a duration function for the loading phase of tasks.
- A task is to be assigned 1 platform, 1 oven, 1 cooler, each during a certain period. Since only one resource of each type is assigned to the task, we may choose function symbols to represent the resources assigned to the task:
 $PlOf(Task) : Platform, OvOf(Task) : Oven, CoOf(Task) : Cooler$
 The intended interpretation of these symbols is obvious.
- Each phase of the execution of a task is characterized by a start and stop time; these are 6 time points. We choose 6 function symbols with the obvious intended interpretation:
 $StartPl(Task) : Time, StopPl(Task) : Time, StartOv(Task) : Time, StopOv(Task) : Time, StartCo(Task) : Time, StopCo(Task) : Time$.
- Finally, object symbols can be used to represent the objects. In our case, 20 tasks $o_1, \dots, o_{20}$, 3 platforms pl_1, pl_2, pl_3 , 2 ovens ov_1, ov_2 , 1 cooler co . With IDP, we can use identifiers to specify them in the theory (using the constructed-from statement), or we leave them unnamed and create a structure containing the data.

Summarized, the vocabulary Σ is as follows:

- Types *Platform, Oven, Cooler, Task, Time*.
- $Dur(Task) : Time$ - the duration of oven time of tasks (all other durations have the same)
- $PlOf(Task) : Platform, OvOf(Task) : Oven, CoOf(Task) : Cooler$
- $StartPl, StopPl, StartOv, StopOv, StartCo, StopCo : Task \rightarrow Time$
- Potentially, object symbols/identifiers for tasks $o_1, \dots, o_{20}$, platforms pl_1, pl_2, pl_3 , ovens ov_1, ov_2 , and cooler co .

The theory consists of the following laws:

- The temporal links between different phases of tasks.

$$\begin{aligned}
 \forall o[Task](StartOv(o) &= StartPl(o) + 1 \\
 &\wedge StopOv(o) = StartOv(o) + Dur(o) - 1 \\
 &\wedge StartCo(o) = StopOv(o) + 1 \\
 &\wedge StopCo(o) = StartCo(o) + 4 \\
 &\wedge StopPl(o) = StopCo(o) + 1)
 \end{aligned}$$

- No simultaneous usage of platforms, ovens, coolers:

$$\begin{aligned}
 \forall pl \forall o_1 \forall o_2 (pl &= PlOf(o_1) \wedge pl = PlOf(o_2) \\
 &\Rightarrow StopPl(o_1) < StartPl(o_2) \vee StartPl(o_1) > StopPl(o_2)) \\
 \forall ov \forall o_1 \forall o_2 (ov &= OvOf(o_1) \wedge ov = OvOf(o_2) \\
 &\Rightarrow StopOv(o_1) < StartOv(o_2) \vee StartOv(o_1) > StopOv(o_2)) \\
 \forall co \forall o_1 \forall o_2 (co &= CoOf(o_1) \wedge co = CoOf(o_2) \\
 &\Rightarrow StopCo(o_1) < StartCo(o_2) \vee StartCo(o_1) > StopCo(o_2))
 \end{aligned}$$

- If we express data in a structure, then adding data to the theory is not necessary and model expansion can be used. Alternatively, if we express data in the theory, formulas expressing exhaustive enumerations, unique name axioms and type closure axioms need to be expressed (or the constructed-from statement can be used)

Exercise 2.5.1 *What form of inference do we need to apply on this theory (and potentially other input objects) to compute a correct schedule for a given steel oven problem?*

2.6 Axiomatising complex domains

2.6.1 Generalising UNA and DCA in FO(.)

In many modelling applications, the knowledge engineer has full knowledge of the entities in the domain (or in some type) and has designed a set τ of object and function symbols so that each entity in the domain (or in that type) is represented by a unique term $t \in \mathbf{Terms}(\tau)$. This is the case in many database applications in which the strings in the tables represent all relevant entities of the database application domain.

As seen in subsection 2.3.2, a Unique Name Axiom ($\mathbf{UNA}(\tau)$) and a Domain Closure Axiom ($\mathbf{DCA}(\tau)$) are needed to get the models right. These axioms are easy to state in FO (although quite long) if τ is a finite set of object symbols. Here, we investigate how to express them when τ contains also function symbols so that $\mathbf{Terms}(\tau)$ is infinite. Such applications arise all the time in logic or functional programming languages where terms are used as composite data structures. Think of lists built from *nil* and *cons/2*.

In such applications, different terms, e.g., *nil*, *cons(1, nil)*, *cons(1, cons(1, nil))*, ..., obviously serve to represent different entities. But another, more essential property of such constructors shows up here: namely, that a structured object should uniquely determine its components. E.g., for the value of the term *cons(1, nil)*, it should be derivable that its first component is the value of 1, and its second component is the value of *nil*. This is a special property that in general functions do not have. E.g., from the value 9 of the compound term $1 + 8$, we cannot derive its first and second components, since 9 is the value of infinitely many terms $0 + 9, 5 + 4, -1000 + 1009, \dots$. Hence, the function $+$ cannot serve to build data structures.

In a sufficiently expressive logic, it is possible to express the condition that a set τ of object and function symbols act as a collection of **constructors** of the domain or a type. Two conditions need to be expressed.

As for the first condition, given a constructed entity e , it should be possible to identify the constructor F and the arguments $\vec{d}$ such that $F(\vec{d}) = e$. The entity e should not be the result of applying another constructor, and it should not be the result of applying F to another tuple than $\vec{d}$. This boils down to a **groupwise injectivity axiom**: the co-domains of different constructors should be disjoint, and each constructor should be injective. A function is injective if different arguments have different mappings. In mathematical notation, function F is injective if $\bar{x} \neq \bar{y}$ implies $F(\bar{x}) \neq F(\bar{y})$, or equivalently, if $F(\bar{x}) = F(\bar{y})$ implies $\bar{x} = \bar{y}$. The groupwise injectivity axioms can be stated in FO.

Definition 2.6.1

The theory $\mathbf{UNA}(\tau)$ for τ consists of all FO axioms (2.14) and (2.13). For each pair of

different symbols $f/n, g/m \in \tau$ with $n, m \geq 0$:

$$\forall x_1 \dots \forall x_n \forall y_1 \dots \forall y_m (\neg(f(x_1, \dots, x_n) = g(y_1, \dots, y_m))) \quad (2.13)$$

Notice that n and m may be zero. For each $f/n \in \tau$ with $n > 0$:

$$\forall x_1 \dots \forall x_n \forall y_1 \dots \forall y_n (f(x_1, \dots, x_n) = f(y_1, \dots, y_n) \Rightarrow x_1 = y_1 \wedge \dots \wedge x_n = y_n) \quad (2.14)$$

If τ is a finite set of object symbols, this definition collapses to the definition of UNA given in subsection 2.3.2.

This axiom(s) has been called the **unique name axiom for τ** , although this is a slight abuse of terminology. The abuse is in the fact that in some contexts, this groupwise injectivity axiom is strictly stronger than the original idea of UNA, which is that different terms should represent different entities.

Example 2.6.1

Take $\Sigma = \{\mathbf{nat}, 0 : \mathbf{nat}, S : \mathbf{nat} \rightarrow \mathbf{nat}\}$ and $\tau = \{0, S\}$. Then $\text{UNA}(\tau)$ is:

$$\begin{aligned} &\forall n (\neg(0 = S(n))) \\ &\forall n \forall m (S(n) = S(m) \Rightarrow n = m) \end{aligned}$$

In the following structure $\mathfrak{A}$, all ground terms over τ have a different value but $\text{UNA}(\tau)$ is not satisfied. Indeed, $S^{\mathfrak{A}}$ is not injective since $S^{\mathfrak{A}}(a) = S^{\mathfrak{A}}(b)$. Notice that a and b are not represented by terms over τ .

- $\mathbf{nat}^{\mathfrak{A}} = \mathbb{N} \cup \{a, b\}$.
- $0^{\mathfrak{A}} = 0 \in \mathbb{N}$
- $S^{\mathfrak{A}} = \{(n, n+1) | n \in \mathbb{N}\} \cup \{(a, b), (b, b)\}$

Proposition 2.6.1

If $\mathfrak{A} \models \text{UNA}(\tau)$, then for every pair of different terms t, s over τ , it holds that $t^{\mathfrak{A}} \neq s^{\mathfrak{A}}$.

Proof. The proof is a beautiful application of the principle of **proof by infinite descent**. Let $\mathfrak{A}$ be a structure satisfying the axioms (2.14) and (2.13) for τ . Assume towards contradiction that there exists syntactically different terms t, s over τ such that $t^{\mathfrak{A}} = s^{\mathfrak{A}}$. Since $\mathfrak{A}$ satisfies Eq. 2.13, the top function symbols of t and s must be the same. Hence, t and s are terms of form $f(t_1, \dots, t_n)$ and $f(s_1, \dots, s_n)$. Since t and s are different terms, it holds that $n > 0$ and there is at least one $i \leq n$ such that t_i and s_i are different terms. But $\mathfrak{A}$ satisfies Eq. 2.14, therefore it holds that $t_i^{\mathfrak{A}} = s_i^{\mathfrak{A}}$. Thus, any pair of different terms with the same value in $\mathfrak{A}$ possesses a pair of strict subterms that are different but have the same value. By iterating this argument, we can construct an infinite sequence of pairs of smaller and smaller terms that are different but have the same value in $\mathfrak{A}$. Such a sequence cannot exist. Contradiction. $\blacksquare$

The second proposition is an inductive definition of the main types of the constructors, to express that they consist of the entities constructed by the constructors in τ .

From now on, every type $\mathbf{t}$ of a vocabulary Σ is also a unary predicate of Σ . In any structure $\mathfrak{A}$, the value $\mathbf{t}^{\mathfrak{A}}$ is the set of entities of type $\mathbf{t}$ in $\mathfrak{A}$.

Definition 2.6.2

The $DCA_{FO(\cdot)}(\tau)$ is an (inductive) definition defining all the types $\mathbf{t}/1$ of the constructors $f : \mathbf{t}_1 \times \dots \times \mathbf{t}_n \rightarrow \mathbf{t}$ of τ . It consists of definitional rules of the form:

$$\forall x_1 \dots \forall x_n (\mathbf{t}(f(x_1, \dots, x_n)) \leftarrow \mathbf{t}_1(x_1) \wedge \dots \wedge \mathbf{t}_n(x_n)) \quad (2.15)$$

If f is an object symbol ($n = 0$), then the definitional rule is $\mathbf{t}(f) \leftarrow$.

The definition constructs the defined types using the constructors.

Example 2.6.2

Let again Σ be $\{\mathbf{nat}, 0 : \mathbf{nat}, S : \mathbf{nat} \rightarrow \mathbf{nat}\}$ and $\tau = \{0, S\}$. Then $DCA_{FO(\cdot)}(\tau)$ is

$$\left\{ \begin{array}{l} \mathbf{nat}(0) \leftarrow \\ \forall n (\mathbf{nat}(S(n)) \leftarrow \mathbf{nat}(n)) \end{array} \right\}$$

Inductive definitions can be expressed in SO. For definitions of types, it takes a slightly special form as illustrated in the following example.

Example 2.6.3

In the case of the inductive definition of the type $\mathbf{nat}$, the SO axiom has the following form:

$$\forall P ((P(0) \wedge \forall n [\mathbf{nat}](P(n) \Rightarrow P(S(n)))) \Rightarrow \forall n [\mathbf{nat}] P(n))$$

It states: for any set P that contains 0 and is closed under application of the successor function, all elements of the type $\mathbf{nat}$ belong to P .

Example 2.6.4

Let $\tau = \{C_1 : \mathbf{t}, \dots, C_n : \mathbf{t}\}$, a finite set of identifiers of type $\mathbf{t}$. For this case, $DCA(\tau)$ is the definition:

$$\left\{ \begin{array}{l} \mathbf{t}(C_1) \leftarrow \\ \dots \\ \mathbf{t}(C_n) \leftarrow \end{array} \right\}$$

This definition is equivalent with:

$$\forall x [\mathbf{t}](x = C_1 \vee \dots \vee x = C_n)$$

This is the typed version of the DCA for vocabularies of object symbols seen in subsection 2.3.2.

Example 2.6.5

Let Σ be $\{\mathbf{nat}, \mathbf{list}, 0 : \mathbf{nat}, 1 : \mathbf{nat}, 2 : \mathbf{nat}, \dots, \mathbf{nil} : \mathbf{list}, \mathbf{cons} : \mathbf{nat} \times \mathbf{list} \rightarrow \mathbf{list}\}$. We want to build a theory T that should entail the following natural facts: $\mathbf{member}(1, \mathbf{cons}(1, \mathbf{nil}))$,

$\neg member(1, nil), \neg member(1, cons(2, cons(3, nil)))$. This obviously cannot work if T has models $\mathfrak{A}$ in which $2^{\mathfrak{A}} = 1^{\mathfrak{A}}$, or $nil^{\mathfrak{A}} = cons(1, nil)^{\mathfrak{A}}$ or $cons(2, cons(3, nil))^{\mathfrak{A}} = cons(1, nil)^{\mathfrak{A}}$ (why not?). Hence, T should include UNA and DCA axioms for the types **nat** and **list**, to make sure that two different list terms have a different value.

Exercise 2.6.1 Write up the UNA and DCA axioms for the types **nat** and **list**.

Example 2.6.6

As an extension of section 2.2.8, we add a new operation $opList$ to the theory of mathematical group. This operation extends the binary group operation to lists of arbitrary length. The operation is defined in the natural inductive way: applied to the empty list, it returns the neutral element; applied to a list $cons(x, t)$ it returns the group element $(x \text{ op } y)$ where y is the result of the list operation on the tail t .

The vocabulary of the original theory was $\Sigma = \{op/2:, e/0:\}$ representing respectively the binary operation of the group and the neutral element. In the original theory, the domain consisted uniformly of group elements. Now, the domain needs to be split up in two types: **group** and **list**. The existing theory can stay as it is, except that the symbols and variables must be typed with **group**. The new type **list** is constructed from the set of constructors $\tau = \{nil : \mathbf{list}, cons : \mathbf{group} \times \mathbf{list} \rightarrow \mathbf{list}\}$. The theory of $UNA(\tau)$ and $DCA(\tau)$ is:

$$\begin{aligned} & \forall x \forall y (\neg (nil = cons(x, y))) \\ & \forall x \forall y \forall x1 \forall y1 (cons(x, y) = cons(x1, y1) \Rightarrow x = x1 \wedge y = y1) \\ & \left\{ \begin{array}{l} \mathbf{list}(nil) \leftarrow \\ \forall x \forall y (\mathbf{list}(cons(x, y)) \leftarrow \mathbf{group}(x) \wedge \mathbf{list}(y)) \end{array} \right\} \end{aligned}$$

The predicate $opList : \mathbf{list} \times \mathbf{group} \rightarrow \mathbb{B}$ is then defined as follows:

$$\left\{ \begin{array}{l} opList(nil, e) \leftarrow \\ \forall x \forall t \forall y (opList(cons(x, t), op(x, y)) \leftarrow opList(t, y)) \end{array} \right\}$$

The type **list is defined in terms of the type **group** which is not defined.** Since **group** is undefined, different models $\mathfrak{A}$ may have different values for **group**. This value can be *any* group. With exception of the neutral element symbol e , there are no terms to represent group elements. The value of **list** depends entirely on the value of **group**. So, in different structures with different values for **group**, there will be different values for the type **list**. Moreover, with exception of the list entities that contain only the neutral element, no list entities have a term representation. Hence, here is a case where DCA holds, but where the domain is **not** the set $\{t^{\mathfrak{A}} | t \in \mathbf{Terms}(\tau)\}$. **This is the consequence of the fact that the type **list** is defined in terms of the type **group** which is not defined, and for which UNA and DCA do not hold. Since models may contain group entities without term representation, they may contain list entities without term representation.**

Term and Herbrand structures So far, symbols and values were separated. In term structures, the domain is a collection $\mathbf{Terms}(\tau)$ of well-typed terms. Term structures can interpret also function symbols not in τ . A special case is the Herbrand interpretation.

Definition 2.6.3: Term and Herbrand structures

Given a set τ of object and function symbols, a **term structure constructed from** τ is a structure $\mathfrak{A}$ such that its universe $D_{\mathfrak{A}}$ is the set of terms over τ and each symbol of τ has the *trivial value*: for $C \in \tau$, $C^{\mathfrak{A}} = C$ and for a function symbol $f/n \in \tau$, $f^{\mathfrak{A}}$ maps tuples $(t_1, \dots, t_n)$ of terms over τ to the term $f(t_1, \dots, t_n)$.

A structure $\mathfrak{A}$ is a **Herbrand structure** if it is a term structure constructed from the set of all object and function symbols interpreted by $\mathfrak{A}$.

The use of UNA and DCA does not guarantee that every entity in the universe or in a defined type has a term representation. This was shown in Example 2.6.6, where the type **list** was defined in terms of the undefined type **group**. However, there are many useful cases where UNA and DCA do guarantee that all entities have a unique term representation.

For a collection τ of constructor symbols, $DCA(\tau)$ contains a definition defining the result types $\mathbf{t}$ of all symbols in τ using rules of the form:

$$\forall x_1 \dots \forall x_n (\mathbf{t}(f(x_1, \dots, x_n)) \leftarrow \mathbf{t}_1(x_1) \wedge \dots \wedge \mathbf{t}_n(x_n))$$

The parameters of this definition are the types $\mathbf{t}_i$ that are argument types but not result types of symbols of τ .

For the next theorem, we assume that $DCA(\tau)$ has no parameter symbols. That is, if a type is defined in terms of a parameter type, then also the parameter type is defined. In this case, every domain element of the defined types is represented by a term over τ . This follows from the following theorem.

Theorem 2.6.1

A Σ -structure $\mathfrak{A}$ satisfies $UNA(\tau)$ and $DCA(\tau)$ iff $\mathfrak{A}$ is isomorphic to a term structure with universe constructed from τ .

Exercise 2.6.2 *Prove this theorem with the following hints. For simplicity, assume there is only one defined type. The direction ($\Leftarrow$) is easy. For the direction ($\Rightarrow$) consider the function $b : \mathbf{Terms}(\tau) \rightarrow D_{\mathfrak{A}}$ that maps a term t to $t^{\mathfrak{A}}$. This is a well-defined total function. First prove that since $\mathfrak{A}$ satisfies $UNA(\tau)$, b is injective. Then prove that since $\mathfrak{A}$ satisfies $DCA(\tau)$, b is surjective. Together, we obtain that b is bijective. Next construct a term structure $\mathfrak{A}'$ with universe constructed from τ and for each symbol σ interpreted by $\mathfrak{A}$, define $\sigma^{\mathfrak{A}'}$ as $b^{-1}(\sigma^{\mathfrak{A}})$. Prove that $\mathfrak{A}'$ is isomorphic to $\mathfrak{A}$.*

2.6.2 Databases from the logical point of view

A database DB is a collection of tables consisting of cells filled with strings.

Example 2.6.7

Consider a simplified student database DB storing courses, prerequisites, lecturers, students and grades.

Instructor		Enrolled		Grade		
Ray	CS230	Jill	CS230	Jill	CS230	A
Hec	CS230	Jack	CS230	Jack	CS230	C
Sue	M100	Flo	CS230	Flo	CS230	AA
Sue	M200	Jill	M100	Jill	M100	D
PassingGrade		Flo	M100	Flo	M100	A
AAA		Jill	M200	Flo	M200	AAA
AA		Flo	M200			
A		Prerequ				
B		CS230	M100			
C		M100	M200			

In typical database applications, the database is a modelling of the actual state of affairs of the application domain.

Exercise 2.6.3 *Test yourself. Before reading on, think about a database as a modelling of the application domain. What is the nature of its informal interpretation $\mathcal{I}$? What information is explicit and what implicatures are there? How would you express the database as a theory (implicatures included)?*

The names of tables P with n columns represent relational concepts $\mathcal{I}(P)$ in the application domain. They correspond to predicate symbols P/n . The strings s in the columns represent individual entities $\mathcal{I}(s)$ in the application domain. They correspond to object symbols, in particular to identifiers, not concepts. For example $\mathcal{I}(\text{Instructor})$ is the relational concept between lecturers and the courses they teach; $\mathcal{I}(\text{Sue})$ is a specific lecturer of the faculty, who teaches two courses with code M100 and M200.

A database possesses three common implicatures: (1) the implicature that object symbols are identifiers; database managers choose a set τ of object symbols as unique identifiers of entities of the application domain. This is formalised by the unique name axioms $\text{UNA}(\tau)$. (2) The implicature that all entities of the application domain have been named by a symbol. This is formalised by $\text{DCA}(\tau)$. (3) The implicature that each table is an exhaustive enumeration of the value of the represented concept. This is formalised by expressing tables as FO explicit definitions or FO(ID) definitions.

Given is a database DB with vocabulary Σ_{DB} including the finite set τ_{DB} of object symbols.

For simplicity, we take the untyped setting (databases are untyped).

Definition 2.6.4

Given a database DB with vocabulary Σ_{DB} including the finite set τ_{DB} of object symbols, $\text{Th}(DB)$ is the FO theory consisting of $\text{UNA}(\tau_{DB})$, $\text{DCA}_{FO}(\tau_{DB})$ and the explicit definitions expressing the table of each table predicate P/n in DB .

Exercise 2.6.4 *For Example 2.6.7, write down $\text{UNA}(\tau_{DB})$, $\text{DCA}_{FO}(\tau_{DB})$ and the explicit definition for Prerequ .*

Exercise 2.6.5 *In the context of the example database of Example 2.6.7, think of a few queries such that the answers returned by a contemporary database system depend on the implicatures expressed in $\text{Th}(DB)$. That is, if these implicatures would not be assumed, the answers would be wrong. Explain where exactly the implicatures are used.*

The closest semantic object in logic to a database DB is the Herbrand structure $\mathfrak{A}_{DB}$ with universe $D_{\mathfrak{A}_{DB}} = \tau_{DB}$, and with value $P^{\mathfrak{A}}$ equal to the set of tuples in the table of P .

Theorem 2.6.2

For every Σ_{DB} -structure $\mathfrak{A}$, $\mathfrak{A} \models \text{Th}(DB)$ iff $\mathfrak{A}$ is isomorphic to the Herbrand structure $\mathfrak{A}_{DB}$.

Proof. ($\Leftarrow$) Do this yourself: prove that $\mathfrak{A}_{DB}$ satisfies $\text{Th}(DB)$ and then prove that each structure isomorphic to $\mathfrak{A}_{DB}$ satisfies $\text{Th}(DB)$.

($\Rightarrow$) Every model $\mathfrak{A}$ of $\text{Th}(DB)$ is isomorphic to a term structure $\mathfrak{A}'$ with universe τ_{DB} (Theorem 2.6.1). The structure $\mathfrak{A}'$ is also a model of $\text{Th}(DB)$ and satisfies the explicit definition of P/n . This means that $P^{\mathfrak{A}'}$ is the set of tuples in the table of P/n . (Prove this!) ■

All models are isomorphic, hence $\text{Th}(DB)$ is a **categorical theory** in FO that identifies the unique state of affairs of the application domain. A consequence is that questions about the database application do not need to be solved using deduction inference applied to $\text{Th}(DB)$, but can be answered using evaluation inference in $\mathfrak{A}_{DB}$. The following proposition is a special case of Proposition 1.3.3.

Proposition 2.6.2

For each sentence φ over Σ_{DB} , $\text{Th}(DB)$ entails φ iff $\mathfrak{A}_{DB} \models \varphi$.

Proof. A sentence φ over Σ_{DB} is true in $\mathfrak{A}_{DB}$ or is false in $\mathfrak{A}_{DB}$. All models of $\text{Th}(DB)$ are isomorphic to $\mathfrak{A}_{DB}$. The isomorphism Proposition 2.2.2 entails that φ is true in every model of $\text{Th}(DB)$, in which case this theory entails φ , or φ is false in every model of $\text{Th}(DB)$, in which case $\neg\varphi$ is true in all of them and $\text{Th}(DB)$ logically entails $\neg\varphi$. ■

In a sense, this is a theorem of immense practical utility, since it is at the basis of database technology. A user who wants to know whether some proposition φ is true in the current state of affairs of the database application, has basically a deductive inference problem: for input φ and T , the theory of the available information about the current state of affairs, the problem is to decide whether T entails φ . In general, deductive inference problems in FO are hard, undecidable even, but here T is categorical, its unique model is $\mathfrak{A}_{DB}$ (modulo isomorphism), and the deductive problem collapses to the evaluation inference problem with input $\mathfrak{A}_{DB}, \varphi$. This problem is efficiently solvable, even if DB (and hence $\text{Th}(DB)$) contains gigabytes of data.

Another consequence of the categoricity of $\text{Th}(DB)$ is that if $\text{Th}(DB) \not\models \varphi$, then $\text{Th}(DB) \models \neg\varphi$. That is, a database contains complete knowledge. That is rarely the case with FO theories. On the other side of the coin, databases are not suitable to express incomplete information. This is a well-known weakness of databases.

Exercise 2.6.6 (*easy, important*) Consider the following alternative attempt to express

$\text{Th}(DB)$, namely as the conjunction of all facts in the tables, i.e., as the sentence ϕ_{DB} :

$$\begin{aligned}\phi_{DB} = & (\text{Instructor}(\text{Ray}, \text{CS230}) \wedge \dots \wedge \\ & \text{PassingGrade}(\text{AAA}) \wedge \dots \wedge \\ & \text{Enrolled}(\text{Jill}, \text{CS230}) \wedge \dots \wedge \\ & \text{Prerequ}(\text{CS230}, \text{M100}) \wedge \dots \wedge \\ & \text{Grade}(\text{Jill}, \text{CS230}, \text{A}) \wedge \dots)\end{aligned}\tag{2.16}$$

Explain in what sense this representation is failing. Show an unintended model of this sentence. Use this model to show a few queries that cannot be solved with this theory. Hint: Example 2.2.10.

What are queries, declaratively? It could be easily overlooked that modelling is not only required for expressing information, but sometimes also for expressing problems or questions.

Take a query to a database in some declarative query language, say SQL. How could a query express information, given that its goal is to ask for information? It is absurd!

But consider this. When a user thinks of a query for the DB , he has in mind a certain informal composite concept $\mathcal{C}$ of which he wants to know the value in DB . E.g., the user wants to know all students having succeeded for all courses in which they were enrolled. Here, the concept $\mathcal{C}$ is “ x is a student having succeeded for all courses in which x is enrolled”. The user’s first act is a modelling act, namely to define $\mathcal{C}$ in terms of the concepts expressed in DB and to express this definition as a formal query $\mathcal{Q}$. This query is given to the database system to compute a value $\mathcal{Q}^{\mathfrak{A}_{DB}}$. And so, indeed a query expresses information, namely the definition of $\mathcal{C}$. As such, a query language (or a fragment of it) is a declarative language for modelling definitions of concepts.

The user expresses a formal definition $\mathcal{Q}$ in the query language of his choice. In industry, this is normally SQL. In the theory of databases, often set expressions are used. For our running example:

$$\mathcal{Q} = \{s : \forall c(\text{Enrolled}(s, c) \Rightarrow \exists g(\text{Grade}(s, c, g) \wedge \text{PassingGrade}(g)))\}$$

Alternatively, the user might use $\text{FO}(\cdot)$ definitions. A new predicate $Q/1$ can be introduced with informal interpretation $\mathcal{C}$ (i.e., $\mathcal{I}(Q) = \mathcal{C}$) and a $\text{FO}(\cdot)$ -definition Δ is developed:

$$\left\{ \forall s(Q(s) \leftarrow \forall c(\text{Enrolled}(s, c) \Rightarrow \exists g(\text{Grade}(s, c, g) \wedge \text{PassingGrade}(g)))) \right\}$$

With this representation, the answer to the query is the value $\mathcal{Q}^{\mathfrak{A}'}$ in the unique expansion $\mathfrak{A}'$ of $\mathfrak{A}_{DB}$ that satisfies Δ . It is computed by definition Δ -expansion inference. An advantage of this method over using FO set expressions is that inductive queries can be posed. (Think of the query for ancestors, or descendants of a person, given a database storing the parent relation). This approach is implemented in the database query language Datalog, a sublanguage of Prolog used for querying. As such $\text{FO}(\cdot)$ explains the declarative nature of the query language Datalog.

Remark 2.6.1: $\text{Th}(DB)$ is not necessarily true

Recall an important fact about modelling: virtually all modellings are approximations of the true state of affairs. This is also often the case for $\text{Th}(DB)$. Users of a database often know that the current state of affairs is not exactly $\mathcal{I}(I_{DB})$. They may know or suspect that some entities are missing in the database (violating DCA); that some tables

are incomplete (in which case the tables are not *exhaustive* enumerations) and that some tables contain plain errors (e.g., some grades for the course M100 in *Grade* might be wrong). Users use this meta-knowledge as a meta-filter to assess the correctness or completeness of answers returned by the database system. E.g., a user may query for the results of students for the course CS230 while knowing that the results for this course are correct but potentially incomplete. This is similar to physicists that are aware that Newtons theory of gravity is not entirely correct and that assess the correctness of predictions by this theory, e.g., by verifying whether there are important “relativistic effects”.^a

Thus, $Theo(DB)$ is not *true* in the absolute sense of the word and it does not contain the meta-knowledge of users about incompleteness and errors. However, it tells the user how the database system itself “thinks” about the current state of affairs, and **this allows the user to understand how to express his questions as formal queries and to understand the answers returned by the database system.** It is then up to the user to apply the meta-knowledge and assess the quality of the answers.

Errors in a modelling do not necessarily mean the end of it. If humanity would stop using all false scientific theories and other modellings, hardly any would be left and we would be back in the stone age.

^aFor example, GPS would be entirely untrustworthy if relativistic effects were not accounted for.

2.6.3 Peano arithmetic

The structure $\mathbb{N}$ of the natural numbers is a fundamental concept in mathematics and formal sciences. It interprets (at least) $\Sigma_{\mathbb{N}} = \{0, S/1:, +/2:, \times/2:\}$. With slight abuse of notation, we will use $\mathbb{N}$ to denote the set of natural numbers, and the structure.

- $D_{\mathbb{N}}$ is the set $\mathbb{N}$ of natural numbers.
- $0^{\mathbb{N}} = 0 \in \mathbb{N}$: the value of the numeral 0 is the natural number 0.
- $S^{\mathbb{N}}$ is the function $\mathbb{N} \rightarrow \mathbb{N} : n \mapsto n + 1$, called the *successor function*. In set theoretic terms, it is the set $\{(n, n + 1) | n \in \mathbb{N}\}$.
- $+^{\mathbb{N}}, \times^{\mathbb{N}}$: the well-known sum and product functions on $\mathbb{N}$. They are the sets $\{((n, m), n + m) | n, m \in \mathbb{N}\}$ and $\{((n, m), n \times m) | n, m \in \mathbb{N}\}$.

Can we express the concept of natural number in logic, and *get the models right*? I.e., can we build a theory $\text{Th}(\mathbb{N})$ such that all its models are isomorphic to $\mathbb{N}$? This was studied for the first time by the Italian mathematician Giuseppe Peano in the paper “*Arithmetices principia, nova methodo exposita*” in 1889.

Peano’s goal was similar as the goal of axiomatising a database by $\text{Th}(DB)$. His theory has a similar layout as $\text{Th}(DB)$. The constructors of $\mathbb{N}$ are the symbols in $\tau_{\mathbb{N}} = \{0, S\}$. The set $\tau_{\mathbb{N}}$ is the minimal vocabulary τ for which $\text{Terms}(\tau)$ is infinite. Peano’s theory consists of axioms corresponding to $\text{UNA}(\tau_{\mathbb{N}})$, $\text{DCA}(\tau_{\mathbb{N}})$ and FO axioms serving to define $+$ and $\times$.

- Peano expressed that 0 and S are “groupwise injective” :

$$\forall n \neg (0 = S(n)). \quad (2.17)$$

$$\forall n \forall m (S(n) = S(m) \Rightarrow n = m) \quad (2.18)$$

Verify that this is $\text{UNA}(\tau_{\mathbb{N}})$.

- Peano expressed an axiom in Second-order logic (SO) called the **induction axiom**.

$$\forall P(P(0) \wedge \forall n(P(n) \Rightarrow P(S(n))) \Rightarrow \forall n P(n))$$

In words, each subset of the universe that contains 0 and is closed under S , contains all elements of the universe. Verify that this is $DCA_{SO}(\tau_{\mathbb{N}})$, the domain closure axiom expressed in SO.

- Peano “defined” the function $+$ by a set of two FO axioms:

$$\forall m(0 + m = m) \quad (2.19)$$

$$\forall n \forall m(S(n) + m = S(n + m)) \quad (2.20)$$

This has the flavour of an inductive definition of $+$. It is expressed through FO equalities.

- Likewise, Peano “defined” the function $\times$ by a set of two FO axioms:

$$\forall n(0 \times n = 0) \quad (2.21)$$

$$\forall n \forall m(S(n) \times m = m + (n \times m)) \quad (2.22)$$

Idem.

The above axioms, including the SO axiom, are called Peano’s axioms. We denote this theory PA^{SO} . It consists of six FO axioms and one SO axiom. Below, a term of the form $S(S(\dots(S(0))\dots))$ with n occurrences of the symbol S is denoted S^n .

Theorem 2.6.3

$\mathfrak{A} \models PA^{SO}$ iff $\mathfrak{A}$ is isomorphic to $\mathbb{N}$.

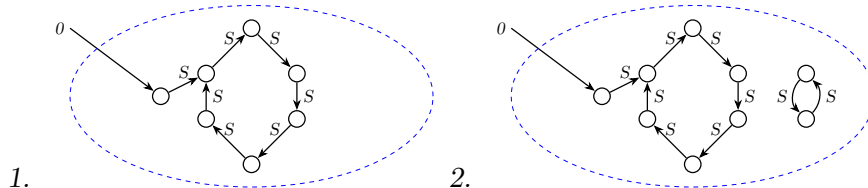
Proof. It follows from Theorem 2.6.1 that a structure $\mathfrak{A}$ that satisfies PA^{SO} is isomorphic to a term structure for $\tau_{\mathbb{N}}$. Take the term structure $\mathfrak{A}$ where $+^{\mathfrak{A}}$ is the function that maps pairs (S^n, S^m) to S^{n+m} and $\times^{\mathfrak{A}}$ is the function that maps pairs (S^n, S^m) to $S^{n \times m}$. We should prove that $\mathfrak{A}$ is the only term structure satisfying the axioms of $+$ and $\times$. Do this yourself; both proofs are by induction on n . Thus, all models of PA^{SO} are isomorphic to this term structure $\mathfrak{A}$. In the final step, we should prove that $\mathbb{N}$ satisfies PA^{SO} . Do this yourself. Thus, all models are isomorphic to $\mathbb{N}$ (by transitivity of isomorphism). ■

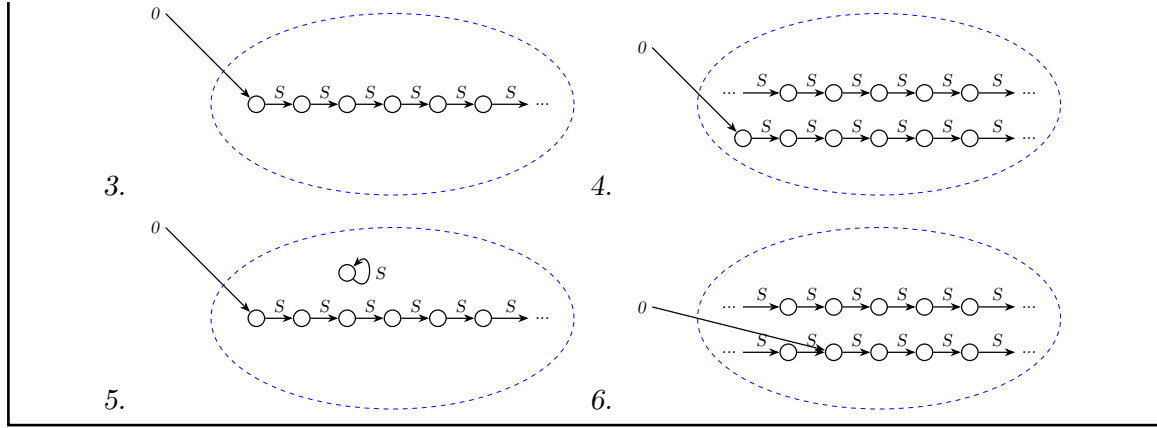
Thus, PA^{SO} is a categorical theory.

In $FO(\cdot)$, the SO induction axiom can be expressed as (using the type **nat**):

$$\left\{ \begin{array}{l} \text{nat}(0) \leftarrow \\ \forall n(\text{nat}(S(n)) \leftarrow \text{nat}(n)) \end{array} \right\} \quad (2.23)$$

Exercise 2.6.7 In which of the following structures is the $DCA(\tau_{\mathbb{N}})$ satisfied? In which of them is $UNA(\tau_{\mathbb{N}})$ satisfied?





The FO induction schema Peano's theory is an SO theory due to the induction axiom:

$$\forall P(P(0) \wedge \forall n(P(n) \Rightarrow P(S(n))) \Rightarrow \forall n P(n))$$

This property states that a subset P of the universe that contains 0 and is closed under S , contains all elements of the universe.

For certain applications, in particular for theorem proving, it is desirable to have a FO version of this theory. Indeed, SO has no sound and complete proof system, while FO has one (in fact, it has many of them). Proof systems will be discussed in Chapter 6. A first order version of Peano's theory can be obtained by replacing the induction axiom by the so called *induction schema*. The idea is simple : rather than stating the induction property for all subsets P of the universe (quantifying $P/1$ universally), the induction property is stated only for subsets of the universe that can be expressed using some FO formula $\varphi[x]$.

Definition 2.6.5: Induction schema

The *induction schema* is the countably infinite set of formulas:

$$(\varphi[0] \wedge (\forall x(\varphi[x] \Rightarrow \varphi[S(x)]))) \Rightarrow \forall x \varphi[x]$$

where $\varphi[x]$ is any formula with free variable x .

The FO theory consisting of the six FO axioms augmented with the induction schema is countably infinite and is called *Peano arithmetic*. It will be denoted PA^{FO} .

The induction schema, although infinite, should be expected to be weaker than the induction axiom. Indeed, an infinite universe has *non-countably* many subsets P . The induction axiom states the induction property for all of them ($\forall P$). On the other hand, the induction schema states the induction property only for a countable set of subsets of the universe, those that are expressible with a formula $\varphi[x]$ (more precisely, the set $\{x : \varphi[x]\}$). That is weaker.

In Chapter 6 it is shown that Peano arithmetic PA^{FO} does not get the models right. It has non-standard models in which $DCA(\tau_{\mathbb{N}})$ is not satisfied. Hence, non-standard models are not isomorphic to $\mathbb{N}$. The universe in these structures contains many more objects than the natural numbers. The proposition about $\mathbb{N}$ that cannot be expressed in FO is “*the set of natural numbers consists of $0, 1, 2, \dots, n, n+1, \dots$ and nothing else*”. Stated in other words, it is the property that each natural number can be reached from 0 by iterated application of the successor function.⁹

⁹I remember the moment, long time ago, when I understood this for the first time. Till that time, I had held FO to be the universal language suitable to express all mathematical thoughts in a precise way. I vividly remember my disappointment that such a simple property could not be expressed in FO.

To the defense of the induction schema and of PA^{FO} , most properties of interest about the natural numbers can be proven from PA^{FO} . The instances of the induction schema correspond to the principle of *proof by induction*. Thus, each time a theorem is proven by induction, an instance of the induction schema is used.

In modelling mathematical structures, mathematical logicians using FO will “try” to build theories that get the models right. E.g., the FO logic theory T_{group} of group of Section 2.2.8 succeeds in this, since its models are exactly the groups. However, most of the time it is impossible to get the models right. Rather than to move to a stronger logic such as SO, it has been mainstream in mathematical logic to build (infinite) FO theories that have “non-standard models” but are strong enough to prove most properties of interest. This is also the case with the foundational Zermelo-Fraenkel Set Theory, considered by many to provide the foundation for mathematics. It seems fair to describe the methodology of mathematical logic as the method of building theories strong enough to prove theorems of interest. Too bad if they do not get the models right.

In formal science, arguably the guiding rule for modelling is to get the models right. It is the same in some branches in computational logic. E.g., in Constraint programming, the problem solving method is to encode application knowledge as a theory called the constraint theory such that solutions (called satisfying assignments) are abstractions of states of affairs satisfying the theory. If such a theory has “non-standard models”, i.e., models abstracting impossible states of affairs, a constraint system will produce an erroneous answer. In general, a modelling to be used for solving different types of computational problems, needs to get the models right. The use of this methodology requires the use of more expressive logics than FO.

2.6.4 Logic Programming as a declarative language

Prolog is a declarative logic programming language. Initially, the declarative view on a Prolog program was that it was a set of material implications, called Horn clauses. Thus, from a declarative point of view, logic programming was seen as Horn clause logic, a sublogic of FO. It is still often explained that way. This view faltered in the second part of the seventies when the negation as failure inference rule was added to Prolog systems. Consider the member program:

```
member(X, [X|T]).
member(X, [_|T]):- member(X,T).
```

It was interpreted as the Horn theory:

$$\begin{aligned} &\forall x \forall t (member(x, [x|t])) \\ &\forall x \forall h \forall t (member(x, [h|t]) \Leftarrow member(x, t)) \end{aligned}$$

Now consider the following Prolog queries and their answers:

```
?- member(1, [2,3,4]).
no
?- not member(1, [2,3,4]).
yes
```

Of course, 1 is not a member of the list [2,3,4]. The problem is that the Horn theory does not entail $\neg member(1, [2,3,4])$. Hence, the Prolog system returns a **logical error**! One ended up in the uneasy and confusing situation that Prolog with negation as failure derived theoretically unsound answers which however were intuitively correct.

Exercise 2.6.8 [important, easy] Show an (undesirable) model of the member Horn theory in which $\text{member}(1, [2, 3, 4])$ is true. Hint: take the final structure as defined in Example 2.2.10.

Since then, the nature of the declarative logic underlying Prolog has been a topic of debate. Research was conducted for alternative formal and informal semantics of logic programs that provided a better explanation of the meaning of Prolog programs, one that could explain why its inferences are intuitively correct.

One declarative view that solves this problem is that a logic program is a $\text{FO}(\cdot)$ theory consisting of UNA + DCA applied to all its object and function symbols, and an $\text{FO}(\cdot)$ definition corresponding to the set of Prolog rules. This definition defines *all* predicate symbols that occur in the program. The UNA and DCA part formalises the programmers intuitions of function symbols as constructors of (recursive) data structures. The definition part interprets rules as definitional rules which entail the Horn clauses but are much stronger.

In this view, the member program corresponds to the $\text{FO}(\cdot)$ theory consisting of UNA and DCA and the following inductive $\text{FO}(\text{ID})$ definition:

$$\left\{ \begin{array}{l} \forall x \forall t (\text{member}(x, [x|t]) \leftarrow \mathbf{t}) \\ \forall x \forall h \forall t (\text{member}(x, [h|t]) \leftarrow \text{member}(x, t)) \end{array} \right\} \quad (2.24)$$

This is a categorical theory. All its models are isomorphic to a term structure $\mathfrak{A}$, even a Herbrand structure. The theory entails $\neg \text{member}(1, [2, 3, 4])$ since this formula is false in $\mathfrak{A}$. In this view, the negation of logic programming is standard objective negation. What is non-standard is the view of function symbols as constructors and the view of rules as definitional implications.

The following remarks hold:

- The informal view of logic programs as definitions was underlying early semantic work in the field, e.g., in [6]. But this work used the weak completion translation to FO. Inductive definitions cannot be expressed in FO.
- In [27], an important formal semantics called the *well-founded semantics* was proposed for logic programs, but no informal semantics. In various papers [11, 12, 13], my coauthors and I have argued that the corresponding informal semantics is a combination of UNA+DCA (treating function symbols as constructors) and the principle of inductive definition (allowing for negation in the condition). The formal semantics of definitions in $\text{FO}(\cdot)$ is an extension of the well-founded semantics.
- Logic programs have a procedural interpretation. The SLDNF query inference algorithm induces a procedural interpretation of Prolog programs. Executing a query implements this specific form of inference: query inference. On the other hand, many forms of inference can be applied to a $\text{FO}(\cdot)$ theory, which prevents that a procedural interpretation for $\text{FO}(\cdot)$ can exist.
- Prolog extends logic programming with side effects based on the procedural interpretation. Programs containing such side effects, have no declarative informal semantics.
- $\text{FO}(\cdot)$ is the only successful integration of FO and logic programming. It is a modelling logic as defined in this course.
- In a logic program, all object and functions symbols are interpreted as constructors. In $\text{FO}(\cdot)$, one can choose.

- A logic program contains only one definition, and all predicates are defined in it. In $\text{FO}(\cdot)$, a theory can contain many definitions, even of the same concepts, and a definition can have parameters.
- Bodies of $\text{FO}(\cdot)$ definitions have a much richer syntax than in Logic Programming. E.g. FO rule bodies, universal quantifiers, aggregates. Definitions such as those in Example 2.4.16 (terminating nodes of a graph) and Example 2.4.20 (company control) cannot be expressed as Prolog programs.
- Prolog is untyped, $\text{FO}(\cdot)$ is typed.

There is currently no unified view on logic programming as a declarative logic. There are several reasons why so far the view of logic programs as a particular kind of $\text{FO}(\cdot)$ theory is not universally accepted.

- The concept of non-monotone inductive definition, with negation in the body, was not scientifically understood in the first decennia of logic programming. There was no scientific theory of non-monotone definitions that could be used to explain logic programs with negation.
- An influential trend in knowledge representation and in logic programming is common sense reasoning. Its objective is to explain KR languages in terms of common sense, and its adherents currently believe that mathematical language concepts such as inductive definitions do not belong to common sense.

The logic $\text{FO}(\cdot)$ is the result of efforts of myself, colleagues and PhD students to study inductive definitions, their relation to logic programming, and to extend logic programming to make it suitable for knowledge representation and problem solving. It aims to integrate the best of worlds in one formalism. E.g., FO , types, aggregates, identifiers, domain and type closure, definitions, ...

2.7 Important for the exam

The theoretical part of the exam consists of small and large questions.

Small questions: examples

- explain/correct some tricky logical formula (e.g., those from the pragmatics section);
- the difference between a definition and its material implications, or its FO predicate completion.
- evaluate a formula in a structure; show non-equivalence of two formulas by a structure (possible world analysis)
- explain the induction axiom in $\text{Th}(\text{NAT})$; what is its link with DCA? With the reachability problem? with inductive definitions? With HO ?
- what are the three components of $\text{Th}(\backslash\text{DB})$, the theory of databases? Explain why they are needed and what they express

Knowledge of semantics of FO and extensions of it

- values of symbols in FO and extensions of it : domains, tuples, relations, functions, the hierarchy of values

- structures: in graph notation, in symbolic notation, as in IDP;
- isomorphic structures

Knowledge of

- definitions of terms versus formulas
- definition of satisfaction relation
- definitions of the main semantical concepts (satisfiable, valid/tautological, contradictory, entailment, equivalence, categorical theories) (these concepts were introduced in Chapter 1)

Understanding of pragmatics:

- important implicatures (UNA, DCA, definitions)
- the role of UNA, DCA and definitions to cope with it.

Understanding of extensions of FO and being able to use them (no formal definitions): types, arithmetic, aggregates, definitions

Understanding of the SO induction axiom and the FO induction schema

Understanding of the difference between an inductive definition and sets of FO implications or FO equivalences.

understanding of the relation between

Knowledge of Th(DB): the theory of a database

Understanding of the Peano axioms; knowledge of the induction axiom and the induction schema and the relation to the domain closure axiom.

Understanding of Prolog as FO(.); what is the problem of Horn clauses.

Chapter 3

Modelling dynamic systems in classical logic

3.1 Introduction: Modelling dynamic systems in LTC

So far, only static applications were modelled in $\text{FO}(\cdot)$. In this chapter, we use $\text{FO}(\cdot)$ for modelling dynamic worlds with actions and dynamic relations and functions that change over time.

Many special purpose logics are designed to specify dynamic worlds. E.g., the logics of the next chapter: Event-B, CTL, and LTL. Time is implicit in these logics. In $\text{FO}(\cdot)$, there is no support for time, hence time must be made explicit, as a parameter of dynamic predicates and functions. In the AI community, this is sometimes called the *method of temporal arguments (MTA)*.

This method has advantages and disadvantages. The main advantage is didactic: by using a logic without built-in support for time, the laws of time need to be made explicit, yielding better insight in what these laws are. A second advantage is flexibility: since time and laws of time are explicit in the ontology and the logical axioms, they can be easily modified. In temporal logics, certain versions of laws of time are built-in properties of the logic and cannot be changed. Thus, for the purpose of insight, using a universal logic like $\text{FO}(\cdot)$ for a first course on dynamic worlds and temporal reasoning is the best option. Of course there are some disadvantages as well: theories are more verbose, and the reasoning tools are less efficient.

Modelling dynamic domains in AI The problem of modelling temporal domains is historically an important research topic in knowledge representation (KR) and AI. Different methods have been developed, such as the *situation calculus* and the *event calculus*. The method presented in this chapter borrows from both: it is called the *linear-time calculus (LTC)*.

An LTC theory is a typed $\text{FO}(\cdot)$ theory formalising a dynamic domain. The time type T is interpreted by the set $\mathbb{N}$. This results in a *linear time topology*: for each pair t_1, t_2 of time points, it holds that $t_1 = t_2 \vee t_1 < t_2 \vee t_2 < t_1$.

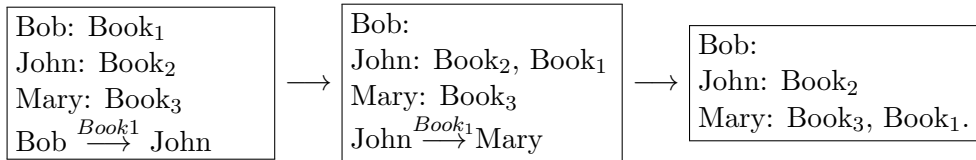
In other temporal modelling languages, time is modelled as a *tree* of times or states. One state may have multiple successor states, representing alternative ways in which the world may evolve. Each branch in this tree represents an alternative potential history. Such a time topology is called *branching time*. A well-known formalism of branching time is the

situation calculus. CTL, seen in the next chapter, also supports branching time: one can use it to express that while an elevator actually moves between the 2nd and the 5th floor, it always has the *possibility* to move to the ground floor (the elevator has the possibility but never uses it). Such propositions cannot be expressed in LTC nor in other linear time logics.

The first and main difficulty in modelling dynamic worlds lies in modelling how actions change the world.

Example 3.1.1

As a running example, the following temporal domain contains people and books. The only temporal relation is of a person owning a book. The only sort of action is that of a person giving away a book to another person. The concrete scenario considered here is the one where initially, Bob, John and Mary own book 1, respectively book 2 and book 3. Bob gives book 1 to John. Then John gives book 1 to Mary. This scenario results in a sequence of state transitions:



Actions such as *giving books away* are sometimes seen as *state transformers*.

Definition 3.1.1

An LTC vocabulary Σ_a consists of

- A set of types, including T (time).
- A set of *static* predicates and function symbols, without argument of type T.
- A set of *dynamic* predicate symbols that have exactly one argument of T.
- Interpreted symbols¹ of T:
 1. numerals $0, 1, 2, \dots : T$
 2. successor function $S(T) : T$
 3. addition $T + T : T$

For now, dynamic function symbols are not considered.

Example 3.1.2

The LTC vocabulary of the running example:

- Types: *Person*, *Book*, T
- Static:
 - *Bob*, *Mary*, *John* : *Person*

¹Beware: some temporal inference methods of the IDP system expect you to use **Start** instead of 0, and the successor time function **Next(Time):Time** instead of +1 or S(·).

- $Book_1, Book_2, Book_3 : Book$
- An example of a static predicate in this context: $HasAuthor(Book, Person)$.
- Dynamic:
 - $Owns(Person, Book, T)$: $Owns(p, b, t)$ means “person p owns book b at time t ”.
 - $Gives(Person, Book, Person, T)$: $Gives(g, b, r, t)$ means that “person g (the giver) gives book b to r (the receiver) at time t ”.

Dynamic predicates can often be categorized as:

- *Action* predicates. E.g., giving an object to some person, moving from one place to another, putting one object on top of another ...
- *Fluent* predicates represent state. E.g., **Owns**, **Location**, **On**, **Off**, **Running** ...

In the LTC, we do not make an explicit distinction. However, we will see that they are often axiomatized in a distinctive way.

Some examples of expressions

- *Timed axioms*. E.g.,

$$\begin{aligned} &Owns(Bob, Book_1, 0) \\ &Gives(John, Book_1, Mary, 1) \end{aligned}$$

- *Action preconditions* express necessary conditions for actions to happen:

$$\forall t \forall g \forall b \forall r (Gives(g, b, r, t) \Rightarrow Owns(g, b, t))$$

- *Action concurrency axioms* express what actions cannot happen simultaneously:

$$\forall t \forall g \forall b \forall r_1 \forall r_2 (Gives(g, b, r_1, t) \wedge Gives(g, b, r_2, t) \Rightarrow r_1 = r_2)$$

- *Effect axioms* express the effect of actions.

$$\begin{aligned} &\forall t \forall g \forall b \forall r (Gives(g, b, r, t) \Rightarrow Owns(r, b, t + 1)) \\ &\forall t \forall g \forall b \forall r (Gives(g, b, r, t) \Rightarrow \neg Owns(g, b, t + 1)) \end{aligned}$$

Example 3.1.3

A first attempt at formalising the book example:

$$\begin{aligned} &Owns(Bob, Book_1, 0). \\ &Owns(John, Book_2, 0). \\ &Owns(Mary, Book_3, 0). \\ &\forall g \forall b \forall r \forall t (Gives(g, b, r, t) \Rightarrow Owns(g, b, t)) \\ &\forall g \forall b \forall r_1 \forall r_2 \forall t (Gives(g, b, r_1, t) \wedge Gives(g, b, r_2, t) \Rightarrow r_1 = r_2) \\ &\forall g \forall b \forall r \forall t (Gives(g, b, r, t) \Rightarrow Owns(r, b, t + 1)) \\ &\forall g \forall b \forall r \forall t (Gives(g, b, r, t) \Rightarrow \neg Owns(g, b, t + 1)) \\ &Gives(Bob, Book_1, John, 0) \wedge Gives(John, Book_1, Mary, 1) \end{aligned}$$

Is this theory a precise description of the scenario? **Not even close.**

What is missing?

- UNA, DCA for $Person, Book$. Why these axioms are important, also here, was ex-

plained in the previous chapter.

Exercise 3.1.1 *Think of possible worlds in which UNA, DCA are not satisfied and how they differ from the intended world. Think of propositions true in the scenario but false in models of the theory that do not satisfy these axioms. E.g., what if Bob = John? What if there is another book?*

- There is no precise definition of the initial state nor of the set of actions that happen. The scenario possesses exact knowledge of the initial state and which events happen, but this knowledge is not expressed.
 - This theory does not express that, initially, Bob only owns $Book_1$, John only $Book_2$, Hence this theory cannot entail that $\neg Owns(Bob, Book_3, 0)$ or $\neg \exists b Owns(Bob, b, 2)$.
 - Likewise, it is not expressed that there are only two events. This theory does not exclude $Gives(Mary, Book_3, John, 1)$. Therefore, it does not entail $Owns(Mary, Book_3, 2)$.

This problem can be solved with the methods of the previous chapter, by using definitions of $Owns/3$ and $Gives/4$. However, in the case of $Owns/3$ there is a catch: we can define its initial state at time 0, but not (easily) the later states.

- The theory does not represent that the only effect of giving a book away is that the giver loses ownership and the receiver gains ownership of the book.

We need to add *inertia* axioms, to express what is not affected by the actions in successive states.

$$\begin{aligned} \forall t \forall g \forall b \forall r \forall p \forall b_1 (Owns(p, b_1, t) \wedge Gives(g, b, r, t) \wedge \neg(g = p \wedge b = b_1) \\ \Rightarrow Owns(p, b_1, t + 1)) \\ \forall t \forall g \forall b \forall r \forall p \forall b_1 (\neg Owns(p, b_1, t) \wedge Gives(g, b, r, t) \wedge \neg(g = p \wedge b = b_1) \\ \Rightarrow \neg Owns(p, b_1, t + 1)) \end{aligned}$$

But what if there are multiple fluents and actions? For every combination of an action and a fluent that is not affected by the action, it seems that an inertia axiom of this kind is required. But typically, an action affects only very few fluents. What if in a complex domain, there are many tens, hundreds of actions and fluents? This is too complex! We need to find a better way.

Exercise 3.1.2 ^aApply the IDP system on the given (incomplete) theory to verify the above claims about its incompleteness.

^a <https://idp.cs.kuleuven.be/idp/?present=Book1>

3.2 A historical note on the frame problem in early AI

In 1955, John McCarthy, Marvin Minsky, Nathaniel Rochester and Claude Shannon submitted a proposal for organising the Dartmouth summer project. This proposal is credited with introducing the term “artificial intelligence”. The proposal was accepted and the Dartmouth summer project in 1956 is seen by many as the start of AI research.

In a paper in 1963, McCarthy presented an initial version of the situation calculus in FO. A few years later in 1969, in a seminal paper [22], McCarthy & Hayes pointed to several problems with this early situation calculus and with FO in general. They observed that it is

fairly easy to represent the effects of actions in FO but that this does not suffice (as we saw in the book example). The non-effects of actions should also be described, they noted: the properties that remain unchanged when the action occurs. Such axioms were called *inertia axioms*. Confer the axioms in the running example in the previous section, e.g.,

$$\begin{aligned} \forall t \forall g \forall b \forall r \forall p \forall b_1 (& \text{Owns}(p, b_1, t) \wedge \text{Gives}(g, b, r, t) \wedge \neg(g = p \wedge b = b_1) \\ & \Rightarrow \text{Owns}(p, b_1, t + 1)) \end{aligned}$$

This is not a simple axiom to understand or to write. Moreover, in a complex world, most actions affect only a small set of properties, hence, leave many properties unchanged. Thus, it seemed that a very large number of inertia axioms would be required in FO.

Worse even, McCarthy and Hayes expected that an AI theory about a complex domain would have to be updated many times with new dynamic properties and actions. Each subsequent update seems to require a growing number of inertia axioms for all new combinations of new or existing properties and actions, entirely opposed to the principle of modularity. Maintenance of such a theory seems unfeasible. McCarthy later called this the *elaboration intolerance problem*, and argued that elaboration tolerance is an essential property for knowledge representation.

Definition 3.2.1

(from John McCarthy (1998)) A formalism is *elaboration-tolerant* if it is convenient to modify a set of facts expressed in the formalism to take into account new phenomena or changed circumstances.

Early AI researchers such as McCarthy and Hayes thought about large scale AI, with complex logical theories to build AI systems. The complexity of the real world is huge due to the number of properties, actions, agents, the number of unexpected events and behaviours, mutual influences, ramifications, etc. McCarty and Hayes coined the problem of describing such complex dynamic worlds as *frame problem*. The problem of describing inertia rules is only a small part of the full frame problem.

The full frame problem remains unsolved, and is considered to be unsolvable by many. In this course, we focus on dynamic worlds of small dimensions. However, even for limited dynamic applications, the problem of formalising inertia is a difficult one that took several decades to be solved. For us, the frame problem is as follows.

Definition 3.2.2

The *frame problem* is the problem of expressing effects and non-effects of actions.

Between 1969 until the late 1980s, it was thought to be impossible to express inertia in a concise and elaboration-tolerant FO theory.

It took until around 1990 before a correct, elegant and general solution for the frame problem for situation calculus in FO was found. This was mainly the work of Ray Reiter. [24]

The method presented in this course is inspired by Reiter's method, but differs as follows:

- We use a linear-time calculus instead of situation calculus.
- We use FO(ID) definitions to express inertia and causal rules rather than FO. However, the completion of these definitions correspond to Reiter's solution.

John McCarthy (1927-2011)

- coined the term *artificial intelligence* with, a.o., Shannon and Minsky (1955);
- proposed a hypothetical computer program “the advice taker” that should contain a knowledge base of “common sense information” expressed in a suitable formal language (most likely a part of FO) and be able to infer conclusions;
- developed the first functional language Lisp (1960);
- developed the first approach to formal modelling of dynamic systems: *Situation calculus* (1963);
- created the Stanford AI lab, educating many good AI researchers;
- published with Hayes (1969) a seminal paper on the frame problem in FO. This way, he became founding father of the field of Nonmonotonic Reasoning, a subfield of Knowledge Representation.
- one of the first to prove the correctness of a compiler (1970);
- Turing Award 1971;
- remained influential in the 80s and 90s.

**3.3 Expressing inertia through causal laws****Definition 3.3.1**

Inertia is the property that a fluent’s value does not change in time unless it is explicitly affected by an action.

We distinguish between two types of dynamic symbols:

- *Inertial* symbols: its values persist unless directly affected by an action. Fluent symbols typically have inertia. E.g.: **Owns/3**.
- *Non-inertial* symbols: its values tend not to persist in a next state. Action predicates typically do not have inertia. E.g.: **Gives/4**.

Most fluents are inertial. Most actions have relatively few effects, which tend to be easy to understand and relatively easy to represent. E.g.,

$$\begin{aligned} \forall t \forall g \forall b \forall r (Gives(g, b, r, t) \Rightarrow Owns(r, b, t + 1)) \\ \forall t \forall g \forall b \forall r (Gives(g, b, r, t) \Rightarrow \neg Owns(g, b, t + 1)) \end{aligned}$$

If we now want to add other actions, how can we represent the inertia of fluents like **Owns/3** – even if they have nothing to do with it? We hope for compact representations, in which we can focus on the actions’ effects and get the inertia for free.

Causality When we express:

$$\begin{aligned} \forall t \forall g \forall b \forall r (Gives(g, b, r, t) \Rightarrow Owns(r, b, t + 1)) \\ \forall t \forall g \forall b \forall r (Gives(g, b, r, t) \Rightarrow \neg Owns(g, b, t + 1)) \end{aligned}$$

what we really want to say is this:

$$\begin{aligned} \forall t \forall g \forall b \forall r (Gives(g, b, r, t) \text{ “causes” } Owns(r, b)) \\ \forall t \forall g \forall b \forall r (Gives(g, b, r, t) \text{ “causes” } \neg Owns(g, b)) \end{aligned}$$

In natural language, we frequently use conditionals to express that something causes something else. However, to say that x *causes* y is not the same as the material implication $x \Rightarrow y$! Causal conditionals are yet another type of conditional. The question is how we can express that x *causes* y in $\text{FO}(\cdot)$.

The key is to specify exactly what the effects of actions are, that is, what the changes are that they bring about: what fluents they cause to become true and what fluents they cause to become false. We can then define the fluents at every state point in terms of these effects in the previous state.

A solution for the frame problem To specify what causal effects take place, we introduce for each fluent P two new auxiliary *causality* predicates, to express causes to make P true, and causes to make P false. We specify the existing effects in a precise way by *defining* these auxiliary symbols. Then we specify the inertial fluents by *defining* its value in a state by defining it in terms of its previous state and the effects that happened in the previous state.

Per n -ary inertial predicate $P(\bar{x}, t)$, we introduce three new predicates with the following intended interpretation:

- $I_P(\bar{x})$: Initially, at $t = 0$, $P(\bar{x}, t)$ is true.
- $CT_P(\bar{x}, t)$: At t , $P(\bar{x}, t)$ is Caused to become True
- $CF_P(\bar{x}, t)$: At t , $P(\bar{x}, t)$ is Caused to become False.

The core of an LTC theory is one large $\text{FO}(\cdot)$ definition Δ defined next.

Definition 3.3.2: LTC Definition

The $\text{FO}(\cdot)$ definition Δ defines every inertial fluent P/n and its causality predicates $CT_P/n, CF_P/n$ by simultaneous induction on the standard order on time points. The rules are of the following kind:

$$\begin{aligned} \forall \bar{x} (P(\bar{x}, 0) &\leftarrow I_P(\bar{x})) \\ \forall \bar{x} \forall t (P(\bar{x}, t+1) &\leftarrow CT_P(\bar{x}, t)) \\ \forall \bar{x} \forall t (P(\bar{x}, t+1) &\leftarrow P(\bar{x}, t) \wedge \neg CF_P(\bar{x}, t)) \\ \dots \text{Effect rules as definitional rules for } CT_P, CF_P \dots \end{aligned}$$

The resulting definition Δ defines multiple predicates by simultaneous induction. It is a definition over an induction order, being the standard order of natural numbers on $\mathbb{T}$. The definition is nonmonotone due to the negative occurrence $\neg CF_P(\bar{x}, t)$ of a defined predicate in the third rule.² Recall the subtleties of the nonmonotone definition of $\mathfrak{A} \models \neg\varphi$ in terms of $\mathfrak{A} \models \varphi$.

The first 3 sorts of rules are called the *frame rules*:

- the base case defines P in the initial state;
- the 2nd inductive rule defines P to be true if there is a cause for it in the previous state; it is an inductive rule since CT_P is a defined predicate of Δ ;
- the 3rd inductive rule defines that P remains true unless it is caused false in the previous state. This rule is called the *inertia rule* or *inertia law*. It is a nonmonotone rule.

²If, in turn, $CF_P(\bar{x}, t)$ is defined by expressions containing P , then these two are defined by mutual nonmonotone induction.

Effect rules are definitional implications of the causality predicates and specify exactly what effects do occur; no other effects occur.

The recursion underlying this definition can be shallow or deep. I call it shallow if all effect rules are non-recursive. In that case, the only recursion is due to the inertia rule. E.g., the definition Δ for the running example would be shallow, since there is only one action *Give* and its effect rules are non-inductive:

$$\begin{aligned} \forall g \forall b \forall r \forall t (CT_Owns(r, b, t) &\leftarrow Gives(g, b, r, t)) \\ \forall g \forall b \forall r \forall t (CF_Owns(g, b, t) &\leftarrow Gives(g, b, r, t)) \end{aligned}$$

We now formally lay out these and the other expressions in an LTC theory.

Rules and axioms of the LTC Given is an LTC vocabulary Σ_a .

Definition 3.3.3

A formula or definitional rule φ over Σ_a is a (*single-*)*state* formula or rule in variable t if t has only free occurrences and the only term of type **T** in φ is t .

A state formula is a proposition describing the state at time t .

- E.g., $\forall g \forall b \forall r (Gives(g, b, r, t) \Rightarrow Owns(g, b, t))$.

Definition 3.3.4

A formula or definitional rule φ over Σ_a is a *bi-state* formula or rule in variable t if t has only free occurrences and the only terms of type **T** in φ are t and $t + 1$.

A bi-state formula is a proposition connecting the states at two successive times t and $t + 1$.

- E.g., $\forall g \forall b \forall r (Gives(g, b, r, t) \Rightarrow \neg Owns(g, b, t + 1))$

A single-state formula is a bi-state formula in which $t + 1$ does not occur. Sometimes, the term $S(t)$ or $Next(t)$ is used instead of $t + 1$.

Definition 3.3.5

An *action effect rule* is a definitional implication of the form:

$$\begin{aligned} \forall t \forall \bar{x} (CT_P(\bar{a}, t) &\leftarrow Act(\bar{a}', t) \wedge \varphi[\bar{x}, t]) \\ \forall t \forall \bar{x} (CF_P(\bar{a}, t) &\leftarrow Act(\bar{a}', t) \wedge \varphi[\bar{x}, t]) \end{aligned}$$

where Act is an action predicate symbol, $P/(n + 1)$ is inertial, $\bar{a}$ and $\bar{a}'$ are tuples of terms of suitable length and type, $\varphi[\bar{x}, t]$ is a single-state formula in t and $\bar{x}$ is the set of all non-temporal free variables in the rule (in $\bar{a}$, $\bar{a}'$ or $\varphi[\bar{x}, t]$).

We call an action effect rule *context-independent* if $\varphi[t] = true$. Otherwise, it is *context-dependent*. Some context-independent effect rules are:

- $\forall t \forall g \forall b \forall r (CT_Owns(r, b, t) \leftarrow Gives(g, b, r, t))$
- $\forall t \forall g \forall b \forall r (CF_Owns(g, b, t) \leftarrow Gives(g, b, r, t))$

Some context-dependent rules:

- $\forall t \forall x (CT_Broken(x, t) \leftarrow Drop(x, t) \wedge Fragile(x))$
- E.g., picking up a block from a stack of blocks makes the underlying block clear:
 $\forall t \forall b \forall b_1 (CT_Clear(b, t) \leftarrow Pick(b_1, t) \wedge On(b_1, b, t))$

Definition 3.3.6

An *action precondition axiom* for action predicate Act is of the form

$$\forall \bar{x} \forall t (Act(\bar{x}, t) \Rightarrow \varphi[\bar{x}, t])$$

where $\varphi[\bar{x}, t]$ is a state formula.

It expresses a necessary condition for the action to occur. E.g., to give a book away, you need to own it.

$$\forall t \forall g \forall b \forall r (Gives(g, b, r, t) \Rightarrow Owns(g, b, t))$$

Definition 3.3.7

The *no-concurrency axiom* for action predicates $Act_1, \dots, Act_n$ expresses that at most one action takes place at the same time.

This axiom is used to express that no actions of different types occur simultaneously, but also that at most one instance of each type of action occurs simultaneously.

In FO, if there are n action predicates, $n + \frac{n \times (n-1)}{2}$ axioms are required: one per action predicate, one per pair of different action predicates. In $FO(\cdot)$, it could be expressed by a single axiom of length n :

$$\forall t (\#\{\bar{x}_1 : Act_1(\bar{x}_1, t)\} + \dots + \#\{x_n : Act_n(\bar{x}_n, t)\} \leq 1)$$

E.g., in the running example:

$$\forall t (\#\{(g, b, r) : Gives(g, b, r, t)\} \leq 1)$$

or

$$\forall g \forall b \forall r \forall t (Gives(g, b, r, t) \wedge Gives(g_1, b_1, r_1, t) \Rightarrow g = g_1 \wedge r = r_1 \wedge b = b_1).$$

Definition 3.3.8

A base *Linear Time Calculus* over Σ_a is a theory consisting of an LTC definition Δ_a consisting of *frame rules* and *action effect rules* for every inertial fluent, of *action preconditions axioms*, the *no-concurrency axiom* and *initial state expressions* and static expressions (definitions and/or sentences).

Example 3.3.1

Expressing the running example as LTC:

$$\begin{aligned}
& \text{UNA}(\text{Person}) \wedge \text{DCA}(\text{Person}) \\
& \text{UNA}(\text{Book}) \wedge \text{DCA}(\text{Book}) \\
& \left\{ \begin{array}{l} I_Owns(\text{Bob}, B1) \leftarrow \\ I_Owns(\text{John}, B2) \leftarrow \\ I_Owns(\text{Mary}, B3) \leftarrow \end{array} \right\} \\
& \left\{ \begin{array}{l} Gives(\text{Bob}, B1, \text{John}, 0) \leftarrow \\ Gives(\text{John}, B1, \text{Mary}, 1) \leftarrow \end{array} \right\} \\
& \left\{ \begin{array}{l} \forall p \forall b (Owns(p, b, 0) \leftarrow I_Owns(p, b)) \\ \forall p \forall b \forall t (Owns(p, b, t+1) \leftarrow CT_Owns(p, b, t)) \\ \forall p \forall b \forall t (Owns(p, b, t+1) \leftarrow Owns(p, b, t) \wedge \neg CF_Owns(p, b, t)) \\ \forall g \forall b \forall r \forall t (CT_Owns(r, b, t) \leftarrow Gives(g, b, r, t)) \\ \forall g \forall b \forall r \forall t (CF_Owns(g, b, t) \leftarrow Gives(g, b, r, t)) \end{array} \right\} \\
& \forall g \forall b \forall r \forall t (Gives(g, b, r, t) \Rightarrow Owns(g, b, t)) \\
& \forall t (\# \{(g, b, r) : Gives(g, b, r, t)\} \leq 1)
\end{aligned}$$

Shorter presentations Under some circumstances, shorter representations are possible.

E.g., if the initial state of a fluent P/n is known, then its definition can be added to the definition of P/n in the form of rules specifying $P(\dots, 0)$. With this, the new predicate P_{init} need not be introduced. This applies to the running example.

E.g., if y in fluent $P(x, y, t)$ is functional in terms of x in the sense of the following property:

$$\forall x \forall t (\# \{y : P(x, y, t)\} \leq 1)$$

then if there is a cause for $P(x, y, t)$ to become true, there will be a cause for the existing value $P(x, y', t)$ to become false. We can then define CF_P in terms of CT_P . Alternatively, we can drop CF_P and rewrite the inertia rule in the definition of P/n in terms of CT_P :

$$\forall x \forall y' (P(x, y', t+1) \leftarrow P(x, y', t) \wedge \neg \exists y CT_P(x, y, t))$$

Also this is the case for the running example. A book has at most one owner. Therefore, if person p is made owner of book b then the previous owner of b is disowned of b .

Exercise 3.3.1 Rewrite the theory of the running example exploiting these shorter

representations.

$$\begin{array}{l}
UNA(Person) \wedge DCA(Person) \\
UNA(Book) \wedge DCA(Book) \\
\left\{ \begin{array}{l} Gives(Bob, B1, John, 0) \leftarrow \\ Gives(John, B1, Mary, 1) \leftarrow \end{array} \right\} \\
\left\{ \begin{array}{l} Owns(Bob, B1, 0) \leftarrow \\ Owns(John, B2, 0) \leftarrow \\ Owns(Mary, B3, 0) \leftarrow \\ \forall p \forall b \forall t (Owns(p, b, t+1) \leftarrow CT_Owns(p, b, t)) \\ \forall p \forall b \forall t (Owns(p, b, t+1) \leftarrow Owns(p, b, t) \wedge \neg \exists p' CT_Owns(p', b, t)) \\ \forall g \forall b \forall r \forall t (CT_Owns(r, b, t) \leftarrow Gives(g, b, r, t)) \end{array} \right\} \\
\forall g \forall b \forall r \forall t (Gives(g, b, r, t) \Rightarrow Owns(g, b, t)) \\
\forall t (\# \{ (g, b, r) : Gives(g, b, r, t) \} \leq 1)
\end{array}$$

IN THE PROJECT AND THE EXAM, IT IS PROHIBITED TO ELIMINATE THE AUXILIARY PREDICATES UNLESS IT IS CLEARLY STATED THAT IT IS ALLOWED TO DO SO.

Representation of LTC in the IDP language ³In the online IDP code, you'll see:

- It uses the type name `Time` to represent the time type T .
- IDP can only handle finite domains, hence *Time* should be restricted to finite type:
 - In the declaration of the vocabulary, we insert `type Time isa Int` to obtain arithmetical symbols for type `Time`.
 - In the declaration of the IDP structure, declare `Time = {0..N}`, with N some numeral.

One needs to pay attention with universal quantification over `Time` in formulas of the IDP language! Suppose $P(\text{Time})$ is declared. Consider the formula

$$P(0) \wedge \forall t[\text{Time}] : P(t) \Rightarrow P(t+1).$$

This sentence is unsatisfiable in IDP since it entails $P(N+1)$ which is a contradiction with P 's type declaration. Indeed, the latter entails that P is a subset of `Time` which does not contain $N+1$.

Such errors occur in quantified formulas over a finite integer type, where the quantified variable t also occurs in atoms $P(t+1)$. This may lead to **inconsistency** or **the unintended disappearance** of models. They may be difficult to discover. The correction is:

$$P(0) \wedge \forall t[\text{Time}] : t < \text{MAX}[: \text{Time}] \wedge P(t) \Rightarrow P(t+1).$$

The extra condition takes care that t is quantified over $[0, N-1]$. The problem does not occur in definitions. E.g.,

$$\left\{ \begin{array}{l} P(0) \leftarrow . \\ \forall n : P(n+1) \leftarrow P(n). \end{array} \right\}$$

³ <https://idp.cs.kuleuven.be/idp/?present=Book2>

The inductive rule seemingly violates the finite-time constraint, but IDP internally interprets this rule as follows:

$$\left\{ \begin{array}{l} P(0) \leftarrow . \\ \forall m[Time] : P(m) \leftarrow \exists n[Time] : m = n + 1 \wedge P(n). \end{array} \right\}$$

Now you can see that this rule cannot derive $P(N+1)$.

Later, we will often use **Start** for 0 and **Next(Time):Time** for $S/1$. Some inference procedures of IDP require this (progression inference, proof of invariance ...).

3.3.1 Discussion of LTC

Asymmetries in LTC An LTC contains causation and inertial rules for P :

$$\begin{aligned} \forall \bar{x} \forall t (P(\bar{x}, t + 1) &\leftarrow CT_P(\bar{x}, t)) \\ \forall \bar{x} \forall t (P(\bar{x}, t + 1) &\leftarrow P(\bar{x}, t) \wedge \neg CF_P(\bar{x}, t)) \end{aligned}$$

Strikingly, there are no symmetrical causation and inertial rules for $\neg P$ of the following kind:

$$\begin{aligned} \forall \bar{x} \forall t (\neg P(\bar{x}, t + 1) &\leftarrow CF_P(\bar{x}, t)) \\ \forall \bar{x} \forall t (\neg P(\bar{x}, t + 1) &\leftarrow \neg P(\bar{x}, t) \wedge \neg CT_P(\bar{x}, t)) \end{aligned}$$

Such rules are **not legal FO(ID) syntax**. Indeed, rules with negated head are not allowed in definitions.⁴

However, let us consider the corresponding material implications:

$$\begin{aligned} \forall \bar{x} \forall t (CF_P(\bar{x}, t) &\Rightarrow \neg P(\bar{x}, t + 1)) \\ \forall \bar{x} \forall t (\neg P(\bar{x}, t) \wedge \neg CT_P(\bar{x}, t) &\Rightarrow \neg P(\bar{x}, t + 1)) \end{aligned}$$

The first expresses causation of $\neg P$, the second inertia for $\neg P$. The question is then whether these axioms are implied by the frame rules of LTC. Well, it can be proved easily that the second implication, expressing inertia for $\neg P$ is indeed entailed by the fluent definitions with its frame rules: a fact that is false at t , remains false at $t + 1$ unless there is a cause to make P true.

Exercise 3.3.2 *Explain this intuitively.*

However, the first implication which states that a cause for $\neg P$ leads to $\neg P$ at $t + 1$ does not always hold. It does not hold when contradictory effects exists, that is, when at some time t , both $CT_P(\bar{x}, t)$ and $CF_P(\bar{x}, t)$ are true. In such a case the second frame rule unambiguously derives that P is true at $t + 1$, even though there is a cause to be false. One could say that a cause for P to be true *dominates* a simultaneous cause for P to be false.

However, under a reasonable condition, the LTC theory entails the first implication:

$$\forall \bar{x} \forall t (CF_P(\bar{x}, t) \Rightarrow \neg P(\bar{x}, t + 1))$$

The reasonable condition is that the LTC has no contradictory effects, i.e., the LTC entails

$$\forall \bar{x} \forall t (\neg (CT_P(\bar{x}, t) \wedge CF_P(\bar{x}, t)))$$

⁴A frequent error made during exams is to include definitional rules with negation in the head in an LTC theory. That is a basic mistake against the syntax of definitions in FO($\cdot$).

Example 3.3.2

Surprisingly, the book LTC does not exclude contradictory effects. It does not entail

$$\forall t \forall p \forall b (\neg (CT_Owns(x, b, t) \wedge CF_Owns(x, b, t)))$$

Exercise 3.3.3 Find a situation in which *Owns* is caused to be true and false simultaneously.

^aFor help, see the online IDP code. At the bottom, it contains the proposition that an *Owns* fact is both caused to be true and false simultaneously. Compute a model of this theory and discover a situation where the same *Owns* fact is caused to be true and false simultaneously.

^a <https://idp.cs.kuleuven.be/idp/?present=Book3>

When contradictory effects $CF_P(\bar{x}, t)$ and $CT_P(\bar{x}, t)$ hold, the second dominates the first, and $P(\bar{x}, t + 1)$ will be derived. This may be sign of an error. To avoid, one might insist that the LTC entails, for every inertial predicate, the following proposition:

$$\forall t (\neg (CT_P(\bar{x}, t) \wedge CF_P(\bar{x}, t)))$$

If an LTC does not entail this, then it must be rewritten.

Exercise 3.3.4 Find out why the Book-LTC allows contradictory effects, and then correct it.

Historical note An application of temporal reasoning is planning. One of the first automated planners was STRIPS (*Stanford Research Institute Problem Solver*) developed by Richard Fikes and Nils Nilsson in 1971 at SRI International. It used a simple special purpose planning language, also called STRIPS. Later planning languages were built as extensions of this language. One example is the language PDDL (“Planning Domain Definition Language”) which is used in the yearly AAI planning competitions. There are many variants and extensions of PDDL most of which are generalized in LTC.

Example 3.3.3

A monkey is at location A in a lab. There is a box in location C. The monkey wants the bananas that are hanging from the ceiling in location B, but it needs to move the box and climb onto it in order to reach them. The STRIPS program expressing this is as follows:

Initial state: At(a), Level(low), BoxAt(c), BananasAt(b)

Goal state: Have(Bananas)

Actions:

```
Move(X, Y)           // move from X to Y
  Preconditions: At(X), Level(low)
  Postconditions: not At(X), At(Y)
ClimbUp(Loc)         // climb up on the box
  Preconditions: At(Loc), BoxAt(Loc), Level(low)
  Postconditions: Level(high), not Level(low)
```

```

ClimbDown(Loc)    // climb down from the box
  Preconditions: At(Loc), BoxAt(Loc), Level(high)
  Postconditions: Level(low), not Level(high)
MoveBox(X, Y)     // monkey moves box from X to Y
  Preconditions: At(X), BoxAt(X), Level(low)
  Postconditions: BoxAt(Y), not BoxAt(X), At(Y), not At(X)
TakeBananas(Loc)  // take the bananas
  Preconditions: At(Loc), BananasAt(Loc), Level(high)
  Postcondition: Have(bananas)

```

In STRIPS, every dynamic symbol is either inertial or an action, there are no concurrent actions, UNA + DCA hold for all types, and lastly, the effect of an action does not depend on the state. Therefore the corresponding effect definitional rules in LTC have a simple format:

$$\forall x \forall t (CT_P(\bar{x}, t) \leftarrow Act(\bar{y}, t))$$

This makes the translation of a STRIPS program into LTC straightforward. See ⁵ for the LTC theory expressing the Monkey STRIPS specification.

3.3.2 Generalizations of the LTC

Now we can extend LTC in various ways to accommodate for special features of the dynamic system.

Delayed effects A delayed effect rule:

$$\forall p \forall t (C_Receive(p, t + 5) \leftarrow Send(p, t))$$

It takes 6 days for a message p to be received.

Ramifications Sometimes, effects cause other effects: they propagate. We call such derived effects *ramifications*. Such ramifications can often be expressed elegantly, by causal rules with causality predicates in the body.

Example 3.3.4

E.g., a suitcase has a spring opening mechanism and two locks. The action of opening one lock has the ramification that it opens the suitcase if the other lock is open already. We can write rules like:

$$\begin{aligned}
&\forall t (CT_OpenSuitCase(t) \leftarrow CT_OpenLock1(t) \wedge OpenLock2(t)) \\
&\forall t (CT_OpenSuitCase(t) \leftarrow CT_OpenLock2(t) \wedge OpenLock1(t)) \\
&\forall t (CT_OpenSuitCase(t) \leftarrow CT_OpenLock2(t) \wedge CT_OpenLock1(t))
\end{aligned}$$

Note that ramifications expressed like this occur simultaneously with the effects that cause them. A simpler representation is by a ramification rule:

$$\forall t (CT_OpenSuitCase(t) \leftarrow OpenLock1(t + 1) \wedge OpenLock2(t + 1))$$

Any set of actions leading to two locks being open will cause the suitcase being open simultaneously.

⁵ <https://idp.cs.kuleuven.be/idp/?present=Monkey>

Cyclic ramifications Ramification may lead to cycles.

Example 3.3.5

Take a set of interconnected gears. An action $Rotate(Gear, T)$ causes the gear to turn. All connected gears are then caused to turn immediately.

Using a predicate $Conn/2$ to represent the symmetric graph of connected gears, we represent this:

$$\left\{ \begin{array}{l} \forall g \forall t (Turn(g, 0) \leftarrow I_Turn(g)). \\ \forall g \forall t (Turn(g, t+1) \leftarrow CT_Turn(g, t)). \\ \forall g \forall t (Turn(g, t+1) \leftarrow Turn(g, t) \wedge \neg CF_Turn(g, t)). \\ \forall g \forall t (CT_Turn(g, t) \leftarrow Rotate(g, t)) \\ \forall g \forall g_1 \forall t (CT_Turn(g, t) \leftarrow CT_Turn(g_1, t) \wedge Conn(g, g_1)) \end{array} \right\}$$

Notice that the last of these $FO(\cdot)$ rules is a reachability rule. Hence, this LTC cannot be expressed in FO.

^aThe IDP code illustrates cyclic ramifications and also that the weaker theory obtained by applying predicate completion to the above definition is too weak and has unintended models.

^a <https://idp.cs.kuleuven.be/idp/?present=Gear.simpler>

Prevoyant action preconditions axioms In rare cases, it might be useful to state an action precondition in terms of the future state. E.g., a chess player pl is not allowed to move a piece p to position pos if pl would be in check as a result:

$$\forall t \forall pl \forall p \forall pos (ChessMove(pl, p, pos, t) \Rightarrow \neg Check(pl, t+1))$$

Expressing the no-concurrency axiom in the IDP language The no-concurrency axiom forbids more than one action per time point. As we saw, it is tedious to express the no-concurrency axiom in FO if there are multiple actions symbols. It is easier to express with aggregates:

$$\forall t : \#\{x : Action_1(x, t)\} + \dots + \#\{x : Action_n(x, t)\} \leq 1$$

It can be further simplified by reifying the action predicates, i.e., by introducing a type $Action$, constructors for all actions without the time argument, and a predicate $Happens(Action, T)$ to express that some action occurs.

Example 3.3.6

We declare

type Action constructed by $\{Gives(Person, Book, Person) : Action\}$

All atoms $Gives(g, b, r, t)$ are replaced by $Happens(Gives(g, b, r), t)$. No-concurrency can be expressed compactly:

$$\forall t : \#\{a : Happens(a, t)\} \leq 1$$

Relaxing the no-concurrency axiom The no-concurrency axiom forbids more than one action and is sometimes too strict. However, typically not all actions can occur simultaneously. Therefore, *action concurrency axioms* are necessary. E.g., an axiom to express that simultaneous *Gives* actions are allowed, but not of the same book.

$$\forall t \forall g \forall b \forall r_1 \forall r_2 (Gives(g, b, r_1, t) \wedge Gives(g, b, r_2, t) \Rightarrow r_1 = r_2)$$

Defined fluents A *fluent definition* is a definition of a **non-inertial** predicate symbol with definitional rules of the form:

$$\forall \bar{x} \forall t (P(\bar{x}, t) \leftarrow \varphi[t])$$

where $\varphi[t]$ is a state formula.

- E.g., a definition of a book owner: $\left\{ \forall p \forall t (Owner(p, t) \leftarrow \exists b Owns(p, b, t)) \right\}$
- E.g., an object is clear if nothing is on top of it: $\left\{ \forall b \forall t (Clear(b, t) \leftarrow \neg \exists b_1 On(b_1, b, t)) \right\}$
- E.g., a player is in check if his king is attacked by a piece of the opponent:

$$\left\{ \forall t \forall pl (Check(pl, t) \leftarrow \exists p Attacks(p, King(pl), t)) \right\}$$

A defined fluent evolves with time, due to changes on its parameters. Often it is possible to express a defined fluent as a standard fluent symbol by frame rules together with certain effect rules. E.g., for the fluent *Owner/2*, the LTC definition can be extended with the three frame rules for this fluent together with two effect rules defining:

$$\begin{aligned} \forall p \forall t (CT_Owner(p, t) &\leftarrow \exists g \exists b Gives(g, b, p, t)) \\ \forall p \forall t (CF_Owner(p, t) &\leftarrow \exists r \exists b (Gives(p, b, r, t) \wedge \forall b_1 (Owns(p, b_1, t) \Rightarrow b_1 = b))) \end{aligned}$$

However, it is simpler to express the definition than to express these effects rules.

Nondeterministic causation Some actions have nondeterministic effects, meaning that in the same context, the effects of an action may be different. A prototypical example would be throwing a dice: its effect is a face up of a number from one to six. Other examples are a lottery where the effect is a prize for some lucky participant, or sending a message to a receiver over a lossy channel ... The problem is that causality predicates are deterministically defined in terms of the action predicates and the state they occur in. This does not work for nondeterministic effects. The challenge is also to have a *local, modular* solution: one that does not require to modify the frame rules, and that correctly expresses the (deterministic or nondeterministic) effect of all actions by separate causal rules.

To this end, we introduce for each non-deterministic effect of an action *A* on a fluent *P* the *undefined causality predicates* *CT_P_A/n* and/or *CF_P_A/n* which serve to capture exactly the effect of action *A* on *P*. A solution:

1. Introduce causality predicates *CT_P_A* (and/or *CF_P_A*) for each combination of action *A* and fluent *P* nondeterministically affected by *A*.
2. To include the non-deterministic effects on *P*, we add the following rules to the definition of *CT_P* and *CF_P* in the core causality definition:

$$\forall \bar{x} \forall t (CT_P(\bar{x}, t) \leftarrow CT_P_A(\bar{x}, t))$$

$$\forall \bar{x} \forall t (CF_P(\bar{x}, t) \leftarrow CF_P_A(\bar{x}, t))$$

As such, it is expressed now that the new non-deterministic effects on *P* will be taken into account by the frame rules.

3. Add suitable FO axioms on $CT_P_A(\bar{x}, t)$ and/or $CT_P_A(\bar{x}, t)$ that express the non-deterministic effects of A on P .

This solution preserves the LTC core causation definition and yields a modular and elaboration-tolerant solution.

Example 3.3.7: Russian roulette

Reconsider the effect of shooting on being alive as expressed by the core causality definition:

$$\left\{ \begin{array}{l} Alive(0) \leftarrow I_Alive \\ \forall t(Alive(t+1) \leftarrow CT_Alive(t)) \\ \forall t(Alive(t+1) \leftarrow Alive(t) \wedge \neg CF_Alive(t)) \\ \forall t(CF_Alive(t) \leftarrow Shoot(t)) \end{array} \right\}$$

Now, we want to add a nondeterministic shooting action of Russian roulette, represented by the action predicate $RR(t)$, which may kill or have no effect. Following the proposed strategy, we introduce $CF_Alive_RR(t)$ to capture the effect of the Russian roulette action on $Alive$. We extend CT_Alive with this new effect by adding the following rule to the LTC core causation definition:

$$\forall t(CF_Alive(t) \leftarrow CF_Alive_RR(t))$$

What remains is to express the non-deterministic effect of Russian Roulette as a formula relating $RR(t)$ and $CF_Alive_RR(t)$. If $RR(t)$ is false, then $CF_Alive_RR(t)$ is false. If $RR(t)$ is true, then $CF_Alive_RR(t)$ is true or false. This is expressed by the material implication:

$$\forall t(CF_Alive_RR(t) \Rightarrow RR(t))$$

This FO formula is to be added to the FO-part of the LTC theory (not to the definition!) The result is the $FO(.)$ theory:

$$\left\{ \begin{array}{l} Alive(0) \leftarrow I_Alive \\ \forall t(Alive(t+1) \leftarrow CT_Alive(t)) \\ \forall t(Alive(t+1) \leftarrow Alive(t) \wedge \neg CF_Alive(t)) \\ \forall t(CF_Alive(t) \leftarrow Shoot(t)) \\ \forall t(CF_Alive(t) \leftarrow CF_Alive_RR(t)) \\ \forall t(CF_Alive_RR(t) \Rightarrow RR(t)) \end{array} \right\}$$

Example 3.3.8: Lottery in the running example

In the running example, we add the nondeterministic action $Lottery(Book, T)$. A lottery action $Lottery(b, t)$ can only take place if b has no owner. The effect of this action is that it nondeterministically assigns b to an arbitrary person p .

To formalize this, we express the action precondition for $Lottery(b, t)$:

$$\forall b \forall t (Lottery(b, t) \Rightarrow \neg \exists p (Owns(p, b, t)))$$

We add the predicate $CT_Owns_Lott(p, b, t)$ and extend the LTC core causation defi-

dition with the following rule:

$$\forall p \forall b \forall t (CT_Owns(p, b, t) \leftarrow CT_Owns_Lott(p, b, t))$$

It remains to specify the logical relationship between $Lottery(b, t)$ and $CT_Owns_Lott(p, b, t)$. Two conditions need to be expressed: (1) if $Lottery(b, t)$ is false, then $CT_Owns_Lott(p, b, t)$ is false, and (2) if $lottery(b, t)$ is true then for exactly one person p , $CT_Owns_Lott(p, b, t)$ is true. The following FO($\cdot$) axioms specify this:

$$\begin{aligned} \forall p \forall b \forall t (CT_Owns_Lott(p, b, t) \Rightarrow Lottery(b, t)) \\ \forall b \forall t (Lottery(b, t) \Rightarrow \#\{p : CT_Owns_Lott(p, b, t)\} = 1) \end{aligned}$$

It may be interesting to dwell on why the first of these two axioms is not expressed as the following definitional rule added to the LTC core causation definition of $Lottery/2$:

$$\forall p \forall b \forall t (Lottery(b, t) \leftarrow CT_Owns_Lott(p, b, t))$$

But since all rules in the core causation definition are of causal nature, this new rule would suggest that $CT_Owns_Lott(p, b, t)$ *causes* the lottery $Lottery(b, t)$ to happen, which is certainly not true.

Exercise 3.3.5 Do a possible world analysis to show that the following sentence is not equivalent to the solution in the previous example and admits unintended models.

$$\forall b \forall t (Lottery(b, t) \Leftrightarrow \#\{p : CT_Owns_Lott(p, b, t)\} = 1)$$

Hint: propose a model in which there are 2 or more people at a time t satisfying CT_Owns_Lott but there is no lottery at t .

Exercise 3.3.6 Insert the lottery action in the IDP formalization of the book-theory and compute a model.

Exercise 3.3.7 Build an LTC to express the action of picking up a die $Pick(Die, T)$ and throwing a die $Throw(Die, T)$. Picking up a die causes the die to be held $Held(Die, T)$. Throwing it causes it lay on the ground showing following a face with a number between 1 and 6 ($Face(Die, \{1..6\}, T)$).

Inertial functional concepts In many applications, fluents show up that are total functional concepts. E.g., in the running example, the concept of ownership represented as the predicate symbol $Owns/3$ is a total function concept in the sense that every book is expected to have exactly one owner at each time point. Thus, the first argument (the owner) is functionally determined by the second and third argument.

Assume that $P/n + 1$ is such a functional fluent (lets assume that the first argument is functionally determined by the rest). In that case, the current value of the function will be terminated exactly when a new value is created. We can exploit this by including the following definitional rule which defines CF_P in terms of CT_P :

$$\forall x_1 \forall y_1 \forall x_2 \dots \forall t (CF_P(x_1, x_2, \dots, x_n, t) \leftarrow P(x_1, \dots, x_n, t) \wedge CT_P(y, x_2, \dots, x_n, t))$$

Even better, the predicate CF_P can be eliminated all together using the following frame

axioms:

$$\left\{ \begin{array}{l} \dots \\ \forall \bar{x} (P(\bar{x}, 0) \leftarrow I_P(\bar{x})) \\ \forall \bar{x} \forall y \forall t (P(\bar{x}, t+1) \leftarrow CT_P(\bar{x}, t)) \\ \forall x_1 \forall y \bar{x} \forall t (P(x_1, \dots, x_n, t+1) \leftarrow P(x_1, \dots, x_n, t) \wedge \neg \exists y CT_P(y, x_2, \dots, x_n, t)) \\ \dots \end{array} \right\}$$

E.g., in the running example, the definition of *CF_Owns* can be dropped and the inertial rule replaced by:

$$\forall p \forall p1 \forall b \forall t (Owns(p, b, t) \leftarrow Owns(p, b, t) \wedge \neg \exists r CT_Owns(r, b, t))$$

Note: One can use a total function symbol F/n rather than $P/n+1$. In that case, the frame axioms can be rephrased as:

$$\left\{ \begin{array}{l} \dots \\ \forall \bar{x} (F(\bar{x}, 0) = I_F(\bar{x}) \leftarrow) \\ \forall \bar{x} \forall y \forall t (F(\bar{x}, t+1) = y \leftarrow CT_F(\bar{x}, y, t)) \\ \forall \bar{x} \forall t (F(\bar{x}, t+1) = F(\bar{x}, t) \leftarrow \neg \exists y CT_F(\bar{x}, y, t)) \\ \dots \end{array} \right\}$$

This is a logical representation of an assignment operation.

There is one potential danger. By choosing function symbol F/n , an implicit constraint that F/n is a total function is added. Experience shows that this often leads to unexpected inconsistencies that are hard to solve. If you do this anyway, take care not to represent CT_F as a total function symbol; this adds the erroneous constraint that at every time point, F is caused to change, which is false and will lead to inconsistency. One can use either the predicate symbol $CT_F/n+1$, or a *partial function symbol* CT_F/n . In the latter case, do not forget to correctly express the inertia rule.

Exercise 3.3.8 Work out the frame axiom for a partial function F/n . Notice that such a function may become undefined in some argument at some time point.

Symmetrical causation rules In case of contradictory causations, we have seen that CT_P dominates CF_F . I.e., if there is cause for P and a cause for $\neg P$, then P will be caused.

It is possible and sometimes useful to modify the LTC such that in case of contradictory causations, the fluent behaves inertially, that is, it retains its current value. This is achieved by replacing the second rule of the definition of P by the *symmetric causation rule*:

$$\forall t \forall \bar{x} (P(\bar{x}, t+1) \leftarrow CT_P(\bar{x}, t) \wedge (CF_P(\bar{x}, t) \Rightarrow P(\bar{x}, t)))$$

Exercise 3.3.9 Test the symmetric causation rule. ^a

^a <https://idp.cs.kuleuven.be/idp/?present=Gear-symmetrical>

Event Calculus style of inertial rules There are other ways to express frame axioms. Here is another one:

$$\begin{aligned} \forall \bar{x} \forall t (P(\bar{x}, t) \leftarrow \exists t_1 : t_1 < t \wedge CT_P(\bar{x}, t_1) \wedge \\ \neg \exists t_2 (t_1 < t_2 < t \wedge CF_P(\bar{x}, t_2))) \end{aligned}$$

In words, P holds at t if it is caused at an earlier time t_1 and it is not caused false during the interval $]t_1, t[$. This axiom was used in another form of temporal theories called the *event calculus*.

3.3.3 Temporal reasoning in the 1970s: the Yale turkey shooting problem

The McCarthy and Hayes paper of 1969 [22] led to the development of a new area of logic called *nonmonotonic reasoning*. By 1980, the logic community had developed several nonmonotonic reasoning formalisms to solve the frame problem. They felt confident. Then, in 1985, two scientists, Hanks and McDermot, threw a bomb into the community. They described an extremely simple problem called the *turkey shooting problem*, and demonstrated errors in no less than 3 different main nonmonotonic formalisms that had been specially designed for solving the frame problem. It was a shock.

After this event, a number of people returned to classical logic. A few years later, Ray Reiter proposed his solution for formalising situation calculus in FO.

The problem Here's an extended Yale turkey shooting problem which exploits several non-basic facilities of the LTC:

Initially, there is a hunter, a living turkey (named Fred) and an unloaded gun. The hunter can load the gun if it is not loaded. He can always shoot, and if the gun is loaded, the turkey dies. There is an action of waiting which can always be done, and which has no effects. () The turkey is dead iff it is not alive. The turkey can start to walk, but only if it is alive, and it can hold still. Any action that kills the turkey stops it from walking.*

The first part of this problem up to (*) corresponds to Hanks and McDermot's original Turkey Shooting problem. It came with a scenario as follows: initially the turkey is alive and the gun unloaded. There are three actions: the hunter loads the gun at time 0, waits at time 1, shoots at time 2. What holds at time 3? A good solution should predict that at 3, the turkey is dead. None of the tested nonmonotonic solutions of the time could produce that result.

Remark 3.3.1

(not for the exam) Here is some explanation of how this error could happen. All temporal reasoning formalisms of those days were based on the intuition that in a model of a temporal specification, all effect rules should be satisfied and in addition, the set of changes that occur between two subsequent states should be *minimal*. That is, any analogous state of affairs in which fewer changes occur between subsequent states, should violate at least one effect rule. An example may clarify that. In the turkey shooting problem, there are three actions *Load*, *Wait*, *Shoot* and three effect rules:

$$\begin{aligned} \forall t (Load(t) \Rightarrow Loaded(t+1)) \\ \forall t (Shoot(t) \Rightarrow \neg Loaded(t+1)) \\ \forall t (Shoot(t) \wedge Loaded(t) \Rightarrow \neg Alive(t+1)) \end{aligned}$$

The expected outcome of the turkey shooting is the following scenario A:

$$(Alive, \neg Loaded) \xrightarrow{Load} (Alive, \mathbf{Loaded}) \xrightarrow{Wait} (Alive, Loaded) \xrightarrow{Shoot} (\neg \mathbf{Alive}, \neg \mathbf{Loaded})$$

The three stressed facts are the effects. Verify that all effect rules are satisfied in A, and it is impossible to drop some effects without breaking an effect rule. So, this was a model according to the logics of the day. Had it been the only model, all these logic theories would have been correct. But Hanks and McDermot showed the existence of a second model B that **nobody had expected**:

$$(Alive, \neg Loaded) \xrightarrow{Load} (Alive, \mathbf{Loaded}) \xrightarrow{Wait} (Alive, \neg \mathbf{Loaded}) \xrightarrow{Shoot} (Alive, \neg Loaded)$$

... *Wait* unloads the gun!? There are no effect rules for *Wait*? How can this be minimal? Well, first verify that all effects rules are satisfied in this scenario B. Second, lets check for minimality. Notice that in scenario B, there are only two changes, which is even less than the three in the scenario A. Assume that the flip of *Loaded* to $\neg Loaded$ due to *Wait* does **not** occur. This leads to scenario C:

$$(Alive, \neg Loaded) \xrightarrow{Load} (Alive, \mathbf{Loaded}) \xrightarrow{Wait} (Alive, \mathbf{Loaded}) \xrightarrow{Shoot} (Alive, \neg Loaded)$$

Now the effect rule deriving $\neg Alive$ is violated in the last state. In general, it is impossible to reduce the number of changes in the scenario B without some effect rule being violated. Therefore, scenario B is a correct model according to the logics of the day. In scenario B, *Wait* has an incorrect, since unjustified effect of unloading the gun, but thanks to this effect, there is no longer need for *Alive* to flip to false in the last step! So, this explains why this was a model according to the logics of the day. The principle of minimization of effects resulted in unintended models and failed to solve the problem of temporal reasoning.

A solution in LTC Since there is only one turkey, gun and hunter, we leave these concepts implicit. There are 4 actions:

- *Load, Shoot* performed by the hunter
- *Walk, Stop* performed by the turkey

There are 3 fluents:

- *Alive, Walking, Dead*: fluents of the turkey
- *Loaded*: fluent of the gun

We will define *Dead* in terms of *Alive*. The remaining fluents *Alive, Walking, Loaded* are inertial. There are two special cases in this example:

- the ramification that getting killed terminates *Walking*;
- *Dead* is defined in terms of *Alive*.

The LTC theory:

- Initial state:

$$I_Alive \wedge \neg I_Loaded$$

- Causal rules:

$$\left\{ \begin{array}{l} \dots (\text{frame rules for } Loaded, Walking, Alive) \dots \\ \forall t (CT_Loaded(t) \leftarrow Load(t)) \\ \forall t (CT_Walking(t) \leftarrow Walk(t)) \\ \forall t (CF_Walking(t) \leftarrow Stop(t)) \\ \forall t (CF_Loaded(t) \leftarrow Shoot(t) \wedge Loaded(t)) \\ \forall t (CF_Alive(t) \leftarrow Shoot(t) \wedge Loaded(t)) \\ \forall t (CF_Walking(t) \leftarrow CF_Alive(t)) \end{array} \right\}$$

Notice that we did not express that shooting terminates walking, but we have a more general rule that any time in which alive is caused to be terminated, also walking is caused to be terminated. When later, the action *TurkeyHeartAttack* is introduced, its effect on walking is covered by the current ramification rule.

- Preconditions:

$$\begin{array}{l} \forall t (Load(t) \Rightarrow \neg Loaded(t)) \\ \forall t (Walk(t) \Rightarrow Alive(t)) \end{array}$$

- No-concurrency axiom: ...

- Definition of *Dead*

$$\left\{ \forall t (Dead(t) \leftarrow \neg Alive(t)) \right\}$$

Exercise 3.3.10 Without no-concurrency axiom, there is a problem if at the same time, the turkey is shot with a loaded gun and starts to walk. Check out what happens and solve this with a relaxed concurrency axiom.

Exercise 3.3.11 Adapt the problem such that the hunter can miss the turkey; i.e., make shooting nondeterministic.

Example 3.3.9

Transmission of signals

A sender sends signals to a receiver. Signals may get lost. The receiver receives the signal with certain delay. Signals are received in order of transmission. Each transmitted signal is different.

There is only one sender and one receiver, so we will not make them explicit.

```

vocabulary V {
  Type Signal
  Send(Signal,T)      % action
  Receive(Signal,T)   % action
  Lose(Signal,T)      % action
  OnMedium(Signal,T)  % action
}
theory T: V {
  {
    ...
     $\forall s \forall t (CT\_OnMedium(s,t) \leftarrow Send(s,t))$ 
     $\forall s \forall t (CF\_OnMedium(s,t) \leftarrow Receive(s,t))$ 
     $\forall s \forall t (CF\_OnMedium(s,t) \leftarrow Lose(s,t))$ 
  }
  % Signal is sent only once
   $\forall s \forall t \forall t_1 (Send(s,t) \wedge Send(s,t_1) \Rightarrow t = t_1)$ 
  % Action preconditions:
   $\forall s \forall t (Receive(s,t) \Rightarrow OnMedium(s,t))$ 
   $\forall s \forall t (Lose(s,t) \Rightarrow OnMedium(s,t))$ 
  % Signal ordering is preserved.
   $\forall s \forall s_2 \forall t \forall t_2 (Send(s,t) \wedge Send(s_2,t_2) \wedge t_2 < t \Rightarrow$ 
     $\forall t_1 (Receive(s,t_1) \Rightarrow \neg OnMedium(s_2,t_1))$ 
  }

```

Exercise 3.3.12 Fix the action concurrency axiom in this theory.

The last formula is not a state or bi-state formula. Strictly spoken, it falls outside the basic LTC. The verification method that we will see in the later section does not work for this specification.

Exercise 3.3.13 A software package dependency system consists of a dynamic collection of software components each importing a number of required “import” services from other software components and providing a number of “export” services to other software components. E.g., think of a software component as a software library, and of a service as a procedure or function. Libraries import services from and export services to other libraries.

- Software components exist in different versions as specified by a version number.
- For each software component, the required import and export services are given. Imported and exported services of all versions of the same component are the same and are described by static predicates. The same service may be exported by different components (e.g., two libraries providing two implementations of the same abstract data type).
- At each time, each software component should be “installed”: each of its import services should be imported from some other component in the system. This induces a dependency relation on the components.
- The component dependency should be acyclic.
- There are two actions: to add and to remove a component with some version

number.

- The system uses a highest version policy: each component imports services from the most recent version of a provider component.
- When different components offer the same exported service, a component that imports that service has the choice from where it imports it.

Express this in LTC.⁶ In this problem, there are no inertial predicates. (You keep reinstalling components every time step.)

3.4 Inference on LTC

For static $\text{FO}(\cdot)$ theories we saw that many useful problems can be solved by applying various forms of inference on the domain specification. This is even more so for dynamic $\text{FO}(\cdot)$ LTC theories. There is a zoo of problems in various fields of computer science and AI that can be explained in terms of existing and some new forms of inference in LTC. This includes some approaches to verification.

We will give an overview of the many tasks and problems that can be solved on the basis of an LTC theory.

3.4.1 Applications of Finite Model Generation

Observation: The value of the time type $\mathbf{T}$ can be set to infinite time $\mathbb{N}$ or to finite time $[0, N]$. Most LTC theories have a special property:

- any model $\mathfrak{A}$ with $\mathbf{T}^{\mathfrak{A}} = [0, N]$ can be extended at the tail to a model with infinite time $\mathbf{T}^{\mathfrak{A}} = \mathbb{N}$, and vice versa,
- any model $\mathfrak{A}$ with $\mathbf{T}^{\mathfrak{A}} = \mathbb{N}$ truncated to $\mathbf{T}^{\mathfrak{A}} = [0, N]$ yields a model with finite time.

Hence, for such theories, we can study the LTC with infinite time by restricting time to finite intervals.

Exercise 3.4.1 Check that Peano's theory does not have this property.

Applying finite model generation on LTC Take an LTC theory, restrict time to some finite interval, add additional desired or undesired propositions, and apply a finite model generator or a finite model expander like IDP.

- A model is an execution of the system that obeys all our constraints.
- We get information from a model or from the absence of models.

This is a flexible method to explore many different scenarios, desired or undesired.

- Specify the details that you are interested in, the initial state or vice versa, constraints on intermediate or final states, or both. Specify some of the actions. Then the solver search for a possible state of affairs. This way, one gets insight in broad range of possible behaviours of the system.
- Add constraints that are the inverse of what the dynamic system should do or of what we expect the system can do. If a model is found, an error has been detected.

To use IDP, the suitable form of inference is model expansion:

⁶This example is presented in [17].

LTC model expansion

Input: LTC theory T , finite structure $\mathfrak{A}_i$ specifying finite time $[0, N]$, finite domains for all types, and possibly more data.

Output: a model $\mathfrak{A}$ of T expanding $\mathfrak{A}_i$

To handle the finite domain, T is to be adapted to quantify bi-state formulas φ over the interval $[0, N - 1]$, as explained earlier on page 119.

Reducing temporal problems to finite model generation inference AI problems that can be approached this way:

- *Prediction* $(init, act) \mapsto (out)$: Given an initial state and a sequence of actions, what is the final state
- *Postdiction* $(out) \mapsto (init, act)$: Given an observation on some final state, explain in terms of possible initial state and actions.
- *Planning* $(init, out) \mapsto (act)$: Given initial state and a desired goal, find a plan, a sequence of (potentially concurrent) actions that transforms initial state in a state satisfying the goal proposition.

The running book example and the Yale turkey shooting problem are both examples of prediction. Note also that prediction and postdiction can be nondeterministic due to unknown initial state or nondeterministic actions.

AI planning by model expansion The problem with solving a planning problem using finite model expander like IDP is that the input fixes the number of time points. Yet we do not know how many time points will be needed for a planning problem. Many AI planning systems do not have this limitation.

For STRIPS, it can be shown that in the worst case, an exponential number of time points (hence, actions) is needed, exponential in the number n of fluent atoms of the planning domain.

Proof. sketch of upper bound: If there is a plan solving a planning goal that contains repetition of states, the actions between repeating states can be dropped, without effect on the correctness of the plan.⁷ With n fluent atoms, only 2^n different states exist. Therefore, if there is a plan, there exists one of at most 2^n steps. It is possible to construct planning problems such that the shortest path is 2^n .⁸ ■

Exercise 3.4.2 *Think of a planning problem that takes exponentially many actions; such a planning problem proves the exponential worst case lower bound.*

A solution for this problem is by iterated calls of model expansion while increasing the time interval in $\mathfrak{A}_i$. In IDP, this is achieved by iterated model expansion calls with an input structure including

$$\text{Time} = \{0..N\}$$

for $N = 1, 2, 3, \dots$ until a plan is found. When a plan is found with this methodology, it will be a shortest plan; if no plan exists, the method does not terminate. This is an effective

⁷This is analogous to cutting loops in the pumping lemma.

⁸An example is the towers of Hanoi planning problem.

strategy, certainly to compute the shortest plan. Solvers based of this strategy do well in the AI planning competition in the category “shortest plan” and have won many of them.

To implement this effectively, it is necessary that systems can incrementally reuse computations for N at $N + 1$. This is a feature that IDP does not (yet) offer.

Example 3.4.1

A classical planning problem: *the towers of Hanoi*. Go to <https://idp.cs.kuleuven.be/idp/?chapter=intro/9-IDPD3> and select The Towers of Hanoi.

Exercise 3.4.3 Another classic is the blocks world problem. It is a domain of blocks and a table and one robot that can pick up and put down blocks on the table or on other blocks. To pick up a block, the block must be clear and the robot free. Choose a vocabulary and express in LTC.

3.4.2 Progression inference and (interactive) simulation inference

Progression inference is the inference that takes as input a description of the state of the world at one particular point of time, and that outputs potential states of the world at the next time point. By iterating progression inference, an LTC theory can be used to simulate evolution over time.

Definition 3.4.1

Given an LTC vocabulary Σ_a , we define its *state vocabulary* Σ_a^s as the set of symbols obtained from Σ_a by dropping all time arguments.

E.g., $\text{Owns}(\text{Person}, \text{Book}, \text{T})$ becomes $\text{Owns}(\text{Person}, \text{Book})$.

Structures of Σ_a^s represent snapshots of the LTC world (without knowing the time). A sequence $\langle \mathfrak{A}_0^s, \mathfrak{A}_1^s, \mathfrak{A}_2^s \dots \rangle$ of Σ_a^s -structures that all have the same domain corresponds one-to-one to a Σ -structure $\mathfrak{A}$ where $\text{T}^{\mathfrak{A}}$ is $[0, n]$ (finite sequence of length n) or $\mathbb{N}$ (infinite sequence) such that for each $t \in \text{T}^{\mathfrak{A}}$ the state at time t in $\mathfrak{A}$ is as given by $\mathfrak{A}_t^s$.

Now, we are ready to define progression inference. It operates on a bi-state LTC theory that consists of bi-state axioms and describes the connection between two successive states – a simulation of one time step.

Progression inference
Input: a bi-state LTC theory T_a over LTC vocabulary Σ_a and a Σ_a^s -structure $\mathfrak{A}_i^s$
Output: a Σ_a^s structure $\mathfrak{A}_o^s$ such that $\langle \mathfrak{A}_i^s, \mathfrak{A}_o^s \rangle$ corresponds to a model $\mathfrak{A}$ of T_a with $\text{Time}^{\mathfrak{A}} = \{0, 1\}$

At the end of the current chapter, we will discuss the LogicBlox system, a commercial logic-based system that operates using progression inference.

Remark 3.4.1

Progressing inference (and heavyweight invariant checking - see Section 3.4.4) have been implemented in IDP, but only for LTC theories that satisfy certain syntactic conditions:

- The LTC theory uses **Time** as the time type, **Start:Time** as the initial state time

point, and $\text{Next}(\text{Time}) : \text{Time}$ as the successor time function. One should not use $T, 0, t + 1, S(t)$.

- The LTC theory consists of static axioms and definitions (no temporal arguments), initial state axioms (using $\text{Start} : \text{Time}$), single-state and bi-state axioms, and definitions of temporal predicates (fluents, causality predicates, action predicates) consisting of single-state rules of the form:

$$\forall t \dots (P(\dots, t) \leftarrow \varphi[t])$$

where $\varphi[t]$ is a single-state formula, and bi-state rules of the form:

$$\forall t \dots (P(\dots, \text{Next}(t)) \leftarrow \varphi[t])$$

where $\varphi[t]$ is a bi-state formula.

More information on progression inference with the IDP system can be found at <https://dtai.cs.kuleuven.be/krr/files/progression.pdf>.

An application of progression: interactive simulation By iteratively applying progression inference, systems can be simulated with or without interaction.

Algorithm($T_a, \mathfrak{A}$)

T_a : LTC theory

$\mathfrak{A}$: a Σ_a^s -structure specifying initial state

1) Generate a set of initial states satisfying the initial state and state axioms.

2) Loop:

Let the user select a state from the given set of states.

Set the selected state as current state.

Apply progression inference on the current state.

Present the possible successor states to the user.

Go to 2.

Example 3.4.2

For an interactive simulation of the Pac-Man problem in IDP, see: <https://idp.cs.kuleuven.be/idp/?chapter=intro/9-IDPD3> and select Pacman.

In the web interface, only a few steps can be executed due to a resource consumption limitation on the server. To execute well, use a local installation of the IDP web-IDE on your computer. A single LTC specification was used for solving multiple problems:

- Interactive simulation inference: interactively run a Pac-Man scenario;
- Optimization inference: search for the shortest path covering all the pellets.

This is an illustration of the knowledge-base paradigm (as explained in chapter 7) in the context of dynamic worlds.

3.4.3 Lightweight verification by finite model generation

This section is inspired by the Alloy language and tool (<http://alloytools.org/>). Alloy is a language and tool for abstract specification and verification of dynamic systems. It was developed at MIT by Daniel Jackson. The language is a syntactic variant of FO. It performs reasoning by bounded model generation. Alloy theories need to specify a maximal size of various types.

The verification problem After building a formal modelling of a domain, we want to analyze it. We expect *emergent* properties of the system: properties that will be satisfied by the described system. Such properties should be entailed by the specification.

A *verification problem* then consists of verifying if such a property is *entailed* by the formal specification or not.

One frequently useful kind of property that needs to be verified is called an *invariant*. It is a property about a snapshot in time, and the goal is to prove that it is true at each instant of time.

Definition 3.4.2

A state formula $\varphi[t]$ is an *invariant* of an LTC theory T_a if $T_a \models \forall t \varphi[t]$.

Abusing terminology, we sometimes call $\forall t \varphi[t]$ an invariant.

E.g., in the running *Book* example, an expected emergent property is that at time t no book is owned by two persons simultaneously, for each time instant t . That is, we expect that this property is an invariant of the book LTC. Another expected invariant is that a book always has an owner.

However, proving such an emergent property is a non-trivial deductive inference problem. Indeed, recall that T_a is not an FO theory, since it contains the domain closure axiom for time, which is DCA(N) (see Section 2.6.3). This axiom cannot be expressed in FO (this will be shown in Chapter 6). Thus, theorem provers for FO cannot be used for verification from T_a . Worse, it will be shown in Chapter 6 that for any logic in which DCA(N) can be expressed, no complete theorem provers can be built.

For this reason, Alloy focusses on simpler forms of reasoning based on finite model generation. It cannot prove invariants from the specification. However, it may suffice to prove that some property is not an invariant. It may be used to find errors in specifications. It may increase the confidence that the specified system is correct. The collection of such methods is called *light weight verification*.

Guided simulation Beside searching for bugs in a specification, there are many other uses of finite model generation to explore an LTC theory:

- Simulate the dynamic system by generating models;
- Find an “execution” that violates a desired invariant;
- Check if a given sequence of actions is possible, and if so, what the states are at different time points. E.g.: propose a sequence of request actions in an elevator specification, search for a finite model, and inspect the intermediate states; automatic generation of test values in C# ...

In Alloy, this is called *guided simulation*.

3.4.4 Heavyweight verification of invariants

Verification of desired system properties is an important concern. So far, we have only seen lightweight methods based on finite model generation which allow us to find bugs in specifications (or increase our confidence in the correctness of the system). However, finite model generation of LTC theories cannot ensure that some property holds at all time in all possible executions of the modelled system. Below is a list of properties in various sorts of systems that one may want to prove.

- E.g.: There is exactly one owner per book:

$$\forall t \forall b \forall p_1 \forall p_2 (Owns(p_1, b, t) \wedge Owns(p_2, b, t) \Rightarrow p_1 = p_2).$$

- E.g.: When moving, the doors of the train are closed.
- E.g.: A traffic light is never green in two directions.
- E.g.: It is always possible for the elevator to reach a state with open doors at the ground floor.
- E.g.: For each direction, the traffic light eventually becomes green.
- E.g.: The system is never in a deadlock.

Looking at these propositions, we see that the first three are propositions that some property about a single state is true at all times. If such a proposition holds, we will call it an invariant of the system. The other propositions are more complex and refer to evolutions of the system. They are more complex to express and more complex to be verified. They will be studied in the next chapter.

Recall Definition 3.4.2. How do we verify that $T_a \models \forall t \varphi[t]$? The first idea that comes to mind is to call a theorem prover to verify $T_a \models \forall t \varphi[t]$. This is problematic: in every structure of the LTC theory T_a , the time type T is supposed to be interpreted as $\mathbb{N}$, and hence the theorem prover should be able to reason about the natural numbers. As described in subsection 2.6.3 and proved in section 6.3, a theory of the natural numbers requires an extension of FO to either SO (to make PA^{SO}), FO(ID) or FO(Arit), so that we can express $DCA(\{0, S/1\})$. This means we need an automated theorem prover for those logics, but unfortunately, today these do not exist.

A second idea is to give a copy of PA^{FO} to a powerful FO theorem prover. This is an FO theory, but it is infinite: instead of the SO induction axiom, it contains the infinitely many instances of the induction schema. There exist contemporary theorem provers supporting inductive proofs using the induction schema, like Isabelle and Coq. Their disadvantage is that neither is fully automatic. They require interaction with the human user to choose the right instances of the induction schema.

Lucky for us, for the task at hand, we might not need all possible instances of the induction schema. There is one particular instance of the induction schema that is really promising. We want to prove $\forall t \varphi[t]$, i.e., that the state formula $\varphi[t]$ is an invariant, so we could try building proofs by induction using the instance:

$$\varphi[0] \wedge \forall t (\varphi[t] \Rightarrow \varphi[S(t)]) \Rightarrow \forall t \varphi[t].$$

We denote this formula as $\mathbf{Ind}(\varphi[t])$. This sentence is entailed by an LTC theory T_a , since T is interpreted by $\mathbb{N}$. So, it is sound to use this instance in a proof. We now develop a methodology for proving invariants $\forall t \varphi[t]$ using $\mathbf{Ind}(\varphi[t])$. We will show that this method is not complete (i.e., we cannot prove all invariants with it) but it is useful nevertheless. It has been implemented in many existing systems.

Bi-state theories and invariants The method is applicable to LTC theories of the following form:

Definition 3.4.3: Bi-state LTC theory

We call an LTC theory a *bi-state LTC theory* if it has the following form:

$$T_a = T_{\text{static}} \cup T_0 \cup T_s \cup T_t$$

where

- T_{static} is an $\text{FO}(\cdot)$ theory of the static symbols,
- T_0 is a set of initial state expressions,
- T_s consists of single-state expressions $\forall t \varphi[t]$ or definitions consisting of single-state rules,
- T_t consists of bi-state formulas and definitions consisting of bi-state rules.

T_s includes action preconditions and no-concurrency or simultaneity axioms and may include fluent definitions which consist of single-state rules. T_t includes the simultaneous definition of fluents and causality predicates by frame rules and effect rules.

Definition 3.4.4

Given any single-state or bi-state theory T and numeral n . The theory denoted $T[n]$ consists of expressions $\varphi[n]$ (rule or FO axiom) for each expression $\forall t \varphi[t]$ in T .

Explained in another way, $T[n]$ is obtained by dropping quantifiers $\forall t$ and substituting n for t in formulas and rules.

Procedure 3.1: Heavyweight verification

1. Input:
 - a bi-state LTC theory $T_a = T_{\text{static}} \cup T_0 \cup T_s \cup T_t$
 - a single-state formula $\psi = \varphi[t]$.
2. The first step serves to establish the base case of the inductive proof. We construct the following theory:
 - $T_{\text{init}} = T_{\text{static}} \cup T_0 \cup T_s[0]$
 This theory contains all information of T_a pertaining to the initial state
3. We then use a theorem prover to verify:
 - $T_{\text{init}} \models \varphi[0]$
 If this call succeeds, T_a entails that φ holds in the initial state. If it fails, report failure.
4. The second step serves to prove the inductive step. Construct the following theory:
 - $T_{\text{ind}} = T_{\text{static}} \cup T_s[0] \cup T_s[1] \cup T_t[0] \cup \{\varphi[0]\}$
 This theory contains all information of T_a that relates two successive states, here represented as 0 and 1. Importantly, the initial conditions in T_0 are absent from this theory.
5. We use a theorem prover to verify:
 - $T_{\text{ind}} \models \varphi[1]$
6. If also this call succeeds, then return that $\varphi[t]$ is an invariant of T_a , otherwise report failure.

The verification method The induction axiom is implicitly used in this method, as explained below.

Theorem 3.4.1: Correctness of heavyweight verification

If $T_{\text{init}} \models \varphi[0]$ and $T_{\text{ind}} \models \varphi[1]$ then $T_a \models \forall t \varphi[t]$.

Proof. Let $\mathbf{Ind}(\varphi[t])$ be the induction schema instance $\varphi[0] \wedge \forall t(\varphi[t] \Rightarrow \varphi[S(t)] \Rightarrow \forall t \varphi[t])$.

We have $T_a \models \mathbf{Ind}(\varphi[t])$ since $\mathbf{T}$ is the interpreted type $\mathbb{N}$ for which all induction schema instances hold.

We have $T_a \models \varphi[0]$ since T_{init} is included in T_a .

Since $T_{\text{ind}} \models \varphi[1]$, the following holds:

- Then $T_{\text{static}} \cup T_s[0] \cup T_s[S(0)] \cup T_t[0] \cup \{\varphi[0]\} \models \varphi[S(0)]$.
- Then $T_{\text{static}} \cup T_s[0] \cup T_s[S(0)] \cup T_t[0] \models (\varphi[0] \Rightarrow \varphi[S(0)])$.
- Then $T_{\text{static}} \cup T_s \cup T_t \models (\varphi[0] \Rightarrow \varphi[S(0)])$.
- Then $T_{\text{static}} \cup T_s \cup T_t \models \forall t(\varphi[t] \Rightarrow \varphi[S(t)])$. Why? 0 does not occur in $T_{\text{static}} \cup T_s \cup T_t$. In classical logic, if $T \models \varphi[c]$ and c is a constant that does not occur in T , then $T \models \forall x \varphi[x]$.
- Then $T_a \models \forall t(\varphi[t] \Rightarrow \varphi[S(t)])$ (T_a includes $T_{\text{static}} \cup T_s \cup T_t$)

Now, application of $\mathbf{Ind}(\varphi[t])$ yields that $T_a \models \forall t \varphi[t]$. ■

Remark 3.4.2

The IDP system supports heavyweight verification for a class of LTC theories that satisfy the syntactical conditions of Remark 3.4.1. The conditions are similar as for IDP's progression inference.

Basically, the LTC theory should consists of an arbitrary static theory, an initial state theory using `Start:Time` to present the initial time, a single-state and bistate theory using `Next(Time):Time` for the successor time function.

An experiment with IDP The method is manually applied on the running example online ⁹. It shows the following:

- how to split up `T_Book` in `T_init` and `T_ind`;
- how to apply the theorem prover in IDP to prove the invariant.
 - The theorem prover is SPASS, developed at the Max Planck Institute for Informatics.
 - SPASS is a FO theorem prover, not $\text{FO}(\cdot)$.
 - IDP translates $\text{FO}(\cdot)$ to FO, if possible.

The example also illustrates the use of the IDP procedure `isinvariant(theory T, invariant  $\varphi$ )` that implements the heavyweight invariant verification method. Sometimes it is useful to prove invariants in specific domain; for this IDP provides the method `isinvariant(theory T, invariant  $\varphi$ , structure  $\mathfrak{A}$ )`. $\mathfrak{A}$ is a structure specifying a finite domain for static types.

⁹ <https://idp.cs.kuleuven.be/idp/?present=BookInvariantsSpass>

IDP will perform the invariant verification method, except that it will limit the search to structures that expand $\mathfrak{A}$. Such an example is found online ¹⁰.

Limitations of the method

The method has several weaknesses explained below.

The method is not complete Two of the cases where it may fail to prove the invariance of an invariant:

- If the invariants are not sufficiently strong: see the light switch example.
- If there are other integer-valued types. E.g., for proving correctness of a program with integer-valued variables (such as the GCD program). In that case, other instances of the induction schema are needed to prove properties of these integer valued types and relations.

The invariant should be strong enough The above methodology may not work if the invariant is not strong enough. It may be that a set of invariants $\forall t \varphi[t]$ are mutually dependent on each other. It might be impossible to prove the invariance of each in isolation of the others.

We illustrate this in a light switching example ¹¹.

The specification:

$$\left\{ \begin{array}{l} \dots \text{frame rules} \\ \forall t (CT_On(t) \leftarrow \neg On(t)) \\ \forall t (CF_On(t) \leftarrow On(t)) \\ \forall t (CT_Off(t) \leftarrow \neg Off(t)) \\ \forall t (CF_Off(t) \leftarrow Off(t)) \end{array} \right\}$$

$$On(0) \wedge \neg Off(0)$$

Two invariants of this theory are the following:

$$\begin{array}{l} \forall t \ On(t) \vee Off(t) \\ \forall t \ \neg On(t) \vee \neg Off(t) \end{array}$$

Remarkably, the method for proving invariance fails to prove invariance of either of these formulas. Suppose we want to prove invariance of $\forall t \ On(t) \vee Off(t)$:

- Step 1 succeeds: $On(0) \vee Off(0)$ is entailed by $On(0) \wedge \neg Off(0)$.
- Step 2 fails. Indeed, one initial state that satisfies this invariant is where On and Off hold at 0. It then follows from T_{ind} that in the successive state 1, it holds that On and Off are both false, hence the invariant is broken.

Actually, it is not difficult to see that $On(1) \vee Off(1)$ follows from $\neg On(0) \vee \neg Off(0)$, and vice versa that $\neg On(1) \vee \neg Off(1)$ follows from $On(0) \vee Off(0)$. Therefore, it is a piece of cake to prove that the conjunction $\forall t (On(t) \vee Off(t)) \wedge (\neg On(t) \vee \neg Off(t))$ is an invariant of this LTC. So, while the method is strong enough to prove the conjunction of the two invariants, it is not strong enough to prove the two invariants separately.

¹⁰ <https://idp.cs.kuleuven.be/idp/?present=BookInvariant>

¹¹ <https://idp.cs.kuleuven.be/idp/?present=Light>

Invariants that cannot be expressed in FO To see that FO does not always suffice to express all invariants of a dynamic system, consider a dynamic system in which at each time point, an object moves around in some graph G ; $Pos(v, t)$ means that at time t the object is at vertex v . Its initial position is the vertex A ($Pos(0, A)$). At each time point, the object may stay where it is or move to an adjacent vertex. What is the invariant of this system? It is that at each time t , the object is at a vertex that is *reachable* from A in graph G . Stated differently, at time, there is a path from A to its current position v through G .

As we shall see in section 6.2, this invariant cannot be expressed in FO, but it can in FO(ID).

Exercise 3.4.4 Express this dynamic system in LTC. Express the invariant using FO(ID), using an inductive definition and prove the invariant using IDP.

Example 3.4.3

A similar problem occurs in the gearwheel example ^a. There, an invariant is that all pairs of gearwheels that are connected directly or indirectly, are in the same state (either turning or not). The online example illustrates how to prove invariants involving inductively defined predicates.

^a <https://idp.cs.kuleuven.be/idp/?present=GearInvariant>

Some verifiable properties are not invariants. Consider the following propositions:
E.g.: The elevator can always reach the state that it is on the ground floor with open doors (but does not need to).

E.g.: At all times, if the system is *Busy*, then, after finite time, it will return to the state *Wait*.

These propositions are about *evolution* of the system, not about a *state*. They might be expressible in FO, but we have no *method* to prove them.

To express properties of evolution of the system, we introduce temporal logic in next chapter.

3.4.5 Example: Combining light- and heavyweight verification

In this example ¹², a strategy is sketched to combine the different verification methods, to refine and correct the no-concurrency axiom of a simple example: an information system for the management of a library, called BibSys. It consists of:

- A database with following relations:
 - *InHoldings*(b): book b is owned by the library.
 - *OnLoan*(b, p): book b is lent out to person p .
 - *Available*(b): book b is available in the library.
- Transactions in BibSys.
 - loan of a book
 - return of a book
 - purchase of a book

¹² <https://idp.cs.kuleuven.be/idp/?present=BookConcurrency>

- deletion of a book.

Not every database instance reflects a correct state of affairs of the BibSys system.

- E.g., a book on loan must be in the holdings of the bib.
- E.g., a book cannot be on loan and available.

Integrity constraints

- *E1*: A book is owned by the bib if and only if it is on loan to some person or it is available: Formally:

$$\forall x (InHoldings(x) \Leftrightarrow Available(x) \vee \exists u OnLoan(x, u))$$

- *E2*: No book is simultaneously available and on loan. Formally:

$$\forall x (InHoldings(x) \Rightarrow \neg \exists u OnLoan(x, u) \vee \neg Available(x))$$

- *E3*: No book is lent out to more than one person. Formally:

$$\forall x (InHoldings(x) \Rightarrow \neg \exists u \exists w (u \neq w \wedge OnLoan(x, u) \wedge OnLoan(x, w)))$$

Transactions

A transaction is a program that executes a number of different logically coherent operations on the database. (e.g., moving a book from *Available* to *OnLoan*). Correct transactions preserve integrity constraints.

BibSys as dynamic system BibSys, like other database applications, is a dynamic system.

- Database relations as fluents
- Transactions as actions
- Integrity constraints as invariants

We model BibSys in LTC, using action predicates:

- *Borrows*(*p*, *b*, *t*): person *p* borrows book *b* at time *t*.
- *Returns*(*p*, *b*, *t*): person *p* returns book *b* at time *t*.
- *Remove*(*b*, *t*): book *b* is removed from library at *t*.
- *Add*(*b*, *t*): book *b* is added to library at *t*.

¹³The theory and invariants are found online. Use IDP to:

- Solve a planning problem
- Use lightweight verification to refine the no-concurrency axiom
- Simulate BibSys
- Verify the invariants with the context-based invariance proof method.

¹³ <https://idp.cs.kuleuven.be/idp/?present=BibSys>

Exercise 3.4.5 Notice that the simulation of BibSys in the IDP system comes very close to an actual execution of this system. Think of what we are missing in IDP to use the BibSys theory to actually run the BibSys system. That is, what should be added/modified so that we can actually have a working BibSys software system to manage the library, on the basis of the logic theory .

3.4.6 Reasoning on programs

A computer program P is an executable description of a class of computer processes; a program can be run or executed on some input i and then gives rise to a single computer process (assuming the program is deterministic); this process may stop with some output, or loop, or terminate with error.

On the other hand, a theory in a declarative logic is a description of a class of possible states of affairs. It cannot be run but a range of inferences can be executed on it. As such there is a big conceptual difference between programs and theories. It is a category clash to confuse them.

However, with an expressive temporal reasoning formalism such as LTC in $\text{FO}(\cdot)$, and using a well-chosen vocabulary Σ and informal interpretation $\mathcal{I}$, it is possible to build a logic theory $\text{LTC}(P)$ whose models are precise representations of the computer processes obtained by running the program P . Of course, such a theory $\text{LTC}(P)$ is not a program and cannot be “run”. Nevertheless, $\text{LTC}(P)$ captures some of the essence of what P expresses. In a theoretical sense, both P and $\text{LTC}(P)$ can be seen as equivalent declarative theories of computer processes. Looking from this perspective, a programming language now acts as a *domain dependent* declarative language, in the sense that this logic is geared towards expressing computer processes.

Imagine the power of a theory like $\text{LTC}(P)$, and the problems that in principle could be solved using various forms of inference, if we had unlimited reasoning power. E.g., proving that for certain input, the program will always terminate, or will satisfy a postcondition, or go into an infinite loop, etc. It cannot be so rosy. Indeed, reasoning on such theories is extremely complex. To express time, $\text{LTC}(P)$ includes the theory of $\mathbb{N}$. As such, it has only infinite models, and moreover, deduction over the natural numbers is undecidable as will be seen in chapter 6.

Still, it is worthwhile to explore this idea, if only in order to give some insight in some existing declarative methods currently in development in the field of programming languages, e.g., for the automated generation of test values. The aim is to offer a glimpse of how programs could be formalised in logic and what kind of problems could be solved using, e.g., light weight verification methods.

Example 3.4.4: Expressing a program in LTC.

The following toy program takes as input two natural numbers stored initially in the program variables n, m , and computes the greatest common divider (GCD) of them. The output is stored in the same variables. The program, denoted GCD, is annotated with program points.

```

 $P_1$  while ( $n \neq m$ ) {
 $P_2$  if ( $n > m$ )
then { $P_3$   $n = n - m$  }
```

```

    else { $P_4$   $m = m - n$  }
  }
 $P_5$ 

```

For this simple program, it is easy to build $LTC(GCD)$ in the logic $FO(\text{Types}, \text{Arit}, \text{ID})$. The vocabulary of the theory is obtained as follows. Program variables n, m are expressed as numerical functions where $n(t), m(t)$ specify the value of the program variables n, m at time t . A type PP represents program points; function symbol $\text{CurrPP}(T) : PP$ specifies the *program point* at time t . Structures of this vocabulary represent traces of this program. Towards designing $LTC(GCD)$, we observe that the functions $n/1 : m/1$ behave as inertial dynamic functions while clearly $\text{CurrPP}/1$ is far from inertial, since it changes at every step. The following LTC-like theory specifies the behaviour of the three functions.

```

vocabulary V { $Tisaint$ 
  Type PP = { $P_1; P_2; P_3; P_4; P_5$ }
  CurrPP(T): PP
   $n(T)$ : int
   $I_n$ : int
   $C_n$ : (int, T)
   $m(T)$ : int
   $I_m$ : int
   $C_m$ : (int, T)
}
theory T: V {
  {
     $n(0) = I_n \leftarrow$ 
     $\forall t(n(t) = x \leftarrow C_n(x, t))$ 
     $\forall t(n(t+1) = n(t) \leftarrow \neg \exists x C_n(x, t))$ 
     $m(0) = I_m \leftarrow$ 
     $\forall t(m(t+1) = x \leftarrow C_m(x, t))$ 
     $\forall t(m(t+1) = m(t) \leftarrow \neg \exists x C_m(x, t))$ 
     $\forall t(C_n(n(t) - m(t), t) \leftarrow \text{CurrPP}(t) = P_3)$ 
     $\forall t(C_m(m(t) - n(t), t) \leftarrow \text{CurrPP}(t) = P_4)$ 
  }

  {
     $\text{CurrPP}(0) = P_1 \leftarrow$ 
     $\forall t(\text{CurrPP}(t+1) = P_2 \leftarrow \text{CurrPP}(t) = P_1 \wedge n(t) \neq m(t))$ 
     $\forall t(\text{CurrPP}(t+1) = P_5 \leftarrow \text{CurrPP}(t) = P_1 \wedge \neg(n(t) \neq m(t))$ 
     $\forall t(\text{CurrPP}(t+1) = P_3 \leftarrow \text{CurrPP}(t) = P_2 \wedge n(t) > m(t))$ 
     $\forall t(\text{CurrPP}(t+1) = P_4 \leftarrow \text{CurrPP}(t) = P_2 \wedge \neg(n(t) > m(t))$ 
     $\forall t(\text{CurrPP}(t+1) = P_1 \leftarrow \text{CurrPP}(t) = P_3 \vee \text{CurrPP}(t) = P_4)$ 
     $\forall t(\text{CurrPP}(t+1) = P_5 \leftarrow \text{CurrPP}(t) = P_5)$ 
  }
}

```

This theory contains two definitions. The first is the standard core causation definition of LTC for inertial functions n and m . It contains two effect rules modelling the assignments $n = n - m$ and $m = m - n$ at P_3 , respectively P_4 . The second definition models the behaviour of the program pointer.

The theory $LTC(GCD)$ exactly captures the execution traces of program GCD . As such, the theory $LTC(GCD)$ could be said to express the *operational semantics* of GCD .

It follows that this theory could be useful for a number of reasoning and analysis tasks, at least in principle, but not necessarily in practice. Some of these tasks could be performed by model generation inference. A few examples follow:

- Compute the GCD of two numbers i, j . Compute a model $\mathfrak{A}$ for $LTC(GCD)$ augmented with the formula

$$n(0) = i \wedge m(0) = j$$

Extract $n^{\mathfrak{A}}(k)$ where k is a number such that $\mathfrak{A} \models currPP(k) = P_5$ (that is, the program has halted at time k).

- For given number g compute numbers i, j with GCD g . Compute a model for

$$\exists t(CurrPP(t) = P_5 \wedge n(t) = g)$$

and extract $n^{\mathfrak{A}}(0), m^{\mathfrak{A}}(0)$.

- To detect non-terminating behaviour of the program, compute a model $\mathfrak{A}$ satisfying:

$$\neg \exists t(CurrPP(t) = P_5)$$

The values of the terms $n(0), m(0)$ in $\mathfrak{A}$ are input for which the program loops. For GCD, such models indeed exist and they are characterised by the fact that one of $n^{\mathfrak{A}}(0)$ or $m^{\mathfrak{A}}(0)$ is zero.

- Give formulas that specify constraints on the trace. E.g., specify that in the first 5 iterations of the while-loop in GCD, the then part of the if-then-else is executed. Extract $n^{\mathfrak{A}}, m^{\mathfrak{A}}(0)$ to obtain input leading to such traces.
- Search for input such that the outcome of P is not the GCD of the input. For this, add a formula to express that at some time t where $currPP(t) = P_5$ holds, $n(t)$ is not the GCD of $n(0), m(0)$.

The flexibility of model generation would be enormous, but unfortunately, models are infinite, and finite model generators are not directly applicable. One would need some new kind of infinite model generator (e.g., one using finite representations of infinite structures).

Also deduction inference would be useful.

- Prove that the program is correct. Call deduction inference to prove $LTC(GCD) \models \forall i \forall j (n(0) = i \wedge m(0) = j \Rightarrow \exists t (currPP(t) = P_5 \wedge \Psi[i, j, n(t)]))$ where $\Psi[i, j, n(t)]$ is a formula expressing that $n(t)$ is the GCD of i and j .
- Prove that the program loops iff n or m are zero. Call deduction inference to prove $LTC(P) \models ((\neg \exists t CurrPP(t) = P_5) \Leftrightarrow (n(0) = 0 \vee m(0) = 0))$.

It is fairly obvious that a broad class of programs can be translated in LTC, moreover that many complex questions about programs can be solved in principle by deduction inference. Indeed, many of the above sorts of computational problems are known to be not computable in general. No miracles can be expected. Deduction inference is undecidable for FO and its extensions. As will be seen in chapter 6, there exists no terminating theorem provers for FO, and no sound and complete theorem provers for FO(types,arit) or FO(ID).

Still, some forms of inference are feasible and some are being actively researched in the area of programming languages.

Exercise 3.4.6 Build a formula $\Psi[i, j, k]$ with free variables expressing that k is the GCD of i and j .

Exercise 3.4.7 *How to use deduction inference to prove (in principle) that the program terminates if the input consists of strictly positive numbers?*

Lightweight verification One can impose finite bounds on the values of n, m and on T , and then call a finite model generator. If the bounds are set large enough, some of the above tasks can be executed. This is worked out in detail online ¹⁴. E.g., we can add finite bounds and an axiom expressing that the program halts but does not satisfy the intended postcondition and apply a finite model generator. If a model is found, the program is certainly not correct. If no model is found, we cannot yet be sure that the program is correct but our confidence in the program has grown.

Execution is iterated progression inference The kind of inference applied to $LTC(P)$ that comes closest to executing program P on some input, is iterated progression inference. For a call of the GCD program with input i, j , one starts from the initial structure $[n : i, m : j, currPP : P_1]$. Iterated progression inference will generate a sequence of structures $\langle [n : i_k, m : j_k, currPP : p_k] \rangle_{k \in \mathbb{N}}$. This constitutes the trace of the program for input i, j . Thus, viewing programs P as declarative process theories, one could say that running the program corresponds to iterated progression inference.

Searching for infinite loops Bounded model generation seems unsuitable for proving non-termination of the program. Indeed, a model with bounded time $[0..N]$ satisfying the non-termination condition $\neg \exists t (CurrPP(t) = P_5)$ does not prove non-termination but that the program needs more than N steps. Nevertheless, there may be other ways to prove non-termination using finite model generation.

E.g., assume we add a formula expressing that after one iteration of the while-loop, the values of n and m have not changed. A model satisfying this would provide a case where the while loop goes into an infinite loop.

Consider the following proposition:

$$m(0) = m(3) \wedge n(0) = n(3)$$

Notice that 0 is the initial time point and 3 is the time after one iteration. IDP finds models $\mathfrak{A}$ for this proposition for a time interval with at least 4 time points. In all cases, one of $n^{\mathfrak{A}}(0), m^{\mathfrak{A}}(0)$ is zero. For such input, the program indeed loops.

Generation of test values Testing is the most common method for verifying correctness of programs. To find useful and sufficiently exhaustive test-input is important, difficult and time consuming. Experience shows that many errors are caused by the fact that certain execution flows of the program were not taken into account. E.g., if there is a while loop with two sequential if-then-else instructions, then there should be test input leading to traces that pass at each of the four combinations of the then and the else parts of both.

Systems are in development that systematically search for test input to test every such different execution flow. The difficult job is to find input for the program that leads to a given execution flow. Some clever people have thought up automated methods to search for such input. We explain the principle in terms of $LTC(P)$. The idea is to use finite model generation for bounded $LTC(P)$ augmented with a formula that constrains the flow of the program. A solution to this problem then includes the test input that is searched for.

¹⁴ <https://idp.cs.kuleuven.be/idp/?present=GGD>

UI event handling

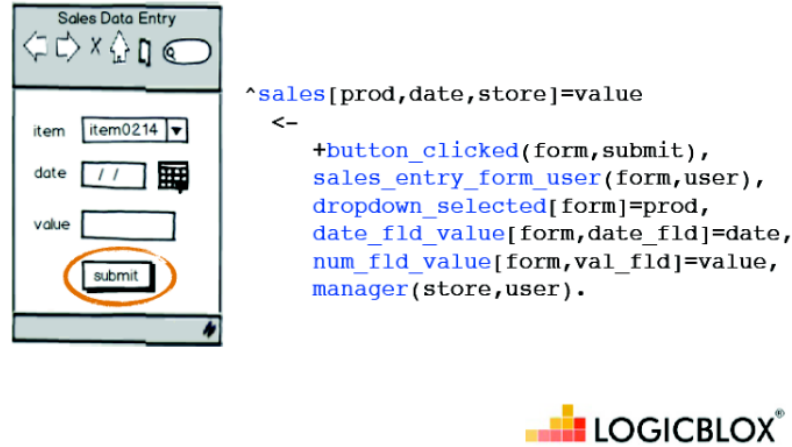


Figure 3.1 – A rule of LogicBlox theory for sales management

We illustrate this for GCD. Suppose we are searching for test input by which the program passes through the *else* instruction during the first iteration, and through the *then* instruction during the second iteration. The corresponding declarative axiom is as follows:

$$\text{CurrPP}(2) = P_4 \wedge \text{CurrPP}(5) = P_3$$

This is added to $LTC(GCD)$ and IDP is called. If a model $\mathfrak{A}$ is obtained, the values $m(0)^{\mathfrak{A}}$ and $n(0)^{\mathfrak{A}}$ is the requested test input. This is experiment (f) online ¹⁵.

Several such systems exist for programming languages, e.g., for Microsoft’s programming language C#. Automated test case generation makes use of Constraint Programming or SMT solvers.

3.5 LogicBlox: software by “running” specifications

Can you imagine that one day large transaction software systems are built as LTC-like logical specifications that are used by performing “execution inference”? I am aware of at least one quite successful American company that is actually doing this: *LogicBlox* (<http://www.logicblox.com/>). The company developed a logic-based system designed to implement large-scale transaction-based software applications: book keeping, stock management, financial management ... A very short summary of the system follows in the context of a sales management application, explaining the system in terms of concepts that we have seen earlier in this course.

Figure 3.1 displays a web page allowing a user to place an order, and the corresponding rule that registers the order. The context is that an earlier action was performed that created this form. To place an order, the user specifies:

- a product,
- a date (of delivery), and
- the required value of that product.

¹⁵ <https://idp.cs.kuleuven.be/idp/?present=GGD>

The displayed rule is then triggered when the user clicks the submit button on the form. To give an idea of what the rule means, we translate it into LTC. Essentially, this is done by turning it into a bi-state rule:

$$\left\{ \begin{array}{l} \dots \\ \forall prod \forall date \forall store \forall value \forall form \forall t \\ C_sales(prod, date, store, value, t + 1) \leftarrow \\ \quad \mathbf{button_clicked}(form, \mathbf{submit}, t) \\ \quad \wedge sales_entry_form_user(form, user, t) \\ \quad \wedge dropdown_selected(form, t) = prod \\ \quad \wedge date_fld_value(form, \mathbf{date_fld}, t) = date \\ \quad \wedge num_fld_value(form, \mathbf{val_fld}, t) = value \\ \quad \wedge \exists store\ manager(store, user, t). \\ \dots \end{array} \right\}$$

This is an effect rule (hence C_sales), specifying an assignment of $value$ to the fluent function term $sales(prod, date, store)$

- $\mathbf{submit}$, $\mathbf{date_fld}$ and $\mathbf{val_fld}$ are constants identifying fields or buttons in the window;
- $form, user, prod, date, value$ and $store$ are variables representing the GUI window identifier, the user that is logged in, the product, date, and value filled in by the user on the form.
- $\mathbf{button_clicked}(form, \mathbf{submit}, t)$: a so-called **pulse atom**, which in this case is an action predicate that is true when the user clicks the $\mathbf{submit}$ button.

LogicBlox theories describe dynamic worlds. They are similar to LTC theories, except that:

- Time is implicit, inertia is built-in (as in Event-B, the system seen in the next chapters).
- Rules express causal rules or definitional rules.
- Distinction is made between *internal* and *external* fluents. *Internal* fluents describe internal state (persistent storage); e.g., $sales, manager \dots$ *External* fluents are linked with the environment (GUI); e.g., $button_clicked, sales_entry_form_user, dropdown_selected, date_fld_value, num_fld_value \dots$

Running this LogicBlox program is done by *simulation/execution inference*: iterated progression inference. A progression step is triggered when the *pulse atom* $button_clicked(form, submit)$ becomes true. The system then computes a *transaction* that results in an update of the state of the system. Once the update is computed, the updated state is stored persistently till the next transaction. The system provides standard transaction management in the cloud. The system scales and is used in large retail chains.

LogicBlox is an *executable, state-based workflow language*:

- a theory specifies a workflow;
- the workflow is executed through iterated progression inference.

One of the ambitions of LogicBlox is to tackle the “company hairball” of the kind shown in Figure 3.2. With the hairball, they refer to the often extremely complex bunch of interacting and interrelated software packages and programs that run company applications. The hairball often leads to terrible maintenance problems. LogicBlox aims to resolve this problem by using its single logical language for almost every task.

LogicBlox claims that they achieve major benefits on the level of development and maintenance: a factor $50\times$ compared to standard Java development for large projects. E.g., the

case study: supply chain, 2% IT footprint

CENSORED

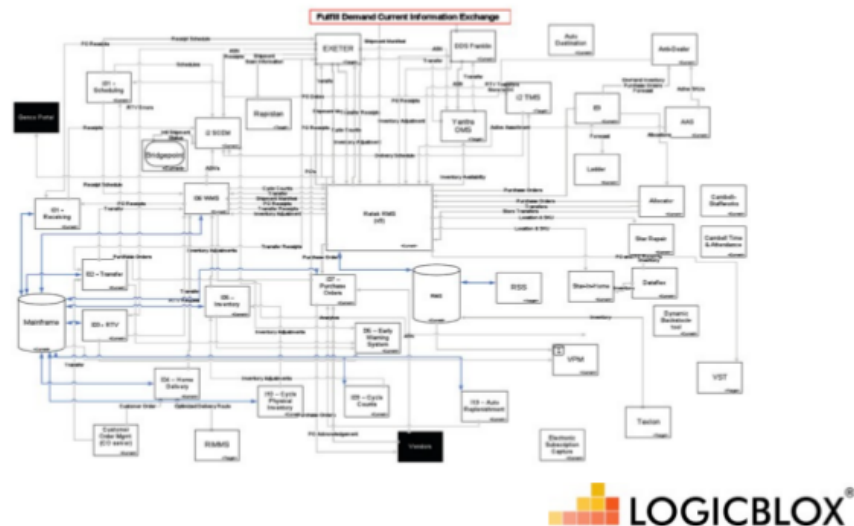


Figure 3.2 – The company hairball

LogicBlox company implemented the stock management of Walgreens (the largest drug retailing chain in the United States) with just 2 developers. This is a big achievement. Some other clients are shown in Figure 3.3.

3.6 Important for the exam

A few general rules that hold for all Chapters.

- Do not make category clashes (e.g., mixing up terms formulas structures theories satisfaction entailment consistency ...). Every category clash costs a penalty 1/20 on the exam. When you study the course, make sure you understand and learn the categories.
- When you phrase theorems, proofs, make sure all terms not broadly used in the course, but introduced in that theorem or proof or just before it are well-defined in your answer. Every unexplained term costs a penalty of 1/20.

Big questions:

- demonstrate and discuss LTC in a basic example scenario, e.g., the Turkey shooting problem.
- discuss different forms of inference in the context of dynamic systems
- heavyweight verification and proof of correctness

Understanding of the concepts and methodology of LTC:

- what the naive representation of Section 1 misses.
- the frame problem
- inertia, causation, actions,
- extensions: definitions, nondeterminism,

some clients



Figure 3.3 – Users of LogicBlox

- link with Strips language

Inference:

- different basic forms of inference of use in various applications.
- progression inference and basic understanding how it is used in LogicBlox
- lightweight verification
- heavyweight verification of invariants for bi-state LTC.
- programs and logic blox as declarative specifications of computer processes.
- execution of programs and logic blox is iterated progression inference.

There will be no detailed questions about LogicBlox.

Formal methods for design and verification of dynamic systems is a broad and heterogeneous area. It began in AI where the logic formalisms of Situation Calculus [24] and Event Calculus [25] were developed. LTC is strongly related to both. Other approaches are Alloy [5], Z, the B-method (see next two Chapters) and many more. Two more recent papers strongly arguing for formal methods are [23] and [21].

Chapter 4

LTL, CTL and verification by model checking

4.1 Model checking

Formal verification tools for dynamic systems comprise three parts:

1. a language (or system) for dynamic system specification;
2. a language for describing propositions to be verified;
3. one or more inference methods to establish whether the described dynamic system satisfies the propositions to be verified.

E.g.: in the previous chapter, we used $\text{FO}(\cdot)$ (adhering to the conventions of bistate LTC) for dynamic system specification, single-state formulas in $\text{FO}(\cdot)$ for goals and invariants, and we used IDP's inference tools for reasoning and problem solving.

In practice, there is often a big difference between dynamic system specification and the propositions one wants to verify. System specification is about transitions, causality, inertia, preconditions, concurrency. System properties to be verified focus on single state properties (invariants) or properties of possible evolutions of the system, such as fairness and deadlock. As such, quite a few systems use different languages for the task of system specification and property specification.

Languages used for *dynamic system specification* are:

- Classical logic or extensions of it as in linear-time calculus, situation calculus, Alloy (variant of predicate logic);
- Z, specification language using set-theoretical notations;
- Special-purpose languages such as traditional programming languages, Petri nets, abstract state machines, B, Event-B, CSP (communicating sequential processes), TLA+, etc.

Languages for describing *system properties*:

- Classical logic;
- Temporal logics, like those in this chapter: linear-time logic (LTL) and computation-tree logic (CTL).

The focus of this chapter is on the temporal logics for system property specification: LTL and CTL. To experiment with these languages, we will use the ProB tool. This tool supports the Event-B language for dynamic system specification and LTL and CTL for system property specification. It supports various forms of inference for these. Therefore, this chapter contains also a brief introduction to Event-B.

Model Checking In the field of (temporal) model checking, the following approach is taken.

- The dynamic system is specified as a *transition structure* $\mathcal{M}$. This is a transition graph between labelled states. Its definition follows. (see earlier on page ??). Various languages have been designed to describe transition structures. Here, the transition model will be specified in the Event-B language of the ProB system.
- System properties are represented by temporal logic formulas. We see three logics: linear-time logic (LTL) and computation tree logic (CTL), and their generalization CTL*.
- The verification inference method is model checking. The inference problem that it solves is defined as follows:

Model checking inference problem

Input: $\mathcal{M}$, state s in $\mathcal{M}$, temporal logic formula φ

Output: $(\mathcal{M}, s \models \varphi)$.

In words, the inference returns **t** if the transition structure $\mathcal{M}$ entails the temporal logic formula φ from state s , and **f** otherwise.

Now, we elaborate this idea.

Transition structures

Definition 4.1.1

A *transition structure* $\mathcal{M} = \langle S, \rightarrow, L \rangle$ over a propositional vocabulary Σ of propositional symbols consists of:

- S : the set of states;
- $\rightarrow$: a binary *transition* relation on S ;
- $L : S \rightarrow 2^\Sigma$: a mapping from states to sets of propositions, representing those true in the state.

We write $s_1 \rightarrow s_2$ to denote that there is a transition from state s_1 to state s_2 . For each state s , $L(s)$ is a structure of Σ representing the snapshot of the world at state s . E.g, for $\Sigma = \{p, q, r\}$, the value $L(s) = \{p, q\}$ represents that p, q are true and r is false in state s . A running example is presented in Figure 4.1.

A path in a transition structure is a representation of a (linear) evolution of the system. In this respect it corresponds to a linear time structure of the LTC. Hence, a transition structure can be seen as a compact representation of a set of linear time structures, namely the set of its paths.

Given an initial state s , a transition structure can be “unwound” from s as a tree of states. Paths from the root s in this tree represent the possible evolutions from s of the dynamic

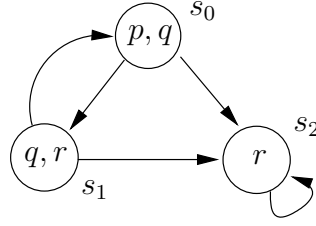
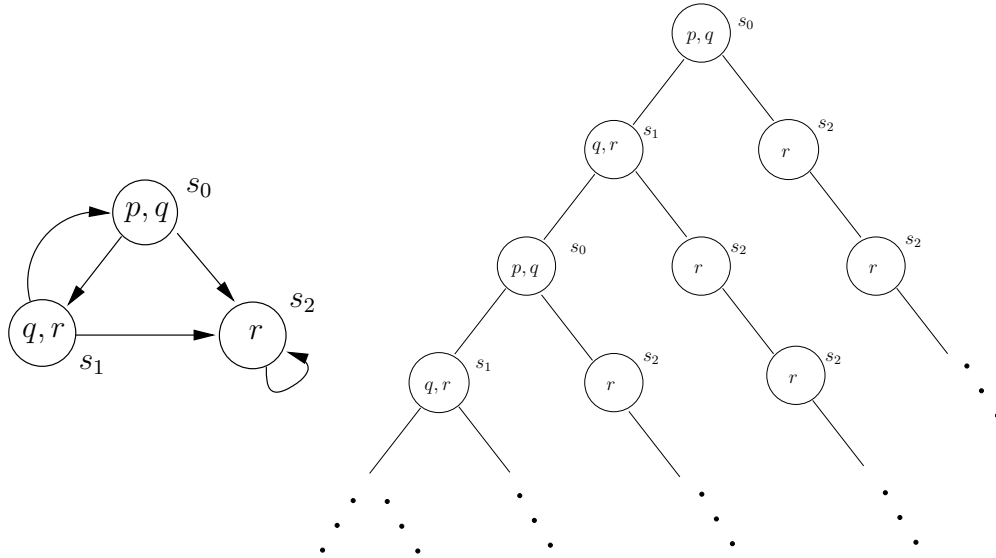


Figure 4.1 – A transition structure

Figure 4.2 – A transition structure and its unwound state tree in s_0 .

system expressed by $\mathcal{M}$. Such paths fulfil the same role as structures of an LTC theory. Figure 4.2 shows the unwinding of the running example from state s_0 .

Several sorts of extensions of transition systems are in use. Some extensions include one or a set of initial states. In some, edges of the transition relation are labelled by actions, sometimes called operations. In this course, we use *action* and *operation* as synonyms. Event-B describes transition structures with multiple initial states and transitions labelled with operations.

The concept of transition structure is strongly related to well-known concepts from theoretical computer science such as finite state machines (FSM) and automata. E.g, an automaton is a transition system extended with an initial state s_0 , a set $\mathcal{A}$ of accept states and labelled edges. It *accepts* a finite string $a_1 \dots a_n$ if this is the sequence of labels on edges in a path $s_0 \xrightarrow{a_1} \dots \xrightarrow{a_n} s_n$ where $s_n \in \mathcal{A}$.

Remark 4.1.1

In the text that follows, the theory of transition structures and temporal logics LTL and CTL is developed for propositional vocabularies only. This is for simplicity reasons. However, the ProB system supports predicate versions of the Event-B language and of LTL and CTL. In examples and exercises, we will sometimes use these predicate vocabularies both for system description and temporal logic formulas.

```

MACHINE Book
Sets
  Person={Bob, John, Mary};
  Book={B1,B2,B3}
VARIABLES Owns
INVARIANT
  Owns: Person Book &
  !b, p1.((p1,b): Owns => ! p2. ((p2,b): Owns => p1=p2))
INITIALISATION Owns := {(Bob,B1), (John,B2), (Mary,B3)}
OPERATIONS
  Give(g,b,r) = PRE (g,b): Owns & r: Person THEN
    Owns = (Owns - {(g,b)}) \ { (r,b) }
  END;
END

```

Figure 4.3 – The running example in ProB

4.1.1 ProB

ProB is an inference system developed at U. Dusseldorf by professor Michael Leuschel (a former PhD of DTAI). The system supports many languages. For dynamic system specification, it supports Event-B but also other languages, e.g. TLA+, Alloy ... It supports LTL and CTL for specification of process properties. ProB also supports fragments of second-order and higher-order logic.

The inference methods of ProB are for LTL and CTL model checking, and furthermore several of those provided by IDP for LTC. The core of ProB is a constraint solver for a very extended language including higher order features. ProB does not support inductive definitions.

A specification of the running example in ProB is given in Figure 4.3. ProB is introduced in the exercise sessions of this course, but here's a summary:

- Machines are theories.
- The Event-B language distinguishes between *variables* (inertial fluents) and *operations* (actions). Intuitively, operations correspond to action predicates in the Linear Time Calculus.
- Its syntax is set-based: “:” denotes the element relation $\in$. E.g., $(g,b):Owns$ expresses the same as $Owns(g,b)$ in FO. The symbol “\” in the assignment to *Owns* represents set union, and the symbol “-” represents set difference.
- The expression *Person Book* denotes the set of relations between *Person* and *Book*, i.e. $\mathcal{P}(Person \times Book)$.
- The syntax is action-oriented. An operation, is expressed in one block of the form “ $\langle operation \rangle(\langle parameters \rangle) = PRE \langle precondition \rangle THEN \langle effects \rangle END$ ”. The figure illustrates this for the operation *Give(g,b,r)*.
- Effects are expressed as assignments. They correspond to causations in LTC.
- Inertia is implicit. It is an implicature of the logic. Variables do not change value over time unless an operation takes place that assigns a value to them.

The transition structure characterized by an Event-B machine is *action-labelled*: transition edges are labelled by an action. Such structures assume that at each transition a unique

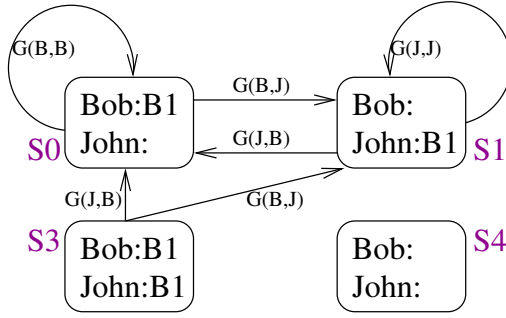


Figure 4.4 – The labelled transition graph of Machine Book

action/event occurs. They exclude concurrency of actions. As such, non-concurrency of operations is built-in in the language. This is an implicit assumption of the Event-B language.

An advantage of the action labelled transition structures is that they are more compact than the unlabelled transition structure. This is because different states which differ only by the action that occurs in it are compressed in one state, and the actions are moved to labels of outgoing edges.

The transition structure for a simplified version of the Event-B machine of Figure 4.3 that assumes $\text{Person}=\{\text{Bob}, \text{John}\}; \text{Book}=\{\text{B1}\}$ is displayed in Figure 4.4.

Transition structures modelled by LTC Transition structures are interesting to model in a finite way a potentially infinite class of temporal structures with infinite time. Intuitively, it is not so difficult to see that also some types of LTC theories T_a induce transition structures. In particular, this is the case if all fluents are inertial, actions have deterministic effects and the no-concurrency axiom (no simultaneous actions) holds. If in addition, a structure $\mathfrak{A}$ is given interpreting all non-time types and all static symbols, then it is easy to see that the class of models of T_a expanding $\mathfrak{A}$ form a forest with roots corresponding to different initial states and with branching due to the selection of different actions at the time points. Such a tree can be compacted in a transition structure. In that case, there is a unique action-labelled transition structure $\mathcal{M}$ that is equivalent with the combination of T_a and $\mathfrak{A}_i$ in the sense that a one-to-one correspondence exist between expansions of $\mathfrak{A}_i$ satisfying T_a , and paths of the transition structure starting at initial states.

A pair of a such a bistate LTC theory and such an input $\mathfrak{A}_i$ induces an equivalent transition structure. Let us call this the *state transition graph of T_a in $\mathfrak{A}_i$* .

Example 4.1.1

The transition structure of Figure 4.4 is the state transition graph of the LTC theory T_{Book} of Chapter 3 in the context of an input structure $\mathfrak{A}_i$ where $\text{Person}^{\mathfrak{A}_i} = \{\text{Bob}, \text{John}\}$ and $\text{Book}^{\mathfrak{A}_i} = \{\text{B1}\}$.

Exercise 4.1.1 Think of a way to compute the action labelled state transition graph for a bistate LTC theory in an input structure $\mathfrak{A}_i$ using the forms of inference for LTC seen in the previous Chapter.

4.2 Linear-Time Logic (LTL)

4.2.1 Syntax and semantics

In temporal logic, time is implicit (unlike in linear-time calculus). Temporal logic propositions do not contain explicit time terms and are evaluated relative to a state; they may be true in one state and false in another. Temporal logic propositions may talk about evolution in time and about the future or past using temporal connectives such as *always in the future*, *sometimes in the future*, *until* ...

We will see two logics implementing different sorts of time topologies.

- Linear-time logic (LTL) takes a linear topology for time. It serves to specify properties on a single path of the transition structure.
- Computation tree logic (CTL) takes a branching time topology. It serves to specify properties about multiple paths, i.e., different evolutions of the system. E.g., in CTL, we can express that some state has a path in which p is always true and also a path in which $\neg p$ is true in the future. This is not a contradiction. E.g., we can express that there is a path in which p is always true although at each time point, it is possible that p becomes false in the future. This means that although p is true at each state of the path, the state at each time point on the path has an alternative future in which p eventually becomes false.

Example 4.2.1

For an example of a structure where the proposition in the previous sentence is true, consider the transition structure in Figure 4.5. The loop at the left through state p has the desired property: p is true at each time point; however, at each time point there is an alternative future off the path in which p becomes false at the next state. This proposition cannot be expressed in LTC nor in LTL because it is not possible there to refer to alternative futures of the same state. It can be expressed in CTL thanks to its branching time topology.

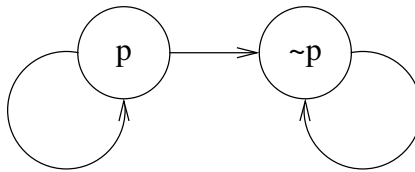


Figure 4.5 – A two-state transition structure

Remark 4.2.1

Do not confuse LTC and LTL.

- LTL (linear-time **logic**): a *propositional* logic with temporal modal operators.
- LTC (linear-time **calculus**): an LTC theory is a theory in an extension of FO expressing a dynamic domain following the *methodology* explained in Chapter 3.

We now proceed to formally define the logic LTL.

Syntax of LTL

Definition 4.2.1

The syntax of LTL sentences (over Σ) is defined by the following BNF:

$$\varphi ::= \begin{cases} \perp \mid \top \mid p \text{ (where } p \in \Sigma) \mid \\ (\neg\varphi) \mid (\varphi \wedge \varphi) \mid (\varphi \vee \varphi) \mid (\varphi \Rightarrow \varphi) \mid \\ (X\varphi) \mid (F\varphi) \mid (G\varphi) \mid (\varphi U \varphi) \mid (\varphi W \varphi) \mid (\varphi R \varphi) \end{cases}$$

The temporal connectives used are: X, F, G, U, W, R. They can be nested too: e.g., $G F p$ means “ p is true infinitely often in the¹ future”.

Informal semantics of LTL connectives An LTL formula is evaluated in a path of $\mathcal{M}$.

- $\perp$: false, $\top$: true
- $(X\phi)$: ϕ is true in the *neXt* state;
- $(F\phi)$: ϕ is true in some *Future* state, starting from now;
- $(G\phi)$: ϕ is *Globally* true. i.e., in all future states of the path, starting from now;
- $(\psi U \phi)$: ψ is true *Until* ϕ becomes true. More precisely, in some future state starting from now, ϕ is true, and in all preceding states starting from now, ψ is true. This expression is satisfied in a path in which ϕ is true in the first state.
- $(\psi W \phi)$: *Weak-Until*: the same as U, except that it is satisfied as well if ψ remains true forever (i.e. ϕ doesn’t have to become true).
- $(\psi R \phi)$: ψ *Releases* ϕ : ϕ is true at least until in a current or future state in which ψ is true. Also satisfied if ϕ stays true forever.

R is similar to W except that the roles of arguments is exchanged and for $\psi W \varphi$, ψ needs to remain true until the state preceding the state satisfying φ , while for $\varphi R \psi$, ψ needs to remain true until and including the state satisfying φ , as if ψ “hands over” its truth to φ . As such it holds that $\psi R \varphi$ and $\varphi W(\varphi \wedge \psi)$ are equivalent.

LTL connectives do not mean the same as in natural language as the following example shows.

$$smoke U sick = \text{I will smoke until I get sick?}$$

Not really. It means that there will come a time that I am sick and before that time, I will smoke. I might go on smoking afterwards. A more correct translation of the natural language statement is $(smoke \wedge \neg sick) W(sick \wedge G \neg smoke)$.

In LTL the future includes the present time. As a consequence, valid statements, true in each path are:

- $G p \Rightarrow p$
- $p \Rightarrow q U p$
- $p \Rightarrow F p$

¹Be careful! This formula does *not* mean that “the truth of p always stays in the future”. Rather, what G does is duplicate the verification of $F p$ to every state on the path, and these copies have no knowledge of each other, nor of G.

Binding conventions of LTL are:

- Unary connectives bind most tightly.
- Then in order $U, R, W, \wedge, \vee, \Rightarrow$.

E.g., $F p \wedge G q \vee \neg q U p$ stands for $((F p) \wedge (G q)) \vee ((\neg q) U p)$.

Formal semantics LTL formulas are evaluated in the context of a transition structure $\mathcal{M}$. The satisfaction relation $\models$ is between paths in $\mathcal{M}$ and LTL propositions.

We call a state s of $\mathcal{M}$ *final* if it has no outgoing edges. In general, we focus on transition structures $\mathcal{M}$ without final states. This entails that every finite path in $\mathcal{M}$ can be extended to an infinite path. Such infinite paths represent possible full, completed histories of the world. The absence of final states embodies the idea that nothing can happen that stops time. This assumption is also made in the LTC, since there the time type T is interpreted by the natural numbers.

The limitation to transition structures without final states is not restrictive. A transition structure $\mathcal{M}$ with final states can easily be turned into an equivalent one $\mathcal{M}'$ without final states, by adding a dummy state s_d and extending $\rightarrow$ with edges to s_d from all final states and from s_d itself. Formally:

- $S' := S \cup \{s_d\}$
- $\rightarrow' := (\rightarrow) \cup \{(s_d, s_d), (s, s_d) \mid s \text{ is a final state of } \mathcal{M}\}$

The semantics of temporal logic is expressed in terms of *infinite paths* in $\mathcal{M}$. This is as in LTC, where time is also infinite.

Definition 4.2.2

A *path* in $\mathcal{M} = \langle S, \rightarrow, L \rangle$ is an infinite sequence of states $\langle s_0, s_1, s_2, \dots \rangle$ such that $s_i \rightarrow s_{i+1}$ for $i \geq 0$.

We use the mathematical variable π to denote paths. Paths are often written informally as $s_0 \rightarrow s_1 \rightarrow \dots$. The state s_i in such a path is called the state at time i in the path. The *suffix path* of a path π starting at s_i is denoted π^i . E.g., π^3 denotes the path $s_3 \rightarrow s_4 \rightarrow s_5 \rightarrow \dots$. A special case is that $\pi^0 = \pi$. A path starting at state s will sometimes be denoted as $s \rightarrow \dots$ or $s \rightarrow s_1 \rightarrow \dots$.

Definition 4.2.3: LTL path satisfaction

Given $\mathcal{M} = \langle S, \rightarrow, L \rangle$ over Σ . Let $\pi = s_0 \rightarrow \dots$ be a path in $\mathcal{M}$ and φ an LTL formula over Σ . We define that π (not s yet) satisfies φ (notation $\mathcal{M}, \pi \models \varphi$) by induction on the structure of φ :

- $\mathcal{M}, \pi \models \top$ (and $\pi \not\models \perp$)
- $\mathcal{M}, \pi \models p$ if $p \in L(s_0)$
- the normal rules for logical connectives $\neg, \wedge, \vee, \Rightarrow$
E.g., $\mathcal{M}, \pi \models (\psi \Rightarrow \varphi)$ if $\mathcal{M}, \pi \models \varphi$ or $\mathcal{M}, \pi \not\models \psi$
- $\mathcal{M}, \pi \models X\varphi$ if $\mathcal{M}, \pi^1 \models \varphi$
- $\mathcal{M}, \pi \models G\varphi$ if, for all $i \geq 0$, $\mathcal{M}, \pi^i \models \varphi$
- $\mathcal{M}, \pi \models F\varphi$ if, for some $i \geq 0$, $\mathcal{M}, \pi^i \models \varphi$

- $\mathcal{M}, \pi \models \psi \cup \varphi$ if there exists $i \geq 0$ such that $\mathcal{M}, \pi^i \models \varphi$ and for all $j = 0, \dots, i-1$, $\mathcal{M}, \pi^j \models \psi$
- $\mathcal{M}, \pi \models \psi \text{ W } \varphi$ (Weak until) if
 - either there exists $i \geq 0$ such that $\mathcal{M}, \pi^i \models \varphi$ and for all $j = 0, \dots, i-1$, $\mathcal{M}, \pi^j \models \psi$,
 - or for all $j \geq 0$, $\mathcal{M}, \pi^j \models \psi$
- $\mathcal{M}, \pi \models \psi \text{ R } \varphi$ (Release) if
 - either there exists $i \geq 0$ such that $\mathcal{M}, \pi^i \models \psi$ and for all $j = 0, \dots, i$, $\mathcal{M}, \pi^j \models \varphi$,
 - or for all $j \geq 0$, $\mathcal{M}, \pi^j \models \varphi$

If $\mathcal{M}$ is clear from the context, we write $\pi \models \varphi$ rather than $\mathcal{M}, \pi \models \varphi$. The place where $\mathcal{M}$ is used in the definition is at the base case: $\mathcal{M}, \pi \models p$ if $p \in L(s)$.

Now we define a second satisfaction relation between states and LTL formulas. An LTL formula is satisfied in state s if it is satisfied in each path from state s .

Definition 4.2.4: LTL state satisfaction

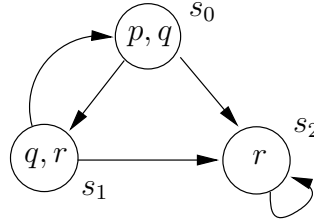
Let $\mathcal{M} = \langle S, \rightarrow, L \rangle$ and $s \in S$.

$\mathcal{M}, s \models \varphi$ if for each path $\pi = s \rightarrow \dots$ in $\mathcal{M}$ starting in s , it holds that $\mathcal{M}, \pi \models \varphi$

Example 4.2.2

What is true?

- $\mathcal{M}, s_0 \models p \wedge q?$ true
- $\mathcal{M}, s_0 \models \neg r?$ true
- $\mathcal{M}, s_0 \models \text{X } r?$ true
- $\mathcal{M}, s_0 \models \text{G } \neg(p \wedge r)?$ true
- $\mathcal{M}, s_0 \models \text{F G } r?$ false; counterexample $(s_0 \rightarrow s_1 \rightarrow)^\infty$
- for all s , $\mathcal{M}, s \models \text{F}(\neg q \wedge r) \Rightarrow \text{F G } r?$ true
- $\mathcal{M}, s_0 \models \text{F G } r \Rightarrow \text{G F } r?$ true



4.2.2 Translation to FO

LTL formulas can easily be translated into FO formulas of LTC. Given a propositional LTL vocabulary Σ , we use the vocabulary Σ_{LTC} consisting of a unary predicate symbol $p(\text{T})$ for every $p \in \Sigma$.

An LTL formula φ is translated into a FO formula that expresses φ is true at some time term t . The corresponding formula is denoted φ^t . This formula is defined by induction:

- $p^t = p(t)$, where $p \in \Sigma$;
- $(\psi \wedge \phi)^t = \psi^t \wedge \phi^t$, and the same for $\vee, \Rightarrow, \neg$
- $(\text{X } \varphi)^t = \varphi^{t+1}$

- $(G \varphi)^t = \forall t_1 (t_1 \geq t \Rightarrow \varphi^{t_1})$
- $(F \varphi)^t = \exists t_1 (t_1 \geq t \wedge \varphi^{t_1})$
- $(\psi U \varphi)^t = \exists t_1 (t_1 \geq t \wedge \forall t_2 ((t \leq t_2 < t_1) \Rightarrow \psi^{t_2}) \wedge \varphi^{t_1})$

Exercise 4.2.1 Extend the translation for W, R .

Example 4.2.3

We apply the translation to compute:

$$(F G r \Rightarrow G F r)^0 = ?$$

The result is:

$$\begin{aligned} \exists t_1 (t_1 \geq 0 \wedge \forall t_2 (t_2 \geq t_1 \Rightarrow r(t_2))) \Rightarrow \\ \forall t_1 (t_1 \geq 0 \Rightarrow \exists t_2 (t_2 \geq t_1 \wedge r(t_2))) \end{aligned}$$

Recall that a path $\pi = s_0 \rightarrow \dots$ corresponds to an LTC structure $\mathfrak{A}_\pi$ such that $L(s_i)$ corresponds to the snapshot in $\mathfrak{A}_\pi$ at time i .

Theorem 4.2.1

$\pi \models \varphi$ iff $\mathfrak{A}_\pi \models \varphi^0$.

No proof.

We see that LTL formulas translate to FO formulas that range over the entire natural numbers. E.g., GFp . In general, these formulas cannot “safely” be evaluated in a finite interval of time points. So finite model generation does not work well. Special techniques were developed to reason efficiently about such formulas. LTL was developed from this work.

4.2.3 Practical patterns of specifications

- $G \neg(\text{started} \wedge \neg \text{ready})$: it’s impossible to get to a state where *started* holds and not *ready*.
- $G(\text{request} \Rightarrow F \text{acknowledged})$: if a *request* of some resource occurs, it will eventually be *acknowledged*.
- $G F \text{enabled}$: a certain device is *enabled* infinitely often.
- $F G \text{waiting}$: a process will run into a state of eternal *waiting*.
- $G(\text{floor}_2 \wedge \text{up} \wedge \text{ButtonPressed}_5 \Rightarrow (\text{up} U \text{floor}_5))$: an upward travelling elevator at second floor does not change direction when it has a request for the 5th floor.

Propositions that cannot be expressed in LTL:

- Statements that there exists a path to some state, or equivalently, that it is *possible* to reach a certain state. E.g. in any state, it is *possible* to get to a *ready* state.
- More in general, statements expressing that there is a path satisfying certain properties. E.g. The lift *could* remain on the third floor with its doors closed forever.
- For this sort of properties, we need CTL.

4.2.4 Important equivalences of LTL

Definition 4.2.5

Two LTL formulas φ, ψ are equivalent (written $\varphi \equiv \psi$), if for all transition structures $\mathcal{M}$ and all paths π in $\mathcal{M}$, it holds that $\mathcal{M}, \pi \models \varphi$ iff $\mathcal{M}, \pi \models \psi$.

As a consequence, when $\varphi \equiv \psi$, it holds that $\mathcal{M}, s \models \varphi$ iff $\mathcal{M}, s \models \psi$.

Duality All connectives and operators have a *dual* connective or operator. E.g., $\wedge$ and $\vee$ are called *dual*, since $\neg(\psi \wedge \phi) \equiv \neg\psi \vee \neg\phi$, and $\neg(\psi \vee \phi) \equiv \neg\psi \wedge \neg\phi$. Similarly, temporal operators have a dual operator:

- $\neg X\phi \equiv X\neg\phi$ is dual to itself (X is self-dual): “not next” = “next not”.
- $\neg G\phi \equiv F\neg\phi$ is dual to $\neg F\phi \equiv G\neg\phi$: “not always” = “at some point not”.
- $\neg(\phi U \psi) \equiv \neg\phi R \neg\psi$ is dual to $\neg(\phi R \psi) \equiv \neg\phi U \neg\psi$

Other useful equivalences are as follows:

- Turning unary into binary operators: $F\phi \equiv \top U \phi$ and $G\phi \equiv \perp R \phi$ or $G\phi \equiv \phi W \perp$
- $\phi U \psi \equiv \phi W \psi \wedge F\psi$
- $\phi W \psi \equiv \phi U \psi \vee G\phi$
- $\phi W \psi \equiv \psi R(\psi \vee \phi)$
- $\phi R \psi \equiv \psi W(\phi \wedge \psi)$

Exercise 4.2.2 Prove these equivalences using the definition of satisfaction.

It follows from these equivalences that every temporal connective operator except X can be expressed in terms of U. In turn, U can be expressed in terms of W as well as in terms of R. It follows that all temporal connectives can be written in terms of those operators as well.

Exercise 4.2.3 Express $p W q$ in terms of U.

Adequate sets of LTL connectives The above equivalences imply that not all connectives need to be implemented in an LTL model checking algorithm. Only a few operators need to be “implemented”, much like NAND and NOR are universal electronic gates that can form all other gates.

X is needed because it cannot be expressed by other connectives. Of the remaining temporal connectives, only one from $\{W, U, R\}$ is required. Each of the following sets is called an *adequate set* of LTL connectives:

$$\{U, X\}, \{R, X\}, \{W, X\}.$$

Each temporal formula can be translated into a formula using only the temporal connectives of an adequate set.

Exercise 4.2.4 Verify that all temporal connectives can be expressed using X and U.

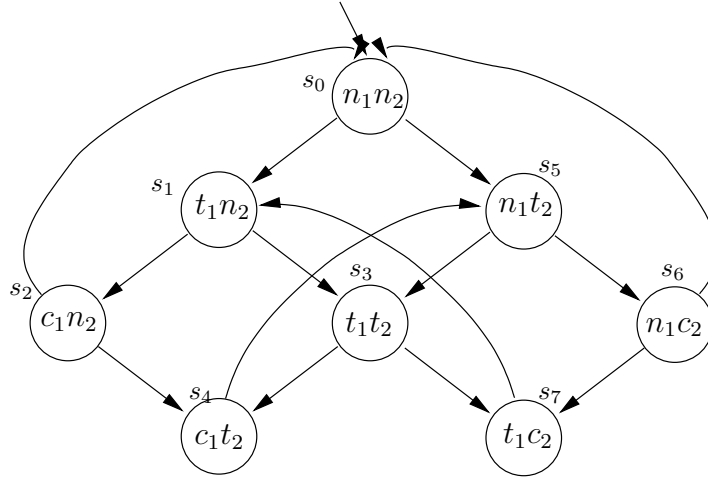


Figure 4.6 – The mutual exclusion transition structure

4.2.5 Example: mutual exclusion (locks)

We apply the modelling methodology on a simple protocol to ensure that concurrent processes do not access a shared resource simultaneously. This is done by identifying *critical sections* of the code, and taking care that only one process can be in its critical section at the same time.

Processes apply to enter their critical section with the protocol. The protocol determines which process is allowed to enter.

Desired properties of the protocol:

- Only one process in its critical section at any time.
- If a process requests to enter, it will eventually be permitted.
- A process can always request to enter its critical section.
- The two processes do not need to alternate in entering their critical section. That is, it should be possible that one process enters its critical section two times in a row.

The transition structure The protocol is expressed as a transition structure $\mathcal{M}$. We assume that there are only two processes 1 and 2, and a single lock.

- Vocabulary $\Sigma = \{n_i, t_i, c_i \mid i \in \{1, 2\}\}$. Each process i can be in one of three phases:
 - n_i : normal,
 - t_i : applying for its critical section (“spinning”),
 - c_i : in its critical section (“holding”).
- A state of the transition structure is made up by a pair (p_1, p_2) , with $p_1, p_2 \in \{n, t, c\}$.

We assume *asynchronous interleaving*, or alternatively *atomicity* of the lock: processes don’t change simultaneously. The proposed transition structure is given in Figure 4.6.

Exercise 4.2.5 Write an LTC theory to represent this application. Introduce types $Process = \{1, 2\}$, $State = \{n, t, c\}$, inertial fluent function symbol $CurState(Process, T)$: $State$ and actions $Request(Process, T)$, $EnterCriticalZone(Process, T)$ and $LeaveCriticalZone(Process, T)$.

Verifying desired propositions

- *Safety*: Only one process in its critical section at any time:

$$G \neg(c_1 \wedge c_2)$$

Is it satisfied in s_0 ? yes.

- *Liveness*: If a process requests to enter the critical section, it will eventually be permitted:

$$G(t_i \Rightarrow F c_i)$$

Is it satisfied in s_0 ? No.

- *Non-blocking*: It is always possible for a process to request to enter its critical section. More precisely, each non-critical state n_i has a requesting next state t_i .
A query for a *possible* path: expressible in CTL but not in LTL!
Is it satisfied in s_0 ? Yes.

- *No strict sequencing*: processes need not enter their critical section in a strict alternation.

We want to verify that there is a path with the property that “*it has two distinct states satisfying c_1 such that c_1 is false in at least one intermediate state and c_2 is false in all intermediate states*”.

In LTL, we cannot express this property directly, since there is no way to express *there is a path such that* However, we can still encode it, namely by expressing the negation of the subproposition “it has two distinct states ...”. The negation is “whenever c_1 is true, then c_1 remains true forever or until a state where c_1 remains false forever or until a state that c_2 is true”. This is encoded by

$$G(c_1 \Rightarrow c_1 W(\neg c_1 \wedge (\neg c_1 W c_2)))$$

There is a path from s_0 that does not satisfy this.

Which one? $s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow s_0 \rightarrow s_1 \rightarrow s_2 \dots$

Therefore $\mathcal{M}, s_0 \not\models G(c_1 \Rightarrow c_1 W(\neg c_1 \wedge \neg c_1 W c_2))$. This entails the desired conclusion, namely that there is no strict sequencing. The falsifying path of this formula is the path we were looking for.

For the last proposition, we applied a coding trick. LTL does not offer the possibility to express properties of the kind “some path leaving from state s satisfies proposition ϕ ”. However, suppose that $\neg\phi$ can be expressed as an LTL formula ψ . Then $\mathcal{M}, s \not\models \psi$ holds exactly when there is a path leaving from s that satisfies $\neg\psi$, hence a path that satisfies ϕ .

This encoding trick exploits the fact that satisfaction in a state is not closed under negation: $\mathcal{M}, s \not\models \phi \not\equiv \mathcal{M}, s \models \neg\phi$.

- $\mathcal{M}, s \not\models \phi$ means that ϕ is false in at least one path from s .
- $\mathcal{M}, s \models \neg\phi$ means ϕ is false in each path from s .

The trick does not work for all properties “there exists a path ...”. The property “*for each path leading to a state t_i , there is a path from that state to a state with c_i* ” is not expressible using this trick because its negation includes “there is a path ...” which cannot be expressed in LTL.

We conclude that the locking protocol contains a bug since it does not ensure fairness.

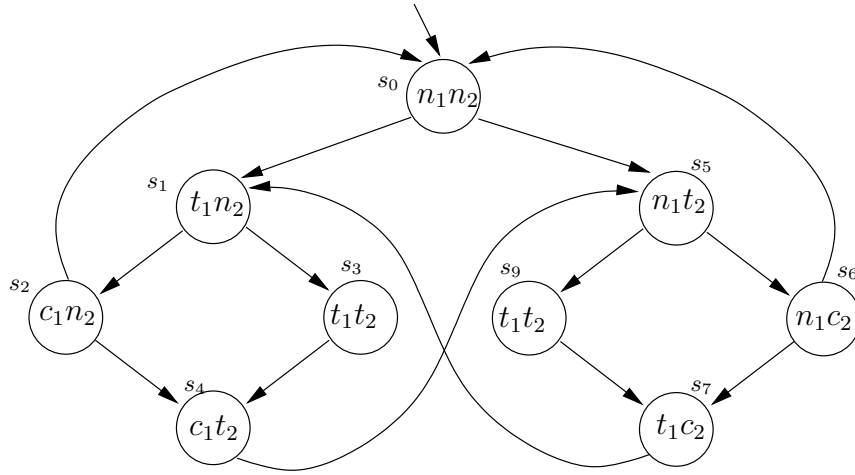


Figure 4.7 – A fair mutual exclusion transition model

A correct model To solve the liveness problem, we may split the state t_1t_2 to remember which process applied first and give it priority over the other. A transition structure corrected for fairness is in Figure 4.7.

4.2.6 Mutual exclusion in ProB

A ProB theory expressing the mutual exclusion protocol is given in Figure 4.8. Some explanation follows:

- **SETS: Process, PrState:** this declares two set symbols and specifies their value. Their use is similar as type domains in IDP.
- **VARIABLES: Turn, State:** they are inertial fluent functions.
- **DEFINITIONS:**
 - **0tPr** is the static function mapping a process to the other process.
 - Note the function notation $p1 \mapsto p2$.
 - LTL statements are defined here: they do not belong to the modelling. Conditions in LTL are written between brackets, as in $GF(\{State(p1)=w\})$. Observe that this extends propositional LTL.
- **INVARIANT:** here the types of fluents are declared together with other state formulas that should be invariant.
 - **Process --> Process** is the set of total functions from Process to Process.
- **INITIALISATION:** the expression contains a concurrent assignment to **Turn** and **State**. The operator $||$ is used to express concurrency, while $:=$ is the standard assignment symbol.
- **OPERATIONS:** this describes preconditions in **PRE** and effects/assignments in **THEN** expressions.
- In the precondition of **Enter**, the expression $\#x. (p(x))$ is an existential quantification.

For more information, see exercise sessions and the following URLs:

http://www3.hhu.de/stups/prob/index.php?title=LTL_Model_Checking

[http://www3.hhu.de/stups/prob/index.php/Mutual_Exclusion_\(Fairness\)](http://www3.hhu.de/stups/prob/index.php/Mutual_Exclusion_(Fairness))

```

MACHINE MutualExclusion
SETS
    Process={p1, p2};
    PrState={n, w, c}
VARIABLES Turn, State
DEFINITIONS
    OtPr == {p1|->p2, p2|->p1};
    ASSERT_LTL == "GF({State(p1)=w}) => GF({State(p1)=c})";
    ASSERT_LTL1 == "GF({State(p2)=w}) => GF({State(p2)=c})"
INVARIANT
    OtPr: Process-->Process &
    State: Process-->PrState &
    Turn: Process &
    not(State(p1)=c & State(p2)=c)
INITIALISATION
    Turn := p1 ||
    State := { p1|->n, p2 |-> n }
OPERATIONS
    Request(p) =
        PRE State(p) = n
        THEN State(p) := w
        END;
    Enter(p) =
        PRE
            State(p) = w &
            not(\#x.(State(x)=c)) &
            (State(OtPr(p))=w => Turn=p)
        THEN
            State(p) := c ||
            Turn := OtPr(p)
        END;
    Release(p) =
        PRE State(p) = c
        THEN State(p) := n
        END
END

```

Figure 4.8 – Mutual exclusion in ProB

4.3 Fairness constraints

Example 4.3.1

A path on which c_2 is true *infinitely often* in Figure 4.7 is

$$s_0 \rightarrow s_1 \rightarrow (s_2 \rightarrow s_4 \rightarrow s_5 \rightarrow s_6 \rightarrow s_7 \rightarrow s_1 \rightarrow)^*$$

We say this path is fair with respect to c_2 . A path on which c_2 is **not** true *infinitely often*:

$$s_0 \rightarrow s_5 \rightarrow s_6 \rightarrow s_0 \rightarrow (s_1 \rightarrow s_2 \rightarrow s_0 \rightarrow)^*$$

This path is “unfair” with respect to c_2 .

Definition 4.3.1: Fairness

A path π is fair with respect to some proposition φ if there are infinitely many $i \in \mathbb{N}$ such that $\pi^i \models \varphi$.

Theorem 4.3.1

A path π of transition structure $\mathcal{M}$ is fair with respect to φ iff $\mathcal{M}, \pi \models \text{GF } \varphi$.

Exercise 4.3.1 Prove this

Many dynamic systems show natural fairness conditions.

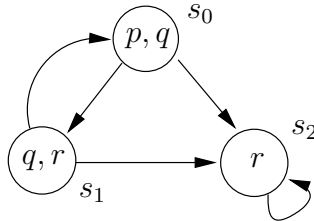
- A process requests access to its critical section infinitely often.
- A process does not stay forever in its critical section. It means $\neg c_i$ is true infinitely often.
- For every floor, an elevator is called to it infinitely often.
- An elevator reaches all floors at regular times.
- Elevator requests are not ignored for eternity.

Fairness conditions, in general, cannot be expressed by a transition system. Therefore, some model checking systems offer the possibility to model the system by a combination of a transition structure $\mathcal{M}$ and a set C of fairness constraints.

Verification tasks of the system are relative to the set of paths of $\mathcal{M}$ that satisfy all fairness constraints $c \in C$.

Example 4.3.2

Assume the running example is extended with the fairness constraint $\neg q$.



The set of possible paths is $\{(s_0 \rightarrow s_1 \rightarrow)^n \rightarrow (s_2 \rightarrow)^\infty \mid n \in \mathbb{N}\}$. This system entails

| F G r while the original system did not.

Exercise 4.3.2 How to express a fairness constraint φ in LTC? Can IDP reason on such constraints?

4.4 CTL and CTL*: branching time logics

The motivation for computation tree logic (CTL) is that propositions that express the existence of a path cannot be expressed in LTL. Sometimes, their negation can be expressed (the encoding trick!), but not in general. Yet, in many applications, existential statements are useful. For example, take the proposition “from each reachable state in which P holds ($\forall$), we can reach a state in which Q holds ($\exists$)”. This cannot be expressed in LTL. Neither can its negation “there is a reachable state in which P holds ($\exists$), from which Q is false on all paths ($\forall$)”.

In branching-time logic, one can quantify over paths, both universally and existentially. We see two branching time logics: CTL (computation tree logic) and CTL*, a generalization of both CTL and LTL. We begin with CTL*.

4.4.1 Syntax of CTL*

CTL* largely shares its syntax with LTL. Two more connectives are introduced: $A[\cdot]$ expresses “for all paths from this state”, and $E[\cdot]$ expresses “there exists a path from this state”. We saw how LTL formulas could be satisfied by paths and by states, where the latter implicitly quantified over all paths from the given state. Since CTL* explicitly quantifies over paths, it is more strict about this: formulas are classified as state or path formulas depending on which they can be evaluated in.

Definition 4.4.1: Syntax of CTL*

We define *state formulas* and *path formulas* over vocabulary Σ by mutual induction using the following BNF:

- State formulas φ

$$\varphi ::= \left\{ \begin{array}{l} \top \mid \perp \mid p \in \Sigma \mid \\ (\neg\varphi) \mid (\varphi \wedge \varphi) \mid (\varphi \vee \varphi) \mid (\varphi \Rightarrow \varphi) \mid \\ A[\alpha] \mid E[\alpha] \end{array} \right.$$

- Path formulas α

$$\alpha ::= \left\{ \begin{array}{l} \varphi \\ (\neg\alpha) \mid (\alpha \wedge \alpha) \mid (\alpha \vee \alpha) \mid (\alpha \Rightarrow \alpha) \mid \\ (X\alpha) \mid (F\alpha) \mid (G\alpha) \mid \\ (\alpha U \alpha) \mid (\alpha W \alpha) \mid (\alpha R \alpha) \end{array} \right.$$

Notice the simultaneous induction of state and path formulas. As mentioned, we discern:

- *State formulas*: propositions about a state, hence to be evaluated in a state.
- *Path formulas*: propositions about a path, hence to be evaluated in a path.

Propositional formulas are about the current state, hence they are state formulas. The path quantifiers $A[\cdot]$ and $E[\cdot]$ quantify over paths starting from the current state, and hence

clearly make state formulas.

The temporal quantifiers $X, F, G, U, (W, R)$ of LTL are evaluated in a path, and thus belong to the path formulas. Furthermore, any state formula can be evaluated in a path, namely in the first state of that path: hence, a state formula is a special path formula.

The inverse is not the case: the only way a path formula could be evaluated in a state is by implicit quantification like in LTL, but we forbade this, so it isn't syntactically allowed in CTL*. E.g., the path formula $F\varphi$ expresses that eventually φ will become true – a property of a single path. To evaluate this in a state, we have to choose between universal and existential quantification over its paths, making the state formulas $A[F\varphi]$ and $E[F\varphi]$ respectively.

Binding priorities for CTL* are almost the same as for LTL

1. First unary connectives: $\neg, X, F, G, A, E$;
2. Then, in order: $U, R, W, \wedge, \vee, \Rightarrow$

E.g., $A[p \wedge q W \neg p]$ is shorthand notation for $A[p \wedge (q W \neg p)]$.

4.4.2 Semantics of CTL*

Definition 4.4.2

Given $\mathcal{M} = \langle S, \rightarrow, L \rangle$. We define the satisfaction relation for state formulas and path formulas by mutual induction on the structure of formulas:

- State formulas: for each state s and state formula φ , we define $\mathcal{M}, s \models \varphi$:
 - the standard rules for $\perp, \top$, atoms, $\neg, \wedge, \vee, \Rightarrow$;
 - $\mathcal{M}, s \models A[\alpha]$ if for each path $\pi = s \rightarrow \dots$ in $\mathcal{M}$, it holds that $\mathcal{M}, \pi \models \alpha$;
 - $\mathcal{M}, s \models E[\alpha]$ if for some path $\pi = s \rightarrow \dots$ in $\mathcal{M}$, it holds that $\mathcal{M}, \pi \models \alpha$;
- Path formulas: for each path $\pi = s_0 \rightarrow \dots$ and path formula α , we define $\mathcal{M}, \pi \models \alpha$:
 - the standard rules for $\neg, \wedge, \vee, \Rightarrow$,
 - the LTL rules for X, F, G, U, W, R .
 - if φ is a state formula: $\mathcal{M}, \pi \models \varphi$ if $\mathcal{M}, s_0 \models \varphi$.

Definition 4.4.3

Two path formulas are equivalent if they are satisfied in the same transition structures $\mathcal{M}$ and paths π . Two state formulas are equivalent if they are satisfied in the same transition structures $\mathcal{M}$ and states s .

Some dualities and LTL equivalences, that again show R and W to be “redundant”:

- $\neg A[\alpha] \equiv E[\neg\alpha]$
- $\neg F\alpha \equiv G\neg\alpha$
- $\neg(\alpha R \beta) \equiv \neg\alpha U \neg\beta$
- $\alpha W \beta \equiv \alpha U \beta \vee G\alpha$

4.4.3 Embedding LTL in CTL*

Syntactically, any CTL* path formula without A, E is an LTL formula. An LTL formula α is evaluated in a path. The corresponding CTL* path formula is α itself.

An LTL formula α can also be evaluated in a state s according to the following rule: $\mathcal{M}, s \models \alpha$ iff for every path $\pi = s \rightarrow \dots$, $\pi \models \alpha$. This shows that the use of an LTL formula α as a state formula corresponds to the state formula $A[\alpha]$.

Proposition 4.4.1

$\mathcal{M}, s \models_{LTL} \alpha$ iff $\mathcal{M}, s \models_{CTL^*} A[\alpha]$.

When using LTL formulas as state formulas, the universal path quantifier is used implicitly. In CTL*, this quantifier is to be explicited. We call $A[\alpha]$ the *embedding* of the LTL formula α as a state formula.

4.5 CTL: a subformalism of CTL*

4.5.1 Syntax and semantics CTL

CTL formulas are CTL* state formulas in which all temporal connectives come in pairs:

- AX and EX
- AF and EF
- AG and EG
- $A[\alpha U \varphi]$ and $E[\alpha U \varphi]$

Definition 4.5.1

Syntax of CTL. We define the formulas of CTL by the BNF:

$$\varphi ::= \left\{ \begin{array}{l} \perp \mid \top \mid p \mid \\ (\neg\varphi) \mid (\varphi \wedge \varphi) \mid (\varphi \vee \varphi) \mid (\varphi \Rightarrow \varphi) \mid \\ AX\varphi \mid EX\varphi \mid AF\varphi \mid EF\varphi \mid AG\varphi \mid EG\varphi \mid \\ A[\varphi U \varphi] \mid E[\varphi U \varphi] \end{array} \right.$$

Each CTL formula is a CTL* state formula.

Informal semantics of CTL Expressions are evaluated in a some node s of the tree:

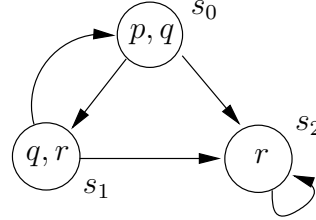
- AX φ : for each path from s , φ holds in the next state;
i.e., in all next states, φ holds
- EX φ : in some path from s , φ holds in the next state;
i.e., in some next state, φ holds
- AF φ : each path goes through a state in which φ is true;
- EF φ : some path goes through a state in which φ is true;
i.e., φ is true in some state equal to or reachable from s ;
- AG φ : in each path, φ is true in all states;
i.e., φ is true in all states equal to or reachable from s ;
- EG φ : in some path, φ is true in all states;

- $A[\alpha U \phi]$: in each path, $\alpha U \phi$ is true;
- $E[\alpha U \phi]$: in some path, $\alpha U \phi$ is true;

The meaning of some CTL expressions is illustrated in Figures 4.9 and 4.10.

Exercise 4.5.1 Which formulas are true in the given transition structure?

- $\mathcal{M}, s_0 \models EX(q \wedge r)$?
- $\mathcal{M}, s_0 \models \neg A(p U r)$?
- $\mathcal{M}, s_0 \models E[(p \wedge q) U r]$?
- $\mathcal{M}, s_0 \models EG(q \wedge EG \neg q)$?
- $\mathcal{M}, s_0 \models EG(q \wedge EX AG \neg q)$?
- $\mathcal{M}, s_0 \models AG(p \vee q \vee r \Rightarrow EF EG r)$?



The following example illustrates how to translate certain propositions in CTL.

Example 4.5.1

Suppose we want to express the following proposition in CTL:

Once $p \wedge q$ becomes true, then possibly $p \wedge q$ remains true forever.

This statement is presumably about the initial state s_0 of the system). The tree of paths from s_0 in is an abstract representation of the class of potential behaviors or starting from s_0 . The first part of the sentence “once $p \wedge q$ becomes true, then ...” states that in all states s reachable from s_0 and satisfying $p \wedge q$, some property P holds. Its translation is:

$$AG(p \wedge q \Rightarrow \varphi_P)$$

The property P that is to hold in all those states s , is “possibly $p \wedge q$ remains true forever”. It states that there is a potential behavior starting at s in which $p \wedge q$ globally holds. Thu, property P can be expressed as $EG p \wedge q$. We obtain:

$$AG(p \wedge q \Rightarrow EG p \wedge q)$$

By breaking up the natural language sentence in pieces, and translating it piece by piece, we obtain the correct CTL proposition.

Take the alternative sentence

Once $p \wedge q$ becomes true, we cannot exclude that $p \wedge q$ remains true.

To express this in CTL, the first part leading to states s is the same as with the previous sentence:

$$AG(p \wedge q \Rightarrow \varphi_Q)$$

Now Q is the statement “we cannot exclude that $p \wedge q$ remains true”.

When is it excluded that the property $p \wedge q$ remains true? If every path leaving s has a future state where $p \wedge q$ is false. This is expressed as $AF \neg(p \wedge q)$. This should not be

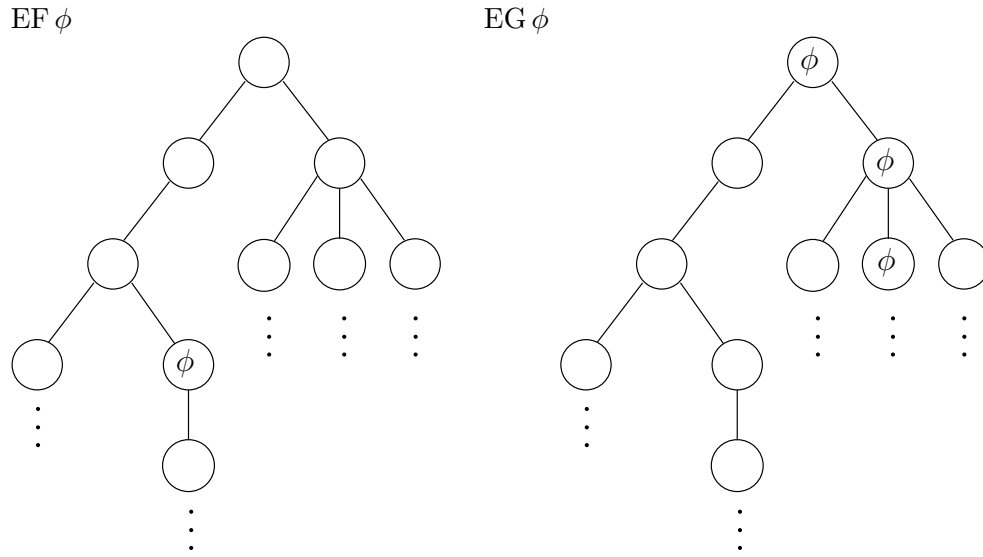


Figure 4.9 – Truth of CTL formulas

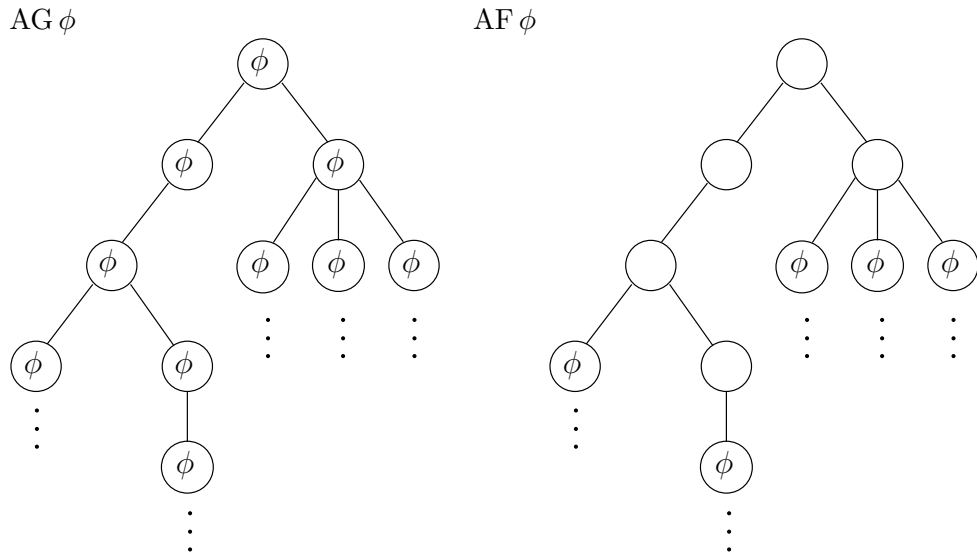


Figure 4.10 – Truth of CTL formulas

the case. Hence, Q is expressed as:

$$\neg \text{AF } \neg(p \wedge q)$$

The full sentence is then:

$$\text{AG}(p \wedge q \Rightarrow \neg \text{AF } \neg(p \wedge q))$$

An equivalent but positive way to rephrase the property q is that in some path leaving s , $p \wedge q$ remains true, expressed as $\text{EG } p \wedge q$. It leads to the following equivalent CTL proposition:

$$\text{AG}(p \wedge q \Rightarrow \text{EG } p \wedge q)$$

It appears that the two natural language statements in this example were different phrasings of the same system property.

4.5.2 Practical patterns of specifications

Assume that a transition system with an initial state is given. Below, a number of frequent informal state propositions are given, and how to translate them in CTL or LTL as state formulas of the initial state.

- All processes will eventually be deadlocked? This property is a universal quantification over all paths, and hence certainly expressible in LTL. It is expressible also in CTL (although there is a catch).

- In CTL: $\text{AF AG } \textit{deadlock}$

- In LTL: $\text{F G } \textit{deadlock}$

Remarkably, these two statements are not logically equivalent. How can they be both correct then? For an explanation, see Remark 4.5.1.

- In its normal state, a process can always request to enter its critical section (using n_i, t_i, c_i)?

- In CTL: $\text{AG}(n_i \Rightarrow \text{EX } t_i)$.

- This property is not expressible in LTL. See Exercise 4.5.2.

This is a statement about arbitrary states in which n_i holds. To express it in CTL, it is stated over all *reachable* states, using the operator AG. This is good enough in practice, but *it does not exclude that there are states in the transition structure (unreachable from the initial state) in which the property does not hold.*

- Processes need not enter critical section in strict alternating sequence?

- In CTL: $\text{EF}(c_1 \wedge \text{EX}(\neg c_1 \wedge \text{E}[\neg c_2 \text{ U } c_1])) \vee \text{EF}(c_2 \wedge \text{EX}(\neg c_2 \wedge \text{E}[\neg c_1 \text{ U } c_2]))$.

- Not in LTL. But as we saw, the negation of this proposition can be expressed in LTL: $\text{G}(c_1 \Rightarrow (c_1 \text{ W } (\neg c_1 \wedge \neg c_1 \text{ W } c_2))) \wedge \text{G}(c_2 \Rightarrow (c_2 \text{ W } (\neg c_2 \wedge \neg c_2 \text{ W } c_1)))$

- From any state it is possible to get to state *restart*:

- $\text{AG EF } \textit{restart}$

- This property is not expressible in LTL. See theorem 4.5.1.

- When the elevator reaches the third floor with doors closed, it can stay there forever:

- $\text{AG}(\textit{floor3} \wedge \textit{doorsclosed} \Rightarrow \text{EG}(\textit{floor3} \wedge \textit{doorsclosed}))$

- Not in LTL. See theorem 4.5.1.
- Every process is enabled infinitely often.
 - In LTL: $G F \textit{enabled}$
 - In CTL: $AG AF \textit{enabled}$.

Proof. Proof. First, assume $G F \textit{enabled}$ is true and $AG AF \textit{enabled}$ is false in the initial state. The latter means that $EF EG \neg \textit{enabled}$ is true: there is a path to a state s having a path in which *enabled* is and remains false and in which $F \textit{enabled}$ is false. By concatenating both paths, we obtain a path from the initial state in which $G F \textit{enabled}$ is false. Contradiction.

Second, assume $G F \textit{enabled}$ is false and $AG AF \textit{enabled}$ is true. The first means that there is a path in which *enabled* is true at most a finite number of times. Take the state s on that path just following the last state where *enabled* is true. s is reachable from the initial state and $AF \textit{enabled}$ is false in s . Hence, $AG AF \textit{enabled}$ is false in the initial state. Contradiction. ■

- If a process is enabled infinitely often, then it runs infinitely often? (using *enabled*, *running*)
 - In LTL: $G F \textit{enabled} \Rightarrow G F \textit{running}$
 - In CTL? What about $AG AF \textit{enabled} \Rightarrow AG AF \textit{running}$? This is not correct!

True, $AG AF \textit{enabled}$ is equivalent to $G F \textit{enabled}$, and the same for $AG AF \textit{running}$ and $G F \textit{running}$. Still,

$$G F \textit{enabled} \Rightarrow G F \textit{running}$$

and

$$AG AF \textit{enabled} \Rightarrow AG AF \textit{running}$$

are certainly not equivalent. The reason is the different quantification over paths. The LTL statement corresponds to the CTL* proposition

$$A(G F \textit{enabled} \Rightarrow G F \textit{running})$$

It quantifies once over every path from the initial state. The CTL expressions contains two separate quantifications over paths.

To see the difference, assume there is one path where *enabled* is not infinitely often true. Then $AG AF \textit{enabled}$ is false and the CTL statement is trivially satisfied since its condition is false.

But the second statement is trivially satisfied only for that one path. It still requires all other paths to satisfy $G F \textit{enabled} \Rightarrow G F \textit{running}$.

This proposition is expressible in LTL and not in CTL.

Informal phrases containing “can” (as in “the elevator can stay on the 3rd floor”) or “possible” (as in “it is possible to reach a state satisfying such or so”) often refer to the existence of different potential futures. This is usually a sign that CTL is the appropriate language.

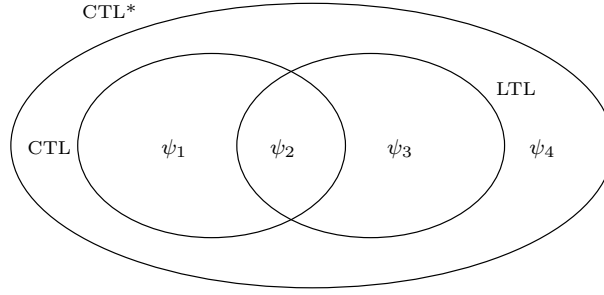


Figure 4.11 – Expressivity comparison

4.5.3 Logical analysis of CTL

Expressivity of LTL, CTL and CTL* Notice that CTL does not have path formulas that are not state formulas. Below, we compare the expressivity of CTL, CTL* and LTL on the level of *state formulas*.

Figure 4.11 shows a comparison of these logics.

- A formula $\psi_1 \in CTL \setminus LTL : \text{AGEF } p$. The proof is simple and illustrative.

We call a transition structure $\mathcal{M}'$ a substructure of $\mathcal{M}$ if $\mathcal{M}'$ is obtained by deleting a set of edges and states from $\mathcal{M}$. Recall that by definition, each node in a transition structure has an outgoing edge. Hence, to obtain a transition substructure by deleting edges and states, the following rules should be obeyed:

- If all edges from a state s of $\mathcal{M}$ are deleted, then also s has to be deleted.
- If a node s is deleted from $\mathcal{M}$, then all edges to and from s need to be deleted.

Clearly, any path π in a substructure $\mathcal{M}'$ of $\mathcal{M}$ is also a path in $\mathcal{M}$.

Theorem 4.5.1

Let φ be a CTL or CTL* state formula such that there exists a transition structure $\mathcal{M}$ with state s and substructure $\mathcal{M}'$ containing s such that $\mathcal{M}, s \models \varphi$ and $\mathcal{M}', s \not\models \varphi$. Then φ is not expressible as an LTL formula.

Proof. Assume towards contradiction that φ is expressible by LTL formula ψ . It means that the CTL* state formulas φ and $\text{A}\psi$ are equivalent. Given that $\mathcal{M}, s \models \varphi$ and $\mathcal{M}', s \not\models \varphi$, it follows that $\mathcal{M}, s \models_{LTL} \psi$ and $\mathcal{M}', s \not\models_{LTL} \psi$. In particular, ψ is true in every path π from s in $\mathcal{M}$ but not on every path from s in $\mathcal{M}'$. This is impossible, since every path from s in $\mathcal{M}'$ is a path from s in $\mathcal{M}$. ■

The theorem suggests a simple method to prove that some CTL or CTL* state formula φ is not expressible in LTL: find a transition structure $\mathcal{M}$ with state s such that $\mathcal{M}, s \models \varphi$ and find a substructure $\mathcal{M}'$ containing s such that $\mathcal{M}', s \not\models \varphi$. Application of Theorem 4.5.1 shows that φ cannot be expressed in LTL.

We illustrate the use of the theorem on the CTL proposition $\text{AGEF } p$. To show that it is not expressible in LTL, it suffices to find a structure and a state in which it is satisfied so that, after deleting some edges (and/or nodes), the proposition is not longer satisfied. In the structure $\mathcal{M}$ and substructure $\mathcal{M}'$ displayed in Figure 4.12, it holds that $\mathcal{M}, s \models \text{AGEF } p$ and $\mathcal{M}', s \not\models \text{AGEF } p$.

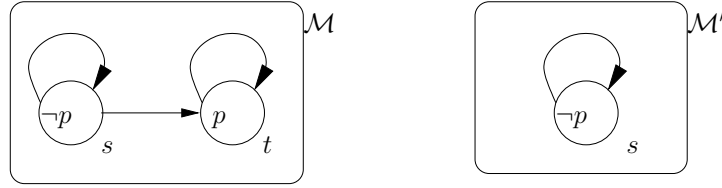


Figure 4.12 – Structure with substructure for $\text{AGEG } p$.

Exercise 4.5.2 Prove that $\text{AG}(n_i \Rightarrow \text{EX } t_i)$ is not expressible in LTL. Prove the same for $\text{EX } P \wedge \text{EX } \neg P$. Think of other CTL formulas that cannot be expressed in LTL.

- $\psi_2 \in \text{CTL} \cap \text{LTL}$:
 - in CTL: $\text{AG}(p \Rightarrow \text{AF } q)$
 - in LTL: $\text{G}(p \Rightarrow \text{F } q)$

Exercise 4.5.3 Prove that these formulas are logically equivalent.

A simpler example is $\text{AG AF } q$ and $\text{GF } q$.

- $\psi_3 \in \text{LTL} \setminus \text{CTL}$: $\text{GF } r \Rightarrow \text{F } a$ (proof omitted)
 - It means that in all paths in which r is infinitely often true, then a will be satisfied. This may be a useful form of *fairness*: infinitely often requested (r) implies eventually acknowledged (a) at least one time. Notice that it does not require that a is true infinitely often.

Another example is $\text{GF } r \Rightarrow \text{GF } a$.

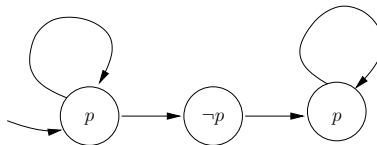
- $\psi_4 \in \text{CTL}^* \setminus (\text{LTL} \cup \text{CTL})$: $\text{E}[\text{GF } p]$ (proof omitted)
 - There is a path with infinitely many p 's.

Exercise 4.5.4 Prove that the property cannot be expressed in LTL.

Sometimes, an LTL statement can be translated into CTL by adding the path quantifier A in front of each LTL connective. E.g., the LTL formula $\psi_2 = \text{G}(p \Rightarrow \text{F } q)$ is equivalent to the CTL formula $\text{AG}(p \Rightarrow \text{AF } q)$ as claimed above.

This translation is not always correct. E.g., the above formula $\psi_3 = \text{GF } r \Rightarrow \text{F } a$ cannot be expressed in CTL, which entails that it is not equivalent to $\text{AG AF } r \Rightarrow \text{AF } a$.

Exercise 4.5.5 Take the LTL statement $\text{F } \text{G } p$ and its attempted mapping to CTL: $\text{AF AF } p$. Show that they are not equivalent by analysing their truth in the leftmost state of the following transition structure:



Exercise 4.5.6 The LTL formula $\text{XF } p, \text{FX } p$ and its CTL translation $\text{AX AF } p$ are equivalent. The LTL formulas $\text{XF } p$ and $\text{FX } p$ are equivalent and hence, $\text{FX } p$ can be

expressed in CTL. However, FXp and the attempted CTL translation $AFAXp$ are not equivalent. Show this.

Remark 4.5.1

On page 172, the proposition “all processes will eventually be deadlocked” was expressed

- in CTL as $AFAG\text{ deadlock}$,
- in LTL as $FG\text{ deadlock}$.

Exercise 4.5.5, showed that these statements are not logically equivalent (taking $p = \text{deadlock}$). How can we explain this apparent contradiction? Well, it is easy to see that $AFAG\text{ deadlock}$ entails $FG\text{ deadlock}$. So, the CTL formula is more restrictive. The transition system in Exercise 4.5.5 is unnatural as a modelling of deadlocks, since it contains a deadlock state from which one can escape (but only to end up in another deadlock state in two steps). For transition systems that include deadlock in a “correct” way (namely as states from which there is no escape), the two formulas are equivalent.

Exercise 4.5.7 Verify that in a transition structure where in every path on which *deadlock* becomes true, it remains true, the CTL and LTL formulas are equivalent.

Useful equivalences of CTL Basic properties of CTL^* such as $\neg A\varphi \equiv E\neg\varphi$ and $\neg G\varphi \equiv F\neg\varphi$, lead to the following important equivalences and dualities:

- $\neg AX\psi \equiv EX\neg\psi$
- $\neg EG\psi \equiv AF\neg\psi$
- $\neg AG\psi \equiv EF\neg\psi$
- $EF\psi \equiv E[\top U \psi]$
- $A[\psi U \phi] \equiv \neg(E[\neg\phi U (\neg\psi \wedge \neg\phi)] \vee EG\neg\phi)$

Adequate sets of CTL Below, paired connectives such as $EG, AU, \dots$ are considered as one CTL connective.

Theorem 4.5.2

A set of temporal connectives in CTL is adequate (i.e., suffices to express all CTL formulas) iff it contains one of $\{AX, EX\}$, at least one of $\{EG, AF, AU\}$ and EU .

Proof is omitted. (For the $\Leftarrow$ half, the proof follows immediately from the useful equivalences. For the $\Rightarrow$ half, it is more difficult.)

4.5.4 On the use of LTL and CTL and CTL^*

There have been years of debate about linear-time versus branching-time logics with extensive literature comparing and arguing both.

There is a tendency towards consensus that LTL is simpler and more intuitive, and better suited to express the most common temporal propositions. Therefore, LTL is used more often in applications. On the other hand, CTL can express information on potential

behaviours. An example:

$$\text{EX AG } p \wedge \text{EX AG } \neg p$$

This is a consistent CTL statement about two potential and clearly incompatible behaviours of the system that cannot be expressed in LTL.

Exercise 4.5.8 Give a transition structure and a state in which this CTL formula is true.

To reason on potential behaviour is sometimes useful and cannot be simulated in LTL. Therefore, some systems support both LTL and CTL.

Why not CTL*? Because its expressivity comes at a price. Model checking of CTL* formulas is computationally much more expensive. LTL and CTL have model checking algorithms with linear time complexity in the size of the transition structure. Model checking algorithms for CTL* have higher complexity. In practical applications, transitions structures are very large. This impedes the practical use of CTL*.

We will return to the actual algorithms for model checking in these logics, as well as their time complexity, in chapter 9.

4.6 Important for exam

Big question for the theoretical part:

- Define the logics LTL, CTL and CTL*:
- formal definitions of syntax and semantics and discussion.

Small questions:

- Prove the criterion for when CTL formulas cannot be expressed in LTL.
- Give formulas of LTL that cannot be expressed in CTL and vice versa.
- Explain some CTL or LTL formulas.
- What is a fairness constraint?
- Evaluate a formula in a transition structure.

What you need to know or understand to answer these questions:

Knowledge of

- transition structure
- LTL syntax and semantics
- CTL* syntax and semantics
- Integration of LTL in CTL*
- CTL as sublanguage of CTL*
- expressivity limitations LTL/CTL; the criterion why many CTL statements cannot be expressed in LTL.

Chapter 5

Refinement in Event-B

5.1 Introduction

The goal of this course is to study modelling techniques and their applications. We study the use of modellings for verification and for solving software problems: we can do the latter by applying a suitable form of inference on our formal theory, or alternatively, we may sometimes be able to compile the formal modelling into a program that solves the software problem.

In this chapter, we see a new key modelling technique: *refinement*. It is a general technique to structure modellings of dynamic systems at different levels of abstraction; at the same time, it has also been used to build modellings that can be compiled into programs.

The modelling methodology seen so far aims to express knowledge directly in some target vocabulary Σ . However, for large systems, this method has its disadvantages. In particular, the vocabulary must be designed at the abstraction level suitable to solve the problems, and this abstraction level may be too refined, too concrete, too detailed. It may lead to potential problems in designing the vocabulary and correspondingly, problems with building the theory at this precision level. Moreover, proving correctness properties of this theory might be challenging.

The technique of step-wise refinement is based on the idea to build a specification in an iterative way, starting from a specification at a high abstract level and gradually refining it. An initial specification is step-wise refined until the desired level of concreteness is obtained. Each of these steps improves the specification in some way, by adding more detail: refining existing concepts or actions, adding new concepts and new actions that implement abstract concepts and actions and that add new functionality or lead to more efficient behaviour. During the refinement process, verification steps take place at all levels of abstraction. In some cases, the result is a specification that is concrete enough to be compiled to an executable program [2].

An important aspect is that step-wise refinement is *incremental*: in the construction of the modelling and in the verification of properties. Refinement steps are constrained such that all verifications proven at higher level continue to hold. As such, the correctness of specifications at concrete levels depends and exploits verifications that were proven at higher abstraction levels. If all these refinements happen correctly the final specification (or program) must be correct by construction.

Compared to “one-shot” methodologies in which a specification is built in one step at the desired level of concreteness (as is currently the case in almost all declarative problem solving

paradigms), the refinement methodology has some important advantages. First, it leads to a more comprehensible, efficient, robust and flexible design process. Abstract specifications are simpler to understand; abstract specifications serve as documentation for more concrete ones. Bugs in a design are detected earlier, at an abstract level. Multiple alternative refinements can be explored, without losing the work done at higher abstraction level. Perhaps the most important advantage is for verification. In a “one-shot” methodology leading to a complex model, verification tasks may be of overwhelming complexity. In the refinement methodology, this complexity is broken up in small steps.

The advantages of modelling at a high abstraction level are ignoring irrelevant detail, focus on core functionality and structure, simplified reasoning. Many aspects of correctness are far more easily proven at more abstract levels than at concrete levels. Abstraction is a key technique to exploit the power of mathematics in empirical formal sciences. Likely, abstraction can be expected to play a key role in mastering the complexity of software and other systems.

But modelling at an *abstract level* is insufficient. Abstract modellings do not contain sufficient information to solve your problem. By applying refinement, details of the problem domain can be brought in in order to *refine specifications from abstract to concrete*. In the end, concrete specifications are obtained from which the computational problems arising in the application field can be solved from the logic specification. This way, refinement combines the advantages of modelling at an abstract level, with those of modelling at a concrete level.

This chapter offers an overview of the refinement technique as it was developed for Event-B.

Historical note The step-wise refinement method was originally developed for programming by Dijkstra and Wirth. Today the approach is used for programs as well as for formal specifications. Step-wise refinement was originally proposed as a constructive approach to program proving. Starting from an initial simplified program, each refinement has to be carried out carefully so that it preserves the correctness of the program, while adding details. If all these refinements happen correctly the final program must be correct by construction.

This technique contrasts with other formal methods, where verification is used to prove program correctness. Using verification, correctness properties of the program or system can be proven *after* their development (e.g., Hoare logic). However, if any changes are made to the verified parts, the entire verification has to be repeated. When using refinement proving the correctness of the initial program as well as that of each refinement step is sufficient to prove the correctness of the final program. Verifying these steps is usually much simpler than the verification of an entire system. [1]

The B method constitutes a formal method which supports the use of refinement when modelling, verifying and developing systems. In particular the Event-B method, an evolution of B, is designed around refinement. Using the ideas of step-wise refinement, Event-B adopts a proof-based development strategy. For each refinement a number of proof obligations, i.e. conditions for correctness, are generated. These guarantee the correctness of the refinement step. Combined with the correctness of the initial modelling, this is enough to guarantee the correctness of the final modelling. These proof obligations are usually proved automatically. [4]

The B method and Event-B method have been used to develop several safety-critical systems, including a driverless metro in Paris. The next section will go into more detail about these examples. Section 5.3 will describe the refinement process.

5.2 Real-world examples

The B method and Event-B are used to develop safety-critical systems. We discuss a few examples.

Metro 14 in Paris using the B method On October 15, 1998 a new metro line opened in Paris, Metro Line 14, that was completely automated. Safety properties on several safety-critical parts of the systems were proved using the B method. To do so, over 100 000 lines of B modelling were written. After proving correctness, no bugs were detected during functional validation, integration validation, on-site testing, nor in the field since it started operating. The success of the B method is apparent when you look at the result. In 2009, the safety-critical software is still in version 1.0.

One aspect of the metro line, the operation of the platform screen doors, was almost completely verified using the B method. The system and software specifications were formalized in B and correctness was proved. Development of the doors' controller lasted four months, after which it was deployed on three platforms for an eight month experiment. No faults were observed during these eight months [31].

Other show cases More recently Event-B is being used for the development of safety-critical and other industrial projects. Event-B was designed as an alternative for B, specifically suited for modelling dynamic systems in their entirety. Event-B is being used by companies including:

Bosch Reliability and safety are essential for embedded control systems in the automotive industry. The required level of safety and dependability is achieved by means of testing. However, due to the continuous increase in system complexity, the effort of testing will grow and will become uneconomical. To evaluate whether formal methods, and in particular Event-B, are suitable for the development and verification of automotive systems, Bosch applied the Event-B methodology to two applications, cruise control and (eco-)start/stop software.

Siemens Transportation Siemens has considerable experience in applying formal methods to software components of railway systems. They have successfully been using the B method for over 15 years. Between 2008 and 2012 Siemens looked into using Event-B for overall system development, applied to the development of transportation systems, results proved promising.

Huld Part of the BepiColombo space probe has been modeled using several levels of refinement. Event-B proved suited for the development of on-board software. The BepiColombo space probe is Europe's first mission to Mercury. It was launched on 20 oktober 2018.

The method of refinement for dynamic systems has not only been developed for the B method. Even prior to the B method, it was developed in the context of Abstract State Machines by Gurevitch starting from 1993. Event-B was developed by Abrial in 1996.

Using the refinement methodology, some spectacular successes have been obtained. However, so far it remains a costly design method which is only applied to mission-critical subsystems.

5.3 Refinement

Refinement is a concept used in many fields and settings, going from topology refinement in mathematics to cultural refinement. In this course refinement is discussed in the context of formal methods, where it denotes transformations of abstract specifications into concrete systems, which preserve correctness and other properties. Step-wise refinement allows this transformation to be done in incremental stages.

While even in the context of formal methods the meaning refinement can vary, the basic idea of refinement is a simple one. It stems from the following principle.

Definition 5.3.1: Principle of substitutivity

If an item can be substituted by another, in such a way that its users can not tell a substitution has taken place, the latter is a refinement of the former.

A refinement B of a specification A has the property that at the abstraction level of A, the behaviour of the system specified by B is undistinguishable from the behaviour of the system specified by A. Refinement is transitive in the sense that if B refines A and C refines B then C refines A. It follows that we can safely build hierarchies of refinements. From the above intuition, in the context of dynamic systems, refinement can be defined as follows:

Definition 5.3.2

Refinement is the process of adding details to a modelling of a dynamic system, in such a way that the newly added information does not “contradict” what was already specified in the modelling. A refinement step links one *abstract* modelling to a more *concrete* modelling.

The word “contract” in the above definition is left somewhat vague. As we will see, this can be interpreted in different ways.

The goal when using refinement is to obtain a zooming effect on the system. Starting from an abstract, simple representation of the system details are added incrementally, to make the modelling more concrete. With each refinement we effectively zoom in on the system, allowing us to specify more details and describe the inner workings of the system. Compare this with a microscope; when you zoom in, you still see the same thing, but in more detail.

Example 5.3.1

To illustrate how refinement works and how it should be interpreted we describe a human using step-wise refinement. A first abstract description is as follows:

A human is an animal.

This implies that humans have all general properties of animals. This entails that, like almost all animals, a human is a living organism able to feed on organic matter, breathe oxygen, reproduce sexually, and respond to stimuli via special sense organs and a nervous system.

We refine this description:

A human is an animal *with the ability of walking*.

This refines the previous description with one additional detail. All properties entailed by the abstract description still hold. In addition, the new specification entails that

humans are automotive and their way of movement is walking.

A human is a *mammal* with the ability of walking.

Since mammals are animals, this refines the previous specification. It entails every proposition already entailed, but also that a human is warm-blooded, gives birth to live young and excretes milk for nourishment.

A human is a primate which walks on two legs and uses language.

This brings us to a more accurate description of what a human is. If this description proves inadequate in a certain context, it can simply be further refined by adding additional, relevant details.

The example suggests that refinement is not only applicable for modelling dynamic systems but is a generally applicable methodology in knowledge representation. Although some initial research was done in the context of Artificial Intelligence, it has not been developed in a systematic way for KR. This is a promising research topic. In this course we will apply it for dynamic systems only.

Throughout this chapter refinement will be discussed in the context of the Event-B method. To support this the Event-B syntax and modelling approach is first described in the following subsection.

5.3.1 Event-B and Rodin

Event-B is based on the B method. It supports an event-driven approach for modelling dynamic systems. An Event-B model represents a system as a finite list of *state variables* which are modified by a finite amount of *events*. Events correspond to actions as seen in LTC. These events describe the possible behaviour of the system. *Invariants* specify properties that must be satisfied in every reachable state of the system, i.e., they must be satisfied in the initial state and maintained by the events.

Rodin is a development environment for refinement in Event-B. It supports interactive Event-B modelling hierarchies by step-wise refinement and enforces a certain refinement strategy upon the user. One important aspect is that it automatically generates *proof obligations* for the user.

The ProB system supports Event-B as a modelling language but does not provide support for refinement. Also, ProB and Rodin use slightly different dialects of Event-B. E.g., *operations* in ProB are called *events* in Rodin.

A proof obligation is a proposition that must be proven to hold. E.g., adding a new *event* generates proof obligations for all existing invariants, namely that the *event* preserves the invariant. The proof obligations are generated automatically by Rodin. Proof obligations are verified semi-automatically by Rodin and may involve interaction with the user.

Modellings are split into static and dynamic parts. The static components are described in *contexts*. Contexts define constants and sets; these sets operate as types for the system. Constants are elements of the domain. The dynamic part of the system is represented by *machines*. These consist of variables, invariants and events, as explained in the previous paragraph. This approach makes for easily readable and understandable modellings of dynamic systems. Figure 5.1 illustrates this. The example shows a simple machine. The single state variable **running** is a boolean variable stating whether the machine is running or not. Two events can take place: the machine can be started or stopped.

Events are made up of *guards* and *actions*. The guards specify the preconditions of the

```

MACHINE
  SimpleMachine >
VARIABLES
  ◦ running >
INVARIANTS
  ◦ running_type: running ∈ BOOL not theorem >
EVENTS
  ◦ INITIALISATION: not extended ordinary >
    THEN
      ◦ off: running = FALSE >
    END

  ◦ Start: not extended ordinary >
    WHERE
      ◦ off: running = FALSE not theorem >
    THEN
      ◦ start: running = TRUE >
    END

  ◦ Stop: not extended ordinary >
    WHERE
      ◦ on: running = TRUE not theorem >
    THEN
      ◦ stop: running = FALSE >
    END
END
END

```

Figure 5.1 – A simple Event-B machine

event. Actions specify an effect of an event. They are specified as assignments to variables. The syntax used in Event-B is a combination of set-theoretical notations and first-order logic.

Notice that the notion of *action* is different in LTC and in Event-B.

5.3.2 Building a copier by refinement

We build a simplified modelling of a *copier*. We start from a very simple model: a machine that can be on or off. We will then specify extra functionalities of a copying machine by bringing detail in internal state and events.

Refinement 0 When modelling systems, it is common, and recommended, to start with an abstract representation of the system. The first, most abstract description of a copy machine is the simple machine represented in Figure 5.1. The state variable **running** is a boolean variable stating whether the machine is running or not. The two events are: the machine can be started or stopped. This is our starting point.

Events are made up of *guards* and *actions*. Guards following **WHERE** are the event’s preconditions. Actions following **THEN** specify the effects of an event.

Invariants and guards can be annotated: e.g., **not theorem** means the proposition is not implied by other propositions (invariants or guards). For an axiom that is **theorem**, Rodin generates a proof obligation. Once this proof obligation is fulfilled, Rodin knows that the truth of this axiom follows from the used set of premises, and will not reactivate the proof obligation anymore as long as these premises exist in the modelling.

Events can be annotated too: e.g., **extended** means that the event is a refinement of an event defined at a higher abstraction level. Guards and actions of the higher level are preserved for it. Events that are **not extended** are new events of the current refinement level.

Refinement 1: Adding amount selection and copy We add two events to select the amount of pages, and to perform the copying. The state of the machine is extended to

express the number of pages:

- `copy_goal`: a new numerical variable

The new events:

- `Select_Amount`: sets `copy_goal` to a specified value
- `Copy`: resets `copy_goal` to zero.

The two original events are left unchanged. The new events (without the new variable `copy_goal` in the `Variables` section) are shown in Figure 5.2.

```

Select_Amount: not extended ordinary >
ANY
  ◦ amount >
WHERE
  ◦ on: running = TRUE not theorem >
  ◦ amount_type: amount ∈ N1 not theorem >
THEN
  ◦ set_goal: copy_goal = amount >
END

Copy: not extended ordinary >
WHERE
  ◦ on: running = TRUE not theorem >
  ◦ goal: copy_goal > 0 not theorem >
THEN
  ◦ end_copy: copy_goal = 0 >
END

```

Figure 5.2 – `Select_Amount` and `Copy` events of the simple copier

The new events are **not extended** and have no abstract counterpart in the original machine. They are viewed as refinements of the null event `skip`. This means that like `skip`, they should not have an effect that can be observed at the abstract level (ref. 0). Therefore, they should only change *new* state variables (`copy_goal`). Otherwise, they would violate the *principle of substitutivity* as they would cause a state change at the abstract level without visible action (since the new events are not visible at the abstract level).

In the refinement, we now want to assert a new invariant: the copier should be on if it has a goal:

$$\text{copy_goal} > 0 \Rightarrow \text{running} = \text{TRUE}$$

For this new invariant, Rodin generates a *proof obligation* for each existing event, namely to prove that the event preserves the invariant.

Now, we observe a problem with the event `Stop`: it does not preserve the new invariant. Indeed, `Stop` sets `running` to false. When executed when `copy_goal > 0` holds, `Stop` violates the invariant. Therefore, we need to refine this event.

For now, we interrupt the copier example, and introduce some more theoretical concepts that are needed to finish this example. After that, we will return to it.

5.3.3 Proof obligations

Rodin automatically generates proof obligations for Event-B modellings. These proof obligations are used to make sure the modelling has certain properties, e.g., that guards, actions and invariants are well defined and that invariants are preserved by the modelling. To show that the machine never violates the invariant, it must be proven that each event preserves the invariant. Thus for each combination of an event and an invariant, a proof obligation is generated.

When refining a machine, a number of proof obligations are generated to incrementally maintain a state where each event preserves each invariant. When adding a new event e , there are new proof obligations for each existing invariant i . Likewise, when adding a new invariant i , proof obligations are generated for each existing event e . These are called proof obligations for invariant preservation.

Since **Initialization** is also an event, it has a proof obligation for every invariant. Proving all these proof obligations ensures that the invariants hold in the initial state and are preserved by all events. Hence, by induction, the modelling of the system entails the specified invariants.

We define the proof obligation for invariant preservation.

Definition 5.3.3: Invariant preservation

A proof obligation for *invariant preservation* for an event e and invariant i takes the following form:

$$\forall s, s', x : A \wedge I(s) \wedge H_e(x, s) \wedge T_e(x, s, s') \Rightarrow I_i(s')$$

where

- A is the set of static axioms on constants and sets in the context associated with the machine;
- $I(s)$ is the conjunction of all invariants of the machine at state s ;
- H_e is the guard of event e ;
- $T_e(s, s')$ is the bstate formula expressing that state s' results from s by applying the actions of e ;
- i is the invariant; $I_i(s')$ states that i is true in s' .
- x represents the event parameter(s).

This is the same idea as the verification method for proving invariants in bstate LTC theories.

Returning to our running example, we see that the abstract (ref. 0) invariant *running* $\in$ *BOOL* is preserved by the abstract events as well as by the two new concrete events. On the other hand, the concrete (ref. 1) invariant *copy_goal* $> 0 \Rightarrow$ *running* = **TRUE** is not by the abstract event **Stop**. We need to refine this event.

5.3.4 Event refinement

As shown by the problem with the **Stop** event, even when only new, independent information is added during a refinement it is seldom enough to only add new state variables and events. Changes to existing events are also often required.

In the running example, the abstract **Stop** event violates the concrete invariant. We do not want to change the original abstract machine. Therefore, we need to refine the **Stop** event in the refined machine.

Adding details to an abstract event during a refinement step is called *event refinement*. Event refinement is performed by two sorts of modifications: *guard strengthening* and *action simulation*. Both need to satisfy certain conditions to be correct. Rodin will generate these as proof obligations.

When the *guard strengthening* and *action simulation* conditions hold, the refinement e of an abstract event preserves all invariants i preserved by the abstract event. Thus, there is no need to generate proof obligations for invariant preservation of e and these invariants i .

Definition 5.3.4: Guard strengthening

The *guard strengthening condition* for an event e that is refined is the proposition expressed below that states that the concrete event can only be enabled if the abstract event is enabled. The formula is:

$$\forall x, s : A \wedge I(s) \wedge J(s) \wedge H_e(x, s) \Rightarrow G_e(x, s)$$

where

- A is the set of axioms in the context
- I and J represent the invariants of the abstract and concrete machine, respectively.
- H_e is the conjunction of all guards of the concrete event and G_e is the conjunction of the guards of the abstract event.

The guard strengthening condition expresses that in any reachable state where the concrete refined event can take place, also the abstract event can take place.

Definition 5.3.5: Action simulation

The *action simulation condition* for an abstract action a of an abstract event e that is being refined is the proposition that states that this action a “simulates” the concrete actions of the refined e . The formula is:

$$\forall x, s, s' : A \wedge I(s) \wedge J(s) \wedge H_e(x, s) \wedge T_e(x, s, s') \Rightarrow Q_a(x, s, s')$$

Where T_e is the bstate formula expressing the effects of the refinement of e and $Q_a(x, s, s')$ the bstate formula expressing the effect of the abstract action a . Here, $A, I(s), J(s), H_e(x, s)$ are as in the previous definition.

T_e and is called the *before-after predicate* of the concrete event and Q_a that of the abstract action a . This action simulation condition ensures that the event’s abstract behaviour is not contradicted by its concrete behaviour.

Rodin generates a proof obligation for the guard strengthening condition of each refined event e , and for the action simulation condition for each action a of the abstract event e .

5.3.5 Building a copier by refinement, continued

Refinement 1 rounded up The abstract event **Stop** violates the invariant

$$\text{copy_goal} > 0 \Rightarrow \text{running} = \text{TRUE}$$

In Figure 5.3 two solutions are specified:

- to strengthen the guard of **Stop** by requesting `copy_goal = 0`; hence, a machine cannot be stopped until printing is finished;
- to add the action `copy_goal := 0` to **Stop**.

<pre> ◦ Start: extended ordinary > REFINES ◦ Start WHERE ◦ off: running = FALSE not theorem > THEN ◦ start: running = TRUE > END ◦ Stop: extended ordinary > REFINES ◦ Stop WHERE ◦ on: running = TRUE not theorem > ◦ no_goal: copy_goal = 0 not theorem > THEN ◦ stop: running = FALSE > END </pre>	<pre> ◦ Start: extended ordinary > REFINES ◦ Start WHERE ◦ off: running = FALSE not theorem > THEN ◦ start: running = TRUE > END ◦ Stop: extended ordinary > REFINES ◦ Stop WHERE ◦ on: running = TRUE not theorem > THEN ◦ stop: running = FALSE > ◦ end_goal: copy_goal = 0 > END </pre>
--	---

Figure 5.3 – Two solutions for Stop

```

CONTEXT
  Copier_State >
SETS
◦ states >
CONSTANTS
◦ off >
◦ copying >
◦ ready >
AXIOMS
◦ states_partition: partition(states, {off}, {copying}, {ready}) not theorem >
END

```

Figure 5.4 – Context specifying the possible states of the copier

Both are possible. The first way is to refuse **Stop** if the specified `copy_amount` is non-zero. The second is that **Stop** sets this value to 0. The second seems the more reasonable solution since it makes it possible to stop the copier even when it has not finished printing.

Refinement 2 We introduce a second refinement: *pages are copied one by one, not all at the same time*. Hence, copying becomes a *process*.

Events do not represent processes. An important assumption made in Event-B is that events take no time, i.e. they happen *instantaneously*. This means only one event can happen at a time and there is no concept of “during” an event. Specifically this means events can not represent processes.

The solution in Event-B is to simulate a process by introducing two or more events: a begin event for the process and an end event for it, and possibly intermediate state transition events of the process. In addition, a state variable to represent the progress of the process. A process may run for ever, in which case the end event will not occur.

We apply this technique to **Copy**, splitting it into three subevents: *begin*, *print page*, *end*. In the refined machine, the copier can be in three different states: *off*, *on and in rest*, and *on and printing*. It follows that the variable `running` is no longer sufficient precise. A novel form of refinement is needed: *variable refinement*. We refine the variable `running` by a new variable `state`, with possible values: `off`, `ready` and `copying`. These values are introduced in a *context*, as shown in Figure 5.4. The context in Figure 5.4 declares set `states` and three constants. It also contains an axiom:

$$\text{partition}(\text{states}, \{\text{off}\}, \{\text{copying}\}, \{\text{ready}\})$$

This expresses that $\{\{\text{off}\}, \{\text{copying}\}, \{\text{ready}\}\}$ is a *partition* of the set `states`: the union of the three sets is `states` and the intersection of the three sets is empty. The latter

implies that $\text{off} \neq \text{copying}$, $\text{off} \neq \text{ready}$, $\text{copying} \neq \text{ready}$. This is the Event-B way to express UNA+DCA for the type `states`, and expresses the same as the “constructed from” statement in IDP.

Now, we refine the abstract variable `running`. The variable `running` has to be replaced by `state` in all invariants. Rodin enforces a gradual strategy for this. This strategy does not immediately allow you to remove `running` from the concrete (ref. 2) machine since this would contradict the abstract (ref. 1) machine. Instead, the new, concrete variable `state` has to be *linked* to its abstraction `running` by a *linking invariant*. In this case, this is the formula:

$$\text{running} = \text{TRUE} \Leftrightarrow \text{state} = \text{copying} \vee \text{state} = \text{ready}.$$

This defines the value of the abstract variable in terms of the concrete variable. It links the abstract machine with the concrete machine: the abstract copier is running iff the concrete copier is ready or copying.

Now that the abstract variable `running` is defined in terms of the concrete variable, Rodin allows to replace `running` in all concrete events and invariants by `state`.

In the next step, the event `Copy` will be refined to represent the process of copying. For this, two new events are introduced `Start_Copy` and `Copy_Page`. These, respectively, signify the start of the copying process and the completion of one copy. The two new events are not, and can not be, refinements of the `Copy` event, since they occur before the end of the copy process. In addition, a new event `Finish_Copy` is introduced as a refinement of the abstract `Copy` event. It is this event that sets `copy_goal` to zero.

Since `Copy_Page` does not refine an abstract event it is not allowed to change the abstract (ref. 1) variables, in particular the `copy_goal` variable. Still, it should be possible to keep track of how many more copies are required to reach the goal. For this reason the `copy_job` variable is introduced in the concrete (ref. 2) machine, which stores the progress of the copying process. At the start of the copying process this variable will be set to the `copy_goal`, after which it will be decremented every time a copy is made. When this variable reaches zero, the copying process can be stopped by the `Finish_Copy` event. These events are listed in Figure 5.5.

To complete Refinement 2, additional invariants can/should be added for correct operation of the copier. They are presented in Figure 5.6. The first three invariants specify the types of the new variables and supply the link between the abstract variable `Running` and concrete variable `State`. The last three invariants specify properties of the copier which should always be true, e.g., that the value for `copy_job` can not exceed that of `copy_goal`. These invariants are quite obvious, but as stated before it is good practice to specify them. Due to the mechanism of proof obligations, they help protect against modelling mistakes, e.g., incrementing `copy_job` rather than decrementing in the `Copy_Page` event.

All generated proof obligations succeed for this refinement step, proving the copier works as intended.

5.3.6 Correctness of Refinement

Refinement builds a sequence $M_0, M_1, \dots, M_n$ of machines, each M_{i+1} refining M_i . What is the desired correspondence between these machines?

Each machine specifies a collection of paths (linear-time structures), collected in a state graph or a transition structure on states. Two main desired correctness properties exist between the collections of paths of two theories M_i and its refinement M_{i+1} .

```

Start_Copy:    not extended ordinary >
WHERE
  ◦ ready: state = ready not theorem >
  ◦ goal:  copy_goal > 0 not theorem >
THEN
  ◦ copy:  state = copying >
  ◦ copy_job: copy_job = copy_goal >
END

Copy_Page:    not extended ordinary >
WHERE
  ◦ copying: state = copying not theorem >
  ◦ copies_left: copy_job > 0 not theorem >
THEN
  ◦ copy:  copy_job = copy_job - 1 >
END

Finish_Copy:  not extended ordinary >
REFINES
  ◦ Copy
WHERE
  ◦ ready: state = copying not theorem >
  ◦ done:  copy_job = 0 not theorem >
THEN
  ◦ end_copy: copy_goal = 0 >
  ◦ finish: state = ready >
END

```

Figure 5.5 – Events related to the copying process.

```

INVARIANTS
◦ state_type: state ∈ states not theorem >
◦ linking_invariant: running = TRUE ⇔ state = copying ∨ state = ready not theorem >
◦ copy_job_type: copy_job ∈ N not theorem >
◦ copy_job_max: copy_job ≤ copy_goal not theorem >
◦ copying_job:  copy_job > 0 ⇒ state = copying not theorem >
◦ copying_goal: state = copying ⇒ copy_goal > 0 not theorem >

```

Figure 5.6 – Specified invariants of the copier machine

Definition 5.3.6: Correctness property 1

Each path/linear time model of M_{i+1} abstracts into a path/linear time model of M_i .

Mnemonic: $\forall \pi_{\text{concrete}} : \exists \pi_{\text{abstract}}$.

When we say that π_{concrete} abstracts into π_{abstract} , we mean the the abstract path is obtained by ignoring new variables at the concrete level, replacing refined events by the corresponding abstract events, and replacing new concrete events by the “skip” event. This property ensures that each behaviour at the concrete level $i + 1$ matches a behaviour at the level i .

The different steps of event and variable refinement that were introduced so far, with guard strengthening and action simulation conditions, suffice to prove correctness property 1.

Theorem 5.3.1

Let $M_0, M_1, \dots, M_n$ be a refinement sequence such that all proof obligations were satisfied, then all invariants at all levels are entailed.

Definition 5.3.7: Correctness property 2

Each path/linear time model of M_i is the abstraction of a path/linear time model of M_{i+1} .

Mnemonic: $\forall \pi_{\text{abstract}} : \exists \pi_{\text{concrete}}$.

This property ensures that each behaviour at the abstract level remains possible at the concrete level. It’s not entailed by the refinement steps that we saw. Hence, not every

behaviour of the abstract machine is possible at the refined machine. Some behaviors get “lost”. Sometimes this is desirable. Other times it is an indication of a bug. In a later section, we will see other proof obligations that will entail correctness property 2.

Example 5.3.2

Assume that we make a mistake in Refinement 2 and define:

```
Copy_Page
WHERE
...
THEN
    copy_job := copy_job-0
END
```

The concrete machine loops. A run of the abstract machine where **Copy** happens, cannot be simulated, since **Finish_Copy** can never be executed. Hence, correctness property 2 does not hold.

On the other hand, correctness property 1 holds. Each run of the refined machine is still simulated at the abstract level. E.g., a run with an infinite number of **Copy_Page** events corresponds to a run with infinite number of **skip** events.

The above example shows a problem of *divergence*: an infinite loop of new events. Alternatively, it is also possible that due to guard strengthening, refined events are not allowed to take place when their abstract versions can take place.

Additional correctness properties need to be imposed to obtain Correctness property 2. This will be seen in a later section.

Remark 5.3.1

A refinements can satisfy 0,1, or both correctness properties. Both correctness properties are generally desirable properties of a refinement. Sometimes though it is desirable for a refinement to violate property 2, reducing the set of potential behaviors.

Do note that there is a slight difference between the notion of refinement here in the course notes and the notion of refinement from the Rodin User’s Handbook, since the latter only considers concretizations that satisfy at least correctness property 1 as refinements (https://www3.hhu.de/stups/handbook/rodin/current/html/tut_refinement.html).

5.4 More about proof obligations

We introduce a new running example. Given is a function $f : [1, n] \rightarrow [0, M]$ specified in some Event-B context. We build a machine to compute the maximum of the range of f , which is the set $\{f(i) \mid i \in [1, n]\}$.

```
Variable m
Invariant
    Inv0_1: m in Nat
Events
    INITIALISATION
```

```

    THEN m := 0
  END
  Maximum
  THEN m := max(ran(f))
  END

```

Here, $\text{ran}(f)$ denotes the range of f . The proof obligations at level 0 are:

- Invariance of $\text{Inv0_1}: m \in \mathbb{N}$. We need to prove that **INITIALISATION** establishes it and **Maximum** preserves it.
- *Well-definedness* of expression $\text{max}(\text{ran}(f))$ in the action $m := \text{max}(\text{ran}(f))$ of the event **Maximum**. max is only defined on non-empty sets. Therefore, we need to prove that

$$\text{ran}(f) \neq \emptyset$$

Rodin generates these three proof obligations and tries to prove them, possibly with assistance of the user.

Refinement 1 Now, we refine the abstract machine. First we, define its invariants:

```

Variable m p q
Invariant
  Inv0_1: m ∈ ℕ
  Inv1_1: p ∈ 1..n
  Inv1_2: q ∈ 1..n
  Inv1_3: max(ran(f)) ∈ f([p,q])
Events
...

```

We use a naming convention to specify the level of invariants. E.g., Inv0_1 is at level 0, Inv1_1 at level 1. One invariant is of refinement level (0), three are of level (1). Next, we define the refinements and new events.

<pre> INITIALISATION (0) THEN m:=0 p:=1; q:= n END; </pre>	<pre> Maximum (0) WHEN p=q THEN m:=f(p) END; </pre>
<pre> increment (1) WHEN p < q f(p) ≤ f(q) THEN p:=p+1 END; </pre>	<pre> decrement (1) WHEN p < q f(p) > f(q) THEN q:=q-1 END; </pre>

(0) and (1) are the levels where the events were introduced.

An example simulation of this system is given in Figure 5.7. The matrix below denotes the sequence of states with at each state the action on the right. The colored cells are those to which p and q point at: p points at the left one and q at the right one. The simulation ends with $m = 10, p = q = 5$.

1)INIT	7	3	10	8	10	9	$p = 1; q = 6; m = 0$
2)Increment	7	3	10	8	10	9	$p = 2; q = 6; m = 0$
3)Increment	7	3	10	8	10	9	$p = 3; q = 6; m = 0$
4)Decrement	7	3	10	8	10	9	$p = 3; q = 5; m = 0$
5)Increment	7	3	10	8	10	9	$p = 4; q = 5; m = 0$
6)Increment	7	3	10	8	10	9	$p = 5; q = 5; m = 0$
7)Maximum	7	3	10	8	10	9	$p = 5; q = 5; m = 5$

Figure 5.7 – A simulation of the machine

The concrete events at level (1) and their abstract events at level (0) is depicted as follows (using abbreviations for event names):

$\pi_{(0)}$:	INIT	skip	skip	skip	skip	skip	Max
	↑	↑	↑	↑	↑	↑	↑
$\pi_{(1)}$:	INIT	Inc	Inc	Dec	Inc	Inc	Max

The new actions refine **skip**. This refinement generates several proof obligations at level (1):

- 3×4 invariant preservation proof obligations: there are 3 new invariants $\text{Inv1}_{\{1, 2, 3\}}$ of level 1 and 4 events.
- Event refinement proof obligations. Two events were refined, and for both each, a guard strengthening and action simulation conditions proof obligations arise.
- The well-definedness proof obligation for $\text{max}(\text{ran}(\mathbf{f}))$.

This suffices to prove the invariants of level (0) and (1). It suffices to prove the Correctness property 1 but not the Correctness property 2.

Pathologies for Correctness 2 A potential problem with refining an event is that the refined events may never be executable. Indeed, there is no guarantee that the guard of the refined event will ever become true. Two sorts of problems may arise.

- *Early deadlock*: the guard of the new events could become false without reaching a state in which the refined guard of refined events becomes true, and hence none can be executed. In our running example, a sequence of increment and decrement actions would then lead to a state where their guards and that of **Maximum** are all false.

INIT Inc Inc Dec (nothing possible)

- *Divergence*: an infinite sequence of new events takes place without ever reaching a state where the refined event applies.

INIT Inc Inc Dec (Max never possible)

To avoid these pathologies, additional proof obligations need to be imposed for Correctness property 2. They are called *trace refinement* proof obligations. To prevent early deadlock, a new proof obligation is added:

Definition 5.4.1: Trace refinement condition 1

The *first trace refinement condition* is the disjunction of the guards of new and refined events.

This way, some precondition is always satisfied: when no new event can take place, a refined event can take place. This prevents early deadlock.

To prevent divergence, it must be proven that new events are *convergent*. The user proposes a *variant*: a numerical expression v that must be proven to be a positive natural number and that strictly decreases in value when a new event takes place. In the example, a suitable variant is $q-p$. Two new sorts of proof obligations are introduced:

Definition 5.4.2: Trace refinement condition 2 & 3

The *second trace refinement condition* is the invariant that $v \in \mathbb{N}$.

The *third trace refinement condition* is that v decreases in value when a *new* event takes place.

Hence, an infinite sequence of new events would lead to an infinite descending sequence of values for v . This is impossible due to the invariant $v \in \mathbb{N}$. The three trace refinement conditions ensure that divergent sequences are impossible. After a finite sequence of new events, a refined event takes place.

A suitable *variant* in the running example is the numerical term $q-p$. This variant consists of the following components:

- The invariant $q - p \in \mathbb{N}$ which must be added. To prove it, a new invariant $p \leq q$ is introduced. This and the existing invariants imply invariant preservation of $q - p \in \mathbb{N}$.
- A before-after predicate needs to be proved, expressing that each new event **Increment** or **Decrement** decreases $q - p$.

Final comment It is quit clear that while the first refinement is quite close to an algorithm that shifts variables p, q to the last occurrence of the maximum of f . In the next and final section, it is illustrated how this specification is translated into a program.

5.5 Compiling Event-B to programs (not for exam)

Event-B specifications that are sufficiently “concrete” can be compiled to a program. This section illustrates the essential steps in the context of the running example.

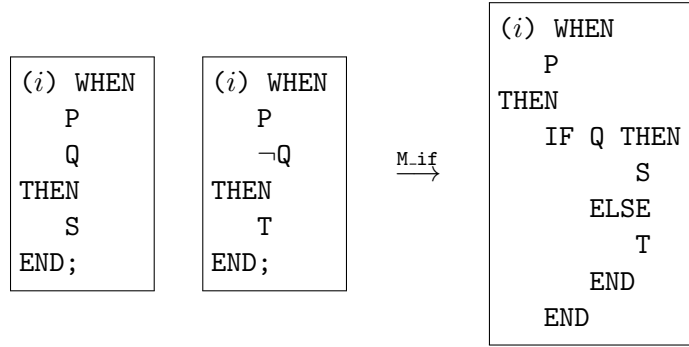
Pidgin programming language Here’s a simplified language with 4 sorts of statements:

```
statement ::= WHILE condition DO statement END
           | IF condition THEN statement ELSE statement END
           | statement ; statement
           | variable list := expression list
```

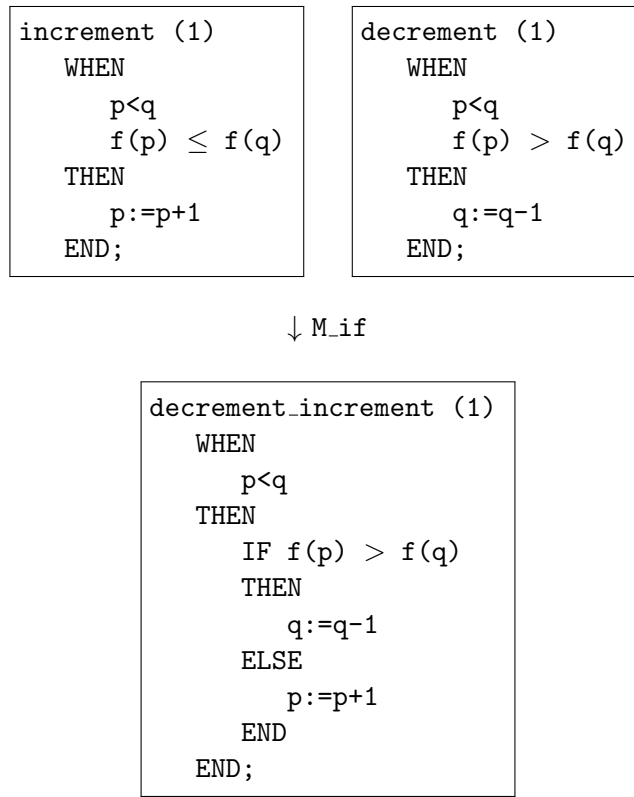
The assignment expresses concurrent assignments. E.g., $a, b := a + 1, a$ turns $a = 1, b = 0$ into $a = 2, b = 1$. This is not equivalent with $a := a + 1; b := a$.

Merging rules Merging rules transform sets of events into an equivalent set of events of fewer but more complex events. By iterated application of merging rules, an Event-B specification is gradually transformed into a Pidgin program.

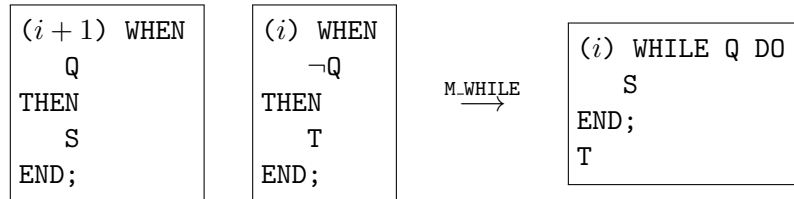
Merging rule M_if Two events with mutually exclusive guards are combined into one:



The two original events must have been introduced at the same refinement level i . We apply this to the increment and decrement events of the previous section:



Merging rule M_WHILE Two events with mutually exclusive guards are combined in one:



Condition: the first event must have been introduced at one refinement step below the second one. More general cases of this rule exist. Applied to the example of the previous section:

```

decrement_increment (1)
  WHEN
    p < q
  THEN
    IF f(p) > f(q) THEN
      q := q - 1
    ELSE
      p := p + 1
    END
  END;

```

```

Maximum (0)
  WHEN p = q
  THEN m := f(p)
END;

```

↓ M.WHILE

```

decrement_increment_maximum (0)
  WHILE p < q DO
    IF f(p) > f(q)
    THEN
      q := q - 1
    ELSE
      p := p + 1
    END
  END;
  m := f(p)

```

Final rule *M_INIT* Once we have obtained an “event” without guard, we add it to the event INITIALISATION. We then obtain the final program. Applied to the running example:

```

INITIALISATION (0)
  THEN m := 0
    p := 1;
    q := n
  END;

```

```

decrement_increment_maximum (0)
  WHILE p < q DO
    IF f(p) > f(q)
    THEN
      q := q - 1
    ELSE
      p := p + 1
    END
  END;
  m := f(p)

```

↓ M_INIT

```

(0) m := 0;
p := 1;
q := n;
WHILE p < q DO
  IF f(p) > f(q)
  THEN
    q := q - 1
  ELSE
    p := p + 1
  END
END;
m := f(p)

```

Conclusion about compiling Event-B refinement hierarchies to programs Compiling an event-B specification into a program requires careful planning from the knowledge engineer. It is the end-point of a goal directed strategy of gradually and carefully extending the event-B refinement hierarchy in the direction of a program. We can see this in the *While* merging rule:

- Two carefully tuned events must be designed
- at two subsequent refinement levels (i), (i+1)
- with opposite guards $\neg Q, Q$ that yield the test of the while Q
- the event at (i+1) is an abstraction of the iteration code
- the event at (i) is an abstraction of the operation following the while.

5.6 Conclusion

Formal specifications of software systems are used in different ways.

Some formal specifications serve only as documentation. Formal verification for such specifications can be useful for e.g. early detection of bugs. Indeed, the later a bug is discovered, the (exponentially) more costly it can be to remove it. Sadly, there is no use of the formal specification for implementing the tasks of the software. After the specification phase is finished, the software system is to be built from scratch, and there are plenty of occasions for human programmers to reintroduce bugs.

Some formal specification methods operate on the abstraction level of the software itself. This happens in systems and languages supporting proof of correctness of program code, e.g., based on Hoare logic and its many descendants. But the very low abstraction level of software, with all its detail, is cumbersome for proving correctness in case of somewhat large systems. Besides, writing the actual program and the specification in parallel takes double the human labour.

Refinement is a method that seems to bridge the gap between these ways of using formal specifications. Abstraction is good. But to be practically useful, we need a way to link abstract descriptions to more concrete “executable” ones. Refinement is a missing link as it allows to close the gap between abstract specifications and concrete ones. It allows a modelling to be built gradually, from abstract to concrete, through a number of refinement steps. Here abstract specifications are reused. It leads to a localized and incremental method for proving correctness. It has been shown that if these refinement steps are proven correct, the final modelling will satisfy all invariants of the full model at all abstraction levels.

We have seen that this method can result in bug-free code, e.g. in the Paris metro. This is a major achievement indeed. On the other hand, the method is not a panacea, as shown by the fact that the formal specification is longer than the resulting program. It is still a very costly method, only useful for mission-critical subsystems. Much progress is still needed before this method becomes broadly applicable.

But it is an interesting principle. Refinement is an instance of a general, important, useful idea that is applicable not only in dynamic systems but in all systems. The idea is to build precise and detailed concrete specifications by a sequence of concretization steps.

Finally, it is important to realize that there are other logical paradigms where formal specifications are used directly for solving the computational tasks (namely through inference engines); these formal specifications may be far more compact than equivalent software

written in standard programming languages. For example, the SQL queries in databases are (often) formal declarative descriptions (of sets): imagine that all SQL queries would have to be programmed as standard programs! As another example, formal specifications in constraint programming may be far more compact, readable and comprehensible than the equivalent software written in standard programming languages. We also refer back to the discussion on LogicBlox in chapter 3.

There is apparently much that we do not yet understand here. In the later chapter on the knowledge-base paradigm, we return to the use of logic specifications for problem solving.

5.7 Important for the exam

You need to understand the principles of refinement, but not the details of the translation of Event-B to program code as seen in the section 5.5.

What you need to know:

Understanding of

- the principle of refinement
- different kinds of proof obligations
- the two correctness conditions.

I may sketch a situation and ask what sorts of proof obligations arise, or which correctness conditions 1 and 2 are satisfied or not.

Theoretically, I may ask how to ensure correctness criterion 1 or 2 in theory or in an example.

Chapter 6

Classical results on Predicate Logic: Provability, Expressivity, Decidability, Incompleteness

6.1 Proof theory

6.1.1 Background: deductive inference

The following are deductive inference problems:

- The problem of deciding tautologicity (logical validity) of a formula:

Tautology checking
Input: a formula φ
Output: **t** if φ is tautological (aka logically valid) ($\models \varphi$),
and **f** otherwise

- The problem of deciding that a theory is unsatisfiable:

Unsatisfiability checking
Input: a formula or theory T
Output: **t** if T is unsatisfiable, **f** otherwise

- The problem of deciding logical consequence:

Deduction
Input: a formula or theory T , formula φ
Output: **t** if T logically entails φ ($T \models \varphi$), and **f** otherwise

For finite theories T these problems can be reduced to each other. E.g., φ is logically valid iff $\neg\varphi$ is unsatisfiable; T is unsatisfiable iff $T \models \mathbf{f}$; and $T \models \varphi$ iff $\wedge(T) \Rightarrow \varphi$ is logically valid. Here, $\wedge(T)$ denotes the conjunction of the formulas of T . These equivalences are shown in Figure 6.1.

Exercise 6.1.1 Explain these equivalences.

Deductive inference (or deduction for short) is the typical form of reasoning used in mathematical proofs. As mentioned in chapter 2, the study of deduction started for over 2000

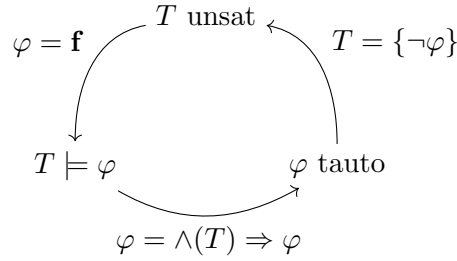


Figure 6.1 – Relationship between unsatisfiability, deduction and tautology checking.

years, with Aristotelian syllogisms. First order predicate logic FO grew out of attempts to formalize mathematical proofs. Even today, in the mind of many people, FO remains entangled with deductive inference and is called “*deductive logic*”.

6.1.2 Hilbert Proof systems

This section gives a fast (and shallow) introduction to proof systems. There are different types of proof systems for FO, but we focus on the original type, Hilbert proof systems.

Definition 6.1.1: Proof system

A proof system $\mathcal{P}$ consists of *axiom schemes* and *inference rules*. Axiom schemes are *logic schemata*, i.e., formula variables $\alpha, \beta, \dots$ or formulas containing only logical symbols ($\wedge, \neg, \forall \dots$) and formula variables. Inference rules are of the form:

$$\frac{\alpha_1, \dots, \alpha_k}{\beta}$$

where $\alpha_1, \dots, \alpha_k, \beta$ are again logic schemata.

A $\mathcal{P}$ -proof of a formula ψ (without formula variables) from a theory T (without formula variables) is a **finite** sequence $\langle \psi_0, \dots, \psi_n \rangle$ of formulas (without formula variables) such that $\psi_n = \psi$ and each $i \leq n$

- ψ_i is an instance of an axiom scheme α of $\mathcal{P}$; this means there is an assignment θ of concrete formulas to formula variables such that $\psi_i = \theta(\alpha)$;
- or $\psi_i \in T$ (we say that ψ_i is *introduced from T* in this step)
- or there is an inference rule $\frac{\alpha_1, \dots, \alpha_k}{\beta}$ of $\mathcal{P}$ such that ψ_i is obtained by applying this inference rule on a set of propositions that were proven before; this means that there is an assignment θ such that ψ_i is the instance $\theta(\beta)$, and the instances $\theta(\alpha_j), j \leq k$ belong to $\{\psi_0, \dots, \psi_{i-1}\}$.

An axiom scheme serves to express domain independent general truths. A good example is:

$$\alpha \vee \neg \alpha$$

This axiom scheme is called the *axiom (scheme) of excluded middle*. Different instances are $P \vee \neg P$ (using $\alpha := P$), $Q(x) \vee \neg Q(x)$ (using $\alpha := Q(x)$), and $\exists x(x = x) \vee \neg(\exists x(x = x))$ (using $\alpha := \exists x(x = x)$). In a proof system containing this axiom scheme, each such instance has a proof of length 1.

An example of an inference rule is the following one:

$$\frac{\alpha, (\alpha \Rightarrow \beta)}{\beta} \tag{6.1}$$

This inference rule is called *modus ponens*. Assume that $P(a, b)$ and $P(a, b) \Rightarrow P(b, c)$ have been proven earlier in the proof, then application of this inference rule with $\alpha := P(a, b)$ and $\beta := P(b, c)$ infers $P(b, c)$.

Example 6.1.1: The Hilbert-Ackermann proof system

A well-known Hilbert proof system is the 1928 Hilbert-Ackermann system. It is designed for the subset of FO formulas built from $\Rightarrow$, $\neg$ and $\forall$, which is an adequate set for FO.¹ It consists of 9 axiom schemes, and 1 inference rule. For formula variables ϕ , ψ , and ξ , the axiom schemes are:

$$\begin{aligned}
 & \phi \Rightarrow \phi \\
 & \phi \Rightarrow (\psi \Rightarrow \phi) \\
 & (\phi \Rightarrow (\psi \Rightarrow \xi)) \Rightarrow ((\phi \Rightarrow \psi) \Rightarrow (\phi \Rightarrow \xi)) \\
 & (\neg \phi \Rightarrow \neg \psi) \Rightarrow (\psi \Rightarrow \phi) \\
 & \forall x(\phi) \Rightarrow \phi[x := t] \\
 & \forall x(\phi \Rightarrow \psi) \Rightarrow (\forall x(\phi) \Rightarrow \forall x(\psi)) \\
 & \phi \Rightarrow \forall x(\phi) \quad \text{where } x \text{ is not free in } \phi \\
 & x = x \quad \text{for every variable } x \\
 & (x = y) \Rightarrow (\phi[z := x] \Rightarrow \phi[z := y])
 \end{aligned} \tag{6.2}$$

Above $\phi[z := x]$ denotes the formula obtained from ϕ by substituting x for all free occurrences of z .

The first four are the axiom schemes for propositional logic. The next three logical axiom schemes provide ways to add, manipulate, and remove universal quantifiers. The final axiom schemes are required for formulas involving the equality symbol.

The proof system has one inference rule, the modus ponens (Eq. 6.1).

Other systems have fewer axiom schemes but more inference rules. E.g., we may replace the fifth axiom scheme by the following inference rule:

$$\frac{\forall x(\phi[x])}{\phi[x := t]} \tag{6.3}$$

Another well-known proof system for clausal logic is the proof system consisting of no axiom schemes and one inference rule, the resolution inference rule.

A proof Pr of φ from theory T often does not use all axioms of T . Let S be the set of formulas that are introduced from T in Pr . Notice that S is always finite. The following proposition is obvious.

Proposition 6.1.1

If Pr is a proof of φ from theory T then Pr is also a proof of φ from S .

Definition 6.1.2: Provability, $\mathcal{P}$ -consistency and $\mathcal{P}$ -validity

Let $\mathcal{P}$ be a proof system.

- A formula φ is $\mathcal{P}$ -provable from a FO theory T (denoted² $T \vdash_{\mathcal{P}} \varphi$) if there is a

¹Alternatively, the Hilbert-Ackermann system can be easily extended with inference rules for $\wedge$, $\vee$ and $\exists$.

$\mathcal{P}$ -proof of φ from T .

- A FO theory T is $\mathcal{P}$ -consistent if there is no sentence φ , such that $T \vdash_{\mathcal{P}} \varphi$ and $T \vdash_{\mathcal{P}} \neg\varphi$.
- A formula φ is $\mathcal{P}$ -valid if there is a $\mathcal{P}$ -proof of φ from the empty theory.

Different proof systems $\mathcal{P}$ resp. $\mathcal{P}'$ induce different provability relations $\vdash_{\mathcal{P}}$ and $\vdash_{\mathcal{P}'}$ and different notions of $\mathcal{P}$ -consistency and $\mathcal{P}$ -validity. Two proof systems are called *equivalent* if they induce the same provability relation. In that case, they induce the same notions of consistency and validity.

The “good” proof systems are those that formalize correct mathematical proofs. But which are those? Well, mathematical proofs serve to prove entailed properties. In the context of logic, entailment is formalized as logical entailment $T \models \varphi$ (φ true in every model of T). Thus, “good” proof systems $\mathcal{P}$ should have the property that $T \vdash_{\mathcal{P}} \varphi$ (φ has a $\mathcal{P}$ -proof from T) entails $T \models \varphi$. This property of $\mathcal{P}$ is called **soundness**. Even better is if the inverse is true: if $T \models \varphi$ then $T \vdash_{\mathcal{P}} \varphi$, i.e., φ has a $\mathcal{P}$ -proof from T . This property is called **completeness**. If $\mathcal{P}$ is sound and complete, then the provability relation $\vdash_{\mathcal{P}}$ is the same as the entailment relation $\models$.

Notice how different $\models$ and $\vdash_{\mathcal{P}}$ are defined. There is an immense difference between verifying that a formula φ is true in all models of T , and building a proof of φ from T using some proof system. The first requires to evaluate φ and T in a potentially infinite class of structures; the second is obtained by mechanical symbol manipulation. It is far from obvious that a logic has a sound and complete proof system.

Remark 6.1.1: Automated theorem proving

Importantly, checking proofs of a proof system is mechanizable. That is, there exist algorithms that take as input any sequence of formulas and decide whether it is a proof of the given proof system. Better, there exist algorithms that search for a proof of tautologicity of α or for a proof of α from T and that, in principle,³ will find it if one exists. This is the basis for the domain of automated theorem proving (ATP).

ATP for FO is the scientific field involved in developing automated theorem provers. Building good theorem provers is very challenging. Some well-known theorem provers are Otter, SPASS, Vampire, ACL2, and Waldmeister.

Theorem provers are applied for automated verification in mission-critical subsystems. An early success was the use of ACL2 to prove the correctness of the floating-point division operation of the AMD K5 microprocessor [18] in the wake of the Intel Pentium FDIV disaster: Intel’s early Pentium processors had shipped with a floating-point arithmetic unit that made slight errors when dividing numbers in a certain range, and had to recall all chips, which cost the company around 600 million dollars.

The unsolved Robbins problem stated in 1933 on Boolean algebra was the first open mathematical problem that was solved by a computer. It was cracked in 1996 by the theorem prover EQP, taking 8 days. Mainstream media picked up the story [19].

However, whereas humans lost the battle to computers in chess and Go, mathematicians still beat computers in solving mathematical problems, by far. In general, for creative and elegant solutions to hard mathematical and other problems, computers have not been able to beat the human mind.

²When $\mathcal{P}$ is clear from the context, we write $\vdash$ rather than $\vdash_{\mathcal{P}}$.

Semi-automatic theorem provers are also in development; they need interaction with humans. Humans could for example guide the search for a proof of a proposition by breaking up the proposition into easier and useful lemmas. Well-known systems are Coq and Isabelle. They support richer logics than FO and are used more and more in program verification. They support proofs by induction which is important in verification of programs and other dynamic systems.

6.1.3 Soundness and completeness

The desired connections between proofs ($\vdash_{\mathcal{P}}$) and entailment ($\models$)⁴ are formalized in the concepts of soundness and completeness of a proof system.

Definition 6.1.3: Soundness and completeness

- A proof system $\mathcal{P}$ is *sound* if $T \vdash_{\mathcal{P}} \alpha$ implies $T \models \alpha$.
- A proof system $\mathcal{P}$ is *complete* if $T \models \alpha$ implies $T \vdash_{\mathcal{P}} \alpha$.

In terms of relations, soundness means $(\vdash_{\mathcal{P}}) \subseteq (\models)$, while completeness means $(\vdash_{\mathcal{P}}) \supseteq (\models)$. Soundness is usually much easier to prove than completeness.

For a proof system to be sound, a necessary condition is that its axiom schemes correspond to tautological (logically valid) propositions after replacing formula variables by logical expressions. If not, the proof system will be unsound. For example, assume we add the axiom scheme $\neg\phi \Rightarrow \phi$ to the Hilbert-Ackerman system of Example 6.1.1. The new proof system allows for the single-step proof (from the empty theory) using $\phi := p$:

$$\langle (\neg p \Rightarrow p) \rangle. \quad (6.4)$$

But the proven formula is clearly not a tautology! The extended proof system is unsound. Likewise, sound inference rules $\frac{\beta_1, \dots, \beta_m}{\alpha}$ correspond to tautological propositions (again replacing formula variables by appropriate propositional or predicate symbols):

$$\beta_1 \wedge \dots \wedge \beta_m \Rightarrow \alpha. \quad (6.5)$$

If the proof system has soundness and completeness, then provability ($\vdash$) and entailment ($\models$) coincide. What's more: $\mathcal{P}$ -consistency and satisfiability also coincide in that case, as do $\mathcal{P}$ -validity and tautologicity (logical validity).

Proposition 6.1.2

If the proof system $\mathcal{P}$ is sound and complete, then $\mathcal{P}$ -consistency and satisfiability coincide.

Proof. If T is not consistent, then for some φ , $T \vdash_{\mathcal{P}} \varphi$ and $T \vdash_{\mathcal{P}} \neg\varphi$. Assume towards contradiction that T is satisfiable and $\mathfrak{A}$ is a model of T , then by soundness of $\vdash_{\mathcal{P}}$, $\mathfrak{A} \models \varphi$ and $\mathfrak{A} \models \neg\varphi$. However, no structure satisfies a formula and its negation. Contradiction, hence T is unsatisfiable.

Vice versa: if T is not satisfiable, then it has no models. This means that trivially, every formula φ is true in all of T 's models (all zero of them), which is the definition of $T \models \varphi$. The same reasoning applies to the formula $\neg\varphi$, so $T \models \neg\varphi$. Now, by completeness, $T \vdash_{\mathcal{P}} \varphi$ and $T \vdash_{\mathcal{P}} \neg\varphi$. We conclude that T is $\mathcal{P}$ -inconsistent. ■

³Under the assumption of unbounded time and memory.

⁴Recall that the symbol $\models$ is overloaded. Pay attention that in " $T \models \alpha$ ", the $\models$ does not refer to the satisfaction relation $\models$ of Definition 2.2.8 but to the entailment relation of Definition 1.2.2.

Proof systems of FO In the past century, many Hilbert proof systems for FO have been developed, consisting of different combinations of logical axiom schemes and inference rules. Other types of proof systems have been defined too, such as tableaux proof methods (e.g. the KE proof method in the logic course in the informatics bachelor), sequent calculus, and natural deduction proof systems.

As mentioned, showing that a proof system is sound is usually easy. Showing completeness is more difficult. In 1930, for the first time, Gödel proved the completeness of a proof system in his PhD dissertation. The proof system $\mathcal{P}_0$ in question was proposed by Whitehead and Russell 20 years earlier in their book *Principia Mathematica*. It was known to be sound for a long time.

Theorem 6.1.1: Gödel's completeness theorem (GCT), 1930

For each theory T and for each sentence α in FO, if $T \models \alpha$ then $T \vdash_{\mathcal{P}_0} \alpha$.

This theorem not only proved the completeness of Whitehead and Russell's proof system, but also of several other proof systems that were known to be equivalent to theirs.

Remark 6.1.2

The completeness theorem also holds if the theory T is infinite. This will be important for proving the compactness theorem.

A simple consequence of the soundness and completeness of these FO proof systems is the *compactness theorem*. The compactness theorem has surprising consequences for proving inexpressivity theorems for FO, as we shall see.

Theorem 6.1.2: Compactness theorem of FO

An infinite FO theory Ψ is satisfiable iff each finite subset is satisfiable.

Proof. Equivalently, an infinite theory Ψ is unsatisfiable iff *at least one* finite subset is unsatisfiable. It is the latter statement we prove.

- ($\Leftarrow$) If Ψ has an unsatisfiable subset Ω , then Ψ is unsatisfiable, since every model of Ψ satisfies all its formulas and all its subsets.
- ($\Rightarrow$) According to Gödel's completeness theorem, there exists a sound and complete FO proof system. We select such a proof system and denote it as $\mathcal{P}$.

If Ψ is unsatisfiable, then by Proposition 6.1.2, Ψ is $\mathcal{P}$ -inconsistent. Per definition, there exists a formula φ so that there exists a $\mathcal{P}$ -proof Pr_1 of φ from Ψ , and there exists a $\mathcal{P}$ -proof Pr_2 of $\neg\varphi$ from Ψ .

A proof from Ψ is a finite sequence and uses only finitely many formulas from Ψ : these are the formulas *introduced from* Ψ according to Definition 6.1.1. Take the set Ω of all formulas introduced from Ψ in Pr_1 and Pr_2 . Ω is a finite subset of Ψ . All other formulas in Pr_1 and Pr_2 were obtained as instances of axiom schemas, or derived by an inference rule from formulas earlier in the proof. It is easy to see that Pr_1 is also a $\mathcal{P}$ -proof of φ from Ω and Pr_2 is a $\mathcal{P}$ -proof of $\neg\varphi$ from Ω . Hence, Ω is $\mathcal{P}$ -inconsistent. Again by Proposition 6.1.2, we conclude that Ω is unsatisfiable. ■

6.2 Expressivity analysis in FO: an introduction

Expressivity theory is the formal study of what properties can and cannot be expressed in a modelling logic $\mathcal{L}$.

In an application domain, one determines a relevant set of concepts and chooses symbols to represent them. One obtains a vocabulary Σ and an informal interpretation $\mathcal{I}$ such that for each symbol $\sigma \in \Sigma$, $\mathcal{I}(\sigma)$ is the concept that it represents. This allows to interpret formal propositions in a logic $\mathcal{L}$ as informal propositions in the application domain.

The next question, the one investigated in this section, is: what properties about the application domain can or cannot be expressed in terms of Σ in the logic $\mathcal{L}$? E.g., suppose Σ contains the predicate symbol *parent*/2 to represent the parent relation between a child and its parent. Can we express in FO the informal propositions that a child has exactly two parents, that some person A is a grandparent or an ancestor of person B, that the parent relation does not contain cycles? Or, given a vocabulary Σ about the natural numbers, can we express in FO that the set of natural numbers consists of 0, 1, ..., n, n+1, ... and nothing more?

In such a setting, we may think of an informal proposition Φ phrased in terms of the chosen concepts. The question is if we can express Φ by a formula or theory φ over Σ . But what does it mean that φ expresses Φ ? It means that φ **gets the models right**: a Σ -structure $\mathfrak{A}$ satisfies φ iff Φ is true in $\mathcal{I}(\mathfrak{A})$, a state of affairs abstractly represented by $\mathfrak{A}$.

To turn this informal expressivity question into a mathematical question suitable for mathematical analysis, we need to get rid of all informal notions in the above statement, namely Φ and $\mathcal{I}$. In the mathematical analysis we will study whether a class $\mathcal{C}$ of structures can be expressed by a formula or theory T over Σ in the sense that $\text{Mod}(T) = \mathcal{C}$. For the connection with the informal setting, we should think of $\mathcal{C}$ as the class of structures $\mathfrak{A}$ abstracting states of affairs $\mathcal{I}(\mathfrak{A})$ satisfying Φ .

Now we are ready to define an expressivity question in mathematical form:

Given a class $\mathcal{C}$ of structures interpreting a vocabulary Σ , does there exist a theory T in $\mathcal{L}$ such that $\text{Mod}(T) = \mathcal{C}$.

To address such questions is the goal of the expressivity analysis of a modelling logic $\mathcal{L}$ (with satisfaction relation $\models$). It is the scientific study of what propositions can and cannot be expressed in $\mathcal{L}$.

Definition 6.2.1: Expressing and expressibility of classes

- A formula or theory T of logic $\mathcal{L}$ *expresses* a class $\mathcal{C}$ of structures if $\text{Mod}(T) = \mathcal{C}$.
- A class $\mathcal{C}$ is *finitely expressible* in $\mathcal{L}$ if it is expressed by a formula φ of $\mathcal{L}$.
- A class $\mathcal{C}$ is *expressible* in $\mathcal{L}$ if it is expressed by a (possibly infinite) theory T of $\mathcal{L}$.

If the logic $\mathcal{L}$ possesses conjunction, then any finite theory T is semantically equivalent with a formula, namely the conjunction of the elements of the theory, denoted as $\bigwedge(T)$.

6.2.1 Proving expressivity

Let's start with the “easy” part: proving that some statement Φ is expressible.

Example 6.2.1

The application domain is that of graphs. Take $\Sigma = \{A/0, G/2\}$ with $\mathcal{I}(G)$ being some binary graph, denoted $\mathbf{G}$ and $\mathcal{I}(A)$ being some vertex $\mathbf{A}$. Take the following informal proposition Φ :

$$\Phi = \text{“The vertex } \mathbf{A} \text{ has no outgoing edges in graph } \mathbf{G}.” \quad (6.6)$$

The sentence Φ is a mathematical statement of the usual kind, but it is called informal here because it is phrased in natural language. We want to investigate if this proposition can be expressed in FO.

The proposition Φ mathematically defines a class $\mathcal{C}_\Phi$ of structures, namely all structures $\mathfrak{A}$ interpreting Σ (and possibly more symbols) such that there exists no $d \in D_{\mathfrak{A}}$ such that $(A^{\mathfrak{A}}, d) \in G^{\mathfrak{A}}$. This class formalizes the information content of the non-formal proposition Φ . We then rephrase the expressivity question in purely mathematical terms: does there exist a formula or theory φ over Σ in $\mathcal{L}$ such that its class of models – call it $\text{Mod}(\varphi) = \{\mathfrak{A} \mid \mathfrak{A} \models \varphi\}$ – is exactly $\mathcal{C}_\Phi$?

For the above Φ and the logic FO, the answer is evidently positive. Φ is logically expressed by φ :

$$\neg \exists x G(A, x) \quad (6.7)$$

It is obvious that indeed, $\mathcal{I}(\varphi)$ matches with Φ , but let's check the formal condition that $\text{Mod}(\varphi) = \mathcal{C}_\Phi$. We have that $\mathfrak{A} \models \varphi$ iff $\mathfrak{A} \models \neg \exists x G(A, x)$ iff (using the rules of Definition 2.2.8) there is no $d \in D_{\mathfrak{A}}$ such that $(A^{\mathfrak{A}}, d) \in G^{\mathfrak{A}}$ iff $\mathfrak{A} \in \mathcal{C}_\Phi$.

From the above example, the method can be generalized to Procedure 6.1.

Procedure 6.1: Proof of expressivity

Given a vocabulary Σ with informal interpretation $\mathcal{I}$, and an informal proposition Φ expressed in terms of $\mathcal{I}(\Sigma)$, to be expressed as a formula or theory over Σ .

1. Let $\mathcal{C}_\Phi$ be the class of all structures interpreting Σ that are abstractions of states of affairs in which Φ is true.
2. Formulate a formula or theory T over Σ in $\mathcal{L}$.
3. Prove that $\text{Mod}(\Phi) = \mathcal{C}_\Phi$ (i.e., $\mathfrak{A}$ satisfies T iff $\mathfrak{A}$ is the abstraction of a state of affairs satisfying Φ).

Example 6.2.2

Again in the graph domain, take $\Sigma = \{G/2\}$ with the same informal interpretation as before. Consider the informal proposition:

$$\Phi = \text{“The graph } \mathbf{G} \text{ is reflexive.”} \quad (6.8)$$

1. The corresponding class $\mathcal{C}_\Phi$ consists of structures $\mathfrak{A}$ that interpret at least Σ and in which $G^{\mathfrak{A}}$ is a reflexive relation, i.e. the set $\{(d, d) \mid d \in D_{\mathfrak{A}}\}$ is a subset of $G^{\mathfrak{A}}$.
2. To express Φ we choose the FO theory:

$$T = \{\forall x G(x, x)\}. \quad (6.9)$$

3. To prove: $\text{Mod}(T) = \mathcal{C}_\Phi$.

Proof. Every structure $\mathfrak{A}$ interpreting Σ satisfies $\forall x G(x, x)$ iff $G^\mathfrak{A}$ contains (d, d) for each $d \in D_\mathfrak{A}$ iff $G^\mathfrak{A}$ is reflexive iff $\mathfrak{A} \in \mathcal{C}_\Phi$. ■

Example 6.2.3: Infinitely expressible proposition

Take $\Sigma = \emptyset$, and the informal proposition:

$$\Phi = \text{“The universe is infinite.”} \quad (6.10)$$

Its class, denoted $\mathcal{C}_{\text{Inf.U}}$, is the class of structures with infinite domain. This proposition only talks about the domain of a structure, which exists in every structure, even if it does not interpret any non-logical symbol. To express this proposition, the only predicate symbol that can be used is the logical symbol $=$. Somewhat surprisingly, the proposition is expressed by the following infinite theory:

$$T_{\text{Inf.U}} = \left\{ \begin{array}{l} \exists x_1 \exists x_2 (\neg(x_1 = x_2)), \\ \exists x_1 \exists x_2 \exists x_3 (\neg(x_1 = x_2) \wedge \neg(x_1 = x_3) \wedge \neg(x_2 = x_3)), \\ \dots \\ \exists x_1 \dots \exists x_n (\neg(x_1 = x_2) \wedge \dots \wedge \neg(x_1 = x_n) \wedge \neg(x_2 = x_3) \wedge \dots \wedge \neg(x_{n-1} = x_n)), \\ \dots \end{array} \right\} \quad (6.11)$$

The theory is a set of FO sentences $\{\varphi_n | n \geq 2\}$ where the formula φ_n is an existential quantification of n variables $x_1, \dots, x_n$, stating in one big conjunction that x_i is not equal to x_j when $0 \leq i < j \leq n$. Hence, φ_n expresses that there exists at least n different elements in the universe. The claim is that T expresses Φ , the informal proposition that the universe is infinite.

Proof. Any structure with an infinite universe satisfies φ_n for each $n \geq 2$; hence it satisfies $T_{\text{Inf.U}}$. Vice versa, any structure satisfying $T_{\text{Inf.U}}$ has at least 2 elements in its universe, at least 3, $\dots$, at least n , for arbitrary n . Hence, such a structure has an infinite universe. ■

Exercise 6.2.1 Prove that a structure $\mathfrak{A}$ satisfies the n 'th axiom of this theory iff $D_\mathfrak{A}$ contains at least n elements. Second, if we change $\exists$ in $\forall$ in such a sentence, what does the resulting sentence mean and is it satisfiable? (Hint: No, it is not)

So, yes, we can express that the universe is infinite in FO, but the way how is inconvenient to say the least: the theory is infinite, and the size of the formulas φ_n is quadratic in n . Still, the theory has its use, as shall become clear soon.

Formulas are finite, theories may be infinite. Therefore, we should expect that we can express more with theories than with formulas. In fact, $\mathcal{C}_{\text{Inf.U}}$ cannot be finitely expressed as will be proven below, in Corollary 6.2.1).

6.2.2 Inexpressivity analysis

An inexpressivity result is more difficult to prove, since we need to prove for the infinite set of formulas or theories over Σ that none expresses the informal proposition Φ .

A first result states simple properties of all expressible classes. Recall that an expansion $\mathfrak{A}'$ of a structure $\mathfrak{A}$ is a structure equal to $\mathfrak{A}$ except that it interprets more symbols.

Theorem 6.2.1

For any FO theory T , let Σ be the set of free non-logical symbols of T . The class $\text{Mod}(T)$ satisfies the following properties:

1. All elements of $\text{Mod}(T)$ interpret Σ (and potentially other symbols).
2. $\text{Mod}(T)$ is closed under expansion: if $\mathfrak{A} \in \text{Mod}(T)$ then any expansion $\mathfrak{A}'$ of $\mathfrak{A}$ belongs to $\text{Mod}(T)$.
3. $\text{Mod}(T)$ is closed under Σ -isomorphism: for any pair of structures $\mathfrak{A}, \mathfrak{A}'$ such that $\mathfrak{A} \cong_{\Sigma} \mathfrak{A}'$, it holds that $\mathfrak{A} \in \text{Mod}(T)$ iff $\mathfrak{A}' \in \text{Mod}(T)$.

Proof. (1) The truth value of a theory T in a structure $\mathfrak{A}$ is only defined if $\mathfrak{A}$ interprets all free non-logical symbol of T . (2) If $\mathfrak{A}$ satisfies T then by Corollary 2.2.1, any expansion of $\mathfrak{A}$ satisfies T . (3) If $\mathfrak{A} \cong_{\Sigma} \mathfrak{A}'$ then both interpret Σ and by Proposition 2.2.2, the truth value of T is the same in $\mathfrak{A}$ and $\mathfrak{A}'$. ■

This theorem is an inexpressivity theorem. Indeed, any class of structures that does not satisfy the properties of Theorem 6.2.1 is inexpressible in FO. All classes $\mathcal{C}$ of structures considered in this section are assumed to satisfy the conditions of the above theorem.

These are natural conditions. In FO (and $\text{FO}(\cdot)$), the truth value of formula or theory φ in a structure depends only on the interpretation of symbols that occur free in φ . That is why any expansion of a structure $\mathfrak{A} \in \text{Mod}(T)$ belongs to $\text{Mod}(T)$. Furthermore, two structures are Σ -isomorphic if there is a bijection between their domains that maps the value of a symbol in Σ in one structure to the value of the same symbol in the other structure. Isomorphic structures are abstract representations of the same states of affairs and hence, they should satisfy the same propositions over Σ .

We now introduce the main technique to prove inexpressivity of this course. The compactness theorem was the first technique that was used to prove inexpressivity results. We will use it to prove inexpressivity in FO of three propositions that have appeared earlier in this course. First, some new terminology is introduced.

Definition 6.2.2: Class inconsistency

Let $\mathcal{C}$ and $\mathcal{C}'$ be classes of structures. We say that $\mathcal{C}$ is *inconsistent with* $\mathcal{C}'$ if $\mathcal{C} \cap \mathcal{C}' = \emptyset$ (they don't share any structures).

We say that $\mathcal{C}$ is inconsistent with a theory or formula T if $\mathcal{C}$ is inconsistent with $\text{Mod}(T)$.

Proposition 6.2.1

A structure satisfies a union of two theories iff it is in the intersection of their model classes:

$$\mathfrak{A} \models T \cup T' \text{ iff } \mathfrak{A} \in \text{Mod}(T) \cap \text{Mod}(T')$$

Proof. $\mathfrak{A} \models T \cup T'$ iff $\mathfrak{A} \models T$ and $\mathfrak{A} \models T'$, iff $\mathfrak{A} \in \text{Mod}(T) \cap \text{Mod}(T')$. ■

Theorem 6.2.2: Inexpressivity theorem

Let Σ be a vocabulary, let $\mathcal{C}$ be a class of structures interpreting Σ ($\mathcal{C} \subseteq \mathcal{C}_\Sigma$), and let T_∞ be an infinite FO theory over Σ . If both:

1. $\mathcal{C} \cap \text{Mod}(T_\infty) = \emptyset$ (in words, T_∞ is inconsistent with $\mathcal{C}$),
2. for each finite subset Ω' of T_∞ , it holds that $\mathcal{C} \cap \text{Mod}(\Omega') \neq \emptyset$ (in words, each finite subset of T_∞ is consistent with $\mathcal{C}$)

... then $\mathcal{C}$ is not expressible in FO.

Before we prove this, let's parse what the theorem says: think of $\mathcal{C}$ as the class of structures satisfying the informal proposition Φ . The infinite FO theory T_∞ also expresses an informal proposition through its informal interpretation, say Ψ . So, we have two informal propositions, Φ and Ψ .

The first condition, $\mathcal{C} \cap \text{Mod}(T_\infty) = \emptyset$, means that the conjunction of Φ and Ψ is inconsistent; there are no structures satisfying both. Thus, Φ and Ψ are contradictory statements. The second condition, $\mathcal{C} \cap \text{Mod}(\Omega') \neq \emptyset$, for every finite subtheory Ω' of T_∞ , means that Φ is consistent with every finite subtheory Ω' of T_∞ . The theorem then ensures that under these conditions, Φ is not expressible in FO.

Proof. Assume towards contradiction that $\mathcal{C}$ is expressed by FO theory T ; i.e., $\text{Mod}(T) = \mathcal{C}$. We now prove a contradiction about the theory $T \cup T_\infty$.

On the one hand, it follows from condition (1) that $\text{Mod}(T \cup T_\infty) = \text{Mod}(T) \cap \text{Mod}(T_\infty) = \emptyset$, hence $T \cup T_\infty$ is unsatisfiable.

On the other hand, take any finite subset $\Omega \subseteq T \cup T_\infty$. We have that $T \cup \Omega = T \cup (\Omega \setminus T)$. The set $\Omega' = \Omega \setminus T$ is a finite subset of T_∞ . It follows that $\text{Mod}(T \cup \Omega) = \text{Mod}(T \cup \Omega') = \text{Mod}(T) \cap \text{Mod}(\Omega') = \mathcal{C} \cap \text{Mod}(\Omega')$. By condition (2) of the theorem, this class is not empty. Thus, $T \cup \Omega$ is satisfiable. Since a subset of a satisfiable theory is satisfiable, Ω is satisfiable as well. Since Ω was chosen arbitrarily, all finite subsets Ω of $T \cup T_\infty$ are satisfiable. It follows from the compactness theorem that $T \cup T_\infty$ is satisfiable.

From our initial assumption that $\mathcal{C}$ is expressible as a FO theory T , we have proven the contradiction that $T \cup T_\infty$ is satisfiable and unsatisfiable. Hence, $\mathcal{C}$ is not expressible in FO. ■

Remark 6.2.1

A common mistake in the exam is that students forget the word “finite” in condition (2). Without that word, condition (2) would say that all subsets of T_∞ , including T_∞ itself, are consistent with $\mathcal{C}$. That contradicts condition (1).

A gracious first application of the inexpressivity theorem reveals an interesting property of the theory T_∞ . Below, we assume that Σ , $\mathcal{C}$ and T_∞ satisfy the conditions of Theorem 6.2.2.

Corollary 6.2.1

T_∞ is not finitely expressible.

If the informal proposition expressed by T_∞ could be expressed by one FO sentence φ_∞ , then also the negation of that informal proposition could be expressed in FO, namely by $\neg\varphi_\infty$. But that contradicts with the inexpressivity theorem.

Proof. Assume towards contradiction that T_∞ is finitely expressed by a FO formula φ_∞ over Σ . Then $\neg\varphi_\infty$ expresses the class $\mathcal{C}'$ of all structures that interpret Σ and do not satisfy T_∞ . Since structures in $\mathcal{C}$ do not satisfy T_∞ , we have $\mathcal{C} \subseteq \mathcal{C}'$. We now verify that not only $\mathcal{C}$ but also $\mathcal{C}'$ satisfies the conditions of the inexpressivity theorem: (1) obviously $\mathcal{C}'$ is inconsistent with T_∞ ($\mathcal{C}' \cap \text{Mod}(T_\infty) = \emptyset$); (2) for every finite $\Omega \subseteq T_\infty$, it holds that $\mathcal{C}' \cap \text{Mod}(\Omega) \supseteq \mathcal{C} \cap \text{Mod}(\Omega) \neq \emptyset$. Hence, Ω is consistent with $\mathcal{C}'$. Now the inexpressivity theorem applies. It entails that $\mathcal{C}'$ is not expressible in FO. Contradiction. ■

Hence, expressible informal propositions in FO are not necessarily finitely expressible. The theorem itself doubles as a proof method, and can serve to achieve two things of value:

- To prove that some *informal* proposition is not expressible in FO.
- To prove that some *infinitely expressible* informal proposition is not finitely expressible in FO.

Now, let's consider three applications of the inexpressivity theorem.

Inexpressivity result 1: Finiteness of the universe is inexpressible in FO Let Σ be empty, and consider the informal proposition “**The universe is finite**”. It characterizes the class of structures with a finite universe, which we denote $\mathcal{C}_{\text{Fin.U}}$. The proposition cannot be expressed in FO, as shown next.

Theorem 6.2.3

$\mathcal{C}_{\text{Fin.U}}$ is not expressible in FO.

Proof. Consider the class $\mathcal{C}_{\text{Inf.U}}$ of structures with infinite universe. In Example 6.2.3 we saw how to express this class by $T_{\text{Inf.U}}$ (Eq. 6.11). Let us verify that the conditions of the theorem are satisfied for $\mathcal{C}_{\text{Fin.U}}$ and $T_{\text{Inf.U}}$.

- Both have the same trivial vocabulary, $\Sigma = \emptyset$.
- $\mathcal{C}_{\text{Fin.U}} \cap \mathcal{C}_{\text{Inf.U}} = \emptyset$: obviously.
- $\mathcal{C}_{\text{Fin.U}}$ is consistent with each finite subset $\Omega \subset T_{\text{Inf.U}}$. Indeed, there is a largest number n such that Ω contains the axiom φ_n from $T_{\text{Inf.U}}$ that expresses that the domain contains at least n elements. Any finite structure with domain size n or more satisfies Ω .

Application of the inexpressivity theorem yields that $\mathcal{C}_{\text{Fin.U}}$ is not expressible in FO. ■

Remark 6.2.2

This theorem and its proof is a common exam question. Students should understand this proof; learning by heart is not accepted. A proof should always include the specification of entities used in the proof, here in particular $\mathcal{C}_{\text{Fin.U}}$, $\mathcal{C}_{\text{Inf.U}}$ and $T_{\text{Inf.U}}$.

The proposition “The universe is finite” *can* be expressed in SO, albeit in a somewhat unexpected way. We make use of a property true of all infinite sets and false of all finite sets: that there exists an injective mapping from the set to a strict subset of it. Recall that an injective mapping is one that maps different inputs to different outputs. The following SO formula expresses that there is a unary function F of the universe that is injective such

that some elements of the universe are not in its image:

$$\exists F \left(\underbrace{(\forall x \forall y (F(x) = F(y) \Rightarrow x = y))}_{F \text{ injective}} \wedge \underbrace{(\exists y \neg \exists x F(x) = y)}_{\text{some element of universe is not in } F\text{'s image}} \right) \quad (6.12)$$

This SO sentence over the empty vocabulary is true exactly in all structures with infinite universe. Its negation expresses that the universe is finite.

Exercise 6.2.2 Express the stronger property that the universe contains at most N elements in FO.

Expressivity result 2: Reachability in a graph is inexpressible in FO The proposition is about two nodes A and B in a directed graph G . These concepts are represented by the symbols of $\Sigma = \{A/0, B/0, G/2\}$ (with the obvious informal interpretation $\mathcal{I}$). A can reach B in G iff there exists a path from A to B following edges of G . We show that the property “**A can reach B in G**” cannot be expressed in FO.

Consider first the negation of that proposition “**A cannot reach B in G**”. We propose the following theory:

$$T_{\text{NoPath}} = \left\{ \begin{array}{l} \neg G(A, B), \\ \neg \exists x_1 (G(A, x_1) \wedge G(x_1, B)), \\ \dots \\ \neg \exists x_1 \dots \exists x_n (G(A, x_1) \wedge \dots \wedge G(x_n, B)), \\ \dots \end{array} \right\}. \quad (6.13)$$

We prove that this FO theory expresses that there is no path from A to B in G .

Proof. The propositions in T_{NoPath} express respectively: there is no path of length 1, of length 2, of length 3, ... from A to B . A structure $\mathfrak{A}$ in which there is no path from $A^{\mathfrak{A}}$ to $B^{\mathfrak{A}}$ in $G^{\mathfrak{A}}$ satisfies each of these propositions, and vice versa, a structure that satisfies each of these propositions cannot have a path from $A^{\mathfrak{A}}$ to $B^{\mathfrak{A}}$ in $G^{\mathfrak{A}}$. ■

We will use T_{NoPath} in the following theorem, showing that the negation of the above informal proposition cannot be expressed in FO.

Define the class $\mathcal{C}_{\text{Path}}$ of Σ -structures $\mathfrak{A}$ in which there is a path from $A^{\mathfrak{A}}$ to $B^{\mathfrak{A}}$ in $G^{\mathfrak{A}}$.

Theorem 6.2.4

$\mathcal{C}_{\text{Path}}$ is not expressible in FO.

That is, the proposition “**There is a path from A to B in G**” is not expressible in FO.

Proof. We verify that $\mathcal{C}_{\text{Path}}$ and T_{NoPath} satisfy the conditions of the theorem.

It is obvious that $\mathcal{C}_{\text{Path}}$ is inconsistent with T_{NoPath} . Second, we verify that $\mathcal{C}_{\text{Path}}$ is consistent with each finite subset Ω' of T_{NoPath} . Any such finite subset Ω' consists of finitely many formulas φ_n stating there is no path of length n from A to B . Select m such that φ_m is missing in Ω' . It is not difficult to find a structure $\mathfrak{A}$ that satisfies all sentences in Ω' but has a path of length m between $A^{\mathfrak{A}}$ and $B^{\mathfrak{A}}$, showing that $\mathfrak{A} \in \mathcal{C}_{\text{Path}}$. For example, take $D_{\mathfrak{A}} = [0, m]$, $A^{\mathfrak{A}} = 0$, $B^{\mathfrak{A}} = m$ and $G^{\mathfrak{A}} = \{(i, i+1) | i \in [0, m-1]\}$. This structure contains

exactly one path from $A^{\mathfrak{A}}$ to $B^{\mathfrak{A}}$, namely of length m . It satisfies all axioms of Ω' , therefore it belongs to $\mathcal{C}_{\text{Path}}$. Hence, $\mathcal{C}_{\text{Path}}$ is consistent with Ω' .

It follows from the inexpressivity theorem that reachability cannot be expressed in FO. ■

Here we obtain an answer to a question at the beginning of this section. Given the predicate *parent*/2, can we express that person A is an ancestor of person B ? This is the case if there is a path through the parent relation from A to B . So, this cannot be expressed in FO.

Reachability and transitive closure are very important concepts in many applications, but they cannot be expressed in FO. This is a motivation to extend FO with language constructs that are strong enough to express reachability. We discuss two such extensions: SO and FO(ID).

First, in SO, “ A has a path to B in G ” is expressed by the following SO proposition:

$$\forall H \left(\left(\underbrace{\forall x \forall y (G(x, y) \Rightarrow H(x, y))}_{\text{extends } G} \wedge \underbrace{\forall x \forall y \forall z (H(x, y) \wedge H(y, z) \Rightarrow H(x, z))}_{\text{transitive}} \right) \Rightarrow H(A, B) \right) \quad (6.14)$$

This formula states that each binary relation H that extends G and that is transitive, contains (A, B) .

Exercise 6.2.3 Explain why this is correct. Hint: what is the least transitive relation extending G ?

Second, in FO(ID), “ A has a path to B in G ” can be expressed if an auxiliary defined concept is used. The FO($\cdot$) theory below, T_{Path} , consists of two parts: first, the inductive definition of an auxiliary predicate *Path*/2 as the transitive closure of *G*/2; second, the axiom *Path*(A, B) which states that (A, B) belongs to the transitive closure of G . Both parts are necessary. The FO($\cdot$) theory is the following:

$$\left\{ \begin{array}{l} \forall x \forall y (Path(x, y) \leftarrow G(x, y)) \\ \forall x \forall y (Path(x, y) \leftarrow \exists z (Path(x, z) \wedge Path(z, y))) \end{array} \right\} \\ Path(A, B).$$

Any structure $\mathfrak{A}$ interpreting Σ has a unique expansion that satisfies the definition of *Path*, namely $\mathfrak{A}[Path : R]$ where R is the transitive closure of $G^{\mathfrak{A}}$. The structure $\mathfrak{A}$ belongs to $\mathcal{C}_{\text{Path}}$ iff there is a path from $A^{\mathfrak{A}}$ to $B^{\mathfrak{A}}$ in $G^{\mathfrak{A}}$ iff $(A^{\mathfrak{A}}, B^{\mathfrak{A}}) \in R$ iff $\mathfrak{A}[Path : R] \models Path(A, B)$. Hence, the expansion of $\mathfrak{A}$ to $\mathfrak{A}[Path : R]$ is a one-to-one correspondence between $\mathcal{C}_{\text{Path}}$ and the class of models of T_{Path} . This example illustrates the use of defined auxiliary symbols in FO($\cdot$) to express a proposition.

Inexpressivity result 3: DCA is inexpressible in FO Recall from Chapter 2 that for a set τ of object symbols and function symbols, the set **Terms**(τ) is (inductively defined as) the set of all terms over τ . Recall also that the Domain Closure Axiom for τ is the informal proposition that every object in the universe is represented by a term over τ . The Domain Closure Axiom for τ is denoted $\text{DCA}(\tau)$. It is satisfied in a structure $\mathfrak{A}$ iff

$$D_{\mathfrak{A}} = \{t^{\mathfrak{A}} \mid t \in \text{Terms}(\tau)\}. \quad (6.15)$$

We define the following class of structures:

$$\mathcal{C}_{\text{DCA}(\tau)} = \{\mathfrak{A} \mid D_{\mathfrak{A}} = \{t^{\mathfrak{A}} \mid t \in \text{Terms}(\tau)\}\}. \quad (6.16)$$

E.g., in case of a database, τ is the set of all object symbols appearing in the tables (there are no function symbols in a database), and $\mathbf{Terms}(\tau)$ is identical to the set of symbols in τ . In the database domain, all objects in the application domain are named by some object symbol, so the Domain Closure Axiom for τ holds in this application.

E.g., also logic programs operate on a closed domain satisfying $DCA(\tau)$. E.g., in a logic program operating on natural numbers and lists of natural numbers, τ consists of the empty list symbol $[]$, the list constructor $list/2$ and one symbol for every natural number. The intended domain correspond to $\mathbf{Terms}(\tau)$ which is infinite and contains such terms as $0, 1, \dots, [], list(1, []), list(2, list(5, [])), \dots$.

But there are many other applications where this axiom cannot be imposed, e.g., any application where exact knowledge of the universe is missing.

Anyway, this axiom has many applications and is obviously an important one. Can we express it in FO? Notice that τ contains no predicate symbols, so the only predicate symbol that can be used to express it is the logical equality symbol $=$.

Theorem 6.2.5

If τ contains at least one object symbol and one function symbol then $\mathcal{C}_{DCA(\tau)}$ is not expressible in FO.

This time, the trick of the proof is to define T_∞ as a theory about a new object symbol $a \notin \tau$, stating that a is different from any term over τ . This theory is obviously inconsistent with $DCA(\tau)$.

Proof. We need to find $\mathcal{C}$ and T_∞ for applying the inexpressivity theorem.

We take T_∞ to be $\{\neg(a = t) \mid t \in \mathbf{Terms}(\tau)\}$. This theory is infinite since $\mathbf{Terms}(\tau)$ is infinite; this follows from the fact that τ contains at least one object symbol and one function symbol with arity > 0 .

Let Σ be $\tau \cup \{a\}$. For $\mathcal{C}$, we take the class of structures interpreting at least Σ and satisfying $DCA(\tau)$. This is the subclass of $\mathcal{C}_{DCA(\tau)}$ limited to structures that also interpret symbol a . We prove that $\mathcal{C}$ and T_∞ satisfy the conditions of the inexpressivity theorem.

(1) Every $\mathfrak{A} \in \mathcal{C}$ satisfies $DCA(\tau)$, hence for the domain element $a^{\mathfrak{A}}$, there exists $t \in \mathbf{Terms}(\tau)$ such that $a^{\mathfrak{A}} = t^{\mathfrak{A}}$. It follows that $\mathfrak{A}$ does not satisfy the formula $\neg(a = t)$ which belongs to T_∞ . Hence, $\mathcal{C}$ is inconsistent with T_∞ .

(2) To prove is that each finite subset Ω' of T_∞ has a model $\mathfrak{A}$ that belongs to $\mathcal{C}$. We construct $\mathfrak{A}$ as follows.

Take the unique τ -Herbrand structure $\mathfrak{A}'$. Its universe $D_{\mathfrak{A}'}$ is $\mathbf{Terms}(\tau)$. $\mathfrak{A}'$ satisfies $DCA(\tau)$ since for every element t in its domain, there exists a term, namely t itself, such that $t = t^{\mathfrak{A}'}$. Next, we will expand $\mathfrak{A}'$ with a well-chosen value for a so that Ω' is satisfied. Since Ω' is a finite subset of the infinite theory T_∞ , there must exist some $t \in \mathbf{Terms}(\tau)$ such that $\neg(a = t) \notin \Omega'$. We define $\mathfrak{A} = \mathfrak{A}'[a : t]$. The expanded structure still satisfies $DCA(\tau)$ and it also interprets a , so $\mathfrak{A}$ belongs to $\mathcal{C}$. For every axiom $\neg(a = t') \in \Omega'$, the value of a in $\mathfrak{A}$ is t , and the value of t' in $\mathfrak{A}$ is t' itself. Both values differ, hence $\neg(a = t')$ is satisfied in $\mathfrak{A}$. It follows that $\mathfrak{A}$ satisfies Ω' . Hence $\mathfrak{A}$ belongs to $\mathcal{C}$ and is a model of Ω' . This shows that $\mathcal{C}$ is consistent with Ω' . Since Ω' was chosen as an arbitrary finite subset of T_∞ , $\mathcal{C}$ is consistent with every finite subset of T_∞ .

We now apply the inexpressivity theorem, and obtain that $\mathcal{C}$ is not expressible in FO.

Now suppose $\mathcal{C}_{\text{DCA}(\tau)}$ would be expressible in FO, say by a theory T over τ , then $\mathcal{C}$, the restriction of $\mathcal{C}_{\text{DCA}(\tau)}$ to structures interpreting also a , would be expressed by the theory $T \cup \{a = a\}$. Contradiction. ■

Exercise 6.2.4 *The proof is not correct if τ does not contain at least one object symbol and one function symbol of arity > 0 . Where does it go wrong?*

Let us consider a few special cases for $\text{DCA}(\tau)$.

If τ contains no object symbols, the induction process for terms cannot build any term since there is no base case; hence, $\text{Terms}(\tau) = \emptyset$. Since the universe of a structure of FO is never empty by definition, $\text{DCA}(\tau)$ is unsatisfiable and $\mathcal{C}_{\text{DCA}(\tau)}$ is empty. This class is expressed by any contradictory formula over τ , e.g., $\exists x \neg(x = x)$ or $\forall x \neg(x = x)$.

If τ is a finite set of object symbols $\{C_1, \dots, C_n\}$ then $\tau = \text{Terms}(\tau)$ and $\mathcal{C}_{\text{DCA}(\tau)}$ is expressed by the formula given in Definition ??:

$$\forall x(x = C_1 \vee \dots \vee x = C_n)$$

If τ contains at least one object symbol C and one function symbol f/n of arity $n \geq 1$, then $\text{Terms}(\tau)$ is infinite. Recall from Section 2.6.1 how to express $\text{DCA}(\tau)$ as the SO axiom ?? or as a FO(ID) theory 2.15. Have another look at it. The simplest case arises in Peano's theory of the natural numbers (Section 2.6.3), where $\tau = \{0/0:, S/1:\}$. In this case, $\text{Terms}(\tau)$ is $\{0, S(0), S(S(0)), \dots\}$. If we interpret $S^n(0)$ as a representation of n , the domain closure axiom expresses the following idea:

“the universe consists of 0, 1, 2, ..., n, n+1, ..., and nothing else.”

This is the *SO induction axiom* which is part of Peano's theory. This is not a difficult proposition to understand. It is surprising and disappointing that an “expressive” language such as FO cannot express it. Actually, FO is not so expressive at all.

6.2.3 Summary of inexpressivity results

We saw three implications of the compactness theorem:

- We cannot express *finiteness* of the universe.
- We cannot express *reachability*.
- We cannot express *domain closure*.

These propositions are important in applications. This motivates to extend FO with additional language constructs. We have seen that in SO, and in FO(ID), we can express some of these propositions. ⁵

In the following example, the above results are used for some very simple proofs of other inexpressivity theorems.

⁵In the same effort, we also obtained proofs that the following propositions are infinitely but not finitely expressible in FO:

- the universe is infinite,
- there is no path from A to B in graph G,
- a is different from each term in $\text{Terms}(\tau)$.

Example 6.2.4

- “**Binary relation Tr is the transitive closure of G**” is not expressible in FO.

Proof. If we had a theory T that expressed that $\text{Tr}/2$ is the transitive closure of $G/2$, then reachability from A to B would be expressible by adding $\text{Tr}(A,B)$ to that theory. Since this is inexpressible, so is the transitive closure. ■

- A graph is connected if for every pair of nodes x, y , there is a path from x to y . The informal proposition “**G is a connected graph**” is not expressible in FO.

Proof. The class $\mathcal{C}$ of structures interpreting G as a connected graph is inconsistent with the theory T_{NoPath} that states that A and B are not connected. On the other hand, $\mathcal{C}$ is consistent with each finite subset of T_{NoPath} . It follows that $\mathcal{C}$ is not expressible in FO. ■

Exercise 6.2.5 Express the proposition “**The graph G is acyclic**” in FO and show that the proposition “**The graph G is cyclic**” is not expressible in FO. Express it in SO or FO($\cdot$).

Exercise 6.2.6 Express the proposition “**The graph G is connected**” in SO or FO($\cdot$).

The compactness theorem was historically the first technique to prove that a property cannot be expressed in FO. More advanced techniques exist. E.g., Ehrenfeucht-Fraïssé games. They are outside the scope of the course.

6.2.4 Facultative material: about non-standard models of PA^{FO}

Recall Peano’s vocabulary $\Sigma_{\mathbb{N}} = \{0, S/1:, +/2:, \times/2:\}$. The standard structure of natural numbers over this vocabulary is denoted by $\mathbb{N}$.

Theorem 6.2.6: non-standard models are inevitable

Every FO theory T over $\Sigma_{\mathbb{N}}$ that is satisfied in $\mathbb{N}$ has $\Sigma_{\mathbb{N}}$ -models that are not isomorphic with $\mathbb{N}$.

Stated differently, the class $\mathcal{C}_{\mathbb{N}}$ of $\Sigma_{\mathbb{N}}$ -structures isomorphic to $\mathbb{N}$ is not expressible in FO. This theorem follows directly from the Upward Löwenheim-Skolem theorem (look it up), the proof of which is below.

Proof. It suffices to show that T has a non-countable model. Indeed, a non-countable model is not isomorphic to the countable structure $\mathbb{N}$.

Let A be a non-countable set of new object symbols, and T_A the theory $\{\neg(c = d) \mid c, d \in A\}$ expressing that all these object symbols are different. Any model of T_A is clearly non-countable.

Take any finite subtheory $\Omega \subset T \cup T_A$. It contains only a finite number of disequalities of T_A . It is easy to see that we can extend the structure $\mathbb{N}$ with values for all object symbols in A so that the disequalities in Ω are satisfied. The resulting structure is clearly a model of Ω , hence Ω is satisfiable. Since this holds for any finite $\Omega \subset T \cup T_A$, the compactness theorem entails that $T \cup T_A$ is satisfiable, hence T has non-countable models. ■

Models of a theory T of the natural numbers that are not isomorphic to $\mathbb{N}$ are called *non-standard models*.

Corollary 6.2.2

The FO theory PA^{FO} (Peano arithmetic with the induction schema) has non-countable models that are not isomorphic to $\mathbb{N}$.

In contrast, all models of the SO theory PA^{SO} (the six Peano axioms plus the SO induction axiom) are isomorphic to $\mathbb{N}$. This theorem was mentioned in Chapter 2.

The strongest FO theory of $\mathbb{N}$ is the (infinite) FO theory $\mathbf{theory}(\mathbb{N}) = \{\varphi \mid \mathbb{N} \models \varphi\}$ consisting of *all* formulas that are true in $\mathbb{N}$. But even this theory has non-standard models.

Corollary 6.2.3

$\mathbf{theory}(\mathbb{N})$ has non-countable models that are not isomorphic to $\mathbb{N}$.

When it comes to modelling the natural numbers, it is impossible in FO **to get the models right!**

6.3 Undecidability and Gödel's incompleteness theorem

Recall from Definition 1.3.1 the notions of decidability and recognisability (also known as semi-decidability) of computational problems $\mathcal{M}$.

6.3.1 Undecidability of deduction

Recall that deductive inference is the decision problem $\mathcal{M}_{ded}$ that takes as input a finite theory T and formula φ and returns **t** if $T \models \varphi$, and **f** otherwise. Tautology (also known as validity) checking is the decision problem that takes as input a formula φ and returns **t** if $\models \varphi$ (φ is true in all structures), and **f** otherwise. Figure 6.1 shows that they can be reduced to each other, and hence fall into the same computability class.

The question rises: are these problems recognisable? Are they decidable?

Theorem 6.3.1: Undecidability of FO

The tautology (logical validity) inference problem for FO is undecidable.

This theorem was proved by Church in 1936 and independently by Turing in 1937. It is widely known as the *undecidability of FO*. Do note that this is about a specific sort of inference problem. There are other computational problems in FO (though not deduction, nor unsatisfiability checking) that are decidable, or even polynomial.

There is also good news:

Theorem 6.3.2: Recognisability of FO

The tautology (logical validity) inference problem for FO is recognisable.

I.e., there is an algorithm that for input φ answers **t** if $\models \varphi$ and otherwise returns **f** or does not terminate. The same holds for deduction and unsatisfiability checking.

This result is known as the **semi-decidability** of deductive reasoning for FO.

Proof. The basic idea is simple. A sketch is as follows. According to Gödel's completeness theorem, there exists a sound and complete proof system for FO. Proofs in this proof system can be encoded as finite strings of some vocabulary (e.g., $\Sigma = \{0, 1\}$ will suffice) such that there is a terminating algorithm A to decide for any input string in Σ^* whether it is a proof of that proof system or not.

Now consider the following algorithm: generate exhaustively all strings over Σ ; for each generated string w , run A to check if w is a proof and if so, check if the last proposition in w is equal to the input formula. If so, stop and return **t**. Otherwise, go on.

If the input formula is valid, a proof exists and its encoding will eventually be generated. So the algorithm will terminate with **t**. Otherwise, the algorithm will not terminate. ■

In what follows, it's mainly important that deductive inference in FO is undecidable but recognisable. There is one more interesting result, though.

Corollary 6.3.1: Unrecognisability of satisfiability checking

The satisfiability checking inference problem for FO is unrecognisable.

Proof. Checking whether a theory or formula is unsatisfiable is equivalent to deduction, which is undecidable but recognisable. If satisfiability checking was also recognisable, both would become decidable, which contradicts what we just said. ■

This means that any algorithm made to check whether a theory or formula is satisfiable can loop forever on some theories and formulas that are, in fact, satisfiable.

6.3.2 Gödel's Incompleteness Theorem

Given a sound and complete proof system $\mathcal{P}$, can we build an FO theory T from which every FO formula φ true in $\mathbb{N}$ can be proven? I.e., such that the following holds:

$$\mathbb{N} \models \varphi \Rightarrow T \vdash_{\mathcal{P}} \varphi. \quad (6.17)$$

Well, sure! Take $\text{theory}(\mathbb{N}) = \{\varphi \mid \mathbb{N} \models \varphi\}$. It contains every sentence φ true in $\mathbb{N}$, and every such sentence has a one-step proof from this theory (using the theory introduction step in Definition 6.1.1).

However, $\text{theory}(\mathbb{N})$ is infinite. Can we find a finite theory? Or more generally, can we find a potentially infinite theory that can be represented in a finite way? E.g., Peano's FO arithmetic (PA^{FO}), the FO theory with the induction schema, is infinite, but it is finitely representable by six axioms and one axiom schema.

To answer that question, we first need to carefully define when a theory T is *finitely representable*. The natural solution of computer science is the following: a theory T is finitely representable if T is (*recursively*) *enumerable*, that is, there exists a program P that can generate T , possibly in infinite time (cfr. Definition 1.3.1). In this case, the program P can be seen as the finite representation of T . This is more general than axiom schema's. Indeed, it is clear that for each schema, there exists an algorithm that can generate all instances of that schema. As such, any theory represented as a finite set of axioms and axiom schemas is recursively enumerable. E.g., Peano arithmetic PA^{FO} is recursively enumerable.

The question can now be rephrased as follows: can we build a recursively enumerable FO theory that is satisfied in $\mathbb{N}$ and from which every FO sentence true in $\mathbb{N}$ can be proved? E.g., can every FO proposition true in $\mathbb{N}$ be proven from PA^{FO} ? That sort of questions was solved in general by Gödel. Below, assume that we have chosen a sound and complete proof system of FO inducing a provability relation $\vdash$.

Theorem 6.3.3: Gödel's incompleteness theorem (GIT), 1931

If a theory T

1. is FO, and
2. is recursively enumerable, and
3. is satisfied in $\mathbb{N}$ ($\mathbb{N} \models T$),

then there exists an FO formula α such that α is satisfied in $\mathbb{N}$ ($\mathbb{N} \models \alpha$), yet α is not provable by T ($T \not\vdash \alpha$).

Stated alternatively: a recursively enumerable FO theory that satisfies $\mathbb{N}$ and from which every FO sentence true in $\mathbb{N}$ can be proven, does not exist.

Note that due to soundness and completeness of the proof system (thanks to Gödel's completeness theorem), provability and entailment are interchangeable. Hence, for a theory T satisfying the conditions of Gödel's incompleteness theorem, there exists a formula α true in $\mathbb{N}$ but not entailed by T ($T \not\models \alpha$). Thus, T has a model in which α is false.

The condition that T is recursively enumerable ensures the effective finite representability of the theory. Peano FO arithmetic with induction schema (PA^{FO}) is recursively enumerable and is satisfied in the natural numbers. Hence, the theorem entails that there exist formulas true in $\mathbb{N}$ but not provable from this theory.

What about $\mathbf{theory}(\mathbb{N})$? While being the strongest FO theory of $\mathbb{N}$, it nevertheless has non-standard models (Theorem 6.2.6). Still, $\mathbf{theory}(\mathbb{N})$ contains every FO formula true in $\mathbb{N}$ and hence, entails every such a formula. What does Gödel's theorem say about $\mathbf{theory}(\mathbb{N})$? This question is considered in the next section.

6.3.3 Undecidability of the natural numbers

Let $\mathbb{N}$ be the structure of the natural numbers interpreting at least $\Sigma_{\mathbb{N}} = \{0, S/1 : +/2 : \times/2 : \}$. Many important questions about the natural numbers are instances of the following inference problem.

Model checking in $\mathbb{N}$

Input: a sentence φ over $\Sigma_{\mathbb{N}}$

Output: $\varphi^{\mathbb{N}}$; in words, **t** if $\mathbb{N} \models \varphi$; otherwise **f**.

This is called the *decision problem of the natural numbers*. Is this computational problem decidable? recognisable? co-recognisable? These questions can be rephrased in terms of a familiar set:

$$\mathbf{theory}(\mathbb{N}) = \{\varphi \mid \varphi \text{ is a FO formula interpreted in } \mathbb{N} \text{ and } \mathbb{N} \models \varphi\}$$

The questions are whether membership of $\mathbf{theory}(\mathbb{N})$ is decidable? recognisable? co-recognisable?

Proposition 6.3.1

Model checking in $\mathbb{N}$ is recognisable iff it is co-recognisable.

Proof. This follows from the fact that $\mathbb{N} \not\models \varphi$ iff $\mathbb{N} \models \neg\varphi$. Given an algorithm A that recognises $\mathbb{N} \models \varphi$, one can co-recognise $\mathbb{N} \models \varphi$ by calling A on $\neg\varphi$. Vice versa, given an algorithm B that co-recognises $\mathbb{N} \models \varphi$, one can recognise $\mathbb{N} \models \varphi$ by calling B on $\neg\varphi$. Therefore, $\mathbf{theory}(\mathbb{N})$ is recognisable iff co-recognisable. ■

Corollary 6.3.2: of GIT: Undecidability of the natural numbers

The decision problem of the natural numbers is not decidable, not recognisable and not co-recognisable.

Proof. The theory $\mathbf{theory}(\mathbb{N})$ entails all FO formulas true in $\mathbb{N}$ and hence, it contradicts the conclusion of Gödel's incompleteness theorem (GIT). Therefore, it must violate the premise of Theorem 6.3.3. There are three conditions that can be violated, but since $\mathbf{theory}(\mathbb{N})$ is a FO theory and is satisfied in $\mathbb{N}$, the only possibility is that it is not recursively enumerable. Recursive-enumerability is equivalent to recognisability as explained in Proposition 1.3.1, hence $\mathbf{theory}(\mathbb{N})$ is not recognisable. It is also not co-recognisable, as seen in Proposition 6.3.1. ■

So, not only is the problem of deciding whether a formula φ is true in $\mathbb{N}$ not decidable, it is not recognisable and not co-recognisable. This property is known as the *undecidability of the natural numbers*.

It is a direct consequence of Gödel's incompleteness theorem. However, it was not Gödel who proved it. When he proved his theorem in 1931, computability theory did not exist yet, and the concepts of algorithm and decidability were not yet formalized. In fact, his work was an important spark for the development of such a theory. Five years later, Gödel, Church and Turing each had developed a theory of computability.

6.3.4 Incompleteness of SO and FO(ID)

We have seen that SO and FO(ID) both admit a finite categorical theory of $\mathbb{N}$ – i.e. a theory of which all models are isomorphic to $\mathbb{N}$. For SO, the theory was obtained by replacing the infinite FO induction schema by the SO induction axiom:

$$\underbrace{\forall N}_{\text{Every set } \dots} \left(\underbrace{(N(0) \wedge \forall x (N(x) \Rightarrow N(S(x))))}_{\text{... containing 0 and closed under the successor operation]...}} \Rightarrow \underbrace{\forall x N(x)}_{\text{... contains the whole domain.}} \right) \quad (6.18)$$

It expresses DCA for $\Sigma = \{0/0:, S/1:\}$. The theory PA^{SO} that contains this axiom is a categorical theory of $\mathbb{N}$.

Exercise 6.3.1 Recall how to express $DCA(\tau)$ in $FO(\cdot)$.

The question now is what the GIT tells us about these logics.

Corollary 6.3.3

Any extension $\mathcal{L}$ of FO with a finite (or recursively enumerable) theory $T_{\mathbb{N}}$ of $\mathbb{N}$ that is categorical (i.e., all models of $T_{\mathbb{N}}$ are isomorphic to $\mathbb{N}$), does not have a sound and complete proof system.

Proof. Assume towards contradiction that $\mathcal{L}$ has a sound and complete proof system. Since $\mathcal{L}$ is an extension of FO, it can also prove FO formulas. Proofs are finite strings and can be exhaustively enumerated. Consider the algorithm that exhaustively enumerates proofs from $T_{\mathbb{N}}$ and, for each generated proof outputs the proven formula φ if φ is FO. This is obviously an enumerator of $\mathbf{theory}(\mathbb{N})$, which does not exist since it is not recursively enumerable. Contradiction. ■

In any logic $\mathcal{L}$ with a sound and complete proof system, for any categorical recursively enumerable theory T describing a structure $\mathfrak{A}$, there is a decision procedure to decide all properties of $\mathfrak{A}$ expressible in $\mathcal{L}$: for any input φ , exhaustively enumerate all proofs from T and stop as soon as φ or $\neg\varphi$ is proven. A natural consequence is that in such a logic, only structures with decidable properties expressible in $\mathcal{L}$ can be described categorically. Vice versa, any logic $\mathcal{L}$ with sufficient expressivity to categorically describe a structure $\mathfrak{A}$ of which properties expressible in $\mathcal{L}$ are undecidable, cannot have a sound and complete proof procedure.

In other words, any logic in which we can accurately describe the structure of the natural numbers, cannot have a complete proof system.

In conclusion: the two logics we have seen so far to express the structure of the natural numbers in (SO and FO($\cdot$)) have no sound and complete proof system.

Remark 6.3.1: Historical note: Gödel's proof

Gödel did not present his theorem in the above form. E.g., he did not use the concept of “recursive enumerable” which did not exist yet in 1931. In fact, he proved his theorem only for a specific FO theory, namely Russell and Whitehead's Principia Mathematica. However, in the introduction he had explained that his proof could be generalized for any theory that could somehow be “generated”.

His proof was brilliant (and extremely tedious). He showed how to encode formulas φ by numbers called the Gödel number of φ :

$$\varphi \longrightarrow n_{\varphi}$$

He constructed a formula $\varphi_T[n]$ that is true for a number n iff n is the Gödel number of a provable sentence.

He then showed the existence of a self-referential FO sentence, called the *Gödel sentence*, which intuitively means:

$$\text{“}I \text{ cannot be proven.} \text{”} \tag{6.19}$$

More precisely, he proved that there is a number m that encodes a sentence that is logically equivalent to the sentence $\neg\varphi_T[m]$. (i.e. $n_{\neg\varphi_T[m]} = m$).

This sentence, like any other, is either true or false in $\mathbb{N}$. If it would be false, then its negation $\varphi_T[m]$ would be true. By construction of φ_T , the formula encoded by m (i.e., $\neg\varphi_T[m]$) is provable from T and hence, by the soundness of $\vdash$, true in $\mathbb{N}$. This is a contradiction. Hence, $\neg\varphi_T[m]$ must be true.

Since this sentence is true, and since it says that it cannot be proven, it cannot be proven. Hence, this is a true but unprovable sentence.

Largely inspired by his work and by his suggestion of theories that could be “generated”, Church (1936) and Turing (1937) developed formal definitions of computability, decidability, recursive enumerability and were able to generalize Gödel's theorem to all recursively enumerable theories of $\mathbb{N}$ and state and prove the undecidability of validity and of the natural numbers.

Remark 6.3.2: Impact of Gödel's incompleteness theorem (GIT)

Gödel's incompleteness theorem is perhaps the most famous theorem of mathematical logic.

In mathematics, the meaning of Gödel's incompleteness theorem has been immense. Before Gödel, mathematicians hoped that they could axiomatize every useful structure (such as the natural numbers) and could prove the consistency of the theory and all its theorems. This was called “*Hilbert's program*”. A crucial step in this program was the *Entscheidungsproblem*, the problem of finding an algorithm for deciding validity or satisfiability.

The question whether deciding validity in first-order logic is possible – the classical *Entscheidungsproblem* – was an important open problem in mathematical logic:

Das Entscheidungsproblem ist gelöst, wenn man ein Verfahren kennt, das bei einem vorgelegten logischen Ausdruck durch endlich viele Operationen die Entscheidung über die Allgemeingültigkeit bzw. Erfüllbarkeit erlaubt. (...) Das Entscheidungsproblem muss als das Hauptproblem der mathematischen Logik betrachtet werden. [16]

The “decision problem” is solved, if one knows an algorithm which, given a logical expression, decides by finitely many operations, if the expression is satisfiable or valid, resp. (...) The “decision problem” must be seen as the most important problem in mathematical logic.

Gödel proved that this problem was unsolvable and hence, that Hilbert's program could not be realized.

In philosophical AI, Gödel's incompleteness theorem has given rise to speculation regarding the adequacy of *logic* and the standard *Von Neumann computer architecture* for building AI-systems.

Some have argued along the following lines: any intelligent computer system based on the Von Neumann architecture will be based on a logic proof system. There will be sentences that can be understood to be true by Humans, but not by the computer system, in particular, the *Gödel sentence* of its proof system. Hence, we are smarter than computers. However, this argument is considered erroneous by experts (arguments focus on the way Penrose interprets Gödel's theorem and are “mercilessly technical”).

A strong advocate of this position was Roger Penrose, winner of the Nobel prize Physics, in his book “*The Emperor's New Mind*” (1990). To explain the superiority of the human brain, he (and others) suggested that there are macroscopic quantum phenomena connected to the human brain's neural activity.

6.4 Implications for “deductive logic”

In the view of FO as the logic of deductive reasoning, the results of this section lead to the following uneasy situation. On the one hand, FO lacks expressivity to express certain critical kinds of human domain knowledge. If we decrease the logic in expressivity, the problem will get worse. On the other hand, even now deductive reasoning in FO is not decidable, let alone tractable/efficient. If we extend FO to make it more expressive, this “problem” will only get worse.

This is an instance of the *expressivity/efficiency trade-off*. This trade-off had a strong influence on AI and computational logic. In AI and computational logic, many concluded that FO’s undecidability makes FO unpractical for building effective software solutions for computational problems. Different research directions sprang out from this.

Before we discuss them, let us reflect about the nature of the trade-off. The trade-off does not say that problems in more expressive languages will be solved less efficiently but rather that the class of problems that can be stated in more expressive languages is larger; hence the worst case gets more difficult. Hence, while the increased complexity sounds as a disadvantage for more expressive languages, it is in fact an advantage.

On the other hand, common sense suggests that human software engineers can exploit the expressivity limitations of a less expressive logic $\mathcal{L}_1$ to build more efficient solvers than those for a more expressive logic $\mathcal{L}_2$. Often this is indeed the case. And sometimes the inverse is the case. Sometimes, the particularities of $\mathcal{L}_1$ make it harder to develop good algorithms. Sometimes, the collected efforts of computer scientists developing algorithms for the more general and hence, more interesting inference problem for $\mathcal{L}_2$ lead to systems better than those of $\mathcal{L}_1$. Sometimes these systems, for instances of the inference problem, are better than what can be achieved with “reasonable” effort by hand-made programming. Domains where this arises are, e.g., databases, constraint programming, and semantic web languages.

Now, let us return to the research directions that sprung from the expressivity/efficiency trade-off. One direction was to continue building theorem provers for FO and hope that in practice it will not be so bad. This is what happens in the field of automated theorem proving (ATP) discussed in Remark 6.1.1. This is less unreasonable as it may seem and the ATP community has developed strong theorem provers that at least in certain domains have good use. Thus, ATP is a small but highly specialized community that goes on developing theorem provers for FO, and these solvers are increasingly effective.

Another direction was to restrict FO to a subformalism for which deductive reasoning is decidable or even tractable. This road was taken for instance in the area of *description logics*. Description logics form the basis for the *web ontology languages* like OWL and RDF which are used in Semantic Web applications. The original description logics were small fragments of FO for which deductive reasoning was tractable. In the past two decennia, solvers were developed for gradually more expressive description logics. They are supported by systems such as RACER which, perhaps surprisingly, show tractable behaviour for the early (tractable) description logics and often outperform solvers specifically developed for these low end logics.

A different approach in a similar vein was to restrict FO to a subformalism and develop a proof method such that the human programmer had control over the execution of the proof method. This was implemented in *logic programming (Prolog)*. The Prolog language originated by limiting the expressivity of FO to Horn clause logic. Deductive inference in Horn clause logic was still undecidable, but Prolog’s procedural semantics gave human users a way to understand the behaviour of programs, and hence to predict and control it. Of

course, in both cases, by restricting expressivity, we increase the expressivity problems.

Yet another direction was to develop other, cheaper but practically useful forms of inference. In this setting, it is possible to increase the expressivity of FO. This can be seen to be the case in e.g. databases and constraint solving. This is the topic of chapter 7.

6.5 Important for exam

Big questions:

- compactness theorem and proof
- inexpressivity theorem and proof
- inexpressivity of finite universe, reachability, or of DCA: theorem and proof
- or a combination (e.g., inexpressivity theory plus application for reachability)

In the case of the last sort of question, you should be able to show the axiomatization of the negated concept.

Bonus points could be earned if you can solve a variant problem.

For small questions:

- explain undecidability and semi-decidability of deduction and validity inference
- explain Gödel's incompleteness theorem and its conditions
- explain why it entails undecidability of natural numbers
- explain why H_0 and $FO(.)$ have no sound and complete proof system
- evaluation of FO from the information centered view (separating information from inference)

What I think you need to know of this section to be able to answer these questions:

Section "Deductive inference":

- Understanding of what deduction is, what proof systems and formal proofs are, soundness and completeness.
- Knowledge of Compactness theorem; proof.

Section "Some expressivity limitations of FO"

-Knowledge of:

- + Inexpressivity theorem; proof
- + Inexpressivity of finiteness, reachability and DCA ; proofs

(additional knowledge may lead to bonus points)

Section Undecidability and Gödel's incompleteness theorem

- Understanding of concepts of decidable, recursively enumerable, semi-decidable.
- Knowledge of the main theorems:
 - + Undecidability and semi-decidability of Validity/satisfiability/deduction for FO.
 - + Gödel's incompleteness theorem
 - + its two main corollaries:
 - undecidability of the natural numbers
 - lack of sound and complete proof systems for $FO(.)$ and SO

Chapter 7

Inference and the FO($\cdot$)-KB project

Assume that we built a logic theory of the application domain that contains essential information relevant to certain computational tasks. How can this theory be used to solve the computational tasks that arise in this application domain? This is done by applying one or more suitable forms of inference on the theory.

This is the topic of this chapter. As such this chapter is about the relation between knowledge of an application domain and computational tasks that arise in that domain. The chapter discusses the FO($\cdot$)-KB project, the research project of the *Knowledge Representation and Reasoning (KRR)* group at KU Leuven. We will see the link with various declarative programming paradigms. The chapter is derived from an invited talk at JELIA, the *European Conference on Logics in Artificial Intelligence* in 2016.

7.1 Motivation

What is the most important human resource after air, water, fire, food and shelter? Probably knowledge. Humans experts possess large amounts of (declarative) knowledge. They use it to accomplish tasks and to solve problems. How does this work? This is the basic research question of the field of KRR. If we ever want to be able to build software systems in a principled way, we will need to understand this. This places KRR at the foundations of computer science.

About every area in computational logic is involved in aspects of this question. However, scientific understanding is partial and scattered over the many fields of computational logic and declarative problem solving. One issue that fragments computational logic more than anything else is the reasoning/inference task. In the current state of the art, most logics in the field of computational logic are entangled with a specific form of inference:

Classical first-order logic (FO)	<i>deduction</i>
Deductive databases (SQL, Datalog)	<i>query answering</i>
Logic programming (Prolog)	<i>program execution</i>
Answer set programming (ASP)	<i>answer set computation</i>
Inductive logic programming	<i>inductive inference</i>
Abductive logic programming	<i>abductive inference</i>
Constraint programming (CP)	<i>constraint solving</i>
Temporal logics	<i>model checking</i>
Description logics	<i>subsumption inference</i>
Planning language (PDDL)	<i>planning inference</i>

Thus, for every form of inference a new logic is developed. From the perspective of logic as the study of inference, this is not unnatural. However, from the perspective of logic as a language to express knowledge, the entanglement of logic and inference is unfortunate. I argue this with two cases.

Example 7.1.1

Consider the following proposition: “*Every lecturer teaches at least one course in the first bachelor*”. In FO, with an appropriate vocabulary, this could be transcribed as:

$$\forall x(Lecturer(x) \Rightarrow \exists c(Course(c, bach1) \wedge teaches(x, c)))$$

What is its purpose? What task is it to be used for? It could be a query to a database, a constraint in a course assignment problem, a desired property to be verified from a formal specification of the course assignment domain, etc ... A declarative proposition has no inherent purpose or task. The proposition could be relevant to many different sorts of tasks. In the current state of the art, a different logic is needed to represent this proposition depending on the task to be solved. This is not right.

Example 7.1.2

For a second case, consider the graph colouring problem. What is the central “knowledge” in the graph colouring problem? That no two adjacent vertices have the same colour. In FO, this can be expressed as:

$$\forall x \forall y (G(x, y) \Rightarrow Col(x) \neq Col(y))$$

This is a natural representation of the central piece of knowledge in this problem. How to solve a specific graph colouring problem in FO using this proposition? This question amounts to what kind of inference is needed to apply to this formula to solve this problem. The fact is that until fairly recently, for FO, this question was not even posed. FO was seen as the logic of **deductive reasoning** and continues to be seen this way by many to this day. Yet, deduction is utterly useless for solving a graph colouring problem. Therefore, FO seems useless for this problem.

Instead, new logics equipped with the suitable form of inference were developed to handle problems like this: in particular, constraint programming languages such as Ilog, Zinc, CLP, ... Yet, in fact, the form of inference to solve the graph colouring problem using the FO specification is *model generation/expansion*.

From a scientific point of view, these situations are deeply unsatisfactory. Is declarative knowledge not supposed to be independent of the task (and hence, of a specific form of inference)? Should it not be possible to solve multiple types of tasks using information expressed in the same language?

The FO(·)-KB project: an outline The FO(·)-KB project is the research project of KU Leuven’s KRR group. In this approach, a declarative logic theory is not a program, it cannot be executed, it is not even a description of a problem. It is a bag of (descriptive) information.

On the logical level, the goal is study “knowledge” by a principled development of expressive

KR languages. The goal is to develop KR logics with a clear informal semantics and a formal model semantics that formalizes this informal semantics. The logic is expressive enough such that domain knowledge relevant to solving the problem *can* be expressed in a compact, modular way. Classical first-order logic (FO) is taken as foundation but extended and improved where necessary.

On the application level, the goal is to develop a typology of tasks and computational problems in terms of (the same) logic and inference. We are eagerly searching for novel ways of using declarative specifications to solve problems. We are also searching for applications where different forms of inference on the knowledge base are required to obtain the desired functionality. Such applications show a form of *reuse* of the knowledge base that is impossible to achieve in systems that support only one inference task.

On the inference level, the goal is to build solvers for various forms of inference for FO($\cdot$). To this aim, we are integrating various solving techniques from various declarative programming paradigms into the knowledge-base system IDP – the laboratory for our scientific research.

7.2 Why FO as a foundation?

FO is the outcome of 18th- and 19th-century research in “laws of thought” by Leibniz, De Morgan, Boole, Frege ... FO offers a small set of connectives:

$$\wedge, \vee, \neg, \forall, \exists, \Leftrightarrow, \Rightarrow. \quad (7.1)$$

These are essential for KR and are correctly formalized in FO. As a consequence, every expressive declarative modelling language should be expected to have a substantial overlap with FO although syntactic sugar, conceptology and terminology may sometimes obscure the relationship, e.g., SQL, constraint languages, Alloy ...

Another crucial aspect of FO for KR is its mathematically precise informal semantics. E.g., $\forall x(Human(x) \Rightarrow Man(x) \vee Woman(x))$ means *All humans are men or women*.

On the other hand, FO is not expressive enough for practical KR. To turn FO into a practical KR language, it needs to be extended. In the project, several extensions were studied and integrated to make FO(Types, ID, Agg, Arit, Causation, ...). We call this the FO($\cdot$) language framework.

The expressivity/efficiency trade-off reconsidered Deductive inference in FO is undecidable but still semi-decidable. In extensions, e.g., with second order quantification or with inductive definitions, the situation is worse. How to deal with this?

It is in the nature of knowledge that the same knowledge can be useful or relevant in a variety of problems. The computational complexity of these problems is varying widely. Some of these problems are “easy”, e.g., checking truth of a proposition in a structure; other computational problems are difficult or even unsolvable in full generality, e.g., proving a proposition from a theory for verification. This is the nature of knowledge. In a knowledge-centered approach such as the FO($\cdot$)-KB project, one has to accept that some sort of problems are untractable or even undecidable, while other problems are solvable. It may well be that in practical applications, the required forms of inference are mostly of the easy kind. In the FO($\cdot$)-KB project, the focus is mostly on problems that can be solved within a finite domain.

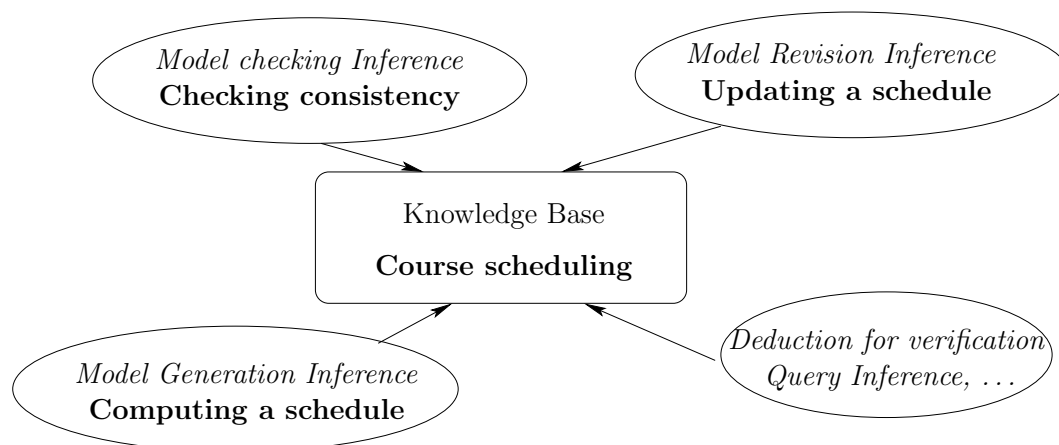


Figure 7.1 – The knowledge base approach to course scheduling

7.3 The knowledge base paradigm

A knowledge base system is a system that manages a knowledge base and supports multiple forms of inference on it. The concept is illustrated in Figure 7.1 in the application domain of university course scheduling.

Knowledge in this domain includes properties such as:

- all lectures take place in a room and at some time point;
- lecturers and students cannot attend different lectures simultaneously;
- rooms cannot host more than one lecture at the same time;
- the capacity of rooms should be sufficient to hold student groups.
- ... In practice, many more constraints arise in such domains.

The knowledge base T_s consists of a theory representing this knowledge.

At several stages during the academic year, different tasks arise that can be solved by applying the appropriate form of inference on the knowledge base. Recall the definition of inference from Definition 1.3.2. It is a computational problem with as input a proposition or theory and its output is invariant under substituting the input formula or theory by a logically equivalent one.

In Definition 1.3.3, we defined different inference problems for arbitrary logic with a satisfaction relation ($\models$): evaluation, model checking, satisfiability checking, model generation, optimization, deduction and possible values problems. Here, we reintroduce these forms of inference and some extra forms of inference in the context of FO(·) and illustrate their application.

Some common tasks that arise in a university scheduling application with theory T_s are the following:

- Verification; does the schedule always follow the property φ ?

Deduction <i>Input:</i> T_s, φ <i>Output:</i> $(T_s \models \varphi)$

- Compute a schedule; a model of T_s comprises a value for predicates *RoomOf*, *TimeOf* representing the schedule.

Model expansion*Input:* $T_s, \mathfrak{A}_i$ with $\mathfrak{A}_i$ a structure representing data.*Output:* a model $\mathfrak{A}$ of T_s expanding $\mathfrak{A}_i$, or UNSAT

Typically such a problem is subject to some optimality criterion. In that case, a variant of this form of inference is needed.

- Check if a manually modified schedule $\mathfrak{A}'$ is still correct.

Model checking*Input:* the modified schedule $\mathfrak{A}'$, T_s *Output:* $(\mathfrak{A}' \models T_s)$

- Compute answer to query φ (boolean) or $\{x \mid \varphi[x]\}$ (set) in schedule $\mathfrak{A}$.

Query answering*Input:* $\mathfrak{A}, \varphi$ or $\{x : \varphi[x]\}$ *Output:* $(\mathfrak{A} \models \varphi)$ or $\{x : \varphi[x]\}^{\mathfrak{A}}$.

- Modify given schedule $\mathfrak{A}$ to satisfy a new proposition φ .

Model revision*Input:* $\mathfrak{A}, \varphi, T_s$ *Output:* structure $\mathfrak{A}'$ close to $\mathfrak{A}$ such that $\mathfrak{A}' \models T_s \wedge \varphi$.

In the following section, we discuss some applications.

7.4 A KB-solution for Interactive Configuration

In many configuration problems, a user is constructing a configuration under complex constraints. E.g., configuring your study program, a schedule for a hospital or university, a computer in an on-line shop, a car, a loan in a bank, a mechanical device, a plan of a building, etc.

Building good configuration software to support the user is difficult. The user chooses, the system supports. During the interaction, a partial configuration is constructed, extended, revised according to the user's wishes. The user is not or only partially aware of the constraints. The system is to support the user in his choices and to take care for maintaining the consistency of the solution.

Industry is aware that good solutions for interactive configuration are extremely difficult to build with standard software technology. Current technologies fall short: spreadsheets, business rule systems, programming languages. Why? The specifications of correct configurations are complex and tend to change rapidly. The range of needed functionalities is high. Snippets of code concerned with some specific constraint spread out at many places in the software, showing a lack of reuse and leading to difficult maintenance.

A solution based on the KB-paradigm The idea is to express the laws of correct configurations in a knowledge base, and to provide different functionalities of the system by various forms of inference. In an interaction between user and system, a partial configuration is built and gradually extended. All knowledge is maintained in one knowledge base; the current partial configuration is modelled as a partial structure. The system performs various forms of inference on the knowledge base and the partial structure to support the user.

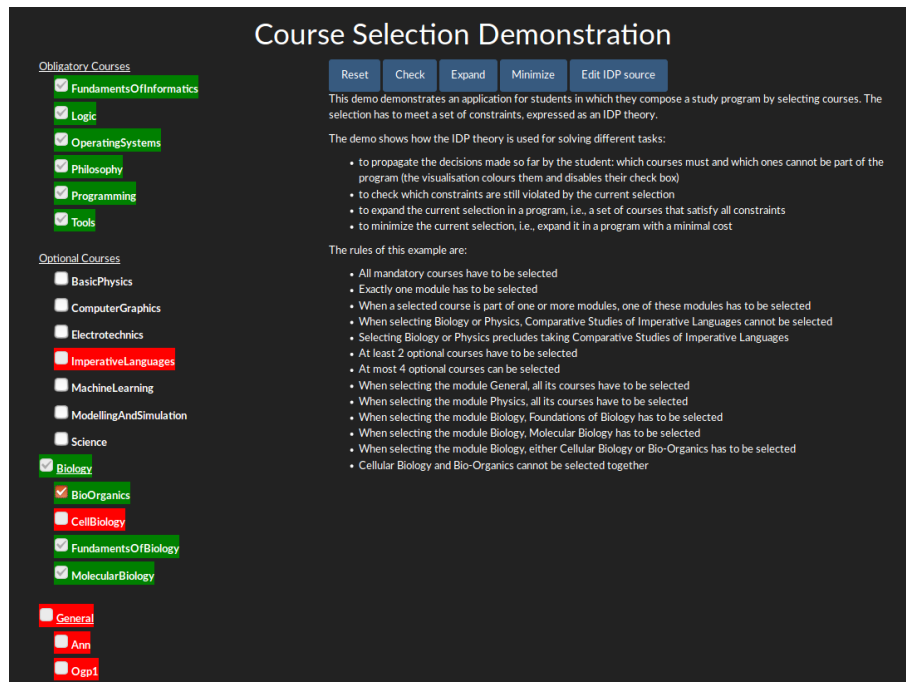


Figure 7.2 – Course selection demo

The idea is illustrated with a demo for the interactive configuration of course selection at a fictitious university. This demo is derived from a challenge problem developed by the VUB (Free University of Brussels) as a test for software companies and methodologies. The demo is implemented with the IDP system. It is available at <https://krr.gitlab.io/demos/courses/>.

The knowledge base T_c contains a number of laws such as:

- compulsory courses have to be selected;
- a number of optional courses have to be selected;
- one out of three possible modules have to be selected;
- each module has a number of module courses which are to be followed exactly if the module is selected.
- etc.

Models of this theory satisfy all laws. The theory can be inspected by clicking the button *Edit IDP source*.

The application is built as a shallow GUI on top of a JSON data structure which communicates information back and forth between the knowledge base system and the user. The GUI is presented in Figure 7.2. The KB system maintains a partial structure $\mathcal{I}$ representing the current choices of the user and the implications that these choices entail. The system supports the users in various ways, using 5 different forms of inference. Below, we describe them.

Definition 7.4.1: Partial structure

A partial structure $\mathcal{I}$ consists of a domain $D_{\mathcal{I}}$, function values for function symbols and *partial sets* for predicate symbols. A partial set is a function from tuples to $\{\mathbf{t}, \mathbf{f}, \mathbf{u}\}$ where $\mathbf{u}$ is called *undefined*.

Definition 7.4.2: Precision

On partial structures, the *precision* partial order relation is defined: $\mathcal{I} \leq_p \mathcal{I}'$ if $\mathcal{I}$ and $\mathcal{I}'$ have the same domain and value of function symbols and, if $P^{\mathcal{I}}(\bar{d}) \neq \mathbf{u}$ then $P^{\mathcal{I}}(\bar{d}) = P^{\mathcal{I}'}(\bar{d})$.

In words, the partial structure $\mathcal{I}$ is less precise than $\mathcal{I}'$ (notation $\mathcal{I} \leq_p \mathcal{I}'$) if the interpretations of predicates in $\mathcal{I}'$ extend those of $\mathcal{I}$. That is, if any atom $P(\bar{d})$ has a defined value in $\mathcal{I}$ (i.e., $P^{\mathcal{I}}(\bar{d}) \neq \mathbf{u}$) then $P(\bar{d})$ has a defined value in $\mathcal{I}'$ as well and $\mathcal{I}$ and $\mathcal{I}'$ agree on its value (i.e., $P^{\mathcal{I}}(\bar{d}) = P^{\mathcal{I}'}(\bar{d})$).

We say that a standard two-valued structure $\mathfrak{A}$ *expands* a partial structure $\mathcal{I}$ if $\mathcal{I} \leq_p \mathfrak{A}$.

Functionalities: inferences on $\mathcal{I}$ and T_c

1. Propagation inference.

Propagation Inference
 Input: $T_c, \mathcal{I}$
 Output: $\mathcal{I}'$ where $\mathcal{I} \leq_p \mathcal{I}'$ and for every model $\mathfrak{A}$ of T_c such that $\mathcal{I} \leq_p \mathfrak{A}$, it holds that $\mathcal{I}' \leq_p \mathfrak{A}$.

In the demo, propagation inference is called initially and whenever the user makes a choice. The effect is that the implications of his choices under T_c are propagated. Forced choices are coloured green, forbidden choices red.

A special form of this inference is *optimal propagation*, where the result $\mathcal{I}'$ is the most precise partial structure such that every model $\mathfrak{A}$ of T_c expanding $\mathcal{I}$ also expands $\mathcal{I}'$.

Stated differently, $\mathcal{I}'$ is obtained from $\mathcal{I}$ as follows:

- an unknown atom A of $\mathcal{I}$ that is true in all models $\mathfrak{A}$ expanding $\mathcal{I}$, is true in $\mathcal{I}'$
- an unknown atom A of $\mathcal{I}$ that is false in all models $\mathfrak{A}$ expanding $\mathcal{I}$, is false in $\mathcal{I}'$.

2. Model checking inference.

This form of inference was defined before. In the demo, it is called when clicking button *Check*. First, the system transforms the current partial structure $\mathcal{I}$ into a total structure $\mathfrak{A}$ by turning every *undefined* atom into *false*. Then *model checking inference* is applied on $\mathfrak{A}$ and T_c to compute whether $\mathfrak{A} \models T_c$.

3. Explanation inference.

At several times, explanations are generated. An explanation is a logical argument why an axiom or group of axioms is violated, why a certain choice is forced or impossible. A simpler form is implemented in this demo: it reports which axioms are violated in the partial structure $\mathcal{I}$ or the structure $\mathfrak{A}$ computed in the model checking step.

4. Model expansion inference.

Model expansion
 Input: $\mathcal{I}, T_c$
 Output: a model $\mathfrak{A}$ of T_c such that $\mathcal{I} \leq_p \mathfrak{A}$.

It serves to complete the current partial structure to a model of the theory. This is useful to fill in the “don’t-cares” of the student. This form of inference is called by pushing the button *Expand*.

5. Optimization inference.

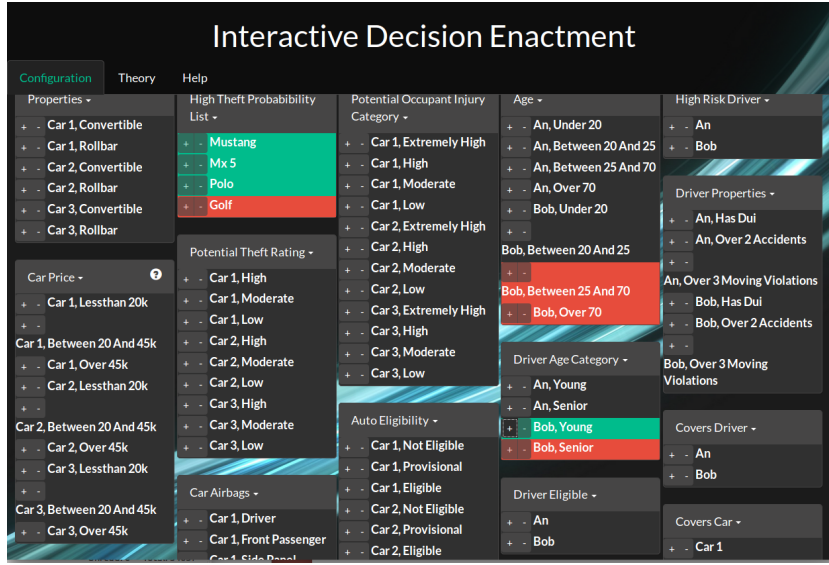


Figure 7.3 – Car Insurance Decision Enactment

Optimization

Input: $\mathcal{I}, T_c, t$ where t is a numerical term

Output: a model $\mathfrak{A}$ of T_c such that $\mathcal{I} \leq_p \mathfrak{A}$ and $t^{\mathfrak{A}}$ is minimal.

This inference is called when clicking button *Minimize*. In this application, the term t expresses the study load of the selected program. Hence, a model with minimal study load is computed. In the theory, the term is declared as the IDP term `optTerm`. It is the following expression counting the total value of selected courses:

$$\text{sum}\{x[\text{Cost}] \mid c[\text{Course}] : \text{SelectedCourse}(c) \ \& \ \text{HasValue}(c)=x : \ x\}$$

Discussion This sort of application is a showcase for the KB paradigm.

The system uses a formal specification to run a software tool. The application shows a perfect *separation* of domain knowledge versus computational problems. As a consequence, changing the specification T_c immediately affects the behaviour all components of the system. One can test this by editing the IDP source.¹

The application shows a strong **reuse** of the knowledge base. This is a form of reuse that cannot (easily) be achieved with standard programming languages or uni-inferential declarative programming languages [28].

Interactive Decision Enactment <https://krr.gitlab.io/demos/autoconfig/>

Another application of the same idea was for interactive configuration of a car insurance in Figure 7.3. This application is a benchmark problem in the area of Business Applications. In this system, a simple GUI for the configuration problem is automatically generated from the vocabulary. By adding new symbols to the vocabulary, the GUI is extended automatically with new fields. This application won the RuleML 2016 challenge [8].

¹E.g., replace $\forall x : \text{MandatoryCourse}(x) \Rightarrow \text{SelectedCourse}(x)$ in axiom 1 of theory T by `true` and save.

7.5 Inference with definitions

Given a definition Δ over Σ with parameter symbols $Param(\Delta)$ and defined symbols $Def(\Delta) = \Sigma \setminus Param(\Delta)$, a special form of model expansion is the following:

Δ-model expansion <i>Input:</i> a $Param(\Delta)$-structure $\mathfrak{A}_p$ <i>Output:</i> the structure $\mathfrak{A}$ satisfying Δ expanding $\mathfrak{A}_p$
--

I.e., compute the unique model of Δ expanding $\mathfrak{A}_p$. This is an instance of the model expansion problem. It is an interesting subproblem. It is deterministic, i.e., there is a unique solution. It has many applications. Moreover, efficient special purpose algorithms exist.

In the IDP3 sytem, this is implemented by the procedure `calculatedefinitions(T,S)`. For an example see <https://idp.cs.kuleuven.be/idp/?chapter=intro/ProblemLibrary>, select the N-queens example.

Another form of inference specific to definitions is *Δ -model revision*. The context is that the unique model $\mathfrak{A}$ of definition Δ for $Param(\Delta)$ -structure $\mathfrak{A}_p$ has been computed, but now, the parameter structure $\mathfrak{A}_p$ is updated, by adding or deleting some tuples to the value of predicates in $\mathfrak{A}_p$. The challenge of model revision is to update $\mathfrak{A}$ in an *incremental way*. Indeed, small updates of $\mathfrak{A}_p$ typically cause small updates of $\mathfrak{A}$. Rather than recomputing $\mathfrak{A}$ from scratch, propagate the given update to modify $\mathfrak{A}$.

Hidden in different syntactical forms, various software systems contain definitions and implement the above forms of inference.

In databases

- Views in databases $\sim$ defined symbols
- View definitions $\sim$ FO(ID) definitions
- *view materialization* $\sim$ Δ -model generation
- *incremental materialized view update* $\sim$ Δ -model revision

In spreadsheets Assume that a field (or a column or row) A is defined in terms of other fields (or columns or rows) B by some symbolic expression. When a spreadsheet computes A from B, it can be seen to perform a form of Δ -model generation. To update A after an update of B is Δ -model revision.

In IDPd3 Δ -model expansion is used in *IDPd3*, the system used in IDP for computing a visualization of structures. An example is found online.²

This application serves to solve Sudoku puzzles and to visualize the outcome.

The theory T expresses the 3 laws of Sudokus: one occurrence of each number in each row, column and block. Model expansion inference solves a puzzle specified in the input structure by returning a model $\mathfrak{A}$ in which *sudoku* ^{$\mathfrak{A}$} represents the solution.

To visualize this solution, the user theory T.D3 below is used. It is a definition of *IDPd3 graphical predicate and function symbols*. These predicates are defined in terms of symbols

² <https://idp.cs.kuleuven.be/idp/?present=SudokuVisualisatie>

interpreted in $\mathfrak{A}$. Δ -model expansion on T_D3 and input structure $\mathfrak{A}$ computes a value for the graphical predicates which is then transferred to the graphical program **d3**.

The operation of IDPd3 illustrated:

Input structure (computed by model expansion)

$sudoku = \{1, 1 \rightarrow 1; \dots; 9, 9 \rightarrow 4\}$

...

$\downarrow$ Δ -model generation on T_D3

$d3_type = \{1, Cell(1, 1) \rightarrow rect; 1, Text(1, 1) \rightarrow 1; \dots\}$

$d3_colour = \{1, Cell(1, 1) \rightarrow white; 1, Text(1, 1) \rightarrow "black"; \dots\}$

$d3_x = \{1, Cell(1, 1) \rightarrow 5; 1, Cell(1, 2) \rightarrow 10; \dots\}$

...

$\downarrow$ translation to d3 input + d3

The program d3 visualizes this input.

In IDP syntax, trailing universal quantifiers may be left implicit, as in Prolog. This feature is

theory T.D3 : V.out { { used below. } } }	$d3_type(1, Cell(r,k)) = rect <-.$ $d3_rect_width(1, Cell(r,k)) = 4 <-.$ $d3_rect_height(1, Cell(r,k)) = 4 <-.$ $d3_colour(1, Cell(r,k)) = \backslash white" <-.$ $d3_x(1, Cell(r,k)) = 5*k <-.$ $d3_y(1, Cell(r,k)) = 5*r <-.$ $d3_type(1, Text(r, k)) = text <-.$ $d3_x(1, Text(r, k)) = 5*k <-.$ $d3_y(1, Text(r, k)) = 5*r + 1 <-.$ $d3_text_size(1, Text(r, k)) = 3 <-.$ $d3_text_label(1, Text(r, k)) = t <-$ $sudoku(r, k) = c \ \& \ toString(c) = t.$ $d3_colour(1, Text(r, k)) = \backslash black" <-.$ $d3_order(1, Cell(r, k)) = 0 <-.$ $d3_order(1, Text(r,k)) = 1 <-.$ } } }	This definition defines slide 1 (argument 1) with: – for each cell (r, c) a rectangular object denoted $Cell(r, c)$, and a text object $Text(r, k)$. – It defines type, width, height, colour, x and y position of the rectangular object. – It defines type, text size and label, colour, x and y position of the text object – The sudoku number at cell (r, c) is the label of the text object.. – that text objects are in front of rectangular objects
--	---	---

Δ -model expansion expands a structure interpreting **sudoku** into a structure interpreting all these graphical symbols. This is fed into **d3**.

For more illustrations, see <https://idp.cs.kuleuven.be/idp/?chapter=intro/9-IDPD3>, select File►9.Visualisations.

7.6 Imperative + Declarative Programming in IDP3

The IDP3 system supports a prototypical knowledge-based programming environment in Lua. The environment includes high level logical objects such as vocabularies, theories, structures. It provides functionalities for manipulation and inference of these integrated and implemented in the language Lua. This supports a novel way of mixing declarative and procedural programming.

This is demonstrated in the following demo for generating Sudoku puzzles (not solutions). A requirement for a Sudoku puzzle is that it has a unique solution. Another condition that is related to its difficulty is that it is minimal. I.e., if a value of the puzzle is deleted, the puzzle has strictly more than one solutions.

In this demo, minimal Sudoku puzzles are generated. The background knowledge base in IDP is:

```
vocabulary sudokuVoc {
  extern vocabulary grid::simpleGridVoc
  type Num isa nat
  type Block isa nat
  Sudoku(Row,Col) : Num
  InBlock(Block,Row,Col)
}
theory sudokuTheory : sudokuVoc {
  ! r n : ?1 c : Sudoku(r,c) = n.
  ! c n : ?1 r : Sudoku(r,c) = n.
  ! b n : ?1 r c : InBlock(b,r,c) & Sudoku(r,c) = n.
  ! b r c : InBlock(b,r,c)  $\Leftrightarrow$  b = ((r-1)/3)*3 + ((c-1)/3) + 1.
}
```

Procedure 7.1: Pseudocode to generate minimal Sudoku puzzles

```
function GENERATESUDOKU
  Puzzle  $\leftarrow$  Empty
  Try to generate 2 solutions for Puzzle;
  while 2 solutions do
    Select random cell where solutions differ;
    Extend Puzzle with the value of the first solution at that cell;
    Try to generate 2 solutions for Puzzle;
  end while
  for each position of Puzzle that contains a value do
    Delete the value at this position;
    Generate at most 2 solutions for Puzzle;
    If there are two solutions, undo the deletion of the value;
  end for
  return Puzzle
end function
Visualize the puzzle and its unique solution.
```

The Lua implementation of part of this procedure is in Procedure 7.2.

This application uses two sorts of inferences: model expansion for generating solutions to puzzles, and Δ -model expansion for visualising puzzles and their solution.

7.7 Various forms of inference in IDP

Queries in IDP The IDP procedure `Query(Q,S)` implements query inference and computes the value of the set expression contained in a query term `Q` in the context of structure `S`. A query declaration associates a symbolic name to a set expression. An example is found

Procedure 7.2: Lua procedure to generate Sudoku puzzles in IDP3

```

vprocedure createSudoku() {
  math.randomseed(os.time())
  local puzzle = grid::makeEmptyGrid(9)

  stdoptions.nrmodels = 2
  local currSols = modelExpand(sudokuTheory,puzzle)

  while #currSols > 1 do {
    repeat
      col = math.random(1,9)
      row = math.random(1,9)
      num = currSols[1][sudokuVoc::Sudoku](row,col)
    until num ~= currSols[2][sudokuVoc::Sudoku](row,col)

    makeTrue(puzzle[sudokuVoc::Sudoku].graph,{row,col,num})
    currSols = modelExpand(sudokuTheory,puzzle)
  end

  printSudoku(puzzle)
}

```

in online.³

It contains the following query declaration:

```
query Q:V { {x[Color]: ?y[Area]:Coloring(y)=x} }
```

Model expansion and optimization inference Model expansion inference in IDP is performed by several procedures including `modelExpand(T,S)` which returns zero, one or more models of T expanding S (the maximum number can be declared via an option).

The procedure `allmodels(T,S)` computes all model expansions.

Another variant is the procedure `minimize(T,S,t)` which performs optimization inference. It computes one or more models of T expanding S that minimize the value of the term t .

These forms of inference are illustrated online.⁴

These procedures may take as input a structure S that is a standard structure of a sub-vocabulary. However, the input may also be a partial structure. An illustration of how to express a partial structure is found in the Einstein puzzle at:

[https://idp.cs.kuleuven.be/idp/?chapter=intro/ProblemLibrary,select Einstein's puzzle](https://idp.cs.kuleuven.be/idp/?chapter=intro/ProblemLibrary,select+Einsteins+puzzle).

It contains a declaration of the form:

```
structure S:V {
  ...

```

³ <https://idp.cs.kuleuven.be/idp/?present=MapColoringMXQuery>

⁴ <https://idp.cs.kuleuven.be/idp/?present=MapColoringMXOpt>

```

OwnsHouse<ct> = {English->Red; }
...
}

```

`OwnsHouse` is declared as a function symbol. The above assignment specifies that this function is all unknown except that `OwnsHouse(English)=Red`. The suffix `<ct>` indicates that this assignment specifies certainly true values. It is also possible to specify certainly false values, e.g.,

```
OwnsHouse<cf> = {Japanese->Blue; }
```

Model expansion is the logical variant of *Constraint Programming*, the declarative programming paradigm for solving search problems. Many constraint programming engines exist such as ILOG, Zinc, etc. An example application of constraint solving/model expansion is the 2 dimensional packing problem where the goal is to put a number of squares of different size on a rectangular grid.

<https://idp.cs.kuleuven.be/idp/?src=31c913a78b077bf6ab89>

A well-known optimization problem is the knapsack problem. The goal is to select a set of objects, each with a value and weight, to put them in a knapsack with a maximal weight. The goal is to optimize the total value of the knapsack. A solution in IDP is found at:

<https://idp.cs.kuleuven.be/idp/?src=ace6dc004bfeff3dcca3>

Propagation inference Propagation inference was introduced in the discussion of the course selection application. Propagation inference is useful to compute possible values of symbols, given a partial structure and a theory. IDP supports different forms of propagation inference: optimal propagation (which is expensive) but also (cheap but incomplete) symbolic and non-symbolic propagation. The procedures are `propagate(T,S)`, `groundpropagate(T,S)` and `optimalpropagate(T,S)`

For a small problem illustrating these three forms of propagation, select the last example (“Propagation & Predicate Table Access”) on <https://idp.cs.kuleuven.be/idp/?chapter=intro/ProblemLibrary>

7.8 Reasoning on LTC theories

In the context of Linear Time Calculus, a range of different extra forms of inference are possible:

- Model expansion and optimization inference can be used for planning (with optimization).
- `initialise(T,S)`, `progress(T,S)`: to compute a set of initial states of a LTC `T` expanding structure `S`, respectively, to compute a set of progressions of `T` from state `S`. These forms of inference can be used for building interactive simulation as demonstrated in a pacman demo. <https://idp.cs.kuleuven.be/idp/?chapter=intro/9-IDPD3>.
- `isinvariant(T,Inv,S)`: proving that `Inv` is an invariant of LTC `T` in the context of models expanding structure `S`.

To the best of our knowledge, IDP is the only system that can use the same formal LTC both to simulate an LTC theory as well as compute (optimal) plans from it.

The company LogicBlox uses progression as a basic inference step to build large business applications. <http://www.logicblox.com/>

7.9 Conclusion

Deductive inference (theorem proving) was the original form of inference studied for logic. It is important for *verification*: verifying that a system satisfies correctness conditions *under all circumstances*. It is a computationally complex form of inference, as is attested by its undecidability in a simple language such as FO. Many common tasks in software products can be achieved by other forms of inference: model checking, query answering, finite model generation, model revision . . . The good news is: each of these forms of inference is *computationally cheaper*, and *practically more often useful* than theorem proving. Deduction is for the design phase (verification); the cheaper forms of inference are for the production phase.

One question that arises is to what extent software systems can be developed by application of logic inference methods on formal specification and what would be the benefit. Of course, the challenges at the computational level remain huge. But where this approach works, there is little doubt that great benefits are feasible on the level of standard software metrics:

compactness – correctness – separation of concerns – reuse – maintainability

The company LogicBlox argues that by using their system, software development is fastened by a factor $50\times$ compared to standard software methodologies. Also we (in KRR) have seen sometimes enormous improvements (one time, replacing a program by a logic theory that was $400\times$ smaller). Another example dating from a few years back was the system Anton for music generation by *model generation* on a variant of FO($\cdot$). The authors claimed that their 500 lines of logic specification performed similar to a program of 88 000 lines of C code.

In a growing range of applications, logical methods become more and more effective. One area for which I believe it is fair to say that logic and inference have found solid ground in industrial applications is database technology. Although many do not longer seem to view database technology as applied logic. Another area where logic gains ground is in Constraint solving.

From a different point of view, I claim that the logic perspective shows overlaps and correspondences in areas of declarative programming – overlaps in the languages, in the inference implemented by the tools, and in the techniques and algorithms used to implement this inference. An enormous amount of technologies have been developed in different areas: database technologies, ontology language technologies, logic programming, constraint solving, SAT, answer set programming, domain-specific configuration languages and solvers, expert systems, business rules technologies . . . The challenge is to integrate and generalize the many existing technologies in generic inference systems for logic.

The specific contribution of the KRR group is to develop applications by applying different inference tools to FO($\cdot$) theories. In this respect, we belong to the pioneers. At present, we are searching for industrial applications for IDP, especially in areas of knowledge-intense applications where multiple forms of inference are of use. The group continues to improve its systems by integrating existing technologies in our systems. You can help us – do a thesis in our group.

7.10 Important for the exam

Inference:

- Understanding of the principle of solving problems by applying inference to specifications.
- Definitions of the most important forms of inference, understanding of their utility.
- Understanding of the knowledge base paradigm as illustrated in interactive configuration.

Large question:

- Give an overview of inference problems or of the knowledge base paradigm. You could study this by making a list of inference problems, their formal definitions, a few examples, other fields where this form of inference is applied.
You could organize your answer on the interactive configuration problem that illustrates 5 forms of inference AND illustrates the separation of information and problem, and the reuse of a theory.

Chapter 8

Algorithms for finite satisfiability checking in propositional and predicate logic

In this chapter, we see algorithms for satisfiability checking of propositional logic and finite model expansion for predicate logic.

8.1 Complexity of satisfiability in propositional logic

8.1.1 Propositional logic

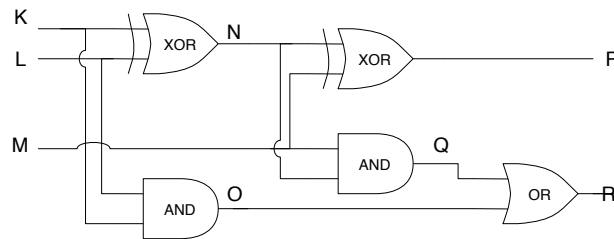
Propositional logic or propositional calculus (PC) is the subformalism of FO obtained by restricting vocabularies Σ to propositional symbols: predicate symbols of arity 0. Propositional logic does not include quantifiers. The connectives are the standard ones:

$$\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow \quad (8.1)$$

The Σ -structures $\mathfrak{A}$ are (but don't contain) boolean functions of Σ . The history of propositional logic goes back to Boole who in 1854 published his work in a paper *An Investigation of the Laws of Thought, on Which are Founded the Mathematical Theories of Logic and Probabilities* [3]. This work introduced Boolean algebra. Propositional logic is a primitive modelling language to which specifications expressed in more expressive languages are frequently translated. It can be considered as the “machine language” of logic.

Example 8.1.1

A boolean circuit example:



This boolean circuit represents 2 formulas

$$(K \text{ XOR } L) \text{ XOR } M \text{ and } (K \wedge L) \vee (M \wedge (K \text{ XOR } L))$$

where XOR is exclusive disjunction. Recall that $(K \text{ XOR } L) \equiv (K \Leftrightarrow \neg L)$.

Example 8.1.2

Some people took drugs. The suspects are: Sam, Michael, Bill, Richard, Matt. There is also Tom but he is innocent.

- Sam said: Michael or Bill took drugs, but not both.
- Michael said: Richard or Sam took drugs, but not both.
- Bill said: Matt or Michael took drugs, but not both.
- Richard said: Bill or Matt took drugs, but not both.
- Matt said: Bill or Richard took drugs, but not both.

Of these 5 statements, 4 are true, one is false.

Tom said: If Richard took drugs, then Bill took drugs. This is true.

Who doped?

We use the following vocabulary. Statements are made by six different people. The intended interpretation of the symbols **ss**, **mis**, **bs**, **rs**, **mas**, **ts** is that the statements made by respectively Sam, Michael, Bill, Richard, Matt and Tom are true (**ss** stands for “Sam Said”). The intended interpretation of the symbols **sd**, **mid**, **bd**, **rd**, **mad** is that respectively Sam, Michael, Bill, Richard, and Matt are doped (**sd** stands for “Sam Doped”).

In terms of this vocabulary, the given information can be specified as follows:

```

ss ⇔ (mid ⇔ ¬bd).
mis ⇔ (rd ⇔ ¬sd).
bs ⇔ (mad ⇔ ¬mis).
rs ⇔ (bd ⇔ ¬mad).
mas ⇔ (bd ⇔ ¬rd).
ts ⇔ (rd ⇒ bd).
ts.
(ss ∧ mis ∧ bs ∧ rs ∧ ¬mas) ∨
(ss ∧ mis ∧ bs ∧ ¬rs ∧ mas) ∨
(ss ∧ mis ∧ ¬bs ∧ rs ∧ mas) ∨
(ss ∧ ¬mis ∧ bs ∧ rs ∧ mas) ∨
(¬ss ∧ mis ∧ bs ∧ rs ∧ mas).

```

8.1.2 Satisfiability inference in PC

In this chapter, we are interested in algorithms implementing *satisfiability checking* and *model generation* for propositional and predicate logic.

Satisfiability checking inference

Input: a PC formula φ

Output: **t** (true) if φ is satisfiable; **f** otherwise

As it turns out, these algorithms do this by computing models of a propositional theory.

That is, they can be used to implement also the following form of inference.

Model generation inference
Input: a PC formula φ
Output: one or more models $\mathfrak{A}$ of φ ; UNSAT if φ is unsatisfiable

The decision problem of satisfiability checking has several sorts of applications. E.g., when we build a theory of integrity constraints in a domain, e.g., in the context of a database application, or a constraint solving problem, or a piece of legislation (a set of laws), etc., a basic correctness test for the theory is that it is satisfiable. E.g., when we want to verify whether a given specification T entails a desired or expected property φ , we test whether $T \cup \{\neg\varphi\}$ is unsatisfiable.

In many applications, the models of a theory provide useful information. In constraint solving applications, the solution to the constraint problem is (part of) a model of the theory. When a verification that $T \models \varphi$ fails because $T \cup \{\neg\varphi\}$ turns out to be satisfiable, a model of the latter theory represents a possible state of affairs in which T is true and the expected proposition φ is false. By inspecting this model, we can detect the error in our reasoning and correct it — by refining T or φ .

In predicate logic, satisfiability checking is unrecognisable. But in propositional logic, the problem is decidable.

Exercise 8.1.1 Explain that satisfiability checking in FO is not semi-decidable.

Procedure 8.1: Bruteforce SAT

```

function BRUTEFORCESAT( $\varphi$ )
  for each structure  $\mathfrak{A}$  interpreting  $\varphi$  do
    if  $\mathfrak{A} \models \varphi$  then
      return  $\mathfrak{A}$ 
    end if
  end for
  return UNSAT
end function
    
```

Consider Procedure 8.1. This exhaustive enumeration is essentially the method of *truth tables* as illustrated below:

p	q	$(p \Rightarrow q) \Leftrightarrow (\neg p \vee q)$			
t	t	t	t	t	t
t	f	t	f	f	f
f	t	f	t	t	t
f	f	f	t	f	f

Each row corresponds to one structure. After one row, the satisfiability of the formula was determined (the outer connective $\Leftrightarrow$ resulted in a truth). After the last row, the tautologicity (logical validity) of the formula was determined (the outer connective $\Leftrightarrow$ resulted in only truths).

Complexity This algorithm is unfeasible for any but the smallest numbers of symbols. With n symbols, 2^n number of structures are to be tested. For $n = 50$, $2^n \approx 10^{15}$, with

$n = 60$, $2^n \approx 10^{18}$. In current applications, CNF formulas with hundred thousands or even million of symbols are no exception.

This exponential character isn't surprising: the satisfiability problem is a prototypical NP-complete problem (in the size of the formula). This is Stephen Cook's 1971 theorem [7].

Remark 8.1.1: P, NP and NP-complete

P is the class of decidable decision problems that can be decided in a time that grows polynomially with the size of the input. NP is the class of decidable decision problems for which at least a proof-of-work can be verified in polynomial time (and the proof-of-work must also be polynomial in the size of the input). The hardest problems in NP are NP-complete; the fastest algorithm to solve those gives the upper time bound for solving all other problems in NP.

A problem whose complement is in P is said to be in co-P, in NP is said to be in co-NP, and so on. A problem's complement is often much harder or much easier itself.

Nobody has been able to prove that $P \neq NP$, i.e., that there are no polynomial algorithms to solve NP-complete problems. Most people believe this to be the case.

" $P = NP$ " is a millennium prize problem, a problem selected by the Clay Mathematics Institute as one of the seven most important problems in mathematics. A proof or a proof of its negation is worth a million dollars.

Exercise 8.1.2 Argue why $P = co-P$.

SAT is in NP because a structure $\mathfrak{A}$ that satisfies φ is sufficient proof-of-work, small enough ($|\Sigma|$) and verifiable in time proportional to the size of φ by filling in the values and evaluating the boolean connectives from the inside outwards.

Note that $\neg SAT(\varphi)$ – SAT's complement, namely $UnSAT(\varphi)$ – is not in NP: there is no simple proof-of-work to show that every $\mathfrak{A}$ failed. That doesn't mean the computation takes longer though, because evidently, computing $\neg SAT(\varphi)$ is upper-bounded by the time it takes to find $SAT(\varphi)$.

Definition 8.1.1: Normal forms of PC

- A *literal* is an atom p or the negation of an atom $\neg p$.
- A formula is in *negation normal form* (NNF) if it contains only $\wedge, \vee$ and $\neg$ and the negation symbol $\neg$ occurs only in literals.
- A *clause* is a disjunction ($\vee$) of literals.
- A formula is in *conjunction normal form* (CNF) if it's a conjunction ($\wedge$) of clauses ($\vee$).

Alternatively, a CNF formula can be viewed as a theory consisting of clauses.

Example 8.1.3: Formula in CNF

$$(p \vee \neg q \vee r) \wedge (p \vee q) \wedge (\neg p \vee \neg q \vee t) \quad (8.2)$$

Almost all algorithms for satisfiability checking of PC require input of CNF formulas. The reason is that algorithms for formulas in CNF are easier to design and implement, and linear transformations from PC to CNF exist.

8.1.3 Complexity of tautologicity checking in CNF

$\text{UnSAT}(\varphi)$ is not the same problem as $\text{SAT}(\neg\varphi)$, which is SAT's dual problem: notice that if this is true, i.e. a formula's negation is satisfiable, then the formula itself is not a tautology. This means $\text{SAT}(\neg\varphi)$ is really $\neg\text{TAUT}(\varphi)$. The latter problem is hence NP-complete too, making $\text{TAUT}(\varphi)$ co-NP-complete.

Since $\text{SAT}(\neg\varphi)$ is $\neg\text{TAUT}(\varphi)$, it also holds that $\text{SAT}(\varphi)$ is $\neg\text{TAUT}(\neg\varphi)$. We can calculate this $\neg\text{TAUT}$ by negating the result of an algorithm for TAUT (just like we did above), even though such an algorithm isn't in NP; this is why it pays to study tautologicity checking in the search of satisfiability checking.

The following simple propositions lead to an efficient algorithm to decide tautologicity of a CNF formula, in the hopes that the overhead of transforming any arbitrary PC formula to CNF is quite efficient too.

Two literals ℓ and $\neg\ell$ are called *complementary*.

Proposition 8.1.1

A clause is tautological iff it contains two complementary literals.

Proof. If a clause has no complementary literals, it is false in the structure making each disjunct false. If a clause has complementary literals ℓ and $\neg\ell$, then in every structure, one of the complementary disjuncts is true. ■

Proposition 8.1.2

A conjunction is tautological iff each of its conjuncts is tautological.

Proof. If $\varphi_1 \wedge \dots \wedge \varphi_n$ is true in a structure, then every conjunct is true in that structure. Hence, if $\varphi_1 \wedge \dots \wedge \varphi_n$ is true in every structure, each conjunct is true in every structure. Vice versa, if every conjunct is true in a structure, then the conjunction is true in the structure. Hence if every conjunct is tautological, the conjunction is tautological. ■

It follows from these two propositions that a CNF formula is tautological iff each of its clauses contains complementary literals.

Proposition 8.1.3

The problem of deciding tautologicity of a CNF formula has linear complexity in the size of the formula.

Proof. The algorithm is to run linearly over φ and verify for each clause of φ whether it contains complementary literals. ■

toCNF The proposition suggests an algorithm to check tautologicity of a formula φ : transform it into an equivalent CNF-formula and apply the linear algorithm. Since there is no free lunch in this world, we can expect that the computation of a formula ψ in CNF equivalent to a given formula φ is a costly worst-case operation. Indeed, if there would be an equivalence-preserving, polynomial transformation to CNF, then $P = \text{co-NP}$.

Procedure 8.2 consists of four successive rewrite phases; in each phase, one or more well-known equivalences are applied as left to right rewrite rules until no rule applies anymore.

We denote the result of applying this algorithm on φ as $\text{toCNF}(\varphi)$.

Procedure 8.2: Conversion to CNF

1. Rewrite every $(\psi \Leftrightarrow \phi)$ as $(\psi \Rightarrow \phi) \wedge (\phi \Rightarrow \psi)$ (*)
 2. Rewrite every $(\psi \Rightarrow \phi)$ as $(\neg\psi \vee \phi)$
 3. Rewrite every $\neg\neg\psi$ as ψ
and $\neg(\psi \wedge \phi)$ as $(\neg\psi \vee \neg\phi)$
and $\neg(\psi \vee \phi)$ as $(\neg\psi \wedge \neg\phi)$ (laws of De Morgan)
- After this step, we obtain formulas in NNF.
4. Rewrite every $(\psi \wedge \phi) \vee \delta$ as $(\psi \vee \delta) \wedge (\phi \vee \delta)$ (*)
distributing disjunction over conjunction

The algorithm terminates. Its output is logically equivalent with the input. Rewrite operations (*) and (*) create two copies of subformulas. Hence, they may double the size of the formula. Therefore, the full rewrite process blows up the formula exponentially in the worst case.

Exercise 8.1.3 *The previous statement is an argument that this algorithm is exponential but not a proof. Give a proof: find a class of formulas for which the exponential blow up occurs.*

Worst-case exponential complexity does not necessarily mean that an algorithm is useless. Indeed, it depends on how often bad cases occur in practice. E.g., the standard linear programming algorithm has exponential worst-case behaviour but until not so long ago it was unbeatable by polynomial algorithms in practical problems. Alas, in practice the above algorithm toCNF indeed behaves exponentially making it quite useless.

Remark 8.1.2

There exists various alternative normal forms for which linear algorithms for satisfiability and tautologicity and other forms of inference exist. One is called *binary decision diagrams*. They are binary tree-like formulas with syntax defined in BNF:

$$\psi ::= \perp \mid \top \mid (\text{if } p \text{ then } \psi \text{ else } \psi) \quad (8.3)$$

where p is a propositional symbol, and such that all symbols in a branch ψ of the tree differ from p . Such a formula is satisfiable iff it has a leaf $\top$ (true) and tautological iff all leaves are $\top$. This shows that linear algorithms for satisfiability and tautologicity exist for this format. Transforming a PC formula into a logically equivalent BDD is also exponential in the worst case, but algorithms exist with much better average behaviour. The field investigating such methods is called *knowledge compilation*.

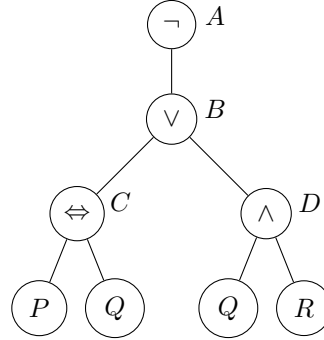
Tseitin transform We now introduce the *Tseitin transformation*, a linear algorithm for transforming a PC formula φ to a closely related CNF formula. One can think of the transform as happening in three steps:

1. Construct the parse tree of φ 's connectives. Label each node.
2. Describe how these labels relate to each other using single-connective formulas.
3. Transform these formulas to CNF.

Let's illustrate this with an example before formalising it.

Example 8.1.4

Consider the formula $\varphi = \neg((P \Leftrightarrow Q) \vee (Q \wedge R))$. We use symbols A, B, C, D as labels for its subformulas. Its parse tree is:



In the table below, we describe the relationships between the labels (first row) and take **toCNF**(second row). The Tseitin transformation of φ is the set of all clauses in the second row.

Relationship ψ	A	$A \Leftrightarrow \neg B$	$B \Leftrightarrow C \vee D$	$C \Leftrightarrow (P \Leftrightarrow Q)$	$D \Leftrightarrow Q \wedge R$
toCNF (ψ)	A	$A \vee B$ $\neg A \vee \neg B$	$\neg B \vee C \vee D$ $B \vee \neg C$ $B \vee \neg D$	$\neg C \vee P \vee \neg Q$ $\neg C \vee \neg P \vee Q$ $C \vee \neg P \vee \neg Q$ $C \vee P \vee Q$	$\neg D \vee Q$ $\neg D \vee R$ $D \vee \neg Q \vee \neg R$

Since this problem lends itself well to recursion, the above three steps can be done recursively instead of sequentially for the whole parse tree. Procedure 8.3 does exactly this, with $\ell(\cdot)$ some injective helper function for generating unique labels.

We can formalize what happens as follows. The set of all subformulas α (nodes in the parse tree, which each represent a single connective or literal in φ) in φ is called S_φ . For each $\alpha \in S_\varphi$, we introduce a new proposition symbol $p_\alpha = \ell(\alpha)$. Furthermore, for each $\alpha \in S_\varphi$, the function **withLabels** generates a new formula from α , denoted α' , by keeping the connective it represents but replacing its components (the subformulas it connects directly) by the corresponding proposition symbols. Finally, we connect these two constructs – the proposition symbol p_α and the single-connective formula α' – by an equivalence: $p_\alpha \Leftrightarrow \alpha'$. This means that wherever p_α appears, it is a stand-in for the original formula of α .

As a small example: if $\varphi = (\gamma) \vee (\beta)$ for some unspecified subformulas γ and β , then we make a new symbol p_φ , a formula $\varphi' = p_\gamma \vee p_\beta$, and assert that $p_\varphi \Leftrightarrow p_\gamma \vee p_\beta$.

The Tseitin transform produces a set of clauses, i.e. a clausal theory. Hence, to complete the process, we need to apply **toCNF** to all the equivalences we generated, and thus get:

$$\mathbf{Tseitin}(\varphi) = \{p_\varphi\} \cup \{\mathbf{toCNF}(p_\alpha \Leftrightarrow \alpha') \mid \alpha \in S_\varphi\} \quad (8.4)$$

Proposition 8.1.4

The Tseitin transformation has linear complexity. It preserves satisfiability, but not tautologicity. Every model of φ can be extended in a unique way to a model of $\mathbf{Tseitin}(\varphi)$, and each model of $\mathbf{Tseitin}(\varphi)$ is a model of φ .

Proof. The Tseitin transformation can be implemented by traversing the parse tree of input φ in one linear pass. While **toCNF** is exponential in general, here it is applied only to formulas

Procedure 8.3: Conversion to CNF via Tseitin

We use ℓ to denote the injective labelling function over formulas.

```

function TSEITIN( $\varphi$ )
  if  $\varphi$  is a literal  $L$  then
    return  $\{\ell(\varphi) \Leftrightarrow L\}$ 
  else
    clauses  $\leftarrow \emptyset$ 
    for component  $\chi$  in  $\varphi$  do
      clauses  $\leftarrow$  clauses  $\cup$  TSEITIN( $\chi$ )
    end for
    clauses  $\leftarrow$  clauses  $\cup \{ \text{toCNF}(\ell(\varphi) \Leftrightarrow \text{WITHLABELS}(\varphi)) \}$ 
    return clauses
  end if
end function

function WITHLABELS( $\varphi$ )
  if  $\varphi$  is  $\neg(\chi)$  then
    return  $\neg\ell(\chi)$ 
  else if  $\varphi$  is  $\chi_1 \wedge \chi_2$  then
    return  $\ell(\chi_1) \wedge \ell(\chi_2)$ 
  else if  $\varphi$  is  $\chi_1 \vee \chi_2$  then
    return  $\ell(\chi_1) \vee \ell(\chi_2)$ 
  else if  $\varphi$  is  $\chi_1 \oplus \chi_2$  then
    return  $\ell(\chi_1) \oplus \ell(\chi_2)$ 
  else if  $\varphi$  is  $\chi_1 \Rightarrow \chi_2$  then
    return  $\ell(\chi_1) \Rightarrow \ell(\chi_2)$ 
  else if  $\varphi$  is  $\chi_1 \Leftrightarrow \chi_2$  then
    return  $\ell(\chi_1) \Leftrightarrow \ell(\chi_2)$ 
  end if
end function

```

with at most two connectives (the $\Leftrightarrow$ and the one in α'). Hence, the size of $\text{toCNF}(p_\alpha \Leftrightarrow \alpha')$ is bounded. The size of the transformation is linear in the size of φ .

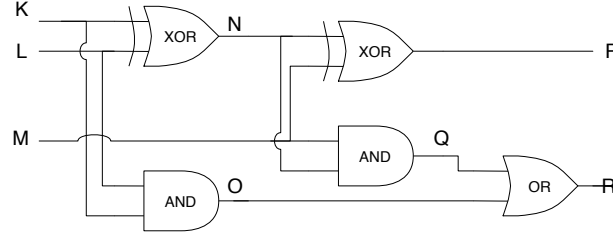
The correctness of the algorithm is based on the following observation. Let φ be a formula with strict subformula α . Let p_α be a symbol not occurring in φ . Let φ' be the formula obtained from φ by substituting the occurrence α by p_α . Then each model $\mathfrak{A}$ of φ can be expanded in a unique way with an interpretation of p_α to a model of $\varphi' \wedge (p_\alpha \Leftrightarrow \alpha)$. Indeed, the unique extension is $\mathfrak{A}[p_\alpha : \alpha^\mathfrak{A}]$. Vice versa, it is easy to see that each model of $\varphi' \wedge (p_\alpha \Leftrightarrow \alpha)$ is a model of φ .

Clearly the Tseitin transformation can be implemented by iterated application of the above transformation rule, followed by equivalence-preserving applications of toCNF . Hence, it follows that each model of φ has a unique expansion to a model of $\text{Tseitin}(\varphi)$ and vice versa, a model of the transformation is a model of the original formula.

While the semantical correspondence of φ and $\text{Tseitin}(\varphi)$ is strong, the transformation does not preserve tautologicity. This is because it introduces symbols that are not constrained by φ but are constrained by $\text{Tseitin}(\varphi)$. In particular, the basic rewrite step introduces a conjunct $p_\alpha \Leftrightarrow \alpha$ which is not tautological. For example, take any structure interpreting α and expand it as $\mathfrak{A}[p_\alpha : (\neg\alpha)^\mathfrak{A}]$. In this structure, the equivalence is not satisfied. ■

It follows that the Tseitin transformation cannot be used in proving tautologicity of φ . However, it can be used for satisfiability checking and model generation inference on φ .

Exercise 8.1.4 Compute the Tseitin transformation of $P \wedge Q$ as given by the boolean circuit example. You may find it useful to break up the XOR into smaller nodes.



Summary We have now seen two algorithms for preprocessing a propositional formula φ to CNF. They come down to the same complexity for finding whether it is satisfiable:¹

- $O(2^n) \text{ toCNF}(\neg\varphi) \rightarrow O(n) \neg\text{TAUT}$.
- $O(n) \text{ Tseitin}(\varphi) \rightarrow O(2^n) \text{ SAT}$.

where n is the size of φ . The Tseitin transformation is widely used. Many refinements exist, like avoiding multiple Tseitinizations of different occurrences of the same formula, or recognising certain specific subformulas (called circuits) and providing an optimal, pre-programmed transformation.

In the end, the CNF is not used for the efficiency of tautologicity inference, for the simple reason that in practice, CNF formulas are basically never tautological. The CNF format is used for satisfiability checking and model generation inference. This inference problem is still NP-complete for CNF, and hence there is no fundamental computational gain. So why is CNF used? Because of its simplicity which is of great help to develop efficient algorithms.

8.2 SAT algorithms

The *CNF-SAT problem* is the satisfiability checking inference problem applied to propositional CNF formulas. This is an NP-complete problem.

The field of SAT is a lively area, with important practical industrial applications, e.g., in verification of computer hardware, in constraint solving. Enormous progress has been made especially since 2000. In the past two decennia, the range of solvable problems went up from a few hundred variables to millions of variables and clauses. Now for some years, progress is slower but still steady.

A major step in this evolution has been the introduction of the CDCL algorithm which below we introduce in several steps.

8.2.1 DPLL: a basic SAT solver

The *Davis-Putnam-Logemann-Loveland (DPLL)* algorithm [9] is a backtracking algorithm for deciding satisfiability of a CNF formula. Although it is not efficient, it is still the basis for the current SAT solvers. First we introduce some basic concepts.

¹Note: the overhead for getting $\neg\varphi$ is $O(n)$.

Definition 8.2.1: Partial structure, conflict clause and unit clause

A partial structure $\mathcal{I}$ for (propositional) vocabulary Σ is a function from Σ to $\{\mathbf{t}, \mathbf{f}, \mathbf{u}\}$. We call $\mathbf{u}$ “undefined”.

A clause C is a *conflict clause* with respect to $\mathcal{I}$ if every literal in C is false.

A clause C is a *unit clause* w.r.t. $\mathcal{I}$ if every literal is false except one literal L that is undefined. This literal is called the *unit literal*.

Partial structures arise in SAT algorithms in intermediate states, when some propositional symbols were assigned a value and others not yet.

It may happen during execution of a SAT algorithm (on a CNF formula φ) that a partial structure $\mathcal{I}$ is constructed such that one of the clauses C of φ is a conflict clause. The current $\mathcal{I}$ cannot be expanded to a model $\mathfrak{A}$ of φ in that case. Backtracking should occur to the assignment that caused the conflict. Likewise, if some clause is a unit clause with unit literal L , then in all models $\mathfrak{A}$ expanding $\mathcal{I}$, L will be true. In this case, DPLL will *propagate* L by setting it true in $\mathcal{I}$. This is called a *unit propagation*.

Unit propagation A unit propagation in the context of a partial structure $\mathfrak{A}$ is the operation of expanding $\mathfrak{A}$ to make a unit literal true. A unit propagation may cause other clauses to become unit clause and hence, may lead to a cascade of unit propagations. This propagation process is implemented by the procedure UP in Procedure 8.4.

If a run of UP assigns values to n variables, this means that $2^n - 1$ possible assignments to these variables are cut from the search space.

The DPLL algorithm A high-level description of DPLL is given in Procedure 8.5. The procedure uses the procedure UP and the following subroutines:

- **Decide:** choose a literal L that is undefined in $\mathcal{I}$ and set it to true: $\mathcal{I}(L) := \mathbf{t}$. L is called a *decision literal* or *choice literal*.
- **Backtrack:** if there is no decision literal, return UNSAT. Otherwise, take the most recent decision literal L and set L to false: $\mathcal{I}(L) := \mathbf{f}$; now $\neg L$ is called a *backtrack literal*. Also, reset every literal L' more recent than L : $\mathcal{I}(L') := \mathbf{u}$. The reset literals are backtrack and unit literals.

Procedure 8.4: Unit propagation

```

function UP( $\mathcal{I}$ )
  while there exists a unit clause with unit literal  $L$  do
     $\mathcal{I}(L) \leftarrow \mathbf{t}$ 
    if there exists a conflict clause then
      return conflictClause
    end if
  end while
end function

```

Example 8.2.1: Example execution of DPLL

Procedure 8.5: DPLL SAT

```

function DPLL( $\varphi$ )
  for symbol  $p$  in  $\varphi$  do
     $\mathcal{I}(p) \leftarrow \mathbf{u}$ 
  end for
  while true do
    conflictClause  $\leftarrow$  UP( $\mathcal{I}$ )
    if conflictClause not null then
      BACKTRACK
    else if  $\mathcal{I}$  is a model then
      print  $\mathcal{I}$ 
      BACKTRACK
    else
      DECIDE
    end if
  end while
  return UNSAT
end function

```

CNF: $\neg P \vee \neg R.$ $\neg Q \vee R.$ $P \vee \neg Q \vee \neg R.$ $P \vee Q \vee R.$	Choice: P – Propagate: $\neg R$ – Propagate: $\neg Q.$ $\rightarrow$ Model! Backtrack: $\neg P$ – Choice: Q – Propagate: R Conflict: $P \vee \neg Q \vee \neg R!$ Backtrack: $\neg Q$ – Propagate: $R.$ $\rightarrow$ Model! Backtrack till end.
---	---

Two models are computed: $\{P, \neg Q, \neg R\}, \{\neg P, \neg Q, R\}$.

The current partial structure Π can be represented as a sequence of true literals $\langle L_0, \dots, L_n \rangle$ according to order in which they were set to true (L_0 being the oldest). The *time* of L_i is its index i in this sequence. The *level* of L_i is the number of decision literals up to and including L_i . All backtrack and unit literals have the same level as the decision literal they follow. Unassigned literals obviously do not have a time or a level yet.

Notice that DPLL starts with a call to UP. If this results in conflict, UNSAT is returned. Literals derived during this initial phase are literals of level 0. The first decision literal is at level 1, and so on.

Example 8.2.2: Times and levels

$P \vee Q \vee R$ $S \vee T \vee Q$ $\neg Q \vee R$ $P \vee \neg Q \vee \neg R$ $\neg S \vee R$	Choice: $\neg P$. – Choice: $\neg S$. – Choice $\neg T$. – Propagate: Q . – Propagate: R . Conflict: $P \vee \neg Q \vee \neg R!$	<table border="1" style="border-collapse: collapse; text-align: center;"> <thead> <tr> <th></th> <th><i>Val</i></th> <th><i>Time</i></th> <th><i>Lev</i></th> </tr> </thead> <tbody> <tr> <td>P</td> <td>0</td> <td>0</td> <td>1</td> </tr> <tr> <td>S</td> <td>0</td> <td>1</td> <td>2</td> </tr> <tr> <td>T</td> <td>0</td> <td>2</td> <td>3</td> </tr> <tr> <td>Q</td> <td>1</td> <td>3</td> <td>3</td> </tr> <tr> <td>R</td> <td>1</td> <td>4</td> <td>3</td> </tr> </tbody> </table>		<i>Val</i>	<i>Time</i>	<i>Lev</i>	P	0	0	1	S	0	1	2	T	0	2	3	Q	1	3	3	R	1	4	3
	<i>Val</i>	<i>Time</i>	<i>Lev</i>																							
P	0	0	1																							
S	0	1	2																							
T	0	2	3																							
Q	1	3	3																							
R	1	4	3																							

The following observations are relevant to understand the improved algorithm that will be introduced below.

1. When a conflict clause arises during DPLL at level > 0 , it contains at least two (false) literals of the last level. Otherwise, the clause would have been a conflict clause or a unit clause at a previous level. In the example, this is $\neg Q$ and $\neg R$.
2. When a unit literal L was derived at the current level > 0 , it was by a unit clause with at least 2 literals of the same level, namely L (which is true) and at least one literal that is false. Otherwise, this clause would have been a unit clause at a previous level. Notice that clauses of length 1 are unit clauses of level 0.
3. A disadvantage of DPLL is that it does not learn; it may make choices for a group of variables that lead to failure, and after backtracking to older levels, it may keep repeating the same bad choices for these variables over and over again.

8.2.2 Conflict-Driven Clause learning (CDCL)

CDCL is an optimization of DPLL dating presented originally in [26]. In a decade, it led to a huge improvement of DPLL and had an immense impact on SAT. Its key idea is that it extends the CNF theory with *learned clauses* obtained in a smart way by *resolution* to cut exponential parts from the search tree.

Definition 8.2.2: Resolution of two clauses

Two clauses $\psi = p \vee L_1 \vee \cdots \vee L_n$ and $\phi = \neg p \vee L'_1 \vee \cdots \vee L'_m$ are *resolved on p* to get the *resolvent* $\delta = L_1 \vee \cdots \vee L_n \vee L'_1 \vee \cdots \vee L'_m$.

Resolution is sound, i.e. the resolvent is logically entailed by the two input clauses: $\psi \wedge \phi \models \delta$. Adding resolvents of clauses of a CNF theory to the theory thus preserves equivalence and has no impact on the set of computed models. However, it may improve the efficiency of the DPLL algorithm considerably. It seems like a good idea to add resolvents to the theory. The question is: which resolvents? We cannot add all resolvents: there are too many of them. The number of resolvents of a CNF theory is exponential in the number of clauses.

In the *conflict-driven clause learning (CDCL)* algorithm, resolvents are constructed that directly aid the solving process. They cut away potentially large parts of the search space by allowing deep backtracking. When a conflict clause arises, resolution is applied to construct a clause that gives the “reason” for the conflict. This clause is called the *learned clause*, and allows to perform *backjumping*, i.e. to backtrack over multiple levels. This may cut away exponential parts of the search tree. Moreover, the clause is stored to avoid that a failed assignment is repeated.

The CDCL procedure is activated when a conflict arises. Its operation is described below.

First resolution step Assume that during execution, a conflict arises due to the conflict clause $c = L_1 \vee L_2 \vee \cdots \vee L_n$. Let us assume that the clause is ordered from young to old according to assignment time, in which case L_1 is the youngest literal and L_2 the second youngest. Recall that the conflict clause contains at least two literals of the current level. Hence, L_1 and L_2 are of the current level. Both are false in the current partial structure $\mathcal{I}$ by definition of a conflict clause. Hence, they are the negation of unit literals or of the decision literal of the current level. ²

²We will see that in CDCL, contrary to DPLL, there are no *backtrack literals*, only *decision* and *unit* literals.

The decision literal of the current level is the oldest literal of the current level. Since L_1 is younger than L_2 , the literal L_1 must be the negation of a unit literal $\neg L_1$, which was derived by a unit clause $u = \neg L_1 \vee L'_2 \vee \dots \vee L'_m$. CDCL resolves the conflict clause c and the unit clause u on L_1 . The result is a new clause which, after removing double literals and ordering from young to old, we denote as $L''_1 \vee \dots \vee L''_k$. This operation is called **ResolveYoungest**.

Procedure 8.6: Resolution of the youngest conflict literal

function RESOLVEYOUNGEST(conflict clause c)
 Reorder the literals in c from youngest to oldest assignment.
 $L \leftarrow$ first literal in c .
 Find the unit clause u that caused propagation of $\neg L$.
return RESOLVE(c, u)
end function

Since $\neg L_1$ was the only true literal in both clauses and was resolved away, this resolution step produces again a conflict clause. Notice that since the conflict clause contained at least two false literals of the current level (L_1 and L_2), the returned conflict clause still contains at least one false literal of the current level (L_2). The youngest literal L''_1 is indeed of the current level and might be L_2 .

Example 8.2.3

Example 8.2.2 continued. It constructs the structure with P, S, T false and Q, R true.

$P \vee Q \vee R.$	Choice: $\neg P$.
$S \vee T \vee Q.$	– Choice: $\neg S$.
$\neg Q \vee R.$	– Choice $\neg T$.
$P \vee \neg Q \vee \neg R.$	– Propagate: Q .
$\neg S \vee R.$	– Propagate: R .
	Conflict: $P \vee \neg Q \vee \neg R!$

ResolveYoungest($P \vee \neg Q \vee \neg R$):

- Reorder to $\neg R \vee \neg Q \vee P$
- Resolve with unit clause $\neg Q \vee R$ of R producing resolvent $\neg Q \vee \neg Q \vee P$.
- Remove double $\neg Q$.
- Return $\neg Q \vee P$.

The returned clause is a conflict clause.

How do we just “find u , the unit clause of $\neg L_1$ ”? That’s easy: at any point of time, the state of DPLL is remembered as a sequence of labelled literals

$$\langle (\ell_0, \kappa_0), (\ell_1, \kappa_1) \dots (\ell_n, \kappa_n) \rangle \quad (8.5)$$

... which represents the order in which the literals ℓ_i were made true. Each literal ℓ_i is derived by decision or by unit propagation: the label κ_i of ℓ_i is either “decision”, or it is the unit clause C that derived ℓ_i .) This is all the information we need.

ResolveYoungest takes as input a conflict clause and returns a new conflict clause with its youngest literal L''_1 a false literal of the current level.

In general, **ResolveYoungest** can be applied to an arbitrary conflict clause $c = L_1 \vee \dots$ with a literal L_1 of the current level, provided this literal is the negation of a unit literal. In that case, $\neg L_1$ was derived by a unit clause $u = \neg L_1 \vee K_2 \vee \dots$. In this case c and u

can be resolved on L_1 . As we have seen, the unit clause u contains at least two literals of the current level. It follows that the resolvent contains again, at least one false literal of the current level: K_2 . Therefore, an invariant of a procedure of iterated application of **ResolveYoungest** is that the produced clauses are conflict clauses with the youngest literal of the current level.

The idea now is to iterate **ResolveYoungest** for a number of steps, at each step resolving away the youngest unit literal of the current level, from the current conflict clause. Such an iteration would definitely have to end (see below), since each iteration replaces its youngest literal by strictly older literals. It definitely ends when the youngest literal of the current level is the negation of the decision literal of the current level – which has no u and hence there can be no more resolution. However, CDCL does something smarter: it iterates **ResolveYoungest** until a conflict clause is obtained with exactly one literal of the current level, *even if it is a unit*. The resulting clause is called the *learned clause* or the *Unique Implication Point (UIP)*. The procedure to compute it is called **ClauseLearning** and is represented in Procedure 8.7. Below, we illustrate this strategy and discuss its merits.

Procedure 8.7: Clause learning

```

function CLAUSELEARNING(conflict clause  $c$ )
  while  $c$  contains  $\geq 2$  literals of current level do    ▷  $\neq$  “while first isn’t decision”!
     $c \leftarrow \text{RESOLVEYOUNGEST}(c)$ 
  end while
  return  $c$ 
end function

```

Example 8.2.4

Example 8.2.2 continued.

$P \vee Q \vee R.$ $S \vee T \vee Q.$ $\neg Q \vee R.$ $P \vee \neg Q \vee \neg R.$ $\neg S \vee R.$	Choice: $\neg P$. – Choice: $\neg S$. – Choice $\neg T$. – Propagate: Q . – Propagate: R . Conflict!
--	--

ClauseLearning($P \vee \neg Q \vee \neg R$):

- Apply **ResolveYoungest**: resolve $\neg R$ with $\neg Q \vee R$.
- We obtain $\neg Q \vee P$. This is the UIP. Note that $\neg Q$ is not the decision literal of the current level, yet it *is* the only literal of the current level.
- **Return this clause.**

Here one resolution step suffices to produce the UIP $P \vee \neg Q$. Its youngest literal is $\neg Q$. It is the only one of the current level. (The second literal in the UIP is P which is the negation of the oldest decision literal.)

All clauses computed during the process are entailed by the theory and are conflict clauses in the current assignment (all literals are false).

At each step, we eliminate the youngest literal of the formula and replace it by older ones, hence the youngest literal in the new conflict clause is strictly older. This cannot go on forever, hence this algorithm terminates. At worst, the algorithm runs until the only literal of the current level is the negation of the current choice literal. But it may stop earlier as illustrated in the example.

The UIP is a clause $L_1 \vee \dots \vee L_k$ with one literal L_1 of the current level and all other literals of older levels. What to do with this clause? There are two uses. First, it is added to the clause database, to avoid that later executions repeat the erroneous derivation. Second, it is used for backjumping, as explained next.

Backjumping: non-chronological backtracking Suppose the learned clause is (ordered in age from young to old):

$$L_1 \vee \mathbf{L}_2 \vee \dots \vee L_k (k \geq 1)$$

L_1 is the youngest, of the current level n ; L_2 is the second youngest, of level $m < n$. At the current level n , this is a conflict clause.

CDCL performs a special sort of backtracking to the level m of $\mathbf{L}_2$. Backtracking to level m ($0 \leq m < n$) means undoing all assignments to variables of level $m+1, m+2, \dots, n$. (In contrast to normal backtracking, all assignments at level m , which we go back to, are kept.)

Since the level of L_1 is n , and the level of all other literals is $\leq m$, the learned clause is a unit clause at all levels $n-1, n-2, \dots, m+1, m$. Thus, m is the oldest level where the learned clause is a unit clause and its unit literal is L_1 . After undoing all assignments of levels $n, n-1, \dots, m+2, m+1$, backtracking proceeds by restarting the unit propagation process at level m with the learned clause as new unit clause. The backjump procedure used by CDCL is given in Procedure 8.8.

Procedure 8.8: Backjumping

function BACKJUMP(learned clause c)

 Determine m from c .

 Undo all assignments to literals of level $m+1, m+2, \dots, n$.

return m

end function

Example 8.2.5: Continuation of the example

	Choice: $\neg P$.
	– Choice: $\neg S$.
	– Choice $\neg T$.
$P \vee Q \vee R.$	– Propagate: Q .
$S \vee T \vee Q.$	– Propagate: R .
$\neg Q \vee R.$	Conflict! Learn $\mathbf{P} \vee \neg \mathbf{Q}$
$P \vee \neg Q \vee \neg R.$	Backtrack over $\neg T$ and $\neg S$.
$\neg S \vee R.$	– Propagate $\neg Q$.
$\mathbf{P} \vee \neg \mathbf{Q}$	– Propagate: R .
	– Choice $\neg S$.
	– Propagate T . Model!

Exercise 8.2.1 Continue the execution.

This is a *backjumping* algorithm, backtracking to level $m+1$. Indeed, rather than switching the choice literal of the backtrack level as in chronological backtracking to level m , CDCL continues with unit propagation at level m , with the guarantee that there is at least one new unit clause, namely the UIP.

A subtle but important difference between CDCL and DPLL is that CDCL has no *backtrack literals*. All values in the partial structure are decision literals or unit literals. Indeed, DPLL created a backtrack literal when switching a choice literal to false (without being able to justify this by a unit clause). In CDCL, such switching does not occur. What happens is that UP is restarted and derives at least one new literal with the UIP as unit clause. Hence, backtrack literals are not created.

Procedure 8.9: CDCL SAT

```

function CDCL( $\varphi$ )
  currentLevel  $\leftarrow$  0
  for  $p \in \Sigma_\varphi$  do
     $\mathcal{I}(p) \leftarrow \mathbf{u}$ 
  end for
  while true do
    conflictClause  $\leftarrow$  UP( $\mathcal{I}$ )
    if  $\mathcal{I}$  is a model then
      print  $\mathcal{I}$ 
      conflictClause  $\leftarrow$  NoModel( $\mathcal{I}$ )
    end if
    if conflictClause not null then
      if currentLevel = 0 then
        return UNSAT
      else
        learnedClause  $\leftarrow$  CLAUSELEARNING(conflictClause)
        ADDTOCLAUSEDATABASE(learnedClause)
        currentLevel  $\leftarrow$  BACKJUMP(learnedClause)
      end if
    else
      Choose an undefined literal  $L$ .
       $\mathcal{I}(L) \leftarrow \mathbf{t}$ 
      currentLevel  $\leftarrow$  currentLevel + 1
    end if
  end while
end function

```

Why is it so good to stop at the first UIP? After all, we could continue the iteration of **ResolveYoungest**. Each conflict clause with one literal of the current level computed by iterating **ResolveYoungest** is a candidate clause for backjumping. If we iterate all the way, we obtain a conflict clause with only one literal of the current level, namely the decision literal. This clause could also be used for backjumping.

It is easy to see that it is best to stop with the UIP. Indeed, during iteration of **ResolveYoungest**, literals of lower level (higher in the tree) are not resolved away. Hence, the set of them grows with each resolution step. This has two disadvantages. First, longer clauses do not propagate as often as short clauses. Second and probably worse, after the UIP was obtained, later resolution steps might introduce literals of a more recent level m' than the backtrack level m of the UIP. In this case, backjumping is less deep than with the UIP, to level m' rather than to level m .

This is illustrated below.

Example 8.2.6

ResolveYoungest past the UIP.

$ \begin{aligned} &P \vee Q \vee R. \\ &S \vee T \vee Q. \\ &\neg Q \vee R. \\ &P \vee \neg Q \vee \neg R. \\ &\neg S \vee R. \end{aligned} $	$ \begin{aligned} &\text{Choice: } \neg P. \\ & - \text{Choice: } \neg S. \\ & \quad - \text{Choice } \neg T. \\ & \quad \quad - \text{Propagate: } Q. \\ & \quad \quad - \text{Propagate: } R. \\ &\text{Conflict!} \end{aligned} $
---	---

ClauseLearning-v2($P \vee \neg Q \vee \neg R$):

- Resolve on $\neg R$ with $\neg Q \vee R$.
- Obtain $P \vee \neg Q$ (the UIP). Reorder by chronology: $\neg Q \vee P$.
- Resolve on $\neg Q$ with $S \vee T \vee Q$.
- Obtain $P \vee S \vee \mathbf{T}$: contains only the choice literal $\mathbf{T}$ of current level. Reordered by chronology: $T \vee S \vee P$.
- **Return this clause.**

The extra resolution step has introduced additional literal S of a non-current level, but S is of younger level than P . If $P \vee S \vee T$ is used for backjumping rather than $P \vee \neg Q$, backjumping will be to the level of S which is less deep.

Deeper backjumping may remove an exponentially larger part of the search tree.

Example 8.2.7: Backjumping with the alternative learned clause.

$ \begin{aligned} &P \vee Q \vee R. \\ &S \vee T \vee Q. \\ &\neg Q \vee R. \\ &P \vee \neg Q \vee \neg R. \\ &\neg S \vee R. \\ &\mathbf{P} \vee \mathbf{S} \vee \mathbf{T} \end{aligned} $	$ \begin{aligned} &\text{Choice: } \neg P. \\ & - \text{Choice: } \neg S. \\ & \quad - \text{Choice } \neg T. \\ & \quad \quad - \text{Propagate: } Q. \\ & \quad \quad - \text{Propagate: } R. \\ &\text{Conflict! Let's add } \mathbf{P} \vee \mathbf{S} \vee \mathbf{T}. \\ &\text{Backtrack over } \neg T. \\ & \quad - \text{Propagate } T. \\ & \quad - \text{Propagate: } Q. \\ &\dots \end{aligned} $
---	--

This is less efficient.

When does CDCL stop? CDCL stops after finding a model or when it discovers unsatisfiability. The latter occurs when it backtracks to level $m = 0$ and subsequent UP computes a conflict clause. In that case, CDCL returns UNSAT, just like DPLL.

Computing multiple models To adapt CDCL to generate multiple models, the standard technique is as follows. When a model $\mathfrak{A}$ is found, a clause is derived that forbids this model. One such clause is $\bigvee_{L \in \mathfrak{A}} \neg L$, the disjunction of negations of true literals of $\mathfrak{A}$. This is a conflict clause in $\mathfrak{A}$. CDCL then proceeds in the standard way after derivation of a conflict clause.

A better clause is the disjunction of the negation of the decision literals of $\mathfrak{A}$. This clause is a conflict clause as well, it is equivalent to the above clause but it is shorter and therefore, will more easily lead to unit propagation. We denote this clause as $\text{NoModel}(\mathfrak{A})$. The full CDCL algorithm for computing all models is Procedure 8.9.

8.2.3 Optimizations of CDCL

Implementation of SAT-solvers The fastest SAT solvers are optimized to the extreme:

- arrays for constant time access;
- datastructures were developed such that important parts fit into the *cache* of the processor as much as possible, to avoid swapping $\Rightarrow 10\text{-}40\times$ faster;
- no automated memory management, no recursive procedures; the program is in control of all memory-management;
- no algorithms more than linear time, except for the basic backtracking algorithm (which is exponential in the worst case);
- implemented in C or C++, no Java.

A number of crucial optimization techniques are discussed now.

Two-watched-literals technique How do we detect conflict and unit clauses without rechecking each clause after each assignment? The *two-watched-literals* technique serves to give access to clauses only when really necessary. The technique “watches” two non-false literals (true or unknown) in each clause. E.g.,

$$\underline{P^u} \vee \underline{Q^u} \vee \neg R^u \vee S^u \quad (8.6)$$

Note: atoms are annotated with their truth value here.

When a watched literal is made **false**, action is required. First, an undefined non-watched literal is searched. If found, it is selected as a new watch. E.g., Q is made false, $\neg R$ is the new watch:

$$\underline{P^u} \vee \underline{Q^f} \vee \neg R^u \vee S^u \quad \longrightarrow \quad \underline{P^u} \vee \underline{Q^f} \vee \neg R^u \vee S^u \quad (8.7)$$

If no candidate watch is found, we detected that the clause is a unit clause or conflict. If the other watched literal is unknown, it is a new unit literal. E.g., in a state where all literals except the watch P are false:

$$\underline{P^u} \vee \underline{Q^f} \vee \neg R^t \vee S^f \quad \longrightarrow \quad \text{unit propagation } P \quad (8.8)$$

Through a hashmap, we have constant-time access to the list of clauses where the negation of any given literal is watched. Hence, when the literal is made true, we immediately know the clauses where potential conflict or unit propagation takes place.

UP Heuristics When executing UP, it may happen that multiple unit clauses demand propagation at the same time. The ramifications of assigning a unit literal aren’t trivial (it can lead to a cascade of unit propagations), and therefore, we consider two polar-opposite heuristics:

- *Breadth-first*: use oldest unit clauses to propagate first.
- *Depth-first*: use youngest unit clauses first

Both strategies are equivalent in the sense that the execution flow after calling UP is the same either way: if neither leads to conflict, they derive exactly the same unit literals, and if one leads to a conflict, so will the other. However, they may lead to a different conflict clause.

Example 8.2.8

- | | |
|----------------------|---|
| $\neg A \vee B$ | |
| $\neg A \vee C$ | • Choose A |
| $\neg B \vee D$ | • Breadth-first UP $\Rightarrow$ conflict: $\neg B \vee \neg C$ |
| $\neg B \vee \neg D$ | • Depth-first UP $\Rightarrow$ conflict: $\neg B \vee \neg D$ |
| $\neg B \vee \neg C$ | |

It may therefore seem that the chosen strategy is of little importance. Unexpectedly, in CDCL it turns out to have a big impact. The reason is that the conflict clause produced by breadth-first tends to contain “older” literals. When combined with conflict clause learning, this causes *deeper* backtracking. Backtracking after depth-first UP is shallower.

Decision Heuristics: VSIDS VSIDS is a heuristic fully named *variable-state-independent decaying sum*. Each literal has a counter initialized to 0. When a conflict clause is added, the counters of its literals are incremented. At a decision point, the unassigned variable with the highest counter is chosen. Periodically, after N choices, all the counters are divided by a constant.

VSIDS is called a conflict-driven decision strategy. The effect of this strategy is that variables appearing in recent conflict clauses get higher priority. As a consequence, search tends to focus on difficult parts of the search space, and this proved to be a major advantage. This strategy dramatically improved performance by an order of magnitude.

The implementation is by maintaining a map indexed on literals. Updating is only needed when adding a conflict clause.

Clause deletions After a while, memory gets full with learned clauses. At regular occasions, clauses that were not recently used for unit propagation are deleted.

Random restarts The order of the selection of variables is important. A wrong initial selection can lead to exponentially larger search trees. Heuristics do not always make correct choices. However, due to the clause learning, more and more information becomes available; heuristic information about what are the important variables is accumulated in the VSIDS counters of literals.

During a run of CDCL, this information is not fully exploited because backtracking never restarts the search from scratch. Therefore, in modern solvers, search is stopped and is restarted using the learned clauses at regular times. The restarts happen at an exponential rate. That is, the n 'th run gets $O(2^n)$ time. Hence, initially the number of restarts is high, it slows down quickly.

8.2.4 Summary

CDCL made SAT solvers applicable to theories that are orders of magnitude larger than was feasible before.

Currently, progress has slowed down but is still steady. New techniques emerge. E.g., symmetry breaking turns out to be powerful tool. The symmetry breaker BreakID developed at KRR combined with state of the art SAT solvers won several times in the SAT competition. New techniques combine CDCL with parallel execution by splitting up the search space in a large number of small search problems. A solver of this kind built a 200-TB math proof [20].

8.3 Other forms of inference in PC

Most research on inference in PC is concentrated on the satisfiability/model generation problem. This problem is called the *SAT problem*.

Other useful forms of inference are:

- Deductive problems: deciding tautologicity and entailment. They can be reduced to SAT problems, as seen in Figure 6.1.
- Maximal satisfiability, a generalization of SAT.

MaxSAT
Input: a PC theory T or CNF formula φ
Output: a $\subseteq$ -maximal PC theory $T' \subseteq T$ that is satisfiable.

This is useful when a problem is overconstrained and we want a solution where as many of the constraints as possible are satisfied.

- Model counting

#SAT
Input: a PC theory T
Output: the number of models of T .

This form of inference is useful in the context of probabilistic reasoning. The ratio of the number of models of $T \cup \{\varphi\}$ over the number of models of T is the probability of φ in the context of the domain axiomatized by T .

8.4 Finite model expansion in IDP

IDP is a knowledge base system that supports multiple forms of inference for $\text{FO}(\cdot)$ theories and structures. The main form of inference is model expansion. We finish this section with a brief description of model expansion in IDP.

Model expansion
Input: theory T , partial structure $\mathcal{I}$
Output: an expansion $\mathfrak{A}$ of $\mathcal{I}$ that is a model of T , or UNSAT if there is no expansion.

The input for the model expansion of IDP is a theory T and a *partial structure* $\mathcal{I}$ of the vocabulary Σ of T . The approach for model expansion in IDP is by *ground and solve*. IDP reduces T together with a $\mathcal{I}$ to a propositional theory $T_{\mathcal{I}}^g$ called the *grounding of T in $\mathcal{I}$* , and then applies (extended) SAT solving to $T_{\mathcal{I}}^g$.

Figure 8.1 shows an application of IDP to the n -queens problem. It contains a vocabulary, theory and structure, as well as a Lua call to the model expansion procedure of IDP.

(Partial) structure A structure declaration in the IDP language specifies a partial structure with a finite set as value for all types. It specifies values or *partial values* for constant, predicate and function symbols.

In Figure 8.1, the structure $\mathcal{I}$ specifies the value $\{1, \dots, 10\}$ for type *pos* and a partial value for *queen(pos, pos)*, namely that *queen*(1, 5) is false and all other atoms *queen*(i, j) are undefined. This is expressed by the notion *queen*(*cf*) where *cf* stands for “certainly false”.

```

vocabulary V {
  type pos isa int.
  queen(pos,pos).
  diag(pos,pos,pos,pos).
}

theory T: V {
  { diag(x, y, x_2, y_2) <- x ~= x_2 & abs(x - x_2) = abs(y - y_2). }
  !x : ?1 y : queen(x,y).
  !y : ?1 x : queen(x,y).
  !x y x_2 y_2 : queen(x,y) & queen(x_2,y_2) => ~diag(x,y,x_2,y_2).
}

structure I: V {
  pos = {1..10}.
  queen<cf> = {1,5}.
}

procedure solve() {
  sol = onemodel(T,I,V)
  print(sol[V::queen])
}

```

Figure 8.1 – Example IDP file.

Model expansion for this input will compute models in which *queen*(1,5) is false. The value of *diag*/4 is unknown in *I*.

Let's now look at the six steps IDP takes to do model expansion.

8.4.1 Type inference.

IDP derives a type for every occurrence of every variable in the theory. This is done (mostly) in the standard way, by matching type declarations of symbols with the terms that occur as arguments in the theory. If the system derives a complete typing, it accepts the theory. Otherwise, it generates a type error. Type inference is useful for debugging and necessary for determining the domains of all logical variables in formulas. The use of subsorts in IDP makes type inference more complex and subtle. The manual discusses it.

8.4.2 Extend the input structure

Prior to grounding, the IDP system applies several procedures to expand/refine the input structure. Expansion does not increase the size of the domains of types but it turns unknown facts into known ones. The benefit of refining the input structure $\mathfrak{A}$ is that grounding leads to a propositional theory of smaller size.

One technique to expand $\mathfrak{A}$ is to compute defined predicates of a definition whose parameters are all known. In the example, *diag*(*pos*,*pos*,*pos*,*pos*) is such a predicate. *diag*(*x*,*y*,*u*,*v*) represents that positions (*x*,*y*) and (*u*,*v*) are on the same diagonal. This relation can be computed prior to solving. The result is stored in the structure as a value of the predicate *diag*.

For an example, consider the following constraint.

`!x y x_2 y_2 : queen(x,y) & queen(x_2,y_2) => ~diag(x,y,x_2,y_2).`

Normally, it would be grounded by instantiating the variables with all possible values, leading to 10^4 ground clauses of the form:

`queen(n_1,n_2) & queen(n_3,n_4) => ~diag(n_1,n_2,n_3,n_4).`

Exploiting knowledge of *diag*, this can be reduced to a much smaller number of smaller clauses, for each pair of positions $(n_1, n_2), (n_3, n_4)$ on the same diagonal:

`\neg queen(n_1,n_2) \lor \neg queen(n_3,n_4).`

Another approach is to refine $\mathfrak{A}$ by (symbolic) propagation methods. E.g., assume that it is given in T or $\mathfrak{A}$ that there is certainly a queen on position $(4,6)$. This can be expressed by an assignment $queen_{ct} = \{(4,6)\}$. By symbolic propagation, new facts can be derived: that all unknown atoms $queen(i,j)$ describing positions on the same row, column, and diagonals are certainly false. A symbolic propagation method is run on the theory to compute such facts and to expand the current partial interpretation $\mathfrak{A}$.

8.4.3 Simplify the input theory

This operation serves to bring the theory in a normalized form to facilitate grounding. The basic idea is as follows. All nested function terms are eliminated from the theory. The resulting theory contains only atoms of the form $P(x_1, \dots, x_n)$ and $F(x_1, \dots, x_n) = x$ with variables, no terms. This can be achieved by iterated application of the following equivalence-preserving rewrite rules:

$$P(\dots t \dots) \longrightarrow \exists x(t = x \wedge P(\dots x \dots)) \quad (8.9)$$

$$t = F'(\dots) \longrightarrow \exists x(t = x \wedge F'(\dots) = x) \quad (8.10)$$

$$t = F(\dots F'(\dots) \dots) \longrightarrow \exists x(F(\dots x \dots) = t \wedge F'(\dots) = x) \quad (8.11)$$

Another technique is to insert bounds for the quantified variables. IDP has a symbolic propagation module to derive bounds for quantified variables. In particular, for each quantified subformula $\forall \bar{x}\psi$ and $\exists \bar{x}\psi$, a bounds formula $\Psi_b[\bar{x}]$ is computed such that the value of $\{\bar{x} \mid \Psi_b[\bar{x}]\}$ is determined by $\mathcal{I}$. These bounds are computed in such a way that the following equivalences hold:

$$\forall \bar{x}\psi \Leftrightarrow \forall \bar{x}(\Psi_b[\bar{x}] \Rightarrow \psi) \quad (8.12)$$

$$\exists \bar{x}\psi \Leftrightarrow \exists \bar{x}(\Psi_b[\bar{x}] \wedge \psi) \quad (8.13)$$

During grounding, the value of the bound formulas are computed and used to instantiate the variables. An often essential component of these bound formulas are types.

Example 8.4.1

To illustrate the principle, assume the theory contains an axiom:

$$\forall x \forall y (R(x, y) \Rightarrow P(x, y)) \quad (8.14)$$

where P is an interpreted symbol of $\mathcal{I}$ and R is not interpreted. The bounds module of IDP will derive that $P^{\mathcal{I}}$ is an upperbound of R . In this case, $P(x, y)$ is a bound formula for

$$\exists x \exists y (R(x, y) \wedge \Psi) \quad (8.15)$$

| During grounding, x, y are substituted only by values $(a, b) \in P^{\mathcal{I}}$.

8.4.4 Ground

The grounding procedure consists of several steps. The first step is to eliminate quantifiers and variables and exploits the bounds as follows:

$$\forall \bar{x} : \Psi_b[\bar{x}] \Rightarrow \varphi[\bar{x}] \quad \longrightarrow \quad \bigwedge_{\bar{a} \in \{\bar{x} | \Psi_b[\bar{x}]\}^{\mathcal{I}}} \varphi[\bar{a}] \quad (\text{a conjunction}) \quad (8.16)$$

$$\exists \bar{x} : \Psi_b[\bar{x}] \wedge \varphi[\bar{x}] \quad \longrightarrow \quad \bigvee_{\bar{a} \in \{\bar{x} | \Psi_b[\bar{x}]\}^{\mathcal{I}}} \varphi[\bar{a}] \quad (\text{a disjunction}) \quad (8.17)$$

The second step is to simplify the resulting formulas and definitions using the Tseitin transformation.

Another simplification is that atoms with known value in $\mathcal{I}$ are replaced by their value, and the formula is simplified. E.g., if $P(d_1)$ is true in $\mathcal{I}$, the formula $P(d_1) \vee \Psi$ is replaced with **t**. In case Ψ is large, a serious benefit is made here. Such techniques are indispensable.

In the final step, atoms $P(d_1, \dots, d_n)$ and $F(d_1, \dots, d_n) = d$ are transformed into propositional symbols. Here $d_1, \dots, d_n, d$ are domain elements specified in $\mathfrak{A}$. The result is a propositional theory in a normalized version of PC($\cdot$), extending CNF with ground versions of aggregate expressions, constraints and definitions.

8.4.5 Solve

The solver of IDP is minisatID. It is called on the ground theory $T_{\mathfrak{A}}^g$ to compute one or more models of it. minisatID is an extension of a SAT solver called minisat, with additional propagators for (ground) definitions, aggregate expressions and constraint variables and constraints.

8.4.6 Post-evaluation

Models of the ground theory are translated back to models of the input theory T .

International context IDP belongs to a family of language and system paradigms with increasingly expressive declarative languages:

- Answer set programming (an extension of logic programming).
- SAT modulo theories: Microsoft built a powerful satisfiability system Z3 that is used in verification.
- Constraint programming systems for rich constraint programming languages such as MiniZinc systems.
- Satisfiability checking systems for verification: ProB, TLA+.

Two aspects are unique for IDP. First, it supports inductive definitions of a very rich kind (only Answer Set Programming has something similar). Second is that IDP is explicitly conceived as a knowledge base system, a system supporting a range of inferences on the same knowledge base.

8.5 Important for exam

Big question: I may ask to execute the CDCL algorithm on a specific SAT theory, and to explain the details. Knowledge of optimizations is a plus.

Small questions:

- What toCNF is and why it is exponential.
- What the Tseitin transformation is and why it is not equivalence-preserving.

What I think you need to know/understand for solving these questions:

- Knowledge of the standard equivalence preserving transformation to CNF; why it is exponential; why it is to be expected that polynomial transformations do not exist.
- Knowledge of Tseitin transformation to CNF; why it is not equivalence preserving; the weaker correctness result.
- Knowledge of DPLL and CDCL

Chapter 9

Algorithms for CTL and LTL model checking

As mentioned at the end of chapter 4, satisfaction of CTL* formulas is quite expensive and hence not often used. LTL and CTL do have their uses, and hence algorithms for checking such formulas will now be discussed.

This chapter is based on the corresponding chapter of the book of Huth and Ryan [17].

9.1 CTL model checking algorithm

9.1.1 Background

In this section, we see an algorithm for solving the following inference problem:

CTL model checking in one state
Input: a transition structure $\mathcal{M} = \langle S, \rightarrow, L \rangle$, a state $s_0 \in S$, a CTL formula ϕ
Output: $(\mathcal{M}, s_0 \models \phi)$ (true if it holds, false otherwise).

Instead, we resolve a related – seemingly harder – problem.

CTL model checking in all states
Input: $\mathcal{M}$, a CTL formula ϕ
Output: $\{s \in S \mid \mathcal{M}, s \models \phi\}$. We denote this set as S_ϕ .

The output of the second problem consists of all states $s \in S$ in which the CTL state formula ϕ is satisfied. Clearly, the model checking problem can be reduced to the second sort of problem. The algorithm presented below solves both sorts of problems.

Adequate set The CTL formulas supported by the algorithm are the formulas built from the adequate set $\{\text{EG}, \text{EU}, \text{EX}, \wedge, \neg, \perp\}$: note that there is no A, no $\vee$ and no $\top$. The following proposition specifies the CTL satisfaction relation as defined in CTL* for this adequate set.

Proposition 9.1.1

- $\mathcal{M}, s \models p$ iff $p \in L(s)$.

- $\mathcal{M}, s \models \psi_1 \wedge \psi_2$ iff $\mathcal{M}, s \models \psi_1$ and $\mathcal{M}, s \models \psi_2$.
- $\mathcal{M}, s \models \neg\psi_1$ iff $\mathcal{M}, s \not\models \psi_1$.
- $\mathcal{M}, s \models \text{EX } \psi_1$ if there exists $s \rightarrow s_1$ such that $\mathcal{M}, s_1 \models \psi_1$.
- $\mathcal{M}, s \models \text{E}(\psi_1 \text{ U } \psi_2)$ iff there exists a finite path $s = s_0 \rightarrow s_1 \rightarrow \dots \rightarrow s_n$ ($n \geq 0$) such that $\mathcal{M}, s_i \models \psi_1$ for every $i \in \{0, \dots, n-1\}$ and $\mathcal{M}, s_n \models \psi_2$.
- $\mathcal{M}, s \models \text{EG } \psi_1$ iff there exists an infinite path $s_0 \rightarrow s_1 \rightarrow \dots$ such that $s = s_0$ and $\mathcal{M}, s_i \models \psi_1$ for every $i \in \mathbb{N}$.

9.1.2 Without fairness constraints

The proposition suggests to compute $\mathcal{M}, s \models \phi$ recursively on the structure of ϕ . The algorithm computes sets $S_\psi = \{s \in S \mid \mathcal{M}, s \models \psi\}$ recursively, for increasing subformulas ψ of ϕ . Below, we discuss the steps in the algorithm and their complexity.

Let V be the number of states and E the number of edges in $\rightarrow$. We make the following complexity assumptions about operations on data structures:

- For each $s \in S$, the sets $\{s' \mid s' \rightarrow s\}$ (departure states from which an edge arrives in s) and $\{s' \mid s \rightarrow s'\}$ (arrival states to which an edge leaves in s) can be generated in time linear in their size. (There are quite a few data structures where this is quadratic.)
- For each $s \in S$ and $p \in \Sigma$, checking $p \in L(s)$ is $O(1)$ (constant time).
- For each S_ψ , checking $s \in S_\psi$ and adding s to S_ψ is $O(1)$.

It is routine to build data structures that satisfy these conditions. Procedure 9.1 computes S_ψ by a case analysis on ψ .

Exercise 9.1.1 In the worst case, the sets S_{ψ_1} and $\{s \mid s \rightarrow s'\}$ contain $O(V)$ elements. Why is the complexity not $O(V^2)$, i.e., quadratic? Argue.

Theorem 9.1.1

For each CTL formula ϕ , $s \in S_\phi$ as calculated in Procedure 9.1 if and only if $\mathcal{M}, s \models \phi$.

No proof. This theorem is proven by induction on the structure of ψ , using the definition of satisfaction of path and state formulas in CTL* (Chapter 4).

Remark 9.1.1

In the book of Huth and Ryan, a different way is used to represent this algorithm. Rather than computing sets S_ϕ , Huth and Ryan extend the set of labels of a state s with CTL formulas that are true in s . Here S_ϕ corresponds to the set of states labeled with ϕ . Of course, this boils down to the same.

Complexity of the algorithm Assume that ψ has f subformulas. A run is needed over all f subformulas of ψ (the size of the parse tree). Each step is at most $O(V \times (V + E))$. It follows that the algorithm has complexity $O(f \times V \times (V + E))$.

This algorithm is quadratic in the size of $\mathcal{M}$. This is too expensive. Indeed, the size of $\mathcal{M}$ tends to be very large. The only non-linear step in the algorithm is for EG ψ_1 . Procedure 9.2 is an alternative algorithm to compute this set.

Procedure 9.1: Recursive CTL model checking

Given a CTL formula ψ and a transition structure $\mathcal{M} = \langle S, \rightarrow, L \rangle$.

- $\psi = \perp$: $S_\psi = \emptyset$;
- $\psi = p \in \Sigma$: $S_\psi = \{s \in S \mid L(s) = p\}$;
To compute S_p , loop over the elements $s \in S$ and check $p \in L(s)$. S_p can be computed in $O(V)$ since checking $p \in L(s)$ and adding s to S_p is $O(1)$.
- $\psi = \neg\psi_1$: $S_\psi = S \setminus S_{\psi_1}$;
Loop over the elements $s \in S$; if $s \notin S_{\psi_1}$ add s to S_ψ . This is $O(V)$.
- $\psi = \psi_1 \wedge \psi_2$: $S_\psi = S_{\psi_1} \cap S_{\psi_2}$;
Loop over the elements $s \in S$; if $s \in S_{\psi_1}, s \in S_{\psi_2}$, add s to S_ψ . This is $O(V)$.
- $\psi = \text{EX } \psi_1$: $S_\psi = \{s \in S \mid \exists s' : s \rightarrow s' \wedge s' \in S_{\psi_1}\}$;
Loop over the elements $s' \in S_{\psi_1}$; for every $s \rightarrow s'$, add s to S_ψ . This is $O(V + E)$.
- $\psi = \text{E}(\psi_1 \text{ U } \psi_2)$: $S_\psi = \{s \in S \mid \exists s_0, \dots, s_n : s = s_0 \rightarrow s_1 \rightarrow \dots \rightarrow s_n \wedge \forall i \in \{0, n-1\} : s_i \in S_{\psi_1} \wedge s_n \in S_{\psi_2}\}$.
The set S_ψ is computed by the following algorithm. It uses a queue datastructure Q for which pushing and popping an element is $O(1)$.

```

 $S_\psi := Q := S_{\psi_2}$ ;
while  $Q \neq \emptyset$  do
    pop  $s$  from  $Q$ ;
    for every  $s' \rightarrow s$ , if  $s' \in S_{\psi_1}$  and  $s' \notin S_\psi$ , add  $s'$  to  $S_\psi$  and to  $Q$ .
end while

```

This algorithm computes S_ψ in $O(V + E)$.

- $\psi = \text{EG } \psi_1$: $S_\psi = \{s \mid \exists \pi = s_0 \rightarrow s_1 \rightarrow \dots : s = s_0 \wedge \forall i \in \mathbb{N} : s_i \in S_{\psi_1}\}$.
The set S_ψ can be computed as follows.

```

 $S_\psi := S_{\psi_1}$ ;
for  $s \in S_\psi$  do
    if  $s$  has no edge  $s \rightarrow s'$  such that  $s' \in S_\psi$ 
    then delete  $s$  from  $S_\psi$ ; restart the for loop.
end for

```

Every time an element of S_ψ is deleted, the for loop is restarted. The for loop takes $O(V + E)$ in the worst case. The for loop is restarted potentially V times. Hence, this algorithm runs in $O(V \times (V + E))$.

Exercise 9.1.2 Explain why this algorithm is correct.

In conclusion, the complexity of the refined algorithm is $O(f \times (V + E))$ and is linear in the size of $\mathcal{M}$. Unfortunately, even though the algorithm is linear, the model $\mathcal{M}$ is typically exponential in the number of variables and the number of processes (i.e., components that execute in parallel). Adding one boolean variable doubles the size of the model and the cost to verify it. A lot of research is going on to overcome this state explosion problem.

9.1.3 With fairness constraints

A fairness constraint ψ is a path constraint that ψ should be true infinitely often (see Definition 4.3.1). As discussed before, such constraints cannot be expressed in a state transition graph.

Some model checking systems support adding explicit fairness constraints to the specifica-

Procedure 9.2: Linear algorithm for $S_{\text{EG } \psi_1}$

Given a CTL formula ψ_1 .

1. Compute $\mathcal{M}'$ by deleting all states in which ψ_1 is false:

$$S' = S_{\psi_1} \quad \text{and} \quad (\rightarrow') = \{(s, s') \in S'^2 \mid s \rightarrow s'\} \quad (9.1)$$

2. Compute all maximal *strongly connected components* (SCCs) of $\rightarrow'$.

A strongly connected component of a graph is a set of states such that each state in it is *reachable* from (but not necessarily directly connected to) each other state. Strongly connected components of a graph can be computed with Tarjan's algorithm in $O(V + E)$.

3. Compute $S_{\text{EG } \psi_1}$ as the set of states in $\mathcal{M}'$ that can reach a strongly connected component.

This algorithm runs in $O(V + E)$.

tion. One example is the system NuSMV. In that case, the specification of the dynamic system is a pair $\langle \mathcal{M}, C \rangle$ of a transition structure $\mathcal{M}$ and a set C of fairness constraints.

Definition 9.1.1: Paths of a transition structure with fairness constraints

Paths of $\langle \mathcal{M}, C \rangle$ are paths of $\mathcal{M}$, but only those that satisfy all fairness constraints $c \in C$.

Indeed, when we quantify over the paths of such a system, the paths that can be walked in $\mathcal{M}$ but don't satisfy C have seemingly vanished.

Definition 9.1.2

c is a *simple fairness constraint* if c is a CTL (state) formula.

Recall that a state formula is also a path formula. Each state formula c characterizes a set S_c of states. A simple fairness constraint c expresses that paths should pass infinitely often through states of S_c .

Definition 9.1.3

The paths of a system $\langle \mathcal{M}, C \rangle$ with C a set of simple fairness constraints are all paths of $\mathcal{M}$ that pass infinitely often through states of S_c , for every $c \in C$.

To implement model checking in LTL and CTL*, it suffices to express fairness constraints as formulas and apply the existing model checking algorithms. As seen before, quantification over fair paths can be expressed in CTL*, using

$$A[(G F c_1) \wedge \cdots \wedge (G F c_n) \Rightarrow \psi] \quad (9.2)$$

$$E[(G F c_1) \wedge \cdots \wedge (G F c_n) \wedge \psi] \quad (9.3)$$

Thus, there is no need to extend model checking algorithms for LTL and CTL* to handle fairness. For CTL formulas φ , the situation is different, as the resulting formula is not a CTL formula. A different approach is followed.

Extending CTL* with fairness quantifiers Given a set C of fairness constraints, we extend CTL* with path quantifiers A_C and E_C . They quantify over fair paths w.r.t. C .

Definition 9.1.4

We extend the standard state formula satisfaction relation with two additional rules:

- $\mathcal{M}, s \models A_C[\psi]$ if for each path π of $\mathcal{M}$ fair to C , it holds that $\mathcal{M}, \pi \models \psi$.
- $\mathcal{M}, s \models E_C[\psi]$ if for some path π of $\mathcal{M}$ fair to C , it holds that $\mathcal{M}, \pi \models \psi$.

Proposition 9.1.2

Given a system $\langle \mathcal{M}, C \rangle$, and φ a CTL* formula, let φ_C be the formula obtained by transforming A into A_C and E into E_C .

$(\langle \mathcal{M}, C \rangle, s) \models \varphi$ iff $\mathcal{M}, s \models \varphi_C$.

CTL with fairness quantifiers The CTL* quantifiers A_C and E_C induce the following additional CTL connectives with fairness:

$$A_C X, \quad E_C X, \quad A_C F, \quad E_C F, \quad A_C G, \quad E_C G, \quad A_C U, \quad E_C U \quad (9.4)$$

One can show as before that the following is an adequate set:

$$\{E_C X, E_C G, E_C U, \wedge, \neg, \perp\}. \quad (9.5)$$

Moreover, the following proposition shows that $E_C X$ and $E_C U$ can be expressed in terms of EX , EU and $E_C G$.

Proposition 9.1.3

- $E_C[\phi U \psi] \equiv E[\phi U (\psi \wedge E_C G \top)]$
- $E_C X \psi \equiv EX(\psi \wedge E_C G \top)$

The proof is based on the fact that a fairness property is a property of the tail of a path. For any tail of a path, it holds that the path is fair iff this tail is fair. Hence, there is a fair path satisfying $\phi U \psi$ if there is a finite path to a state satisfying ψ that can be extended to a path that is fair with respect to C .

Combining the above results, we find that a decision problem $(\langle \mathcal{M}, C \rangle, s) \models \varphi$ can be reduced to the problem $\mathcal{M}, s \models \varphi'$ where φ' uses only standard CTL operators and $E_C G$.

To compute $S_{E_C G \phi}$ we use a variant of Procedure 9.2. In a prior step to model checking in $\langle \mathcal{M}, C \rangle$, we compute S_c , for every $c \in C$. Since c is a CTL formula, the standard algorithm can be used to compute S_c . This need to be done only once for $\langle \mathcal{M}, C \rangle$. Then, in the recursive model checking algorithm (Procedure 9.1 with Procedure 9.2), plug in Procedure 9.3 for computing $S_{E_C G \phi}$.

Exercise 9.1.3 *Given an argument for the correctness of the modified algorithm. In particular, explain how to construct for each state in $S_{E_C G \phi}$ a fair path on which ϕ is globally true.*

Total complexity of the model checking algorithm with fairness is $O(n \times f \times (V + E))$, with n the number of fairness constraints. This algorithm is linear in the size of $\mathcal{M}$.

Procedure 9.3: Linear algorithm for $S_{ECG\phi}$

1. Compute $\mathcal{M}'$ by restricting S to S_ϕ (which has been recursively computed). That is, delete states in which ϕ is false and their edges. This is as before.
2. Compute all maximal SCCs of $\mathcal{M}'$ using Tarjan's algorithm. This is as before.
3. Indicate the subset of “fair” maximal SCCs: a maximal SCC is fair to C if for every $c \in C$, it holds that $SCC \cap S_c \neq \emptyset$. I.e., some state in SCC satisfies the state formula c .
In an SCC, there exists a loop that passes over all its states. It is obvious then that each finite path in $\mathcal{M}'$ that reaches a “fair” maximal SCC can be extended into a path that is fair to C .
4. Finally, compute $S_{ECG\phi}$ as the set of states in $\mathcal{M}'$ that can reach a fair SCC. This is basically as before.

This procedure can be performed in $O(V + E)$.

9.2 An LTL model checking algorithm

In this section, we develop an algorithm implementing the following inference.

LTL model checking

Input: a system $\langle \mathcal{M}, C \rangle$ with C consisting of simple fairness constraints, $s_0 \in S$ and LTL formula Φ .

Output: $(\mathcal{M}, s_0 \models_{AC} \Phi)$.

Recall that a simple fairness constraint is a path constraint characterized by a CTL formula φ . It is the constraint that a path should pass infinitely often through states s such that $\mathcal{M}, s \models \varphi$. The inference problem to be solved is to verify whether $\pi \models \Phi$ for each C -fair path $\pi = s_0 \rightarrow \dots$.

Notice that $(\langle \mathcal{M}, C \rangle, s_0) \models A \Phi$ iff $\mathcal{M}, s_0 \models_{AC} \Phi$. Since an LTL formula Φ does not contain path quantifiers.

The adequate set We solve the inference problem for LTL formulas using the adequate set of LTL connectives $\{\neg, \wedge, X, U\}$. Recall that $F\alpha \equiv \top U \alpha$; $G\alpha \equiv \neg F \neg \alpha, \dots$

LTL model checking: the idea The algorithm presented below was developed by Vardi & Wolper in 1984 [29] and uses techniques from automata theory. The strategy of the algorithm is to search for a C -fair path π in $\mathcal{M}$ such that $\pi \models \neg \Phi$. If it succeeds, then $\mathcal{M}, s_0 \not\models_{AC} \Phi$. If it fails, then $\mathcal{M}, s_0 \models_{AC} \Phi$.

Example 9.2.1

As a running example, consider the LTL model checking problem in Figure 9.1. The problem is to decide $\mathcal{M}, q_3 \models_C \neg(a U b)$. Here the set of fairness constraints C is empty.

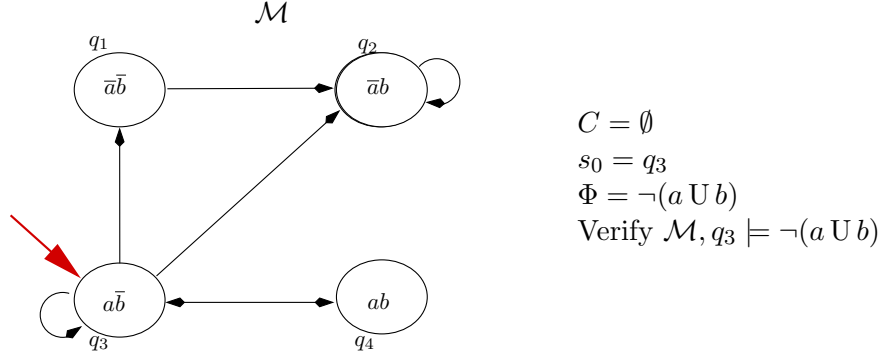


Figure 9.1 – An LTL model checking problem

The answer to this decision problem is **f**: $\mathcal{M}, q_3 \not\models A \neg(a U b)$. Evidence is provided by paths satisfying $a U b$ such as $(q_3 \rightarrow)^* \rightarrow q_4 \rightarrow \dots$ and $(q_3 \rightarrow)^* \rightarrow q_2 \rightarrow \dots$.

9.2.1 Intuition

Definition 9.2.1: Formula set and annotated path

Let $Sub(\Phi)$ denote the set of all subformulas of Φ and the negations of these subformulas. We define the *LTL formula set at i* of path π as the set $Q_i = \{\varphi \in Sub(\Phi) \mid \pi^i \models \varphi\}$. We define the *annotated path* of π as the sequence:

$$\left(\begin{array}{c} Q_0 \\ s_0 \end{array} \right) \rightarrow \dots \rightarrow \left(\begin{array}{c} Q_n \\ s_n \end{array} \right) \rightarrow \dots$$

where Q_i is the LTL formula set of π at i .

Each path π determines one unique annotated path. The strategy of the algorithm is to compute a new transition structure $\mathcal{M}^\times$ and a set $C^\times$ of fairness constraints, such that:

- The states of $\mathcal{M}^\times$ are of the form $\left(\begin{array}{c} Q \\ s \end{array} \right)$ with s a state of $\mathcal{M}$ and $Q \subseteq Sub(\Phi)$;
- The set of $C^\times$ -fair paths $\pi' = \left(\begin{array}{c} Q_0 \\ s_0 \end{array} \right) \rightarrow \dots \rightarrow \left(\begin{array}{c} Q_n \\ s_n \end{array} \right) \rightarrow \dots$ of $\mathcal{M}^\times$ is exactly the set of annotated C -fair paths $\pi = s_0 \rightarrow \dots \rightarrow s_n \rightarrow \dots$ of $\mathcal{M}$.

With $\mathcal{M}^\times, C^\times$ we are then able to solve our problem $\mathcal{M}, s_0 \models A_C \Phi$. Indeed, $\mathcal{M}, s_0 \not\models A_C \Phi$ iff there is a C -fair path $\pi = s_0 \rightarrow \dots$ in $\mathcal{M}$ such that $\mathcal{M}, \pi \models \neg \Phi$ iff there is a $C^\times$ -fair path $\pi' = (s_0, Q_0) \rightarrow \dots$ of $\mathcal{M}^\times$ such that $\neg \Phi \in Q_0$.

An algorithmic solution to the problem is to compute $S_{E_{C^\times} G \top}$ in $\mathcal{M}^\times$, and to check if it contains a state (s_0, Q_0) with $\neg \Phi \in Q_0$.

We have reduced the LTL model checking problem in $\langle \mathcal{M}, C \rangle$ to CTL model checking in $\langle \mathcal{M}^\times, C^\times \rangle$ and $C^\times$.

We now discuss how to construct $\mathcal{M}^\times = \langle S^\times, \rightarrow^\times, L \rangle$.

Construction of $S^\times$. The product state space $S^\times$ consists of certain pairs (s, Q) with $s \in S$ and $Q \subseteq Sub(\Phi)$.

To reduce search space, $S^\times$ is as small as possible. This is done by eliminating pairs (s, Q) that cannot occur in an annotated path; i.e., pairs (s, Q) for which no path $\pi = s \rightarrow \dots$

can exist such that $Q = \{\varphi \in \text{Sub}(\Phi) \mid \pi \models \varphi\}$.

There are two types of inconsistent pairs (s, Q) . One is where Q is a logically inconsistent set. The second one is where s and Q disagree on the value of some propositional symbol p . This is explained below.

Let π be a path and Q the set of elements of $\text{Sub}(\Phi)$ such that $\pi \models \psi$. I.e., $Q = \{\psi \in \text{Sub}(\Phi) \mid \pi \models \psi\}$. Q can be seen to satisfy certain consistency conditions.

- $\pi \models \gamma$ or $\pi \models \neg\gamma$. Hence, for each subformula γ of Φ , either $\gamma \in Q$ or $\neg\gamma \in Q$.
- It holds that $\pi \models \gamma \wedge \delta$ iff $\pi \models \gamma$ and $\pi \models \delta$. Therefore, Q contains $\gamma \wedge \delta$ iff it contains γ and δ .
- If $\pi \models \gamma \cup \delta$, then $\pi \models \gamma$ or $\pi \models \delta$. Hence, if Q contains $\gamma \cup \delta$, then it contains γ or δ .
- If $\pi \models \neg(\gamma \cup \delta)$, then $\pi \models \neg\delta$. Hence, if $\neg(\gamma \cup \delta) \in Q$ then $\neg\delta \in Q$.

These are the conditions for the consistency of an LTL formula sets Q .

Closed formula sets Let $\text{Sub}(\Phi)$ denote the set of all subformulas of Φ and the negations of these subformulas.

Definition 9.2.2

A closed formula set $Q \subseteq \text{Sub}(\Phi)$ of Φ satisfies:

- It contains for each subformula γ of Φ either γ or $\neg\gamma$.
- For each $\gamma \wedge \delta \in \text{Sub}(\Phi)$, $\gamma \wedge \delta \in Q$ iff $\gamma \in Q$ and $\delta \in Q$.
- If $\gamma \cup \delta \in Q$ then $\gamma \in Q$ or $\delta \in Q$.
- If $\neg(\gamma \cup \delta) \in Q$ then $\neg\delta \in Q$.

Denote the set of closed formula sets of Φ by $\mathcal{C}(\Phi)$.

Observe that each closed formula set contains either p or $\neg p$, for each symbol p in Φ .

Definition 9.2.3

We call a state s of $\mathcal{M}$ *consistent* with a closed formula set Q if for each atom p that appears in Φ , $p \in L(s)$ iff $p \in Q$. We denote this by $s \parallel Q$.

If s is not *consistent* with Q then Q contains the negation of a literal that is true in s . Hence, no path in s can satisfy all formulas in Q .

Definition 9.2.4

Define $S^\times := \{(s, Q) \mid s \in S \wedge Q \in \mathcal{C}(\Phi) \wedge s \parallel Q\}$.

Example 9.2.2

The running example: closed formula sets. The closed formula sets are the maximal consistent subsets of $\{a, \neg a, b, \neg b, a \cup b, \neg(a \cup b), \neg\neg(a \cup b)\}$.

We use a compact notation for these sets based on the following notations. We denote $\alpha := (a \cup b)$. We use $\bar{\delta}$ to denote $\neg\delta$. A closed formula set contains $\neg\neg(a \cup b)$ iff it does not contain $\neg(a \cup b)$ iff it contains $a \cup b$. We do not write $\neg\neg(a \cup b)$. It is understood

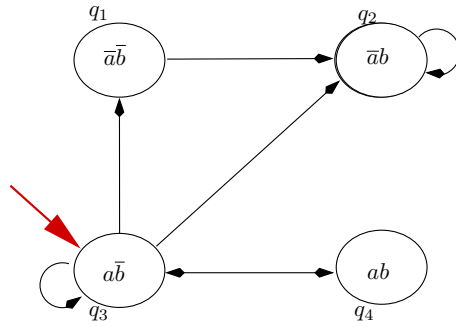
that $\neg\neg(a \cup b)$ belongs to a set if $a \cup b$ does.

E.g., $\bar{a}b\bar{\alpha}$ corresponds to $\{\neg a, b, \neg(a \cup b)\}$. This is not a closed formula set due to b and $\neg(a \cup b)$. E.g., $ab\alpha$ corresponds to $\{a, b, a \cup b, \neg\neg(a \cup b)\}$. This is a closed formula set.

The closed formula sets correspond to all combinations of $\{a, \bar{a}\} \times \{b, \bar{b}\} \times \{\alpha, \bar{\alpha}\}$ except $\bar{a}\bar{b}\alpha$ and $ab\bar{\alpha}$ and $\bar{a}b\bar{\alpha}$. That is, $\mathcal{C}(\Phi) = \{ab\alpha, \bar{a}b\alpha, \bar{a}\bar{b}\bar{\alpha}, \bar{a}b\bar{\alpha}, \bar{a}\bar{b}\bar{\alpha}\}$

Example 9.2.3

The running example: $S^\times$



$$\mathcal{C}(\Phi) = \{ \bar{a}\bar{b}\bar{\alpha}, \bar{a}b\alpha, \bar{a}\bar{b}\alpha, \bar{a}b\bar{\alpha}, ab\alpha \}$$

What are the consistent pairs?

- $q_1 \parallel: \bar{a}\bar{b}\bar{\alpha}$
- $q_2 \parallel: \bar{a}b\alpha$
- $q_3 \parallel: \bar{a}\bar{b}\alpha, \bar{a}b\alpha$
- $q_4 \parallel: ab\alpha$

$$S^\times = \{(q_1, \bar{a}\bar{b}\bar{\alpha}), (q_2, \bar{a}b\alpha), (q_3, \bar{a}\bar{b}\bar{\alpha}), (q_3, \bar{a}b\alpha), (q_4, ab\alpha)\}$$

We denote these pairs more compactly as

$$S^\times = \{q_1, q_2, q'_3, q_3, q_4\}$$

The states q_1, q_2, q_4 correspond to unique states of $S^\times$. q_3 is the only state that was duplicated. Why? Because each of q_1, q_2, q_4 trivially determines whether α is true or not in a path starting in it, and q_3 does not.

In general, if Φ is large, then each state s might be duplicated many times. Vice versa, each closed formula set Q might be consistent with many or with no states $s \in S$, and hence, might be duplicated many times or may not occur at all in $S^\times$. But such situations do not arise in the running example.

Exercise 9.2.1 Think of an example where state sets get duplicated.

Construction of $\rightarrow^\times$. We design $\rightarrow^\times$ as small as possible such that if $(s, Q) \rightarrow^\times (s', Q')$, then such a transition could occur in an annotated path. Cases of impossible transitions are:

- s' is not a possible successor of s : $s \not\rightarrow s'$
- Q' is not a possible successor of Q :
 - $Xp \in Q$ and $\neg p \in Q'$

- $\neg Xp \in Q$ and $p \in Q'$
- $p \cup q \in Q$, $\neg q \in Q$ but $\neg(p \cup q) \in Q'$
- $\neg(p \cup q) \in Q$, $p \in Q$ but $p \cup q \in Q'$

Exercise 9.2.2 Explain.

Definition 9.2.5

For all $(s, Q), (s', Q') \in S^\times$, there is a transition $(s, Q) \rightarrow^\times (s', Q')$ if all following conditions hold:

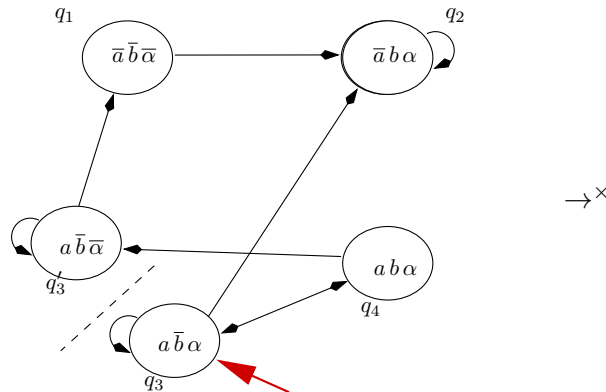
- $s \rightarrow s'$ (consistency with $\mathcal{M}$)
- Q' is a consistent successor of Q :
 - If $X\delta \in Q$ then $\delta \in Q'$.
 - If $\neg X\delta \in Q$ then $\neg\delta \in Q'$.
 - If $\gamma \cup \delta \in Q$ and $\neg\delta \in Q$, then $\gamma \cup \delta \in Q'$.
 - If $\neg(\gamma \cup \delta) \in Q$ and $\gamma \in Q$ then $\neg(\gamma \cup \delta) \in Q'$.

Example 9.2.4

The running example: $\rightarrow^\times$ in $S^\times := \{(q_1, \bar{a}\bar{b}\bar{\alpha}), (q_2, \bar{a}b\alpha), (q_3, a\bar{b}\bar{\alpha}), (q_3, a\bar{b}\alpha), (q_4, ab\alpha)\}$

$\rightarrow^\times$ is:

- $(q_1 =)(q_1, \bar{a}\bar{b}\bar{\alpha}) \rightarrow^\times (q_2, \bar{a}b\alpha)$
- $(q_2 =)(q_2, \bar{a}b\alpha) \rightarrow^\times (q_2, \bar{a}b\alpha)$
- $(q'_3 =)(q_3, a\bar{b}\bar{\alpha}) \rightarrow^\times (q_1, \bar{a}\bar{b}\bar{\alpha}), (q_3, a\bar{b}\bar{\alpha})(= q'_3)$
- $(q_3 =)(q_3, a\bar{b}\alpha) \rightarrow^\times (q_2, \bar{a}b\alpha), (q_4, ab\alpha), (q_3, a\bar{b}\alpha)(= q_3)$
- $(q_4 =)(q_4, ab\alpha) \rightarrow^\times (q_3, a\bar{b}\alpha)(= q_3), (q_3, a\bar{b}\bar{\alpha})(= q'_3)$



Are we done? Unfortunately not.

Recall, we are designing $\mathcal{M}^\times$ such that its paths are annotated paths of $\mathcal{M}$. In particular, it should be the case that if $(s, Q) \rightarrow^\times \dots$ is a path of $\mathcal{M}^\times$, then $Q = \{\varphi \in \text{Sub}(\Phi) \mid (s \rightarrow \dots) \models \varphi\}$.

The transition relation $\rightarrow^\times$ in Example 9.2.4 still has paths $\pi = (s, Q) \rightarrow \dots$ that are not annotated paths of $\mathcal{M}$, that is such that for some $\psi \in Q$, it holds that $\pi \not\models \psi$.

One such a path is the loop $\pi = (q_3, a\bar{b}\alpha) \rightarrow (q_3, a\bar{b}\alpha) \rightarrow \dots$. It holds that $\pi \models a W b$ but $\pi \not\models a U b$.

To eliminate this path, we should forbid paths with an infinite tail containing $a U b, a, \neg b$. How? By using a fairness condition!

Fairness conditions $C^\times$ A fairness condition is specified here as a set c of states. It expresses that a fair path $\pi = s_0 \rightarrow \dots$ should visit infinitely often an element of c . That is, for infinitely many i , $s_i \in c$.

There are two sources of fairness constraints: $C^\times = C^U \cup C'$.

One set of fairness constraints C^U . Their source are the until statements in $Sub(\Phi)$. No path π should have a tail π^i in which two formulas $\gamma U \delta$ and $\neg \delta$ belong to Q_j for all $j \geq i$. Notice that if $\gamma U \delta \in Q_j, \neg \delta \in Q_j$ then since Q_j is a closed formula set, it holds that $\gamma \in Q_j$. Hence, this is a path in which $\gamma W \delta$ is true but not $\gamma U \delta$ then the precondition γ remains true forever and the postcondition δ never becomes true.

Equivalently, any path should visit infinitely often the set of states $c^{\gamma U \delta} = \{(s, Q) \in S^\times \mid \neg(\gamma U \delta) \in Q \text{ or } \delta \in Q\}$.

Definition 9.2.6

Define $C^U = \{c^{\gamma U \delta} \mid \gamma U \delta \in Sub(\Phi)\}$.

Another source of fairness constraints is C , the original set of (simple) fairness constraints of the LTL model checking problem. A simple fairness constraint is a path constraint characterized by a CTL formula φ_c , or equivalently, by the set $S_{\varphi_c} = \{s \in S \mid \mathcal{M}, s \models \varphi_c\}$ of states of $\mathcal{M}$ satisfying φ_c . The fairness path constraint is that the path should pass infinitely often through states of S_{φ_c} .

c is to be translated into a fairness constraint for $S^\times$: $c^\times = \{(s, Q) \in S^\times \mid s \in S_{\varphi_c}\}$.

Definition 9.2.7

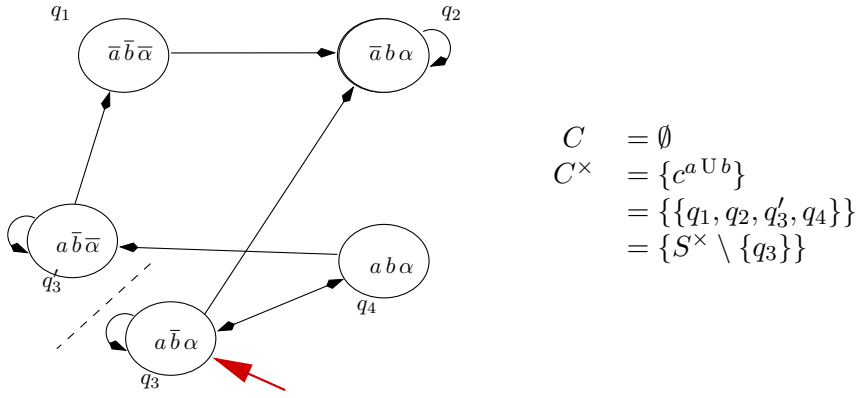
Define $C' = \{c^\times \mid c \in C\}$.

Definition 9.2.8

Define $C^\times = \{c^\times \mid c \in C\} \cup \{c^{\gamma U \delta} \mid \gamma U \delta \in Sub(\Phi)\}$

Example 9.2.5

The running example: fairness conditions $C^\times$



The unfair paths of $\mathcal{M}^\times$ have a tail $(q_3 \rightarrow^\times)^\infty$. E.g. $(q_3 \rightarrow^\times)^\infty$ and $q_3 \rightarrow^\times q_4 \rightarrow^\times (q_3 \rightarrow^\times)^\infty$.

No path of $\mathcal{M}$ corresponds to such unfair paths. Indeed, such an unfair path would correspond to a path in $\mathcal{M}$ with tail $(q_3 \rightarrow)^\infty$. But $\alpha = a \cup b$ is not satisfied in this tail.

The path $(q_3 \rightarrow)^\infty$ of $\mathcal{M}$ matches only the fair path $(q'_3 \rightarrow^\times)^\infty$.

There are infinitely many fair paths of $\mathcal{M}^\times$ in q_3 . E.g. $q_3 \rightarrow^\times q_4 \rightarrow^\times q'_3 \rightarrow^\times q_1 (\rightarrow^\times q_2)^\infty$ matches with $q_3 \rightarrow q_4 \rightarrow q_3 \rightarrow q_1 (\rightarrow q_2)^\infty$ of $\mathcal{M}$.

Example 9.2.6

Summary of the running example. We verify that $\mathcal{M}, q_3 \not\models \Phi$ where $\Phi = \neg(a \cup b)$. Indeed, the state q_3 in the constructed transition graph $\mathcal{M}^\times$ has a fair path, e.g., $q_3 \rightarrow q_4 \rightarrow \dots$. The state q_3 in $\mathcal{M}^\times$ stood for the pair $(q_3, a\bar{b}\alpha)$ where $a\bar{b}\alpha$ is itself a shorthand notation for a closed formula set that contains $\neg\Phi = \neg(\neg(a \cup b))$. This proves that $\mathcal{M}, q_3 \not\models \Phi$.

Theorem 9.2.1

A path π' is a C^U -fair path of $\mathcal{M}^\times$ iff π' is an annotated path of $\mathcal{M}$. A path π' is a $C^\times$ -fair path of $\mathcal{M}^\times$ iff π' is the annotated path of a C -fair path of $\mathcal{M}$.

Summary and complexity

Theorem 9.2.2

$\mathcal{M}, s_0 \models_{AC} \Phi$ iff $\mathcal{M}^\times$ has no $C^\times$ -fair path $(s_0, Q) \rightarrow \dots$ such that $\neg\Phi \in Q$.

Exercise 9.2.3 Prove that this follows from the previous theorem.

At last, the LTL model checking algorithm called to decide $\mathcal{M}, s_0 \models_C \Phi$ is Procedure 9.4.

Complexity of the algorithm Exploiting results for CTL model checking, we see that this algorithm is exponential in the size of Φ . Since the number of closed formula sets is exponential in the size of Φ . However, it is linear in the size of $\mathcal{M}$.

LTL model checking is harder than CTL checking because the construction of $\mathcal{M}^\times$ blows up the transition system.

Procedure 9.4: LTL model checking with fairness

Given are the fairness-constrained transition structure $\langle \mathcal{M}, C \rangle$, the state s_0 and the LTL formula Φ .

1. Compute $S^\times, \rightarrow^\times, C^\times$.
2. Compute the maximal strongly connected components SCC of $\rightarrow^\times$ that are fair to $C^\times$. Such a SCC has the property that $SSC \cap c \neq \emptyset$, for each $c \in C^\times$.
3. If some $(s_0, Q) \in S^\times$ with $\neg\Phi \in Q$ has a path to a fair strongly connected component, then return **f** else return **t**.

9.2.2 Summary of the method
Definition 9.2.9

LTL model-checking is the inference problem that takes as input:

- Σ : propositional vocabulary
- $\mathcal{M} = \langle S, \rightarrow, L \rangle$, with $L : S \rightarrow 2^\Sigma$
- C : set of CTL formulas over Σ
- $s_0 \in S$
- Φ : LTL formula over Σ

and returns $(\mathcal{M}, s_0 \models A_C \Phi)$.

Recall that $\{\neg, \wedge, X, U\}$ is an adequate set of LTL. We assume that Φ uses only the LTL connectives from this set.

Definition 9.2.10

$Sub(\Phi) = \{\varphi, \neg\varphi \mid \varphi \text{ is a subformula of } \Phi\}$

Definition 9.2.11

A set $Q \subseteq Sub(\Phi)$ is a closed formula set of Φ if:

- For each $\gamma \in Sub(\Phi)$ Q contains either γ or $\neg\gamma$.
- For each $\gamma \wedge \delta \in Sub(\Phi)$, $\gamma \wedge \delta \in Q$ iff $\gamma \in Q$ and $\delta \in Q$.
- If $\gamma U \delta \in Q$ then $\gamma \in Q$ or $\delta \in Q$.
- If $\neg(\gamma U \delta) \in Q$ then $\neg\delta \in Q$.

Definition 9.2.12

$\mathcal{C}(\Phi)$ is the set of closed formula sets of Φ .

Definition 9.2.13

$s \parallel Q$ if $s \in S, Q \in \mathcal{C}(\Phi)$ and for each atom p that appears in Φ , $p \in L(s)$ iff $p \in Q$.

Definition 9.2.14

$S^\times := \{(s, Q) \mid s \in S \wedge Q \in \mathcal{C}(\Phi) \wedge s \parallel Q\}$

Definition 9.2.15

$(s, Q) \rightarrow^\times (s', Q')$ if all following conditions hold:

- $(s, Q), (s', Q') \in S^\times$
- $s \rightarrow s'$ (consistency with $\mathcal{M}$)
- Q' is a consistent successor of Q :
 - If $X\delta \in Q$ then $\delta \in Q'$.
 - If $\neg X\delta \in Q$ then $\neg\delta \in Q'$.
 - If $\gamma U \delta \in Q$ and if $\neg\delta \in Q$, then $\gamma U \delta \in Q'$.
 - If $\neg(\gamma U \delta) \in Q$ and if $\gamma \in Q$ then $\neg(\gamma U \delta) \in Q'$.

Definition 9.2.16

$$c^{\gamma U \delta} = \{(s, Q) \in S^\times \mid \neg(\gamma U \delta) \in Q \vee \delta \in Q\}$$

Definition 9.2.17

For any $c \in C$, define $c^\times = \{(s, Q) \in S^\times \mid s \in S_c\}$

Definition 9.2.18

Define $C^\times = \{c^\times \mid c \in C\} \cup \{c^{\gamma U \delta} \mid \gamma U \delta \in \text{Sub}(\Phi)\}$

Theorem 9.2.3: Correctness theorem

$\mathcal{M}, s_0 \models_{AC} \Phi$ iff there is no path through $\rightarrow^\times$ starting in a state $(s_0, Q) \in S^\times$ with $\neg\Phi \in Q$ that is $C^\times$ -fair.

Procedure 9.5: LTL model checking with fairness

Given are the fairness-constrained transition structure $\langle \mathcal{M}, C \rangle$, the state s_0 and the LTL formula Φ .

1. Compute $S^\times, \rightarrow^\times, C^\times$.
2. Compute the strongly connected components of $\rightarrow^\times$
3. Delete all the strongly connected components S' such that $S' \cap c = \emptyset$, for some $c \in C^\times$.
4. If some $(s_0, Q) \in S^\times$ with $\neg\Phi \in Q$ has a path to one of the remaining strongly connected components then answer **f** else **t**.

From the second step on, this is the algorithm for CTL model checking applied to compute $S_{E_{C^\times} \times G \top}$.

Complexity of the algorithm The algorithm is exponential in the size of Φ since the number of closed formula sets is exponential in the size of Φ . The algorithm is linear in the size of $\mathcal{M}$. LTL model checking is harder than CTL checking because composition blows up the transition system.

9.3 Important for exam

Big questions:

- CTL model checking algorithm with fairness

- LTL model checking

Bibliography

- [1] Jean-Raymond Abrial. *Modeling in Event-B: System and Software Engineering*. Cambridge University Press, 2010. DOI: 10.1017/CB09781139195881.
- [2] R.J.R. Back. “On correct refinement of programs”. In: *Journal of Computer and System Sciences* 23.1 (1981), pp. 49–68. ISSN: 0022-0000. DOI: [https://doi.org/10.1016/0022-0000\(81\)90005-2](https://doi.org/10.1016/0022-0000(81)90005-2). URL: <https://www.sciencedirect.com/science/article/pii/0022000081900052>.
- [3] George Boole. *An investigation of the laws of thought: on which are founded the mathematical theories of logic and probabilities*. Dover, 1854. URL: <https://www.gutenberg.org/files/15114/15114-pdf.pdf>.
- [4] Dominique Cansell and Dominique Méry. “Foundations of the B method”. In: *Computing and informatics* 22.3-4 (2003), pp. 221–256.
- [5] Felix Chang. *Alloy analyzer 4.0*. <http://alloy.mit.edu/alloy4/>. 2007.
- [6] Keith L. Clark. “Negation as Failure”. In: *Logic and Data Bases*. Plenum Press, 1978, pp. 293–322. ISBN: 0-306-40060-X.
- [7] Stephen A. Cook. “The Complexity of Theorem-Proving Procedures”. In: *Proceedings of the Third Annual ACM Symposium on Theory of Computing*. STOC ’71. Shaker Heights, Ohio, USA: Association for Computing Machinery, 1971, pp. 151–158. ISBN: 9781450374644. DOI: 10.1145/800157.805047. URL: <https://doi.org/10.1145/800157.805047>.
- [8] Ingmar Dasseville, Laurent Janssens, Gerda Janssens, Jan Vanthienen, and Marc Denecker. “Combining DMN and the knowledge base paradigm for flexible decision enactment”. In: *RuleML* (July 2016).
- [9] Martin Davis, George Logemann, and Donald Loveland. “A Machine Program for Theorem-Proving”. In: *Commun. ACM* 5.7 (July 1962), pp. 394–397. ISSN: 0001-0782. DOI: 10.1145/368273.368557. URL: <https://doi.org/10.1145/368273.368557>.
- [10] Broes De Cat, Joachim Jansen, and Gerda Janssens. “IDP3: Combining symbolic and ground reasoning for model generation”. In: *2nd Workshop on Grounding and Transformations for Theories With Variables*. 2013, pp. 17–24. URL: <https://lirias.kuleuven.be/handle/123456789/413766>.
- [11] Marc Denecker, Maurice Bruynooghe, and Victor Marek. “Logic programming revisited: Logic programs as inductive definitions”. In: *ACM Trans. Comput. Log.* 2.4 (2001), pp. 623–654.
- [12] Marc Denecker and Eugenia Ternovska. “A Logic of Nonmonotone Inductive Definitions”. In: *ACM Trans. Comput. Log.* 9.2 (Apr. 2008), 14:1–14:52. ISSN: 1529-3785. URL: <http://dx.doi.org/10.1145/1342991.1342998>.
- [13] Marc Denecker and Joost Vennekens. “The Well-Founded Semantics Is the Principle of Inductive Definition, Revisited”. In: *KR*. Ed. by Chitta Baral, Giuseppe De Giacomo, and Thomas Eiter. AAAI Press, 2014, pp. 1–10. ISBN: 978-1-57735-657-8. URL: <http://www.aaai.org/ocs/index.php/KR/KR14/paper/view/7957>.

- [14] Alexander Douglas. *The Names of Syllogisms: A Guide*. Apr. 2021. URL: <https://medium.com/genus-specious/the-names-of-syllogisms-a-guide-fe62c7dfd388>.
- [15] Herbert B. Enderton. *A Mathematical Introduction To Logic*. Second. Academic Press, 2001.
- [16] David Hilbert and Wilhelm Ackermann. *Grundzüge der theoretischen Logik*. Vol. 1. Berlin, 1928, p. 73.
- [17] Michael Huth and Mark Ryan. *Logic in Computer Science: Modelling and reasoning about systems*. Cambridge university press, 2004.
- [18] *Interesting applications*. URL: https://www.cs.utexas.edu/users/moore/ac12/current/manual/index.html?topic=ACL2___INTERESTING-APPLICATIONS%7D.
- [19] Gina Kolata. “Computer Math Proof Shows Reasoning Power”. In: *The New York Times* (Dec. 10, 1996). URL: <https://archive.nytimes.com/www.nytimes.com/library/cyber/week/1210math.html%7D> (visited on Aug. 30, 2022).
- [20] Evelyn Lamb. “Two-hundred-terabyte maths proof is largest ever”. In: *Nature* 534.7605 (June 2016), pp. 17–18. ISSN: 1476-4687. DOI: 10.1038/nature.2016.19990. URL: <https://doi.org/10.1038/nature.2016.19990>.
- [21] Leslie Lamport. “Who Builds a House without Drawing Blueprints?” In: *Commun. ACM* 58.4 (Mar. 2015), pp. 38–41. ISSN: 0001-0782. DOI: 10.1145/2736348. URL: <https://doi.org/10.1145/2736348>.
- [22] John McCarthy and Patrick Hayes. “Some Philosophical Problems From the Standpoint of Artificial Intelligence”. In: *Machine Intelligence 4*. Ed. by B. Meltzer and Donald Michie. Edinburgh University Press, 1969, pp. 463–502.
- [23] Chris Newcombe, Tim Rath, Fan Zhang, Bogdan Munteanu, Marc Brooker, and Michael Deardouff. “How Amazon Web Services Uses Formal Methods”. In: *Commun. ACM* 58.4 (Mar. 2015), pp. 66–73. ISSN: 0001-0782. DOI: 10.1145/2699417. URL: <https://doi.org/10.1145/2699417>.
- [24] Ray Reiter. *Knowledge in Action: Logical Foundations for Specifying and Implementing Dynamical Systems*. MIT Press, 2001. ISBN: 9780262527002.
- [25] Murray Shanahan. *Solving the Frame Problem - a Mathematical Investigation of the Common Sense Law of Inertia*. MIT Press, 1997, pp. I–XXXIV, 1–407. URL: <http://mitpress.mit.edu/books/solving-frame-problem>.
- [26] João P. Marques Silva and Karem A. Sakallah. “GRASP - a new search algorithm for satisfiability”. In: *Proceedings of the 1996 IEEE/ACM International Conference on Computer-Aided Design, ICCAD 1996, San Jose, CA, USA, November 10-14, 1996*. Ed. by Rob A. Rutenbar and Ralph H. J. M. Otten. IEEE, 1996, pp. 220–227. DOI: 10.1109/ICCAD.1996.569607. URL: <https://doi.org/10.1109/ICCAD.1996.569607>.
- [27] Allen Van Gelder, Kenneth A. Ross, and John S. Schlipf. “The Well-Founded Semantics for General Logic Programs”. In: *J. ACM* 38.3 (1991), pp. 620–650. DOI: 10.1145/116825.116838. URL: <http://dx.doi.org/10.1145/116825.116838>.
- [28] Pieter Van Hertum, Ingmar Dasseville, Gerda Janssens, and Marc Denecker. “The KB Paradigm and Its Application to Interactive Configuration”. In: *PADL* (2016).
- [29] Moshe Y Vardi and Pierre Wolper. “Automata theoretic techniques for modal logics of programs”. In: *Proceedings of the sixteenth annual acm symposium on theory of computing*. 1984, pp. 446–456. URL: <https://dl.acm.org/doi/pdf/10.1145/800057.808711%7D>.
- [30] Wikipedia. *Harmonic oscillator* — *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Harmonic_oscillator#Equivalent_systems. [Online; accessed 2022-08-25]. 2022.

- [31] Wikipedia. *Paris Métro Line 14* — *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Paris_M%C3%A9tro_Line_14. [Online; accessed 2022-08-29]. 2022.